# UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

## FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

**GRUPO # 11**

# PLATAFORMA INTELIGENTE DE COMERCIO ELECTRÓNICO SaaS MULTITENANT PARA TIENDAS DE ROPA MASCULINA CON ARQUITECTURA OFFLINE-FIRST Y REPORTES GENERATIVOS VÍA INTELIGENCIA ARTIFICIAL — FASHIONSTORE


**INTEGRANTES:**

- **Delgado Rojas Alberto Caleb** **224027204**
- **Mujica Vallejos Andy Mauricio** **224028367**


**MATERIA**: SISTEMAS DE INFORMACIÓN II (SI2) - SA

**SEMESTRE**:2-2026

**Docente**: MSc. Ing. Angélica Garzón Cuéllar

# TABLA DE CONTENIDO (ÍNDICE GENERAL)

## Contenido

CAPÍTULO I. ASPECTOS GENERALES ..... 7
1.1 Introducción ..... 7
1.1.1 La Transformación Digital del Retail Textil y el E-Commerce de Moda Masculina ..... 7
1.1.2 Adopción de la Metodología ICONIX y Reglas de Modelado UML 2.5+ ..... 7
1.2 Planteamiento del problema ..... 8
1.2.1 Contexto Global, Regional y Local del Comercio Minorista de Moda Masculina ..... 8
1.2.2 Diagnóstico de la Situación Actual y Problemáticas Críticas del Modelo Inicial ..... 10
1.2.3 Nuevos Desafíos Estructurales y Necesidad de Escalado hacia el Segundo Parcial ..... 10
1.2.4 Formulación de la Solución Integral con FashionStore SaaS Multitenant ..... 12
1.3 Objetivos del proyecto ..... 13
Objetivo general ..... 13
1.3.2 Objetivos específicos ..... 14
1.4 Justificación ..... 15
1.4.1 Justificación Técnica y Arquitectónica ..... 15
1.4.2 Justificación Comercial, Económica y de Escalabilidad (SaaS) ..... 16
1.4.3 Justificación Social y de Resiliencia Operativa en el Contexto Boliviano ..... 16
1.4.4 Justificación de Inteligencia de Negocios y Machine Learning ..... 16
1.4.5 Justificación Académica y Metodológica (ICONIX vs PUDS) ..... 17
1.5 Alcance y limitaciones ..... 17
1.5.1 Módulos del Sistema ..... 17
1.5.2 Los Seis Desafíos Tecnológicos del Segundo Parcial ..... 23
1.5.3 Planes de Suscripción del Prototipo ..... 24
1.5.4 Matriz de Funcionalidades por Plataforma y Modo de Operación ..... 25
1.5.6 Matriz de Plataformas y Despliegue Operativo ..... 25
1.5.7 Restricciones y Limitaciones del Proyecto ..... 26
1.6 Metodología de desarrollo ICONIX ..... 27
1.6.1 Definición, Principios y Filosofía Pragmática ..... 27
1.6.2 Comparativa Exhaustiva entre PUDS (Primer Parcial) e ICONIX (Segundo Parcial) ..... 27
1.6.3 Las Cuatro Fases de la Metodología ICONIX ..... 28
1.6.4 Trazabilidad Sistemática en ICONIX ..... 31

1.6.5 Plantilla Oficial de Ficha de Caso de Uso Adoptada ....31
1.6.6 Estrategia de Adaptación e Integración de los Tres Ciclos Previos ....31
CAPÍTULO II. ANÁLISIS DE REQUISITOS ....33
2.1 Descripción del Sistema Propuesto ....33
2.1.1 Visión General, Contexto de Negocio y Oportunidad Tecnológica ....33
2.1.2 Diagrama Arquitectónico del Ecosistema Propuesto ....33
2.1.3 Ecosistema de Plataformas y Despliegue Multiplataforma ....35
2.1.4 Los Nueve Pilares Funcionales del Sistema Propuesto ....36
2.1.5 Matriz de Integración Tecnológica, Protocolos y Seguridad ....39
2.2 Identificación de Actores ....40
2.3 Requisitos Funcionales y No Funcionales ....41
2.3.1. Requisitos Funcionales ....41
2.3.2. Requisitos No Funcionales ....42
2.4 Reglas de Negocio ....45
2.4.1 Reglas del Negocio Base ....45
2.4.2 Reglas SaaS y Multitenant ....46
2.4.3 Reglas Offline y Sincronización ....47
2.4.4 Reglas de Cambio y Devolución de Prendas ....48
2.4.5 Reglas de Aprovisionamiento y Compras B2B desde Web Desktop ....49
2.4.6 Reglas de Ofertas de Liquidación y Precios Diferenciados ....50
2.4.7 Reglas de Inteligencia Artificial Generativa y Machine Learning ....50
2.4.8 Matriz Consolidada de Reglas de Negocio (Mapeo RN01 - RN18) ....51
2.5 Diagrama UML de Casos de Uso ....53
2.5.1. Matriz Integral y Lista Consolidada de Casos de Uso del Segundo Parcial ....53
2.6. Especificación de Casos de Uso ....56
CU01: Gestionar Empresas (Tenants) ....56
CU02: Gestionar Planes de Suscripción ....57
CU03: Gestionar Suscripción de Empresa ....59
CU04: Autenticar Usuario Multitenant ....61
CU05: Gestionar Usuarios y Roles por Empresa ....63
CU06: Gestionar Productos y Atributos Textiles Avanzados ....64
CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP) ....66
CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX ....68

3.1. Modelo de Dominio ..... 68
3.2. Diagramas de Robustez (Análisis y Diseño Preliminar ICONIX) ..... 70
CU01: Gestionar Empresas (Tenants) ..... 71
CU02: Gestionar Planes de Suscripción ..... 71
CU03: Gestionar Suscripción de Empresa ..... 72
CU04: Autenticar Usuario Multitenant ..... 72
CU05: Gestionar Usuarios y Roles por Empresa ..... 72
CU06: Gestionar Productos y Atributos Textiles Avanzados ..... 73
CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP) ..... 73
3.3. Diagramas de Secuencia UML ..... 73
CU01: Gestionar Empresas (Tenants) ..... 74
CU02: Gestionar Planes de Suscripción ..... 75
CU03: Gestionar Suscripción de Empresa ..... 76
CU04: Autenticar Usuario Multitenant ..... 77
CU05: Gestionar Usuarios y Roles por Empresa ..... 78
CU06: Gestionar Productos y Atributos Textiles Avanzados ..... 79
CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP) ..... 80
3.4 Diagrama de Clases de Análisis ..... 80
CU01: Gestionar Empresas (Tenants) ..... 81
CU02: Gestionar Planes de Suscripción ..... 81
CU03: Gestionar Suscripción de Empresa ..... 81
CU04: Autenticar Usuario Multitenant ..... 82
CU05: Gestionar Usuarios y Roles por Empresa ..... 82
CU06: Gestionar Productos y Atributos Textiles Avanzados ..... 82
CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP) ..... 83
3.5. Diseño de la Base de Datos (Modelo Físico y Esquemas de Persistencia) ..... 83
3.5.1. Diseño Lógico de la Base de Datos (Modelo Relacional / Dominio — 2do Parcial) ..... 83
3.5.2. Diseño Físico de la Base de Datos (Esquemas de Persistencia DDL Corregidos) ..... 84
3.6. Diseño de interfaces de usuario ..... 95
CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN ..... 96
4.1. Herramientas y tecnologías utilizadas ..... 96
4.2. Diseño de la arquitectura del sistema ..... 97

4.2.1. Diseño Lógico de la Arquitectura: Diagrama de Paquetes en 3 Capas (UML) .....97
4.2.1. Diseño Físico de la Arquitectura: Diagrama de Despliegue (UML) .....99
4.3. Implementación de la base de datos .....101
4.3.1. Configuración de PostgreSQL 15 y Pool Asíncrono (SQLAlchemy 2.0 Asyncio + AsyncPG) .....101
4.3.2. Aislamiento Multitenant en Persistencia .....102
4.3.3. Persistencia Local Móvil (SQLite + sqflite en Flutter) .....102
4.3.4. Persistencia Local Web (IndexedDB + Dexie.js en Angular PWA) .....103
4.3.5. Control de Migraciones del Esquema Relacional (Alembic) .....104
4.4. Desarrollo de los módulos del sistema .....104
4.4.1. Módulo 1: Seguridad, Multitenancy y Administración Global (CU01 al CU05) .....104
4.4.2. Módulo 2: Catálogo Textil e Inventario Multi-Sucursal con CPP y Kardex (CU06, CU07, CU14, CU20) .....106
4.4.3. Módulo 3: Operaciones Offline, Ventas POS y Motor de Sincronización (CU08 al CU12) .....109
4.4.4. Módulo 4: Ventas, Gestión de Cambio de Prenda en Dos Tiempos y Pasarela Stripe (CU13, CU18, CU19) .....111
4.4.5. Módulo 5: Inteligencia Artificial Generativa y Analítica Predictiva con Machine Learning (CU15 al CU17) .....113
4.5. Integración de componentes .....115
4.5.1. Diagrama de Integración General de Componentes del Sistema (UML 2.5+) .....115
4.5.2. Diagramas de Componentes por Paquetes y Subsistemas (UML 2.5+) .....117
4.5.3. Mecanismos de Integración e Interfaz (Protocolos y Contratos REST / JSON) .....121
4.5.4. Entornos de Despliegue y URLs Públicas de Producción .....122
CAPÍTULO V. PRUEBAS Y RESULTADOS .....122
5.1. Plan de pruebas .....122
5.2. Pruebas funcionales .....123
5.3. Pruebas de integración .....123
TI01: Sincronización Offline End-to-End con Reconciliación de Stock .....123
TI02: Flujo Completo de Cambio de Prenda con Pasarela Stripe .....124
TI03: Verificación de Aislamiento Estricto Multitenant .....124
TI04: Generación y Ejecución Segura de Reportes IA .....125
5.4. Corrección de errores .....125
5.5. Resultados de las pruebas .....126
5.6. Casos de prueba .....126

5.6.1. Fichas de Casos de Prueba (Pruebas de Aceptación) ..... 126
CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES ..... 139
6.1. Conclusiones ..... 139
6.2. Recomendaciones ..... 139
BIBLIOGRAFIA ..... 139
ANEXOS ..... 139
Anexo A: Artefactos de la Etapa Fundacional (Primer Parcial) ..... 139
Anexo A.1: Lista de Casos de Uso del Primer Parcial (CU01 al CU25) ..... 139
Anexo A.2: Módulos del Sistema del Primer Parcial (M01 al M19 — Etapa Fundacional) ..... 141
Anexo B: Tabla Comparativa de Planes de Suscripción ..... 150
Anexo C: Referencias ..... 150
Enlace repositorio y QR ..... 152
Enlace despliegue ..... 152

# CAPÍTULO I. ASPECTOS GENERALES

## 1.1 Introducción

### 1.1.1 La Transformación Digital del Retail Textil y el E-Commerce de Moda Masculina

El comercio electrónico (*e-commerce*) ha trascendido de manera irreversible su concepción inicial como un canal comercial accesorio o secundario para consolidarse como el eje motriz de la competitividad, escalabilidad y rentabilidad en la economía digital contemporánea. En la industria global de la confección, indumentaria y moda (*apparel retail*), los canales de comercialización digital experimentan tasas de expansión interanuales vertiginosas, apalancadas por la ubicuidad de los teléfonos inteligentes (*m-commerce*), la sofisticación de los hábitos de consumo en línea y la creciente exigencia de experiencias omnicanales perfectamente fluidas (*unified commerce*).

Dentro de este ecosistema, el segmento específico de la **moda y confección masculina** ha cobrado una relevancia económica sin precedentes. A diferencia de patrones de consumo basados en la navegación recreativa o la compra impulsiva por entretenimiento, las investigaciones de mercado contemporáneas (como los reportes sectoriales de *McKinsey & Company* y *Statista Retail Trends*) demuestran que el comprador masculino moderno opera guiado por premisas altamente estructuradas: **racionalidad operativa, inmediatez en la transacción, certidumbre matemática en la disponibilidad física de las prendas, exactitud en las dimensiones antropométricas y textiles, y personalización estilística orientada a propósitos concretos**. El consumidor masculino busca resolver requerimientos específicos de vestimenta desde sastrería ejecutiva y atuendos corporativos hasta indumentaria informal urbana o prendas estacionales de alta tecnología— optimizando su tiempo personal y exigiendo la total erradicación de la incertidumbre en el calce, los cortes y las tallas.

Sin embargo, la inserción del comercio minorista textil en la economía digital en los países en desarrollo, y muy especialmente en **Bolivia y su contexto regional** (con énfasis en el eje troncal: Santa Cruz de la Sierra, La Paz y Cochabamba), continúa enfrentando profundas asimetrías estructurales:

- **Caos dimensional y falta de estandarización en el tallaje**
- **Informalidad digital y fragmentación transaccional**
- **Desconexión entre sucursales físicas**
- **Distorsión en la valuación de inventarios**
- **Fragilidad de la conectividad a Internet**

### 1.1.2 Adopción de la Metodología ICONIX y Reglas de Modelado UML 2.5+

Para gobernar el desarrollo integral de este ecosistema en un marco de alta rigurosidad y en un periodo intensivo de trabajo, se sustituye el marco PUDS y se adopta formalmente la **metodología ICONIX**, un proceso pesado-ligero centrado en casos de uso que optimiza la carga documental mediante la aplicación del **principio 80/20 de UML** (utilizar aproximadamente el 20% medular de los artefactos del estándar para resolver el 80% de los problemas de modelado y diseño).

El proceso de ingeniería se estructura a través de sus **cuatro fases secuenciales e iterativas**:

- **Fase 1: Análisis de Requisitos:** Delimitación de requisitos funcionales y no funcionales, construcción del Modelo de Dominio conceptual y especificación de fichas de casos de uso

estandarizadas acompañadas de sus prototipos de interfaz de usuario (*wireframes/storyboards*).

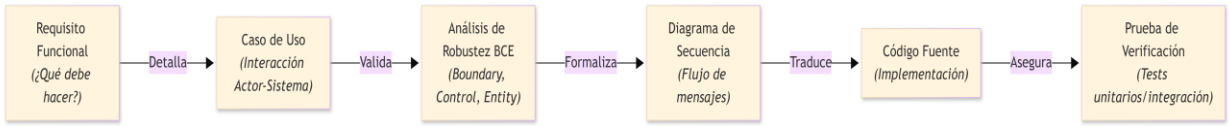
- **Fase 2: Análisis y Diseño Preliminar:** Construcción sistemática de los **Diagramas de Robustez Boundary-Controller-Entity (BCE)**, sirviendo como puente indispensable entre la visión del negocio y la arquitectura técnica. En estricto cumplimiento de las directrices de la cátedra :

$$\text{Actor} \longleftrightarrow \text{Boundary (Interfaz)} \longleftrightarrow \text{Controller (Gestor)} \longleftrightarrow \text{Entity (Datos)}$$

$$\textbf{Regla Inquebrantable: CERO Comunicación Entidad} \longleftrightarrow \textbf{Entidad}$$

  Toda interacción entre entidades de datos debe ser coordinada obligatoriamente por una clase de control (Gestor...).

- **Fase 3: Diseño:** Elaboración de los **Diagramas de Secuencia** con fragmentos combinados (alt, opt, loop). Bajo las directrices de la cátedra, **los fragmentos de repetición (loop) NUNCA deben encerrar a la interfaz de usuario (Boundary) ni a los actores**, circunscribiéndose exclusivamente a la interacción interna entre el Gestor (Control) y las Entidades de datos. Paralelamente, se define el **Diagrama de Clases Lógicas** con tipado de datos completo, métodos, firmas, multiplicidades y el atributo discriminador empresa_id, culminando con el diseño de arquitectura física (Componentes y Despliegue con nodos, puertos y protocolos).
- **Fase 4: Implementación y Pruebas:** Codificación estructurada bajo arquitectura limpia y ejecución de una matriz formal de casos de prueba unitarios, de integración y verificación de sincronización offline, garantizando una **trazabilidad vertical absoluta y comprobable**:




## 1.2 Planteamiento del problema

### 1.2.1 Contexto Global, Regional y Local del Comercio Minorista de Moda Masculina

El comercio minorista de prendas de vestir (*apparel retail*) representa una de las industrias de mayor dinamismo y volumen económico a nivel global. Tradicionalmente estructurada sobre un modelo de interacción eminentemente sensorial —donde el cliente acude físicamente a la tienda, examina la textura y peso de las telas, aprecia la caída de los textiles, utiliza probadores para verificar el ajuste anatómico y recibe la orientación directa de dependientes—, la industria de la moda se ha visto forzada a emprender una profunda metamorfosis tecnológica. El auge del comercio móvil (*m-commerce*), la exigencia de inmediatez y la consolidación de estrategias omnicanales han derribado las barreras entre el espacio comercial físico y las plataformas virtuales.

En el caso específico de la **indumentaria masculina**, las características psicográficas y conductuales del consumidor imponen condiciones muy exigentes. Diversos estudios de comportamiento del consumidor señalan que los varones priorizan la conveniencia, la exactitud y la economía de tiempo. Cuando un comprador masculino busca una prenda de vestir, generalmente persigue un objetivo concreto: renovar su vestuario laboral con camisas formales, adquirir un traje para un evento

específico o proveerse de ropa deportiva o casual con características funcionales definidas. Experimentar fricciones en la experiencia de compra —como colas prolongadas frente a probadores, desorden en estanterías, falta de claridad en las medidas o incertidumbre respecto a si una compra digital le calzará adecuadamente— genera una tasa de abandono que supera el **40%** en tiendas físicas y desmotiva las compras en canales digitales.

**La Realidad Estructural del Mercado Textil en Bolivia**

Al contrastar estas dinámicas globales con el entorno socioeconómico y tecnológico de **Bolivia** (particularmente en las urbes de Santa Cruz de la Sierra, La Paz, Cochabamba y ciudades intermedias), el comercio de moda masculina enfrenta fallas estructurales agudas:

1. **Heterogeneidad Descontrolada y Caos de Tallas:** En el mercado nacional convergen prendas de múltiples orígenes: confecciones de talleres nacionales, manufacturas importadas de países limítrofes (Brasil, Colombia, Perú), prendas de origen norteamericano y un masivo volumen de confecciones asiáticas. Dado que no existe una norma técnica antropométrica boliviana de estandarización de tallas, la etiqueta nominal (S, M, L, XL, o numéricas 38, 40, 42) carece de significado homogéneo. Una camisa talla "L" de origen asiático resulta con frecuencia equivalente a una talla "S" de confección norteamericana. Los clientes que compran por Internet o canales de mensajería sin conocer las **medidas exactas en centímetros (pecho, cintura, largo)** sufren una constante decepción al recibir prendas incompatibles, generando desconfianza hacia el comercio electrónico formal.
2. **La Trampa de la Informalidad Digital:** Gran parte de las empresas textiles que afirman contar con "tienda virtual" carecen de plataformas transaccionales verdaderas. Su operativa descansa en la publicación de catálogos no sincronizados en redes sociales (Facebook, Instagram, WhatsApp). El proceso de venta es manual, precario e ineficiente: un cliente consulta la disponibilidad de una prenda, un empleado debe buscar físicamente la ropa en perchas o bodegas para responder horas más tarde, el pago se efectúa mediante transferencias QR informales que requieren enviar la fotografía del comprobante bancario para su verificación visual, y los despachos se coordinan de forma precaria con servicios de radiotaxi. Este flujo artesanal no escala, genera duplicidad de ventas sobre prendas ya agotadas (*overselling*), carece de trazabilidad contable y proyecta una imagen de poca seriedad que frena la fidelización de clientes de alto valor.
3. **Silos Operativos y Falta de Visibilidad Multi-Sucursal:** Las cadenas comerciales medianas que cuentan con dos o más tiendas físicas dentro de una misma ciudad operan sus locales como compartimentos estancos. Las ventas se registran en cuadernos físicos o en terminales aisladas con bases de datos locales no sincronizadas. Si un cliente solicita una camisa de su talla en la sucursal de una zona comercial y la prenda está agotada, el personal de mostrador no tiene visibilidad sobre si dicha prenda existe en la sucursal ubicada a pocos kilómetros. La venta se pierde irremediablemente, mientras en el otro punto de venta la mercadería permanece inmovilizada hasta convertirse en saldo obsoleto.
4. **Inexactitud Financiera y Vulnerabilidad Tributaria por Omisión del CPP:** Debido a factores macroeconómicos, fluctuaciones en el tipo de cambio de divisas para insumos importados y variaciones en fletes de transporte, los comerciantes de ropa adquieren lotes sucesivos de prendas idénticas a costos unitarios notablemente dispares. La práctica empírica habitual en las PyMEs bolivianas consiste en registrar todo el stock asignándole el valor del último precio de compra recibido. Esta omisión del cálculo riguroso del **Costo**

**Promedio Ponderado (CPP)** distorsiona por completo la valuación de los activos en el balance general, proyecta márgenes brutos de rentabilidad ficticios y coloca a la empresa en situación de contingencia fiscal frente al Servicio de Impuestos Nacionales al no poder sustentar fehacientemente su costo de ventas en el marco de la Ley 843 (IVA 13%, IT 3%).

5. **Inestabilidad de Telecomunicaciones y Vulnerabilidad en Tiendas:** Las redes de telecomunicaciones en Bolivia padecen de recurrentes caídas de servicio, microcortes de fibra óptica e interrupciones del suministro de datos celulares, particularmente severas en los sótanos y zonas interiores de centros comerciales y galerías populares. La mayoría de los sistemas web modernos colapsan ante la pérdida de conexión: la pantalla se congela, el cajero no puede emitir recibos ni consultar precios, las filas de clientes se acumulan y el establecimiento se ve forzado a paralizar sus operaciones, sufriendo cuantiosas pérdidas financieras y daño a su reputación.

### 1.2.2 Diagnóstico de la Situación Actual y Problemáticas Críticas del Modelo Inicial

A partir del análisis técnico del prototipo desarrollado en el primer parcial y de la recopilación de requerimientos del sector comercial, se sintetizan las problemáticas operativas centrales que sustentaron la concepción inicial del sistema:

1. **La Brecha Experiencial Físico-Digital y la Crisis Logística de Devoluciones:** La imposibilidad de ensayar virtualmente las prendas antes del pago, sumada a la ausencia de fichas con medidas anatómicas en centímetros, generaba tasas de devolución superiores al **30%** en ventas digitales, ocasionando absorción de fletes de transporte inverso, deterioro de etiquetas y pérdida de margen comercial.
2. **Desarticulación en el Inventario Multi-Sucursal:** Falta de un kardex centralizado en tiempo real, lo que provocaba desabastecimiento en tiendas de alta afluencia y acumulación de sobrestock en sucursales secundarias.
3. **Distorsión Contable en Existencias:** Carencia de un recálculo automatizado del Costo Promedio Ponderado tras cada compra de abastecimiento a proveedores.
4. **Ineficiencia en Sala de Ventas y Probadores:** Tiempos de espera excesivos de los clientes en tiendas físicas para probarse prendas, sin mecanismos de reserva previa de probadores con preselección digital de ropa.
5. **Experiencia Estática y Falta de Personalización:** Ausencia de asesoría estilística contextualizada a factores climáticos y ocasiones de uso, junto con la falta de esquemas de fidelización por niveles de cliente.

### 1.2.3 Nuevos Desafíos Estructurales y Necesidad de Escalado hacia el Segundo Parcial

Si bien el prototipo del primer parcial solventó con éxito las problemáticas iniciales para una tienda específica, el análisis de escalabilidad técnica y comercial reveló **seis nuevas limitaciones estructurales** que motivan de forma ineludible la evolución hacia el Segundo Parcial:

**A. Modelo Monolítico de Negocio y Ausencia de Escalabilidad Multi-Empresa (SaaS)**

FashionStore fue concebida inicialmente como una solución privativa para una sola marca comercial. Este esquema limita la rentabilidad del desarrollo: no puede ser adoptado por otras cadenas o

boutiques de confección sin duplicar servidores, bases de datos y costos de mantenimiento. En la industria moderna del software, el modelo **SaaS (Software as a Service)** permite comercializar la plataforma a decenas de empresas suscriptoras (**tenants**). Cada empresa debe operar de forma totalmente aislada e independiente (con sus propios roles, empleados, inventarios, precios y clientes), seleccionando planes de suscripción (Básico o Profesional) acordes a su volumen de negocio. El prototipo del primer parcial carecía de esta capacidad multi-empresa.

**B. Vulnerabilidad Crítica ante Interrupciones de Conectividad a Internet (Realidad Boliviana)**

El prototipo inicial dependía rígidamente de una conexión ininterrumpida al backend central en la nube. En la realidad boliviana, las interrupciones del servicio de Internet —debido a fallas de proveedores (Entel, Tigo, AXS), cortes de fibra óptica o saturación en centros comerciales— son frecuentes. Bajo el esquema inicial, un corte de red paraliza la caja POS de la sucursal: no se pueden facturar ventas de mostrador, no se pueden registrar clientes y no se pueden confirmar reservas, provocando filas y pérdidas económicas. De igual modo, si un cliente en la app móvil transita por una zona sin cobertura celular, la aplicación deja de responder. Resulta indispensable dotar a FashionStore de **resiliencia offline**: la web debe operar como una **Progressive Web App (PWA) con IndexedDB** y la app móvil debe incorporar **SQLite local**, permitiendo transacciones de mostrador sin red y ejecutando una **sincronización idempotente posterior** cuando se restablezca la conectividad.

**C. Carencia de Analítica Predictiva, Modelos de Machine Learning y Reportes Generativos**

Los tableros de control del primer parcial ofrecían métricas estáticas basadas en consultas SQL predeterminadas. Los directivos de las empresas no podían solicitar información ad-hoc ni hacer preguntas analíticas en lenguaje natural (ej. *"¿Cuáles fueron los 3 trajes más vendidos en la sucursal Equipetrol durante el último mes?"*). Asimismo, la plataforma carecía de modelos de **Machine Learning** que permitieran anticipar la demanda futura de prendas (modelos supervisados de regresión/series temporales) o segmentar clientes y detectar anomalías en existencias (modelos no supervisados de clustering y aislamiento). Sin un pipeline de datos sintéticos representativo de al menos 6 meses y algoritmos de ML, la toma de decisiones gerenciales continuaba siendo puramente reactiva.

**D. Gestión Incompleta del Ciclo Post-Venta: Políticas de Cambio de Prendas con Doble Registro**

El prototipo inicial implementó la venta y la entrega, pero omitió el tratamiento formal de los **cambios y devoluciones de prendas**. En la práctica del comercio textil, cuando un cliente requiere cambiar una prenda (frecuentemente por falta de calce en talla o preferencia de color), el sistema debe permitir formalizar la operación en mostrador dentro de un plazo normado (ej. 48 horas a 7 días hábiles). Dicho cambio exige emitir contablemente una **Nota de Devolución** (para reingresar la prenda devuelta al stock de la sucursal y recuperar su CPP histórico) y una **Nota de Cambio** (para despachar la nueva prenda entregada), liquidando con exactitud si existe un **diferencial de precio** por variación de talla o modelo, o aplicando una **política de devolución monetaria en 30 días** si el cliente no encuentra un reemplazo de su agrado.

**E. Ausencia de Estrategias de Precios Diferenciados (Mayorista y Liquidación Estacional)**

El modelo original aplicaba un precio de venta rígido y unitario. El retail de moda requiere flexibilizar tarifas para incentivar la venta por volumen:

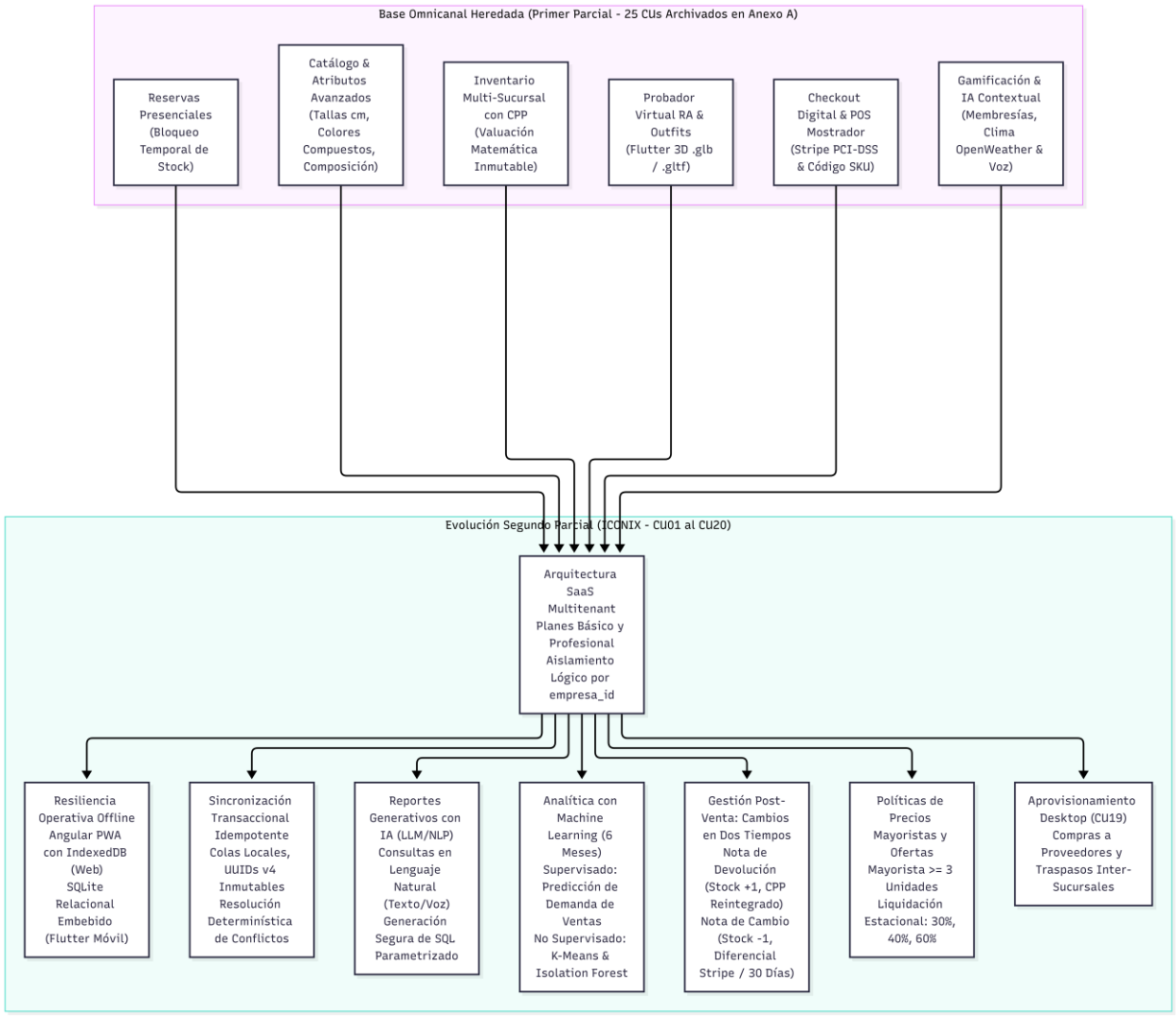
- **Precios al por Mayor:** Aplicables automáticamente a clientes que adquieren compras iguales o superiores a **3 unidades**, dinamizando la venta corporativa y comercial.
- **Ofertas de Liquidación Estacional:** Descuentos promocionales configurables (**30%, 40% y 60%**) para mercadería de fin de temporada, esenciales para desocupar bodegas y recuperar el capital inmovilizado.

**F. Desarticulación del Canal de Compras y Aprovisionamiento Empresarial (CU20)**

Las compras de mercadería a proveedores deben realizarse de manera controlada y centralizada a través del canal **Web Desktop** por el Encargado de Aprovisionamiento o el Administrador de Empresa. El ingreso de lotes de compra debe actualizar de forma atómica el stock físico, registrar los precios pactados y disparar el recálculo automático inmutable del Costo Promedio Ponderado en el kardex central.

### 1.2.4 Formulación de la Solución Integral con FashionStore SaaS Multitenant

Frente a la conjunción de los desafíos de la industria de la moda masculina y las exigencias de escalabilidad, resiliencia y analítica del Segundo Parcial, se formula la arquitectura integral de **FashionStore**:




## 1.3 Objetivos del proyecto

### Objetivo general

Desarrollar y evolucionar la plataforma inteligente de comercio electrónico FashionStore hacia una solución integral de software como servicio (**SaaS**) multitenant, resiliente a contingencias de conectividad y potenciada con analítica avanzada, aplicando rigurosamente la metodología pesada-ligera **ICONIX** y el estándar **UML 2.5+**, con el propósito de dotar a múltiples empresas del sector de confección textil de un ecosistema omnicanal seguro y escalable, capaz de sostener transacciones de venta y reserva ininterrumpidas en modo offline (PWA con IndexedDB y Flutter con SQLite), sincronizar datos de forma idempotente, dinamizar la toma de decisiones gerenciales mediante reportes generativos con inteligencia artificial y modelos de Machine Learning, y gobernar formalmente los procesos de compras de aprovisionamiento, traspasos inter-sucursales y cambios de indumentaria con doble registro documental.

### 1.3.2 Objetivos específicos

Para materializar con éxito el objetivo general, se formulan los siguientes objetivos específicos estructurados según el ciclo de ingeniería del software y los componentes arquitectónicos del sistema:

1. **Modelar la Arquitectura del Sistema bajo la Metodología ICONIX:** Elaborar de forma progresiva y trazable los artefactos medulares de ICONIX (Modelo de Dominio conceptual, especificación estandarizada de Fichas de Casos de Uso con prototipos de interfaz, Diagramas de Robustez BCE sin enlaces directos Entidad-Entidad, Diagramas de Secuencia con fragmentos estructurados alt, opt y loop —asegurando que loop excluya a las interfaces de usuario—, Diagrama de Clases Lógicas tipado y esquemas físicos de base de datos), asegurando la trazabilidad vertical desde los requisitos de negocio hasta las pruebas de aceptación.
2. **Diseñar e Implementar la Infraestructura SaaS Multitenant y Seguridad RBAC:** Construir el modelo relacional multitenant en PostgreSQL mediante clave discriminadora (empresa_id) a nivel de fila y middleware de intercepción en FastAPI, garantizando el aislamiento criptográfico estricto de los datos entre empresas contratantes e implementando la administración de **Planes de Suscripción (Básico y Profesional)** para regular las cuotas de sucursales, usuarios operativos y funcionalidades avanzadas habilitadas.
3. **Construir la Aplicación Web Progresiva (PWA) con Persistencia Local en IndexedDB:** Desarrollar sobre Angular el soporte de Progressive Web App instalable con Service Workers para cacheo de recursos visuales y la API de almacenamiento local **IndexedDB**, permitiendo a los operadores de sala de ventas y cajeros consultar el catálogo de prendas, registrar reservas presenciales y procesar ventas de mostrador de contado de forma autónoma durante interrupciones del enlace a Internet.
4. **Desarrollar la Capa de Datos Móvil Desconectada con Flutter y SQLite:** Implementar un motor relacional embebido local en la aplicación móvil Flutter mediante la biblioteca **SQLite (sqflite)**, posibilitando la precarga de catálogos de indumentaria, la gestión local de apartados en vestidores virtuales y la captura de transacciones comerciales en zonas de débil o nula cobertura celular.
5. **Implementar el Motor de Sincronización Transaccional Idempotente y Resolución de Conflictos:** Diseñar un protocolo de sincronización asíncrona bidireccional basado en identificadores universales únicos (**UUID v4 inmutables**) y colas de despacho local, acompañado de un middleware de verificación de idempotencia en FastAPI que procese transacciones diferidas sin duplicar asientos contables ni cobros, dirimiendo determinísticamente eventuales colisiones de inventario concurrente.
6. **Formalizar el Circuito Logístico de Post-Venta y Cambios de Ropa en Dos Tiempos:** Programar el módulo de cambios de indumentaria en mostrador bajo el principio contable de doble comprobante: emisión de la **Nota de Devolución** (reingreso físico a bodega, incremento de existencias y reintegro del CPP histórico) y emisión de la **Nota de Cambio** (egreso de nueva prenda, descuento de stock y liquidación del diferencial de precio vía pasarela Stripe Sandbox o ejecución de política de reembolso monetario en 30 días).
7. **Automatizar las Políticas Comerciales de Precios Diferenciados y Liquidación:** Implementar el motor de tarificación flexible que aplique automáticamente

descuentos por **precio al por mayor en compras iguales o mayores a 3 unidades**, y descuentos escalonados de **liquidación estacional (30%, 40% y 60%)** para prendas de colecciones pasadas, administrando atributos avanzados de producto (medidas en centímetros, colores compuestos con códigos Hex y propiedades hipoalergénicas).

8. **Integrar el Canal de Aprovisionamiento y Traspasos Inter-Sucursales desde Web Desktop:** Desarrollar el módulo de compras centralizadas a proveedores (**CU19**) accesible exclusivamente desde la aplicación Web Desktop por el Encargado de Aprovisionamiento o Administrador de Empresa, calculando en tiempo real el nuevo Costo Promedio Ponderado del inventario y gestionando las transferencias físicas de mercadería entre tiendas de una misma empresa.
9. **Desplegar el Asistente de Reportes Generativos con Inteligencia Artificial y NLP:** Integrar un motor conversacional de procesamiento de lenguaje natural (consumiendo APIs de modelos de lenguaje LLM) capaz de interpretar solicitudes de información gerencial formuladas en lenguaje natural (escritas o dictadas por voz), traduciéndolas a sentencias SQL parametrizadas, validadas contra inyecciones y restringidas por empresa_id, renderizando cuadros interactivos, gráficos y conclusiones analíticas.
10. **Entrenar y Visualizar Modelos de Machine Learning con Dataset Sintético de 6 Meses:** Construir un pipeline analítico alimentado por más de 6 meses de actividad transaccional sintética realista para entrenar modelos de aprendizaje supervisado (predicción de demanda de ventas para optimizar abastecimiento) y no supervisado (segmentación de clientes mediante K-Means y detección de anomalías en rotación de inventario con Isolation Forest), integrándolos en un dashboard analítico ejecutivo.
11. **Ejecutar Pruebas Integrales de Software y Aseguramiento de Calidad:** Diseñar y ejecutar una matriz exhaustiva de casos de prueba formales (pruebas funcionales por caso de uso, pruebas de integración end-to-end, pruebas de corte y recuperación de red, y pruebas de estrés de sincronización idempotente), certificando el 100% de cumplimiento de los requisitos funcionales y reglas de negocio del sistema.

## 1.4 Justificación

La evolución estratégica y tecnológica de FashionStore se fundamenta rigurosamente en cinco dimensiones de análisis:

### 1.4.1 Justificación Técnica y Arquitectónica

Desde la perspectiva de la ingeniería de software moderna, migrar un sistema propietario o monoproyecto hacia una **arquitectura SaaS Multitenant con base de datos compartida y aislamiento lógico (empresa_id)** constituye el estándar de máxima eficiencia en la industria digital. Permite operar un único código base (*single codebase*), centralizar los despliegues continuos, optimizar la utilización de recursos computacionales en la nube y reducir drásticamente los costos de infraestructura y mantenimiento evolutivo.

Asimismo, la implementación de un paradigma **Offline-First** combinando **Angular PWA (Service Workers + IndexedDB)** en el entorno web y **Flutter (SQLite embebido)** en el entorno móvil resuelve uno de los problemas arquitectónicos más desafiantes en sistemas distribuidos: la tolerancia a fallos de partición de red (*Teorema CAP*). Al incorporar identificadores inmutables **UUID v4** como claves de

idempotencia y despachos atómicos por lotes, el sistema garantiza consistencia eventual, durabilidad transaccional y protección contra duplicidad de datos, demostrando solvencia ingenieril en el diseño de software de misión crítica.

### 1.4.2 Justificación Comercial, Económica y de Escalabilidad (SaaS)

Desarrollar soluciones informáticas a medida y de forma aislada para cada pequeña o mediana empresa del sector textil resulta económicamente inviable, demanda inversiones de capital inasumibles para boutiques locales y restringe severamente el retorno sobre la inversión (ROI) para la empresa de desarrollo.

Al transformar FashionStore en un producto SaaS comercializable bajo planes escalonados (**Plan Básico y Plan Profesional**), se democratiza el acceso a tecnología de frontera (probadores virtuales en Realidad Aumentada, analítica predictiva con Machine Learning y reportes generativos con IA) para cientos de empresas de confección y venta de ropa que de otro modo quedarían marginadas de la economía digital. Para la empresa proveedora de FashionStore, el modelo SaaS instituye un flujo de ingresos recurrentes, predecibles y escalables (MRR - *Monthly Recurring Revenue*), donde la incorporación de nuevos clientes genera un costo marginal cercano a cero. Adicionalmente, las reglas comerciales de precios mayoristas (>= 3 unidades) y liquidación de inventario estacional (30%, 40%, 60%) maximizan el flujo de caja y la recuperación de capital para las tiendas suscriptoras.

### 1.4.3 Justificación Social y de Resiliencia Operativa en el Contexto Boliviano

En el contexto comercial de Bolivia, la infraestructura de telecomunicaciones presenta notorias vulnerabilidades territoriales. En centros comerciales de alta concurrencia, ferias comerciales densas o galerías céntricas, las saturaciones y caídas del servicio de Internet son una constante cotidiana. Imponer a los comercios una dependencia total de la nube significa forzarlos a suspender cobros y cerrar cajas ante el primer corte de red.

Al dotar a FashionStore de capacidad operativa fuera de línea para ventas de mostrador y reservas de probadores, se protege la estabilidad operativa de las tiendas físicas, se salvaguarda el empleo de los dependientes y se previene la fuga económica de clientes. Asimismo, al facilitar la inclusión de tallas anatómicas en centímetros, colores compuestos y especificaciones textiles, se combate la informalidad del comercio informal, promoviendo la formalización del retail de indumentaria y el cumplimiento de las normativas tributarias nacionales mediante la correcta aplicación del Costo Promedio Ponderado.

### 1.4.4 Justificación de Inteligencia de Negocios y Machine Learning

La enorme cantidad de transacciones que genera el retail de moda (ventas, pedidos, reservas, mermas y devoluciones) carece de valor estratégico si los directivos no cuentan con instrumentos ágiles para su interpretación. En las empresas medianas, los gerentes rara vez dominan el lenguaje SQL para extraer reportes ad-hoc de sus bases de datos.

La integración de **Reportes Generativos asistidos por Inteligencia Artificial (LLM / NLP)** democratiza de forma radical la toma de decisiones: un directivo puede formular una pregunta en lenguaje natural desde su computadora o dictarla mediante comandos de voz desde su teléfono inteligente y recibir en segundos un análisis ejecutivo con tablas y gráficos interactivos contextualizados a su empresa. De igual modo, la incorporación de **modelos de Machine Learning** entrenados sobre 6 meses de datos sintéticos sustituye la intuición empírica por rigor estadístico: el modelo de regresión supervisado anticipa la demanda estacional de ventas para evitar

quiebres de stock, el clustering de K-Means segmenta la cartera de compradores para optimizar campañas de fidelización, y el algoritmo Isolation Forest detecta anomalías en existencias antes de que se conviertan en pérdidas financieras.

### 1.4.5 Justificación Académica y Metodológica (ICONIX vs PUDS)

En el ámbito formativo de la carrera de Ingeniería de Sistemas de la Universidad Autónoma Gabriel René Moreno (UAGRM), el proyecto permite contrastar de manera práctica y vivencial dos de las filosofías metodológicas más trascendentes en la ingeniería de software: el Proceso Unificado (PUDS), formal, iterativo y exhaustivo en artefactos, frente a **ICONIX**, ágil, pesada-ligera y focalizada en casos de uso.

La aplicación de ICONIX permite a los futuros ingenieros dominar el **principio 80/20 de UML**, aprendiendo a tender puentes conceptuales sólidos entre los requisitos del negocio y la implementación en código mediante los **Diagramas de Robustez (BCE)**. Demuestra cómo modelar la lógica sin acoplamientos prohibidos (eliminando dependencias directas entre entidades de datos), cómo representar fragmentos interactivos complejos en diagramas de secuencia sin deformar la interacción de interfaces, y cómo garantizar una trazabilidad vertical estricta que asegure la calidad del software entregado.

## 1.5 Alcance y limitaciones

### 1.5.1 Módulos del Sistema

A continuación se especifica exhaustivamente cada uno de los **8 módulos consolidados del Segundo Parcial**, detallando su descripción operativa, estructura de datos clave con restricciones de integridad, funcionalidades nucleares y casos de uso asociados:

#### *1.5.1.1 Módulo M01: Plataforma SaaS Multitenant, Planes y Seguridad RBAC (Desafío VI)*

- **Descripción General:** Administra el ciclo de vida, configuración, aprovisionamiento y aislamiento lógico estricto de las empresas clientes (**tenants**). Gobierna los tiers comerciales de suscripción (Básico, Profesional, Enterprise) con fiscalización de cuotas máximas de sucursales activas y usuarios, y gestiona la seguridad perimetral mediante autenticación JWT cifrada con hash bcrypt y control de acceso basado en roles (RBAC) con discriminador tenant_id en PostgreSQL 15.
- **Datos Principales:**
    - empresas (id_empresa PK UUID, razon_social VarChar 150, nit_identificacion VarChar 30 Unique, subdominio VarChar 50 Unique, email_contacto VarChar 100, estado_empresa Enum: ACTIVA, SUSPENDIDA, INACTIVA, fecha_registro Timestamp).
    - planes_suscripcion (id_plan PK Int, codigo_plan VarChar 20 Unique, nombre_plan VarChar 50, costo_mensual Decimal 10,2, limite_sucursales Int, limite_usuarios Int, acceso_ia_generativa Boolean, acceso_machine_learning Boolean).

- suscripciones (id_suscripcion PK
  UUID, empresa_id FK, plan_id FK, fecha_inicio Date, fecha_vencimiento Date, estad
  o Enum: ACTIVA, VENCIDA, SUSPENDIDA).
- usuarios (id_usuario PK UUID, tenant_id FK, email VarChar 150
  Unique, password_hash VarChar
  255, rol Enum: SUPER_ADMIN, ADMIN_EMPRESA, ENCARGADO_SUCURSAL, C
  AJERO, CLIENTE, ENCARGADO_COMPRAS, intentos_fallidos SmallInt).

- **Funcionalidades Clave:**
  - Aprovisionamiento administrativo de nuevas empresas con validación de NIT y subdominio único.
  - Aislamiento estricto de datos en backend: inyección forzosa del filtro WHERE tenant_id = :current_tenant en el 100% de consultas ORM y queries SQL.
  - Parametrización y control dinámico de cuotas según plan contratado (bloqueo automático ante intentos de creación que excedan el límite de sucursales o usuarios).
  - Middleware de verificación de vigencia de suscripción previo a conceder acceso a endpoints de negocio.
  - Autenticación segura con tokens JWT portando claims de identidad, rol y tenant, con bloqueo de cuenta tras 5 intentos fallidos.
- **Casos de Uso Asociados:** CU01 (Gestionar Empresas), CU02 (Gestionar Planes de Suscripción), CU03 (Gestionar Suscripción de Empresa), CU04 (Autenticar Usuario Multitenant), CU05 (Gestionar Usuarios y Roles por Empresa).

#### *1.5.1.2 Módulo M02: Catálogo de Productos y Atributos Textiles Avanzados*

- **Descripción General:** Centraliza la catalogación técnica de prendas de vestir masculinas bajo especificaciones avanzadas del retail de moda. Erradica la incertidumbre dimensional de calce mediante tablas métricas anatómicas en centímetros, gestiona esquemas de color simples y combinados bicolores con códigos visuales Hexadecimales, especifica la composición de telas (lino, algodón pima, lana merino) con advertencias dermatológicas hipoalergénicas y gestiona modelos tridimensionales (.glb) para el vestidor virtual en Realidad Aumentada.
- **Datos Principales:** id_producto (PK, UUID), tenant_id (FK, UUID), sku (VarChar 50, Unique por Tenant), nombre_comercial (VarChar 120), categoria_id (FK, Int), temporada_id (FK, Int), corte_estilo (Enum: SLIM_FIT, REGULAR_FIT, CUSTOM_FIT), tipo_tejido (VarChar 100), es_hipoalergenico (Boolean), modelo_3d_url (Text), precio_base_menor (Decimal 10,2), precio_mayorista (Decimal 10,2), variantes (talla_cm_json [pecho, cintura, cadera, largo], color_hex_primario, color_hex_secundario).
- **Funcionalidades Clave:**
  - Alta y catalogación de prendas con ficha técnica completa y switch de certificación dermatológica hipoalergénica.
  - Generador matricial de variantes combinando tallas antropométricas en cm y selectores bicolores (ej. Blanco/Negro #FFFFFF / #000000 o Azul/Gris #1A2A3A / #808080).

- Vinculación y optimización de archivos 3D (.glb) para renderizado AR en la cámara de la aplicación móvil Flutter.
- Establecimiento de precios diferenciados: tarifa unitaria minorista estándar y precio al por mayor.

- **Casos de Uso Asociados:** CU06 (Gestionar Productos y Atributos Textiles Avanzados).

#### *1.5.1.3 Módulo M03: Logística de Inventario Multi-Sucursal, Kardex Físico-Valorado y CPP*

- **Descripción General:** Controla las existencias físicas y contables de cada variante textil en cada una de las tiendas de la empresa. Administra el desglose tripartito de stock (físico, reservado y disponible), ejecuta el recálculo matemático inmutable del Costo Promedio Ponderado (CPP) tras cada ingreso de mercadería respaldado por factura, genera auditoría en el libro Kardex, dispara alertas automáticas de reposición por stock mínimo y gestiona la logística de transferencias inter-sucursales.
- **Datos Principales:**
  - inventario_sucursal (id_inventario PK UUID, tenant_id FK, sucursal_id FK, variante_id FK, stock_fisico Int, stock_reservado Int, stock_disponible Int: stock_fisico - stock_reservado, costo_promedio_ponderado Decimal 12,4, ultimo_costo_unitario Decimal 10,2, stock_minimo_seguridad Int).
  - movimientos_kardex (id_kardex PK UUID, tipo_movimiento Enum: ENTRADA_COMPRA, VENTA_POS, DEVOLUCION_CAMBIO, TRASPASO_SALIDA, TRASPASO_ENTRADA, cantidad Int, costo_unitario Decimal 10,2, saldo_fisico Int, saldo_valorado Decimal 12,2, comprobante_factura VarChar 50).
  - traspasos_inventario (id_traspaso PK UUID, sucursal_origen_id FK, sucursal_destino_id FK, guia_remision VarChar 50 Unique, estado Enum: SOLICITADO, EN_TRANSITO, RECIBIDO_CONFORME, DISCREPANCIA).
- **Funcionalidades Clave:**
  - Desglose de existencias en tiempo real por sucursal física (Físico - Reservado = Disponible).
  - Recálculo inmutable del Costo Promedio Ponderado tras cada entrada de lote de compra:

$$CPP_{nuevo} = \frac{(Stock_{ant} \times CPP_{ant}) + (Cant_{ingreso} \times Costo_{ingreso})}{Stock_{ant} + Cant_{ingreso}}$$

$$CPP_{nuevo} = Stock_{ant} + Cant_{ingreso}(Stock_{ant} \times CPP_{ant}) + (Cant_{ingreso} \times Costo_{ingreso})$$

  - Libro de Kardex físico-valorado inmutable para control fiscal y contable bajo la Ley 843.
  - Logística de traspasos entre tiendas físicas: despacho en tránsito (descuento de disponible) y confirmación física en destino con ajuste de CPP local.
  - Alertas automáticas de reposición urgente ante existencias inferiores al umbral mínimo de seguridad.

- **Casos de Uso Asociados:** CU07 (Gestionar Inventario Multi-Sucursal y CPP), CU14 (Gestionar Traspaso Inter-Sucursal de Inventario).

#### *1.5.1.4 Módulo M04: Operaciones Comerciales Offline y Punto de Venta POS (Desafíos II y III)*

- **Descripción General:** Garantiza la continuidad comercial ininterrumpida de las tiendas físicas y la experiencia del cliente ante caídas del servicio de Internet o cortes de energía eléctrica. Integra almacenamiento local (IndexedDB en la PWA Angular y SQLite en Flutter), navegación autónoma del catálogo, reservas presenciales temporales con tickets QR, terminal POS de contingencia para cobros en efectivo al contado con UUID v4 y bandeja de arqueo local.
- **Datos Principales:**
    - catalogo_local_db (Almacén local con snapshot delta de prendas, tallas cm, colores y existencias referenciales).
    - reservas_offline (uuid_reserva PK UUID v4, variante_id, fecha_limite=2h, qr_payload Text, estado='PENDIENTE_SYNC').
    - operaciones_offline (uuid_operacion PK UUID v4, tenant_id, sucursal_id, cajero_id, tipo='VENTA_POS_CONTADO', total_efectivo Decimal 10,2, payload_json Text, estado='PENDIENTE_SYNC', hash_integridad VarChar 64).
- **Funcionalidades Clave:**
    - Descarga periódica delta de catálogos y cacheo de miniaturas WebP mediante Service Workers.
    - Consulta y filtrado 100% desconectado de prendas, fichas técnicas y medidas anatómicas en cm.
    - Generación de reservas presenciales offline con temporizador de 2 horas y código QR escaneable en mostrador.
    - Terminal POS de contingencia: venta presencial de prendas cobradas exclusivamente al contado en efectivo.
    - Generación criptográfica de identificadores UUID v4 inmutables y comprobantes provisionales conteniendo código QR y hash.
    - Módulo de arqueo de caja física para auditar el dinero en gaveta frente a los comprobantes emitidos antes de sincronizar.
- **Casos de Uso Asociados:** CU08 (Descargar y Consultar Catálogo Offline), CU09 (Registrar Reserva Presencial Sin Conexión), CU10 (Registrar Venta POS de Contado Sin Conexión), CU11 (Consultar y Auditar Operaciones Locales Pendientes).

#### *1.5.1.5 Módulo M05: Motor de Sincronización Transaccional Idempotente con UUID v4 (Desafíos II y III)*

- **Descripción General:** Orquesta la transmisión atómica, segura e idempotente de lotes transaccionales acumulados en terminales offline hacia la base de datos central PostgreSQL en el backend FastAPI al restablecerse la conectividad. Aplica cabeceras X-Idempotency-Key basadas en UUID v4 para erradicar cobros duplicados por reintentos de red y ejecuta protocolos deterministas de reconciliación de inventario.

- **Datos Principales:** transacciones_procesadas (idempotency_key PK UUID v4, tenant_id FK UUID, fecha_procesamiento Timestamp, tipo_operacion VarChar 30, payload_hash VarChar 64, response_snapshot_json JSONB, estado_resolucion Enum: CONFIRMADO, DUPLICADO_IGNORADO, CONFLICTO_RESUELTO).
- **Funcionalidades Clave:**
    - Listener reactivo de eventos de red que detecta el estado online y activa el worker de sincronización en segundo plano.
    - Transmisión en lotes (*batch*) securizados con autenticación JWT y clave de idempotencia única por operación.
    - Deduplicación atómica en servidor: detección de UUIDs previamente procesados con retorno del resultado original sin reejecutar inserciones contables.
    - Reconciliación determinista de stock ante sobreventa concurrente (*overselling*): prevalencia de la venta física ya entregada en tienda y generación de alertas de reposición prioritaria.
- **Casos de Uso Asociados:** CU12 (Sincronizar Operaciones Pendientes).

#### *1.5.1.6 Módulo M06: Ventas, Pasarela Stripe, Precios Especiales y Cambios en Dos Tiempos (Desafío IV)*

- **Descripción General:** Gestiona el ciclo comercial omnicanal, pasarelas de pago, políticas de precios promocionales y el proceso postventa de cambios de indumentaria. Automatiza descuentos de liquidación de fin de temporada (30%, 40%, 60%) y tarifas al por mayor (≥3≥3 prendas), procesa pagos tokenizados mediante Stripe Sandbox (cumpliendo PCI-DSS Nivel 1) e implementa el protocolo de cambio de prendas en dos tiempos: **Nota de Devolución** (reingreso a stock) y **Nota de Cambio** (salida de nueva prenda y cobro de diferencial), junto con la política de reembolso en efectivo a 30 días (fin de mes).
- **Datos Principales:**
    - promociones_precios (id_promocion PK UUID, tipo Enum: LIQUIDACION_ESTACIONAL, PRECIO_AL_POR_MAYOR, porcentaje_descuento Decimal 5,2, umbral_minimo=3, fecha_inicio Date, fecha_fin Date).
    - pagos_stripe (id_pago PK UUID, stripe_payment_intent_id VarChar 100 Unique, monto Decimal 10,2, moneda VarChar 10, client_secret VarChar 150, estado Enum: SUCCEEDED, FAILED).
    - cambios_prenda (id_cambio PK UUID, numero_nota_devolucion VarChar 30 Unique, numero_nota_cambio VarChar 30 Unique, id_venta_original FK UUID, prenda_devuelta_id FK UUID, saldo_a_favor Decimal 10,2, nueva_prenda_id FK UUID, diferencial_precio Decimal 10,2, estado_reembolso Enum: CAMBIO_CONCLUIDO, PROGRAMADO_REEMBOLSO_30_DIAS).
- **Funcionalidades Clave:**
    - Motor de liquidación de fin de temporada con rebajas del 30%, 40% y 60% para recuperar capital de ropa estacional pasada.
    - Detección automática en carrito y POS del umbral de compra mayorista (≥3≥3 unidades) aplicando la tarifa especial sin intervención manual.

- Integración tokenizada con Stripe Elements y webhooks asíncronos para ventas directas y cobro de diferencias en cambios de ropa.
- Postventa en dos tiempos: Tiempo 1 (Nota de Devolución e ingreso a stock) y Tiempo 2 (Nota de Cambio y despacho de nueva prenda).
- Protocolo de liquidación monetaria: emisión de comprobante de devolución en efectivo programado al cierre de mes (30 días) ante rechazo de cambio.

- **Casos de Uso Asociados:** CU13 (Gestionar Cambio de Prenda en Dos Tiempos y Devoluciones), CU18 (Gestionar Pasarela de Pagos Stripe), CU19 (Gestionar Ofertas de Liquidación y Precios por Mayor).

#### *1.5.1.7 Módulo M07: Aprovisionamiento y Compras Corporativas desde Web Desktop*

- **Descripción General:** Canal corporativo de compras B2B operado formal y exclusivamente desde computadoras de escritorio (Web Desktop / Laptop) por el Encargado de Compras o Administrador de Empresa. Permite planificar el reabastecimiento, emitir órdenes de compra a proveedores textiles homologados por curva de tallas y lotes, registrar facturas fiscales de adquisición e integrarse directamente con el módulo de inventarios (M03) para actualizar existencias y recalcular el Costo Promedio Ponderado.
- **Datos Principales:** id_orden_compra (PK, UUID), tenant_id (FK, UUID), proveedor_id (FK, UUID), sucursal_destino_id (FK, UUID), numero_orden (VarChar 30, Unique), numero_factura_fiscal (VarChar 50), costo_total_compra (Decimal 12,2), estado_orden (Enum: BORRADOR, EMITIDA, RECEPCIONADA_CONFORME, DISCREPANCIA), fecha_emision (Timestamp), fecha_recepcion (Timestamp), items_orden (variante_id, cantidad_pedida, costo_unitario_compra).
- **Funcionalidades Clave:**
  - Restricción estricta de acceso: validación de dispositivo (exclusivo Web Desktop / Laptop para garantizar auditoría interna; inhabilitado en móviles).
  - Emisión de órdenes de compra formales con especificación de variantes de tallas y costos unitarios pactados.
  - Recepción física en bodega de tienda con verificación de factura fiscal y cotejo de cantidades recibidas.
  - Enlace automático con inventarios: suma de stock físico y recálculo matemático de CPP inmutable en el Kardex de la sucursal receptora (CU07).
- **Casos de Uso Asociados:** CU20 (Gestionar Compras y Aprovisionamiento desde Web Desktop).

#### *1.5.1.8 Módulo M08: Inteligencia de Negocios, Reportes con IA y Machine Learning (Desafío V)*

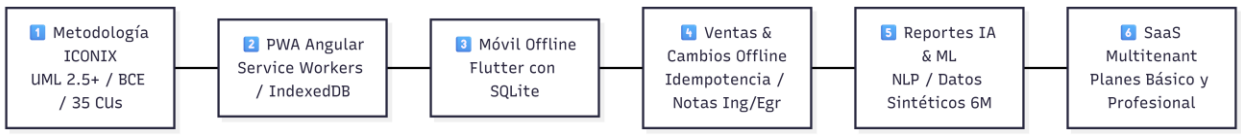
- **Descripción General:** Centraliza la inteligencia estratégica y predictiva de la empresa. Proporciona un asistente conversacional que traduce consultas en lenguaje natural (escritas o dictadas por voz mediante Whisper) en sentencias SQL analíticas de solo lectura sobre PostgreSQL, exporta reportes estructurados a Excel y PDF, y despliega un Dashboard analítico con modelos de Machine Learning entrenados sobre 6 meses de transacciones sintéticas para predecir la demanda semestral de prendas y optimizar el inventario.
- **Datos Principales:**

- reportes_ia (id_reporte_ia PK
  UUID, tenant_id FK, prompt_original Text, audio_transcrito Text, sql_generado Text, validacion_ast_exitosa Boolean, resumen_narrativo Text, config_grafico_json JSON B).
- modelos_ml (id_modelo_ml PK
  UUID, tenant_id FK, tipo_algoritmo Enum: RandomForestRegressor, KMeans, IsolationForest, horizonte_meses=6, metricas_json [R2*R*2, RMSE, Silhouette Score]).
- exportaciones (id_exportacion PK
  UUID, tipo_formato Enum: XLSX, PDF, tamanio_bytes Int, fecha_descarga Timestamp).

- **Funcionalidades Clave:**
  - Procesamiento de consultas estratégicas por texto y dictado de voz procesado con Whisper.
  - Validador AST de seguridad SQL: verificación de consultas exclusivamente SELECT, bloqueo de DDL/DML e inyección forzosa de tenant_id.
  - Exportación automática a libros Microsoft Excel (.xlsx) con fórmulas nativas y documentos ejecutivos PDF (.pdf) con gráficos vectoriales.
  - Inferencia predictiva de demanda a 6 meses vista por categoría y sucursal para anticipar quiebres o sobrestock.
  - Heatmaps de rotación de prendas por combinaciones de talla (XS a XXL) y color.
- **Casos de Uso Asociados:** CU15 (Solicitar Reporte Generativo con IA), CU16 (Exportar Reportes Analíticos a Excel/PDF), CU17 (Gestionar Dashboard Analítico con Machine Learning).

### 1.5.2 Los Seis Desafíos Tecnológicos del Segundo Parcial

El desarrollo del Segundo Parcial responde a seis desafíos técnicos planteados por la cátedra:




| Nro. Desafío | Denominación Oficial | Enfoque Tecnológico y Arquitectónico |
|---|---|---|
| **I** | **Metodología ICONIX** | Sustitución y adaptación integral del PUDS previo hacia ICONIX. Redacción de fichas formalizadas, diagramas de robustez BCE, diagramas de secuencia y clases para los 35 casos de uso. |
| **II** | **Progressive Web App (PWA)** | Evolución de la plataforma web en Angular hacia una PWA instalable con manifiesto de aplicación, Service Workers para cacheo de assets y persistencia en IndexedDB para funcionamiento offline. |

| | | |
|---|---|---|
| **III** | **Persistencia Móvil Offline** | Integración de motor relacional SQLite (sqflite) en la app móvil Flutter para navegación de catálogo y captura de operaciones sin dependencia de Internet. |
| **IV** | **Ventas Offline y Cambios de Ropa** | Venta POS y reservas offline con consistencia transaccional. Flujo completo de cambios de indumentaria por talla/modelo con emisión de notas de ingreso y notas de egreso y resolución de diferencias de precio. |
| **V** | **Reportes Generativos con IA y ML** | Generación de reportes ad-hoc formulados en lenguaje natural (texto y voz). Pipeline de Machine Learning con modelos supervisados y no supervisados sobre dataset sintético de 6 meses de ventas e inventario. |
| **VI** | **SaaS Multitenant y Suscripciones** | Arquitectura multi-empresa en base de datos PostgreSQL compartida con aislamiento lógico estricto por empresa_id. Definición de Planes Básico y Profesional con validación de límites. |

### 1.5.3 Planes de Suscripción del Prototipo

Para validar el funcionamiento del modelo SaaS en el prototipo académico, se parametrizan los siguientes planes comerciales:

| **Característica / Módulo** | **Plan Básico** | **Plan Profesional** |
|---|---|---|
| **Público Objetivo** | Boutiques y tiendas individuales | Cadenas minoristas con múltiples sucursales |
| **Límite de Sucursales Físicas Habilitadas** | Hasta 2 sucursales | Hasta 10 sucursales |
| **Gestión de Catálogo e Inventario CPP** | Ilimitado | Ilimitado |
| **Punto de Venta Presencial (POS)** | Habilitado | Habilitado |
| **Sistema de Reservas Presenciales** | Habilitado | Habilitado |
| **Operación Offline (PWA e IndexedDB / SQLite)** | Habilitado | Habilitado |
| **Sincronización Transaccional Idempotente** | Habilitado | Habilitado |
| **Gestión de Cambios y Devoluciones** | Habilitado | Habilitado |
| **Reportes Convencionales (KPIs y Tableros)** | Habilitado | Habilitado |
| **Reportes Generativos Asistidos por IA (NLP/Voz)** | ✗ No incluido | ☑ Incluido (Ilimitado) |

| | | |
|---|---|---|
| **Módulo Analítico con Machine Learning** | ❌ No incluido | ☑ Incluido (Supervisado + No Supervisado) |
| **Dashboard Analítico Avanzado con ML** | ❌ No incluido | ☑ Incluido |

### 1.5.4 Matriz de Funcionalidades por Plataforma y Modo de Operación

| Módulo / Funcionalidad | Web Angular (Online) | Web PWA (Offline) | Móvil Flutter (Online) | Móvil Flutter (Offline) |
|---|---|---|---|---|
| **Navegación de Catálogo de Prendas** | ☑ | ☑ (Caché local) | ☑ | ☑ (SQLite) |
| **Consulta de Stock en Tiempo Real** | ☑ | ⚠ (Último conocido) | ☑ | ⚠ (Último conocido) |
| **Probador Virtual con Realidad Aumentada** | ❌ (Exclusivo móvil) | ❌ | ☑ | ☑ (Modelos cacheados) |
| **Generación de Reservas de Probador** | ☑ | ☑ (Encolado) | ☑ | ☑ (Encolado SQLite) |
| **Venta de Mostrador en Caja (POS)** | ☑ | ☑ (Encolado) | ❌ | ❌ |
| **Checkout Digital con Pasarela Stripe** | ☑ | ❌ (Requiere pasarela) | ☑ | ❌ (Requiere pasarela) |
| **Gestión de Cambios de Ropa (Notas Ing/Egr)** | ☑ | ☑ (Encolado) | ❌ | ❌ |
| **Sincronización Automática al Conectar** | Automática | Activada al reconectar | Automática | Activada al reconectar |
| **Reportes Generativos con IA (Texto/Voz)** | ☑ | ❌ (Requiere LLM) | ☑ | ❌ (Requiere LLM) |
| **Dashboard con Modelos Machine Learning** | ☑ | ❌ (Requiere cómputo) | ❌ | ❌ |
| **Administración SaaS y Configuración Planes** | ☑ | ❌ | ❌ | ❌ |

### 1.5.6 Matriz de Plataformas y Despliegue Operativo

| Plataforma / Entorno | Tecnologías Utilizadas | Roles que la Utilizan | Operatividad Offline | Funcionalidades Destacadas |
|---|---|---|---|---|
| **Web Desktop (Administrativa)** | Angular 17+, TypeScript, | Super Admin, Admin Empresa, Encargado | Online preferente | Aprovisionamiento de compras (CU19), gestión SaaS, planes de |

| | | | | |
|---|---|---|---|---|
| | Tailwind/Vanilla CSS | Aprovisionamiento | | suscripción, reportes IA texto, dashboard ML. |
| **Web PWA (Punto de Venta)** | Angular PWA, Service Workers, IndexedDB API | Cajeros, Encargados de Sucursal | **Offline Total (IndexedDB)** | Facturación POS de contado, recepción de reservas presenciales, cambios de prendas con notas de devolución y cambio. |
| **Móvil (Clientes y Asesores)** | Flutter 3+, Dart, SQLite (sqflite), ARCore | Clientes, Dependientes de Sala de Ventas | **Offline Total (SQLite)** | Catálogo con tallas en cm, probador virtual RA, reservas en probador, reportes IA por voz, fidelización y compras. |
| **Backend Central** | FastAPI (Python 3.11), SQLAlchemy, AsyncPG, Pydantic | Todos los clientes (vía API REST) | Servidor en la Nube | Aislamiento multitenant, middleware de idempotencia, motor de sincronización, seguridad RBAC con JWT, integración Stripe y LLM. |
| **Base de Datos Central** | PostgreSQL 15 Relacional | Backend FastAPI | Nube (Render / Railway) | Esquema compartido multitenant con discriminador empresa_id, índices B-Tree compuestos, disparadores (triggers) de kardex y funciones PL/pgSQL. |

### 1.5.7 Restricciones y Limitaciones del Proyecto

Con el propósito de mantener el rigor académico y delimitar con precisión el alcance factible en el marco de la asignatura, se declaran las siguientes restricciones formales:

1. **Pasarelas de Pago Electrónico:** La integración con Stripe opera estrictamente en modo de pruebas (**Sandbox**), utilizando números de tarjeta tokenizados provistos por la plataforma con fines de demostración académica. No se procesan cobros monetarios reales.
2. **Ventas y Operaciones Fuera de Línea:** Las ventas ejecutadas en modo offline (PWA e IndexedDB o Flutter y SQLite) se limitan a la modalidad de **pago en efectivo de contado**. Por razones de seguridad bancaria y restricciones técnicas de los procesadores de tarjetas, no se procesan pagos con tarjeta de crédito/débito sin conexión a Internet.
3. **Mecanismo de Suscripciones SaaS:** La contratación y cobro recurrente mensual de las suscripciones de las empresas clientes se gestiona de forma administrativa y simulada dentro de la plataforma; no se implementa un débito automático bancario recurrente real.

4. **Conjunto de Datos para Machine Learning:** Los modelos de Machine Learning (supervisados y no supervisados) se entrenan empleando un pipeline generador de **datos sintéticos representativo de 6 meses de actividad comercial continua**, modelando de forma matemática picos de demanda estacional y patrones de compra realistas.
5. **Alcance Geográfico y Tributario:** La parametrización tributaria (IVA 13%, IT 3% y Costo Promedio Ponderado) y las coberturas geográficas se modelan con base en el marco normativo y territorial del Estado Plurinacional de Bolivia.

## 1.6 Metodología de desarrollo ICONIX

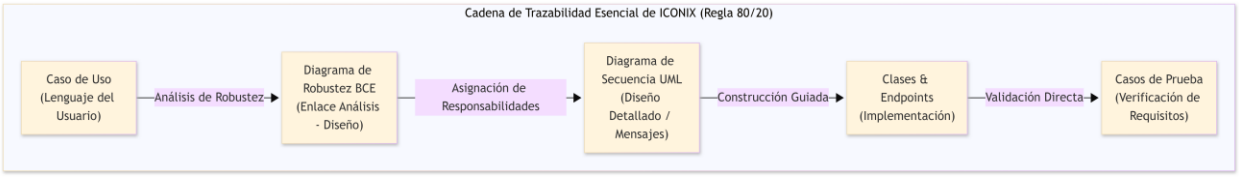
### 1.6.1 Definición, Principios y Filosofía Pragmática

**ICONIX** es una metodología de desarrollo de software clasificada como **pesada-ligera** (*heavyweight-lightweight*), formulada originalmente por Doug Rosenberg. Nace como una respuesta pragmática a los dos extremos metodológicos tradicionales:

- Los procesos formales y pesados como RUP (*Rational Unified Process*) o el Proceso Unificado (PUDS), caracterizados por una exhaustiva producción de artefactos intermedios que frecuentemente derivan en parálisis por análisis.
- Los métodos extremadamente ágiles o ligeros como XP (*eXtreme Programming*), que priorizan la codificación inmediata sacrificando la trazabilidad formal y el modelado arquitectónico.

La piedra angular de ICONIX reside en la aplicación práctica de la **regla del 80/20 de UML**: postula que **el 80% del valor de un sistema de información puede modelarse, analizarse y validarse utilizando únicamente el 20% de los elementos y artefactos de UML**. De esta forma, ICONIX descarta los artefactos redundantes y concentra todo el esfuerzo del equipo de desarrollo en los elementos verdaderamente esenciales para guiar la construcción del código ejecutable.

ICONIX es un proceso **dirigido por casos de uso** e **iterativo e incremental**. Su objetivo supremo es establecer un puente fluido, riguroso y visible que transforme la narrativa de los requisitos expresada en los casos de uso en clases de código implementadas y probadas.




### 1.6.2 Comparativa Exhaustiva entre PUDS (Primer Parcial) e ICONIX (Segundo Parcial)

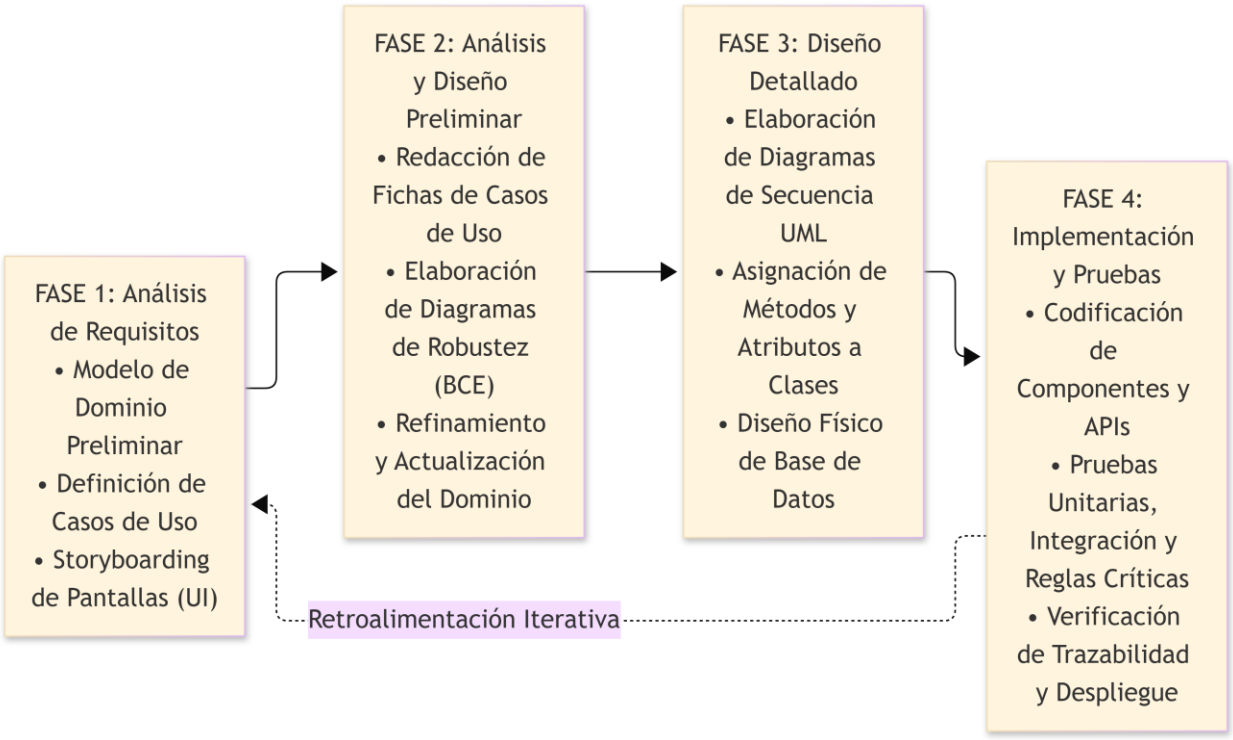
A fin de justificar la transición metodológica aplicada en el proyecto, la siguiente tabla sintetiza las diferencias sustanciales entre ambos marcos de trabajo:

| Dimensión de Comparación | Proceso Unificado (PUDS) — Primer Parcial | Metodología ICONIX — Segundo Parcial |
|---|---|---|
| **Filosofía Base** | Proceso formal pesado, exhaustivo, centrado en la | Proceso ágil-pesado pragmático, centrado en el principio 80/20 de UML. |

| | | |
|---|---|---|
| | arquitectura y dirigido por casos de uso. | |
| **Artefactos Requeridos** | Extensa cantidad de artefactos en cada fase (documentos de visión, modelo de análisis, etc.). | Conjunto mínimo de artefactos indispensables (fichas CU, robustez, secuencias, clases, código). |
| **Estructura del Ciclo de Vida** | 4 fases organizativas (Inicio, Elaboración, Construcción, Transición) con 5 flujos continuos. | 4 fases secuenciales e iterativas por caso de uso (Requisitos, Análisis/Diseño Preliminar, Diseño, Implementación). |
| **Artefacto Distintivo Clave** | Diagramas de comunicación y especificaciones narrativas extensas. | **Diagrama de Robustez (BCE)**, que conecta el análisis con el diseño de clases. |
| **Modelado de Dinámica** | Diagramas de actividades, estados y comunicación paralelos. | Diagramas de Secuencia UML derivados directamente del análisis de robustez. |
| **Complejidad Documental** | Elevada; propensión a la duplicidad de información entre flujos. | Moderada y compacta; trazabilidad vertical limpia Requisito →→ Código. |
| **Cobertura de UML** | Utilización amplia y estricta de todo el estándar UML 2.5+. | Utilización selectiva del subconjunto medular (Casos de Uso, Robustez, Secuencia, Clases). |

### 1.6.3 Las Cuatro Fases de la Metodología ICONIX

ICONIX estructura el ciclo de vida del software en cuatro fases continuas e interrelacionadas:




***Fase 1: Análisis de Requisitos***

En esta fase inicial se define el alcance del sistema en estrecha coordinación con el cliente o los requisitos de la cátedra:

1. **Modelo de Dominio Preliminar:** Se construye un diagrama de clases estructural básico que identifica las entidades conceptuales del negocio del mundo real (ej. *Empresa*, *Sucursal*, *Producto*, *Inventario*, *Reserva*, *Venta*) y sus relaciones asociativas, sin saturarlo de métodos o tipos de datos de programación.
2. **Definición del Modelo de Casos de Uso:** Se delimitan las fronteras del sistema, identificando los actores participantes y los casos de uso que describen el comportamiento esperado.
3. **Prototipado Rápido de Interfaces (Storyboarding):** Se diseñan las pantallas e interfaces gráficas preliminares para que los usuarios comprendan cómo interactuarán con cada caso de uso.

***Fase 2: Análisis y Diseño Preliminar***

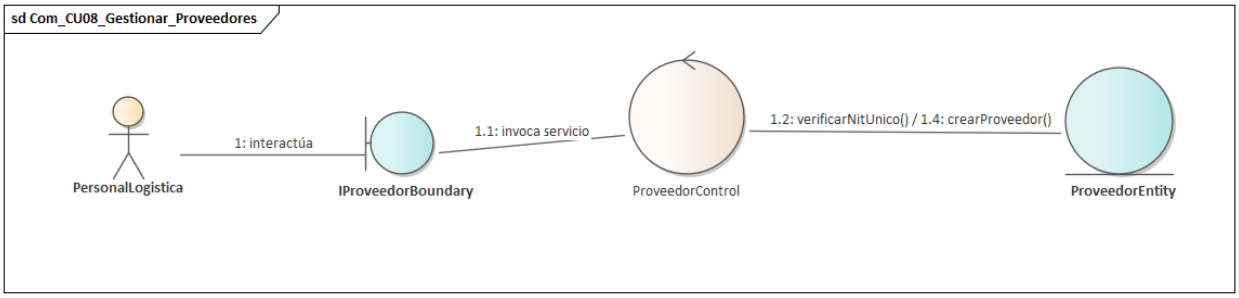
Constituye el núcleo analítico de ICONIX. Su propósito es desentrañar la lógica interna de cada caso de uso antes de entrar a programar:

1. **Fichas Normalizadas de Caso de Uso:** Se redacta el flujo normal y los flujos alternativos en lenguaje transparente y sin ambigüedades técnicas.
2. **Diagramas de Robustez (BCE):** Se construye para cada caso de uso un diagrama gráfico utilizando tres tipos de estereotipos propuestos por Ivar Jacobson:
    - **Clase Frontera (*Boundary* - ┤):** Modela los puntos de contacto entre el actor y el sistema (pantallas, diálogos, lectores de QR, escáneres POS, interfaces de voz).

- **Clase Control (*Controller* - ⚙️):** Modela la lógica de negocio, validaciones transaccionales, consultas de inventario, algoritmos de CPP y coordinación de procesos.
- **Clase Entidad (*Entity* - ○_):** Modela las tablas, registros y estructuras de persistencia que almacenan el estado del sistema en la base de datos PostgreSQL, IndexedDB o SQLite.

**Reglas de Comunicación Estricta en Diagramas de Robustez:**

- Los **Actores** solo pueden comunicarse con objetos de tipo **Frontera (*Boundary*)**.
- Los objetos **Frontera** solo pueden comunicarse con objetos de tipo **Control (*Controller*)** y viceversa.
- Los objetos **Frontera NUNCA** se comunican directamente con objetos de tipo **Entidad (*Entity*)** (salvaguarda de la arquitectura en capas).
- Los objetos **Control** pueden comunicarse entre sí y con objetos **Entidad**.
- Los objetos **Entidad** se comunican con objetos **Control**.




***Fase 3: Diseño Detallado***

En esta fase se toma la estructura conceptual de la Fase 2 y se convierte en una especificación técnica lista para codificar:

1. **Diagramas de Secuencia UML:** Derivan directamente de los diagramas de robustez. Los controladores y entidades se transforman en lifelines (*líneas de vida*) de objetos concretos; los enlaces de comunicación se transforman en mensajes sincrónicos o asincrónicos con parámetros y retornos formales, utilizando fragmentos combinados (alt para condicionales, opt para flujos opcionales, loop para bucles transaccionales).
2. **Diagrama de Clases UML Definitivo:** Se refina el modelo de clases dotando a cada entidad y servicio de sus atributos tipados, modificadores de acceso (+, -, #) y firmas de métodos completas descubiertas durante la diagramación de secuencias.
3. **Diseño de Base de Datos:** Se traslada el diagrama de clases al modelo físico relacional de PostgreSQL, normalizado a Tercera Forma Normal (3FN) con índices y restricciones de integridad.

***Fase 4: Implementación y Pruebas***

1. **Codificación Guiada por Secuencias:** Los desarrolladores implementan los controladores de FastAPI, los componentes de Angular y los widgets de Flutter siguiendo fielmente la interacción de mensajes documentada en los diagramas de secuencia.

2. **Plan y Ejecución de Pruebas:** Se implementan pruebas unitarias y de integración orientadas a verificar los requisitos iniciales. Mediante la trazabilidad, se comprueba que cada escenario de la ficha de caso de uso cuenta con una prueba que valida su correcto comportamiento.

### 1.6.4 Trazabilidad Sistemática en ICONIX

La trazabilidad constituye la mayor fortaleza de ICONIX: garantiza que ningún requisito quede huérfano y que ninguna línea de código exista sin justificación en un caso de uso:



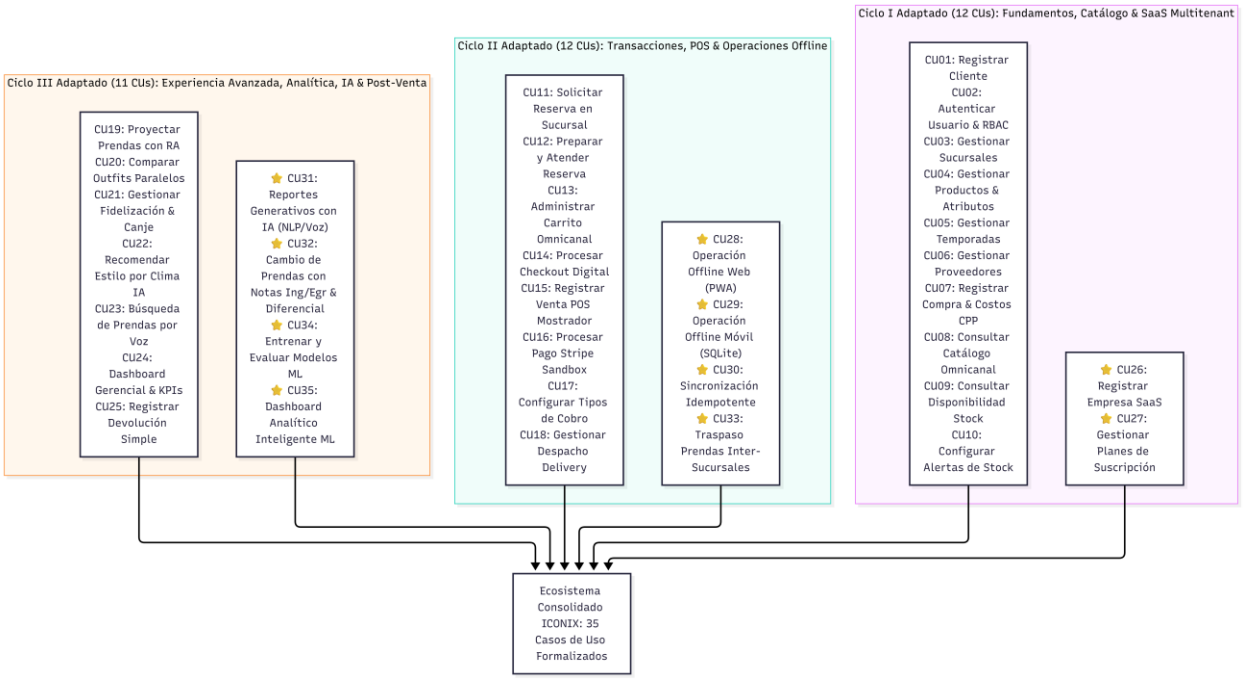

### 1.6.5 Plantilla Oficial de Ficha de Caso de Uso Adoptada

De acuerdo con el estándar establecido por la cátedra para el Segundo Examen Parcial, cada caso de uso se documenta mediante la siguiente ficha tabular estructurada:

| Campo | Descripción y Especificación |
|---|---|
| **Nombre** | Código identificador correlativo y título descriptivo formal del caso de uso. |
| **Autor** | Desarrollador(es) responsable(s) de su especificación e implementación. |
| **Fecha** | Fecha de elaboración original y de última actualización técnica. |
| **Descripción** | Resumen ejecutivo del objetivo y propósito del caso de uso desde la perspectiva del actor. |
| **Actores** | Identificación del actor primario que desencadena el caso de uso y actores secundarios o cooperantes. |
| **Precondiciones** | Estado previo indispensable del sistema o condiciones de negocio que deben cumplirse para iniciar el flujo. |
| **Flujo Normal** | Secuencia ordenada y numerada de pasos cronológicos correspondientes al camino básico exitoso. |
| **Flujo Alternativo** | Enumeración de bifurcaciones, tratamientos de excepción, validaciones fallidas y condiciones de error. |
| **Postcondiciones** | Estado final persistente del sistema, modificaciones en la base de datos o emisiones de comprobantes tras el éxito. |

### 1.6.6 Estrategia de Adaptación e Integración de los Tres Ciclos Previos

Una premisa irrenunciable del Segundo Parcial es que **el proyecto no comienza desde cero**: se trata de la evolución y escalabilidad de un producto de software preexistente. Por lo tanto, los 25 casos de uso concebidos y desarrollados durante los tres ciclos del primer parcial bajo PUDS se integran, homologan y adaptan a las cuatro fases de ICONIX, combinándose armónicamente con los 10 nuevos casos de uso del segundo parcial




Con esta estrategia de integración sistemática, se garantiza que:

1. Ninguna funcionalidad previa quede descontinuada o aislada del nuevo modelo SaaS.
2. Cada uno de los 35 casos de uso cuente con su correspondiente ficha ICONIX normalizada.
3. Se generen y verifiquen los 35 diagramas de robustez (BCE) requeridos para el análisis preliminar.
4. Se generen los diagramas de secuencia UML que gobiernan la implementación y pruebas de todo el sistema.

# CAPÍTULO II. ANÁLISIS DE REQUISITOS

## 2.1 Descripción del Sistema Propuesto

### 2.1.1 Visión General, Contexto de Negocio y Oportunidad Tecnológica

**FashionStore SaaS** (Versión Segundo Examen Parcial) es una solución tecnológica integral de comercio electrónico inteligente, gestión de operaciones de retail omnicanal y administración empresarial B2B bajo la modalidad de **Software como Servicio (SaaS Multitenant)**. La plataforma está concebida específicamente para modernizar, formalizar y escalar las cadenas de tiendas y empresas independientes de confección y comercialización de **indumentaria y sastrería masculina** (*menswear*).

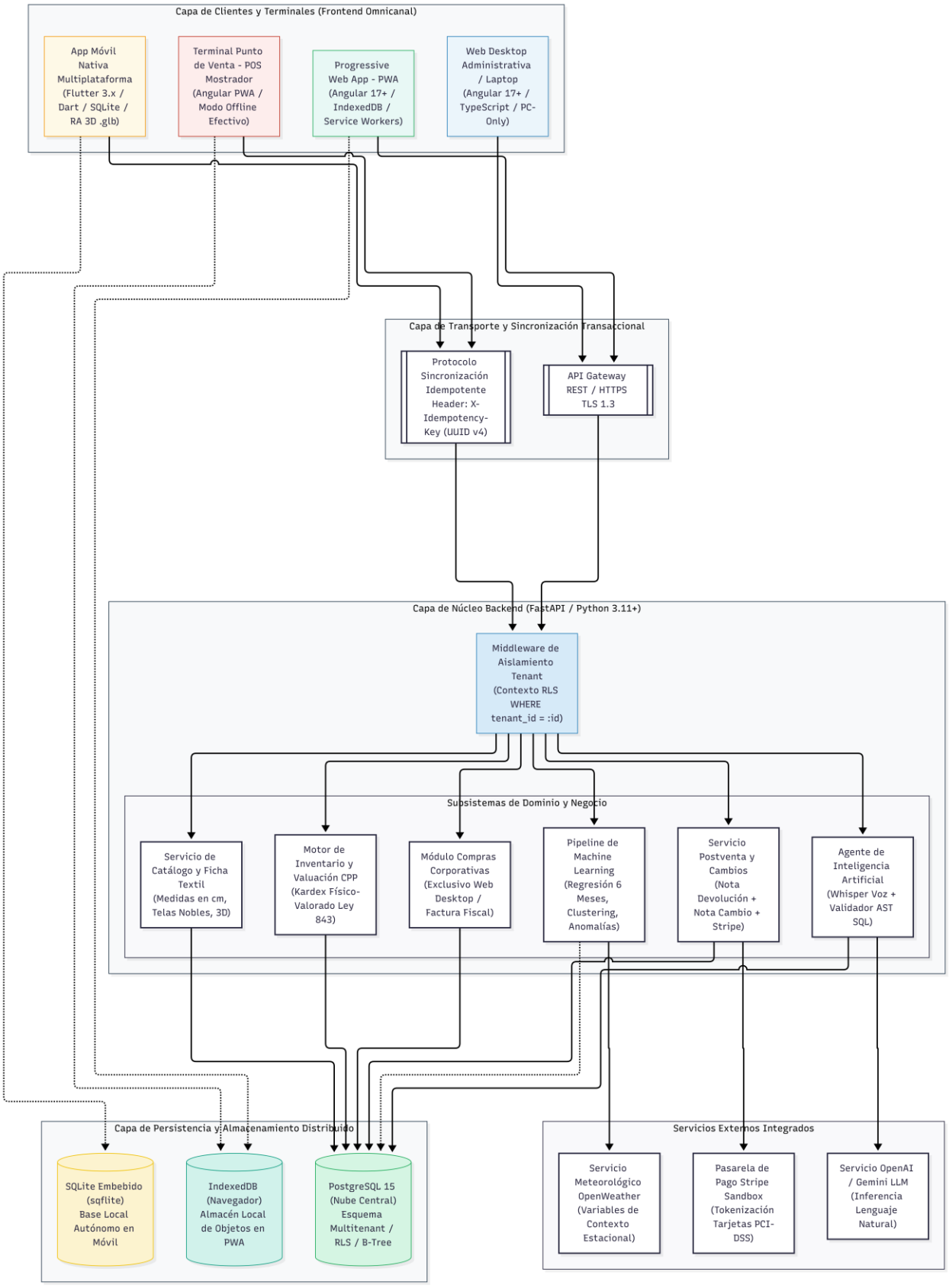
En la industria de la moda masculina, las operaciones comerciales tradicionales enfrentan tres problemáticas estructurales críticas:

1. **Alta tasa de devoluciones y fricción por discordancia de tallaje:** El tallaje genérico comercial (S, M, L, XL) resulta ineficaz frente a la diversidad morfológica masculina y a los diferentes cortes de confección (Slim Fit, Regular Fit, Custom Tailored), provocando devoluciones costosas y desconfianza en canales no presenciales.
2. **Vulnerabilidad e interrupción operativa en tiendas físicas por inestabilidad de telecomunicaciones:** En el contexto boliviano y regional, los centros comerciales y galerías presentan microcortes y caídas prolongadas de enlace a Internet, paralizando los sistemas tradicionales de facturación en la nube y provocando pérdida directa de ventas.
3. **Pérdida de trazabilidad de costos e inventarios:** La ausencia de valuación contable bajo el método del **Costo Promedio Ponderado (CPP)** y el descontrol en los traspasos inter-sucursales erosionan los márgenes comerciales reales e impiden una planificación certera de compras corporativas a proveedores.

Para superar estas limitaciones, FashionStore evoluciona la arquitectura monoproyecto del primer parcial hacia un **ecosistema distribuido de alta disponibilidad, Offline-First, Cloud-Synchronized y AI-Augmented** (resiliencia fuera de línea, sincronización en nube e inteligencia artificial aumentada). El sistema unifica la experiencia del cliente final (B2C), la continuidad transaccional en puntos de venta físicos (POS) y la gobernanza estratégica corporativa (B2B) en una única plataforma gobernada por la metodología ágil y rigurosa **ICONIX**.

### 2.1.2 Diagrama Arquitectónico del Ecosistema Propuesto

El siguiente diagrama en lenguaje UML modela la arquitectura global del sistema, ilustrando la interacción entre las interfaces clientes especializadas, el protocolo de sincronización resiliente, los componentes del backend y los tres niveles de persistencia de datos:


Capa de Clientes y Terminales (Frontend Omnicanal)
App Móvil Nativa Multiplataforma (Flutter 3.x / Dart / SQLite / RA 3D .glb)
Terminal Punto de Venta - POS Mostrador (Angular PWA / Modo Offline Efectivo)
Progressive Web App - PWA (Angular 17+ / IndexedDB / Service Workers)
Web Desktop Administrativa / Laptop (Angular 17+ / TypeScript / PC-Only)
Capa de Transporte y Sincronización Transaccional
Protocolo Sincronización Idempotente Header: X-Idempotency-Key (UUID v4)
API Gateway REST / HTTPS TLS 1.3
Capa de Núcleo Backend (FastAPI / Python 3.11+)
Middleware de Aislamiento Tenant (Contexto RLS WHERE tenant_id = :id)
Subsistemas de Dominio y Negocio
Servicio de Catálogo y Ficha Textil (Medidas en cm, Telas Nobles, 3D)
Motor de Inventario y Valuación CPP (Kardex Físico-Valorado Ley 843)
Módulo Compras Corporativas (Exclusivo Web Desktop / Factura Fiscal)
Pipeline de Machine Learning (Regresión 6 Meses, Clustering, Anomalías)
Servicio Postventa y Cambios (Nota Devolución + Nota Cambio + Stripe)
Agente de Inteligencia Artificial (Whisper Voz + Validador AST SQL)
Capa de Persistencia y Almacenamiento Distribuido
SQLite Embebido (sqflite) Base Local Autónomo en Móvil
IndexedDB (Navegador) Almacén Local de Objetos en PWA
PostgreSQL 15 (Nube Central) Esquema Multitenant / RLS / B-Tree
Servicios Externos Integrados
Servicio Meteorológico OpenWeather (Variables de Contexto Estacional)
Pasarela de Pago Stripe Sandbox (Tokenización Tarjetas PCI-DSS)
Servicio OpenAI / Gemini LLM (Inferencia Lenguaje Natural)

### 2.1.3 Ecosistema de Plataformas y Despliegue Multiplataforma

Para responder a las demandas heterogéneas de los distintos actores del sistema y garantizar la continuidad del negocio, FashionStore despliega un ecosistema compuesto por cuatro plataformas interconectadas:

1. **Web Desktop Administrativa y Corporativa (Angular 17+ / TypeScript):**
    - **Público Objetivo:** Super Administradores de la plataforma, Administradores de Empresa y Encargados de Compras y Aprovisionamiento.
    - **Alcance Funcional:** Diseñada y optimizada exclusivamente para pantallas de computadoras de escritorio y laptops (resolución ≥1280imes800≥1280*imes*800 px con teclado y ratón). Centraliza la gobernanza de inquilinos (*tenants*), configuración de suscripciones, emisión formal de órdenes de compra a proveedores textiles con recepción de facturas comerciales, exportación masiva a Excel/PDF y análisis visual del Dashboard predictivo de Machine Learning.
    - **Control de Acceso:** Restricción a nivel de interfaz que valida el agente de usuario (*User-Agent*) y resolución de pantalla, bloqueando transacciones corporativas complejas desde dispositivos móviles para prevenir errores operativos.
2. **Progressive Web App - PWA (Angular 17+ / Service Workers / IndexedDB):**
    - **Público Objetivo:** Clientes web y operadores de caja en mostradores de tiendas físicas.
    - **Alcance Funcional:** Aplicación web moderna instalable en cualquier navegador compatible con el estándar PWA (Google Chrome, Microsoft Edge, Safari). Emplea Service Workers para el almacenamiento en caché de activos estáticos y miniaturas WebP, y la API **IndexedDB** para mantener una réplica estructurada local del catálogo de prendas, variantes y cola de transacciones. Permite consultar el catálogo y registrar reservas presenciales y ventas directas en mostrador ante la pérdida absoluta de conexión a Internet.
3. **Aplicación Móvil Nativa Multiplataforma (Flutter 3.x / Dart / SQLite):**
    - **Público Objetivo:** Clientes finales y dependientes de ventas en sala.
    - **Alcance Funcional:** Aplicación compilada nativamente para Android e iOS. Integra el motor relacional embebido **SQLite** (sqflite) para la persistencia local de prendas, categorías y reservas temporales con código QR. Incorpora el **Vestidor Virtual en Realidad Aumentada (RA)**, permitiendo proyectar modelos tridimensionales (.glb) a escala real sobre la silueta del cliente a través de la cámara móvil mediante ARCore/ARKit.
4. **Núcleo de Servicios Backend y Persistencia Relacional (FastAPI / PostgreSQL 15):**
    - **Arquitectura:** API RESTful asíncrona de alto rendimiento desarrollada sobre Python 3.11+ y **FastAPI**. Implementa inyección de dependencias para la resolución dinámica del contexto de tenant, validación estricta de esquemas con Pydantic v2 y autenticación robusta mediante tokens JWT con firmas asimétricas y hash criptográfico bcrypt.
    - **Persistencia Central:** Motor relacional **PostgreSQL 15** configurado bajo el esquema de base de datos compartida con discriminador lógico (tenant_id), garantizando

aislamiento estricto mediante políticas de seguridad a nivel de filas (*Row-Level Security* - RLS) e índices B-Tree compuestos.

### 2.1.4 Los Nueve Pilares Funcionales del Sistema Propuesto

La arquitectura del sistema se articula sobre nueve pilares de ingeniería de software que resuelven los desafíos técnicos del Segundo Parcial y las demandas del negocio:

**1. Arquitectura SaaS Multitenant y Gobernanza de Suscripciones (Desafío VI)**

La plataforma permite alojar múltiples empresas textiles independientes sobre una infraestructura compartida, garantizando la total confidencialidad e inviolabilidad de sus datos. Cada solicitud autenticada extrae el identificador del tenant desde el token JWT e inyecta obligatoriamente la cláusula WHERE tenant_id = :id en cada consulta relacional. La monetización se gestiona mediante tres tiers de suscripción:

- **Plan Básico:** Diseñado para pequeños talleres y boutiques; habilita hasta 2 sucursales físicas, catálogo offline básico y restringe el acceso a módulos de IA generativa.
- **Plan Profesional:** Orientado a marcas consolidadas; soporta hasta 10 sucursales, modelos 3D en RA y reportes generativos de IA en texto.
- **Plan Enterprise:** Pensado para franquicias y cadenas industriales; sucursales ilimitadas, reportes generativos por voz (Whisper) y Dashboard completo de Machine Learning a 6 meses.

**2. Resiliencia Operativa Offline-First en Web y Móvil (Desafío II)**

Para blindar las operaciones de venta ante la precariedad de las telecomunicaciones en ferias y centros comerciales bolivianos:

- En la PWA web, **IndexedDB** almacena de manera no volátil el catálogo de productos, variantes de color/talla y precios.
- En la aplicación móvil Flutter, la base de datos embebida **SQLite** resguarda las prendas y colecciones. Los clientes y vendedores pueden consultar fichas técnicas y generar **reservas presenciales temporales de prendas** (con vigencia paramétrica de 2 horas) obteniendo un ticket digital con código QR firmado localmente sin consumir megabytes ni requerir enlace activo.

**3. Terminal Punto de Venta (POS) Autónomo y Cobro al Contado en Efectivo (Desafío III)**

Ante una caída total de red o fluido eléctrico en la tienda física (operando con batería o UPS):

- El terminal POS del cajero conmuta de forma transparente al **modo contingencia desconectado**.
- Permite registrar ventas presenciales de mostrador cobradas **exclusivamente al contado en efectivo**, calculando subtotales, aplicando descuentos de liquidación de temporada o tarifas mayoristas, descontando el stock local y generando un comprobante provisional con código QR y un identificador criptográfico **UUID v4** inmutable.

**4. Protocolo de Sincronización Transaccional Idempotente con UUID v4 (Desafíos II y III)**

Al reanudarse la conectividad a Internet:

- Un agente reactivo dispara la sincronización en segundo plano de las transacciones encoladas.
- Cada registro se transmite mediante una petición HTTP POST acompañada de la cabecera X-Idempotency-Key: <UUID_v4>.
- El backend FastAPI valida la presencia previa del UUID en la base central; si ya existe (por reintentos de red), devuelve la confirmación almacenada sin duplicar cargos ni asientos contables.
- **Resolución Determinista de Conflictos (Overselling):** Si una prenda vendida offline en mostrador fue adquirida simultáneamente por un cliente en el e-commerce online, **se prioriza la venta física presencial**, dado que la prenda ya fue entregada en manos del cliente. El sistema cancela automáticamente la orden virtual, gestiona el reintegro de fondos y emite una alerta de reabastecimiento urgente a bodega.

**5. Catalogación Textil de Precisión Anatómica, Telas Hipoalergénicas y 3D RA**

El módulo de catálogo trasciende el tallaje estándar implementando una **ficha técnica textil avanzada**:

- **Medidas anatómicas en centímetros:** Parámetros biométricos exactos para cada talla (contorno de pecho, contorno de cintura, contorno de cadera, largo total y longitud de manga), erradicando las dudas de calce.
- **Esquemas de colores compuestos:** Selector cromático dual para prendas combinadas (ej. Camisa Oxford Blanco/Azul Marino) con códigos Hexadecimales dobles (#FFFFFF / #002366).
- **Composición de tejidos y advertencia dermatológica:** Detalle de telas nobles (lino orgánico, algodón pima peruano, lana merino) e interruptor (*toggle*) para marcar prendas con **telas hipoalergénicas**, advirtiendo compatibilidad con pieles sensibles o con afecciones dermatológicas.
- **Vestidor Virtual en RA:** Vinculación de archivos tridimensionales (.glb) optimizados para renderizado en tiempo real a escala humana sobre la cámara del dispositivo móvil.

**6. Valuación de Inventarios Multi-Sucursal con CPP Inmutable y Traspasos Logísticos**

El inventario se segrega por tienda física en tres métricas concurrentes: **Stock Físico**, **Stock Reservado** y **Stock Disponible** (Disponible = Físico - Reservado). Cada recepción de lote de mercadería con factura fiscal dispara el recálculo matemático inmutable del **Costo Promedio Ponderado (CPP)**:

$$CPP_{\text{nuevo}} = \frac{(\text{Stock}_{\text{ant}} \times CPP_{\text{ant}}) + (\text{Cant}_{\text{ingreso}} \times \text{Costo}_{\text{ingreso}})}{\text{Stock}_{\text{ant}} + \text{Cant}_{\text{ingreso}}}$$

El sistema registra el último costo unitario pagado, asienta cada ingreso y egreso en el **Kardex físico-valorado** dando estricto cumplimiento a la legislación tributaria boliviana (Ley 843) y gestiona la logística de **traspasos inter-sucursales** con estados formales (SOLICITADO, EN_TRÁNSITO, RECIBIDO_CONFORME) para balancear existencias entre tiendas sin pérdidas operativas.

**7. Postventa en Dos Tiempos, Reembolso en 30 Días y Pasarela Stripe PCI-DSS (Desafío IV)**

El ciclo de cambios de prendas se estructura mediante un flujo riguroso de **dos tiempos y dos notas independientes**:

- **Tiempo 1 (Nota de Devolución):** El cliente presenta la prenda en mostrador dentro del plazo de garantía (7 días hábiles). El cajero inspecciona las condiciones físicas (etiquetas intactas, sin signos de lavado o uso), registra el reingreso de la prenda al inventario (+1 stock disponible o a cuarentena) y emite formalmente una **Nota de Devolución**, acreditando el saldo monetario a favor del cliente.
- **Tiempo 2 (Nota de Cambio):** Con la Nota de Devolución, el cliente selecciona una nueva prenda. Si el valor es idéntico, se liquida a saldo cero; si la nueva prenda tiene mayor valor (por talla, tela premium o modelo superior), el cliente abona el excedente en línea a través de la pasarela **Stripe Sandbox** (tokenización segura con cumplimiento de la norma PCI-DSS). Finalmente, el sistema descuenta la nueva prenda (-1 stock) y genera la **Nota de Cambio** de despacho final.
- **Política de Reembolso en Efectivo a 30 Días:** Si el cliente no encuentra una prenda de su interés y exige la devolución de su dinero en efectivo, **el cajero no entrega dinero de inmediato en mostrador**. El sistema emite una orden de liquidación contable programada para el cierre mensual de la empresa (a los 30 días calendario), garantizando la cuadratura del arqueo de caja de la sucursal y la auditoría fiscal.

**8. Aprovisionamiento y Compras Corporativas Exclusivas desde Web Desktop**

Respondiendo a la instrucción de que las adquisiciones corporativas a proveedores son operaciones de alto impacto financiero ejecutadas en horarios de oficina por personal administrativo, se implementa un módulo accesible **únicamente desde computadoras de escritorio y laptops (Web Desktop)**. Permite planificar órdenes de compra por curvas de tallas, negociar con proveedores textiles registrados, cargar facturas comerciales fiscales autorizadas y disparar automáticamente el ingreso de stock al Kardex valorado y la actualización del Costo Promedio Ponderado.

**9. Inteligencia Analítica Híbrida: Reportes NLP/Voz con IA y Machine Learning a 6 Meses (Desafío V)**

La plataforma provee herramientas avanzadas de toma de decisiones para los niveles ejecutivos de las empresas suscriptoras:

- **Asistente Conversacional con IA Generativa:** Permite a directivos consultar métricas de ventas, rotación de tallas o rendimiento de sucursales formulando preguntas en lenguaje natural escrito o mediante dictado por voz (transcrito por el modelo **Whisper**). Un LLM traduce la solicitud a sentencias SQL analíticas, las cuales son analizadas por un **validador sintáctico AST de seguridad** (que prohíbe sentencias destructivas DROP, DELETE, UPDATE, INSERT, restringe la consulta exclusivamente a SELECT e inyecta forzosamente la cláusula tenant_id). La consulta se ejecuta en una réplica de lectura y se sintetiza en resúmenes ejecutivos con gráficos y descarga directa en formatos corporativos **Excel (.xlsx)** y **PDF (.pdf)**.
- **Dashboard Predictivo con Machine Learning:** Alimentado por un pipeline de más de 6 meses de datos sintéticos que reproducen fielmente la estacionalidad del comercio textil masculino boliviano (invierno, primavera-verano, festividades, picos de fin de año), despliega modelos de regresión para proyectar ventas a 6 meses vista, clustering (K-Means) para

segmentación de clientes y algoritmos de detección de anomalías (Isolation Forest) para auditar pérdidas de existencias.

### 2.1.5 Matriz de Integración Tecnológica, Protocolos y Seguridad

La siguiente matriz consolida los protocolos de comunicación, tipos de datos y barreras de seguridad implementadas en cada uno de los nodos del ecosistema FashionStore:

| Componente | Plataforma / Framework | Protocolo de Transporte | Formato de Carga Útil | Mecanismos de Seguridad y Aislamiento |
|---|---|---|---|---|
| **Web Desktop Corporativa** | Angular 17+ / TypeScript / SCSS | HTTPS (TLS 1.3) / REST | JSON / Multipart (Archivos) | Autenticación JWT Bearer, verificación de resolución ≥1280≥1280 px y *User-Agent* PC. |
| **Progressive Web App (PWA)** | Angular 17+ / Service Workers | HTTPS / Cache API / IndexedDB | JSON / Binario WebP | Almacenamiento local aislado en navegador, validación de permisos de cámara y ubicación. |
| **Móvil Flutter** | Flutter 3.x / Dart / ARCore-ARKit | HTTPS / SQLite (sqflite) | JSON / Binario 3D (.glb) | Sandbox de aplicación móvil, cifrado de base SQLite local, tokens JWT renovables. |
| **Punto de Venta POS Desconectado** | Angular PWA / Service Worker | Localhost / Eventos DOM / IndexedDB | JSON con firma criptográfica UUID v4 | Cierre de transacciones con UUID v4 inmutable, bloqueo de cobros que no sean en efectivo. |
| **Protocolo de Sincronización** | Background Fetch API / FastAPI Sync | HTTPS POST por lotes (*batching*) | JSON (X-Idempotency-Key: UUID) | Deduplicación transaccional en tabla processed_transactions, resolución determinista de stock. |
| **Núcleo de Servicios Backend** | FastAPI / Python 3.11+ / SQLAlchemy | RESTful / Asíncrono ASGI (Uvicorn) | JSON estricto con Pydantic v2 | Inyección de dependencias get_current_tenant, hashing de contraseñas con bcrypt (12 rondas). |
| **Base de Datos Central** | PostgreSQL 15 Relacional | Conexión TCP con pool asyncpg | Tuplas relacionales normalizadas | Row-Level Security (RLS) forzoso por tenant_id, índices B-Tree compuestos, roles de solo lectura. |
| **Pasarela de Pagos Excedentes** | Stripe SDK / Webhooks | HTTPS / REST TLS 1.3 | JSON Tokenizado (sin datos PAN/CVV) | Tokenización directa en cliente (Stripe.js), cumplimiento normativo PCI-DSS Nivel 1. |

| | | | | |
|---|---|---|---|---|
| **Auditoría e IA Generativa** | Whisper API / LangChain / SQLGlot | HTTPS / Streaming de Audio / SSE | Audio WAV/MP3 $ ightarrow$ SQL $ | Validador sintáctico AST (*Abstract Syntax Tree*), filtrado estricto de solo lectura, inyección forzosa de tenant. |

## 2.2 Identificación de Actores

| Actor | Tipo | Descripción |
|---|---|---|
| **Administrador de Plataforma (Super Admin)** | Interno | Máxima autoridad del SaaS. Gestiona todas las empresas contratantes, planes de suscripción y la salud global de la plataforma. |
| **Administrador de Empresa** | Interno | Administrador local de cada empresa (tenant). Gestiona usuarios, roles, sucursales, productos, inventarios, ventas y reportes de su propia empresa únicamente. |
| **Encargado de Sucursal** | Interno | Responsable operativo de la sucursal. Gestiona el inventario físico local, reservas presenciales, traspasos inter-tiendas y, desde la Web Desktop (PC/Laptop), las órdenes de compra y aprovisionamiento a proveedores textiles con recepción de facturas comerciales e impacto en CPP (facultad compartida con el Administrador de Empresa). |
| **Cajero** | Interno | Operador de punto de venta (POS). Registra ventas presenciales, procesa cobros y gestiona cambios/devoluciones de prendas en mostrador. |
| **Cliente** | Externo | Consumidor final. Interactúa por web (Angular/PWA) y móvil (Flutter). Consulta catálogo, reserva, compra, solicita cambios y recibe recomendaciones IA. |
| **Proveedor** | Externo | Empresa que suministra prendas textiles. Recibe y confirma órdenes de compra corporativas. |
| **Sistema de Sincronización** | Sistema | Agente interno que gestiona la sincronización idempotente de operaciones offline (web y móvil) con el backend central. |
| **Pasarela de Pago (Stripe)** | Sistema Externo | Procesa pagos electrónicos con tokenización PCI-DSS sin almacenar credenciales bancarias. |
| **Servicio de IA (OpenAI / Gemini)** | Sistema Externo | Procesa solicitudes de reportes en lenguaje natural y genera recomendaciones contextuales. |
| **Servicio Meteorológico (OpenWeatherMap)** | Sistema Externo | Provee datos climáticos para enriquecer los modelos de machine learning y recomendaciones de temporada. |

## 2.3 Requisitos Funcionales y No Funcionales

### 2.3.1. Requisitos Funcionales

| ID | Requisito Funcional | Desafío |
|---|---|---|
| **RF01** | Gestionar empresas contratantes (tenants): registrar, activar, suspender | VI - SaaS |
| **RF02** | Gestionar planes de suscripción (Básico, Profesional, Enterprise) con límites configurables | VI - SaaS |
| **RF03** | Asignar y verificar vigencia de suscripción de cada empresa con alertas de expiración | VI - SaaS |
| **RF04** | Aislar datos entre empresas: un usuario de Empresa A no puede ver datos de Empresa B | VI - SaaS |
| **RF05** | Administrar usuarios, roles y perfiles por empresa con autenticación segura (JWT, bcrypt) | VI - SaaS |
| **RF06** | Gestionar productos, ficha técnica con medidas en cm, telas nobles/hipoalergénicas y 3D RA | Heredado+SaaS |
| **RF07** | Gestionar inventario multi-sucursal con valuación contable inmutable por CPP y Kardex Ley 843 | Heredado+SaaS |
| **RF08** | Consultar catálogo digital con filtros multicriterio, stock disponible y precios diferenciados | Heredado |
| **RF09** | Descargar y almacenar catálogo estructurado en IndexedDB/SQLite para operación offline | II/III - PWA/Offline |
| **RF10** | Registrar reservas presenciales de prendas con ticket QR firmado sin conexión a Internet | II/III - PWA/Offline |
| **RF11** | Registrar ventas POS de contado en efectivo sin conexión emitiendo comprobante con UUID v4 | IV - Ventas Offline |
| **RF12** | Consultar y auditar operaciones locales pendientes de sincronización en terminal y móvil | II/III - PWA/Offline |
| **RF13** | Sincronizar transacciones pendientes por lotes al restablecerse la conectividad a Internet | II/III - PWA/Offline |
| **RF14** | Detectar y resolver conflictos de sincronización mediante cabecera X-Idempotency-Key y prioridad física | II/III - PWA/Offline |
| **RF15** | Gestionar solicitud de cambio de prendas dentro del plazo de garantía (7 días hábiles) | IV - Ventas |
| **RF16** | Generar Nota de Devolución (Tiempo 1) acreditando saldo a favor y Nota de Cambio (Tiempo 2) | IV - Ventas |
| **RF17** | Calcular y cobrar excedente de precio cuando la nueva prenda es de mayor valor mediante pasarela | IV - Ventas |

| | | |
|---|---|---|
| **RF18** | Emitir orden de liquidación monetaria en efectivo programada a 30 días si el cliente no acepta cambio | IV - Ventas |
| **RF19** | Gestionar traspasos de inventario inter-sucursal con estados formales (Solicitado, En Tránsito, Recibido) | Heredado+Nuevo |
| **RF20** | Solicitar reportes analíticos mediante lenguaje natural escrito o dictado por voz (Whisper) | V - IA |
| **RF21** | Interpretar solicitud con LLM, validar sintaxis SQL con filtro AST seguro y ejecutar en réplica | V - IA |
| **RF22** | Exportar reportes ejecutivos generados a formatos corporativos Excel (.xlsx) y PDF (.pdf) | V - IA |
| **RF23** | Demostrar reportes con datos reales multitenant y filtros de sucursales, prendas y fechas | V - IA |
| **RF24** | Generar pipeline de datos sintéticos históricos (mínimo 6 meses) con estacionalidad de retail | V - ML |
| **RF25** | Entrenar y desplegar modelos de regresión supervisada (proyección semestral) y clustering K-Means | V - ML |
| **RF26** | Visualizar métricas predictivas, mapas de calor y detección de anomalías en Dashboard ML | V - ML |
| **RF27** | Integrar pasarela de pagos Stripe Sandbox con tokenización PCI-DSS para liquidación de saldos | IV - Ventas |
| **RF28** | Gestionar ofertas de liquidación de temporada (30%, 40%, 60%) y precios mayoristas (>= 3 prendas) | Heredado+Nuevo |
| **RF29** | Gestionar compras y aprovisionamiento corporativo exclusivo desde Web Desktop (PC/Laptop) con CPP | Heredado+Nuevo |
| **RF30** | Notificar al cliente y emitir comprobantes digitales con códigos QR para auditoría transaccional | II/III - PWA/Offline |

### 2.3.2. Requisitos No Funcionales

| ID | Requisito No Funcional | Descripción |
|---|---|---|
| **RNF01** | Seguridad | Autenticación JWT + bcrypt, autorización por roles, HTTPS/TLS obligatorio |
| **RNF02** | Aislamiento de datos | Ninguna operación permite acceder a información de otro tenant. Restricción en backend, no solo en UI |
| **RNF03** | Persistencia offline | Operaciones sin conexión sobreviven al cierre y reinicio de la aplicación |
| **RNF04** | Integridad transaccional | Ventas confirmadas y movimientos de inventario mantienen consistencia |

| | | |
|---|---|---|
| **RNF05** | Idempotencia | El reenvío de una operación confirmada no genera registros duplicados |
| **RNF06** | Recuperación ante fallos | Operaciones pendientes se conservan ante errores de comunicación |
| **RNF07** | Mantenibilidad | Código organizado por capas (presentación, negocio, persistencia, sincronización) |
| **RNF08** | Compatibilidad | PWA en navegadores modernos; Flutter conserva capacidades móviles |
| **RNF09** | Rendimiento | Consultas de catálogo y reportes evitan operaciones innecesariamente costosas |
| **RNF10** | Observabilidad | Sincronización registra información suficiente para diagnosticar fallos |
| **RNF11** | Calidad | Pruebas automatizadas que verifican reglas críticas del sistema |
| **RNF12** | Coherencia documental | Modelos UML representan la solución realmente implementada |

### 2.4. Reglas de Negocio

| ID | Regla de Negocio | Descripción |
|---|---|---|
| **RN01** | Plazo estricto de cambio | El cliente dispone de hasta 7 días hábiles desde la recepción de la prenda para solicitar formalmente su cambio. Vencido dicho plazo, el sistema bloquea cualquier solicitud de cambio. |
| **RN02** | Flujo de cambio en dos notas y dos tiempos | El proceso de cambio no es directo ni lineal. Se descompone en dos tiempos operativos: **1) Tiempo de Reingreso:** Al arribar la prenda a la sucursal, se genera la **Nota de Devolución**, reingresando físicamente la prenda al inventario (stock += 1) y registrando el monto original pagado como saldo a favor del cliente. **2) Tiempo de Despacho:** Al seleccionarse la nueva prenda (ej. talla L de mayor valor), el sistema notifica vía push al cliente la diferencia de precio (saldo adicional), el cual debe abonarse mediante la pasarela de pagos (Stripe). Solo tras la confirmación del pago se emite la **Nota de Cambio** y se decrementa el stock (stock -= 1). |
| **RN03** | Política de reembolso monetario | Si la prenda solicitada para cambio no existe en el inventario de la empresa, o si el cliente no acepta el pago de la diferencia de precio ni escoge una prenda sustituta, la empresa procederá a la devolución íntegra del dinero pagado en un plazo de hasta 30 días calendario (coincidiendo con el cierre y arqueo contable mensual). |
| **RN04** | Condiciones físicas de la prenda | Solo son admitidas prendas en perfecto estado: sin uso, limpias, con etiquetas de fábrica intactas y empaque original. El encargado de sucursal realiza la verificación física antes de emitir la Nota de Devolución. |

| | | |
|---|---|---|
| **RN05** | Reingreso al CPP histórico | La prenda devuelta se reincorpora al inventario valorada al costo promedio ponderado (CPP) histórico de la venta original, evitando distorsiones contables. |
| **RN06** | Idempotencia en operaciones offline | Presionar reiteradas veces el botón de confirmación de venta o reserva sin conexión a Internet genera una única transacción en el backend gracias a la generación de un UUID v4 unívoco por operación local. |
| **RN07** | Validación de stock y pago prioritario | En ventas digitales y cambios con saldo, el inventario se reserva temporalmente y solo se descuenta de forma definitiva al recibir el webhook de confirmación exitosa de la pasarela de pagos (Stripe). En caso de conflicto de sincronización offline, se adjudica al primer pago/sincronización confirmada en el servidor central; a las demás se les rechaza y se restituye el monto. |
| **RN08** | Aislamiento y gobernanza SaaS | Cada tenant accede únicamente a sus sucursales, empleados, catálogo, inventario y ventas. El plan contratado (Básico o Profesional) delimita las sucursales permitidas y las funciones avanzadas (IA generativa, ML). |
| **RN09** | Ofertas y liquidación de temporada | Las prendas de temporadas concluidas (ej. ropa de invierno en primavera) pueden ofertarse con descuentos especiales (30%, 40%, 60%) definidos por el Administrador de Empresa para recuperar capital de inversión. Los artículos nuevos de temporada vigente mantienen su precio regular. El catálogo digital incluye una sección destacada con filtros de ofertas y porcentaje de rebaja. |
| **RN10** | Precios diferenciados (Menor vs. Mayor) | El sistema maneja dos tipos de precio para cada producto: precio por menor (unitario) y precio por mayor (aplicable automáticamente a partir de 3 unidades idénticas: cuarta, media docena o docena). |
| **RN11** | Traspasos inter-sucursal | Los movimientos de stock entre sucursales solo son válidos entre sucursales del mismo tenant. Requieren una Nota de Traspaso en estado EN_TRANSITO (origen descuenta) y confirmación de recepción en destino para incrementar stock. |
| **RN12** | Modalidad exclusiva de venta offline | Las ventas efectuadas sin conectividad a Internet son estrictamente de contado (efectivo). No se permite el procesamiento de pagos electrónicos offline por restricciones de seguridad PCI-DSS. |
| **RN13** | Atributos de prendas y composición textil | Cada producto debe detallar SKU, nombre, categoría, temporada, tallas, medidas corporales (pecho, cintura, cadera, largo), combinaciones de colores múltiples (ej. blanco/negro) y composición textil (tipo de tela: algodón pima, lino, lana, fibras sintéticas, con advertencias hipoalergénicas). |
| **RN14** | Compras centralizadas vía Web Desktop | El aprovisionamiento y compras a proveedores textiles se realizan exclusivamente desde la aplicación Web en computadoras de escritorio o laptops por personal autorizado (Encargado de Sucursal / Admin Empresa), dada la complejidad técnica, contractual y financiera de las órdenes de compra. |

## 2.4 Reglas de Negocio

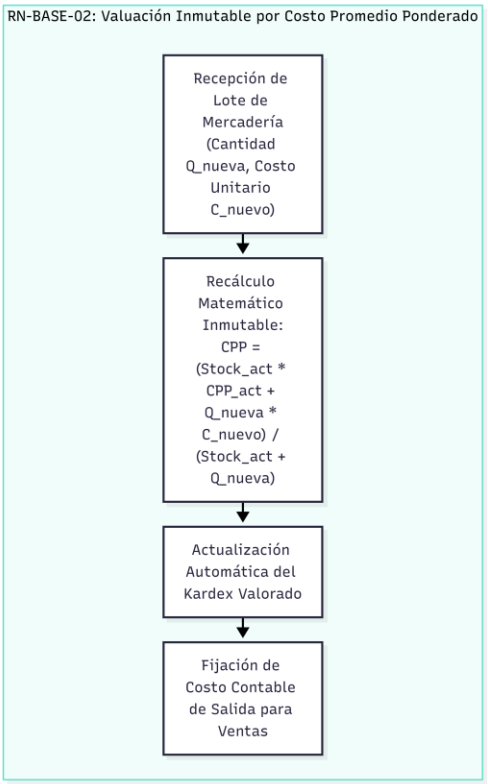
Las reglas de negocio definen las directrices operacionales, restricciones contables, políticas de seguridad y flujos transaccionales inmutables que gobiernan a FashionStore SaaS en su versión del Segundo Parcial. Para garantizar rigor formal y trazabilidad, se estructuran en siete categorías operativas acompañadas de sus respectivos diagramas lógicos de decisión:

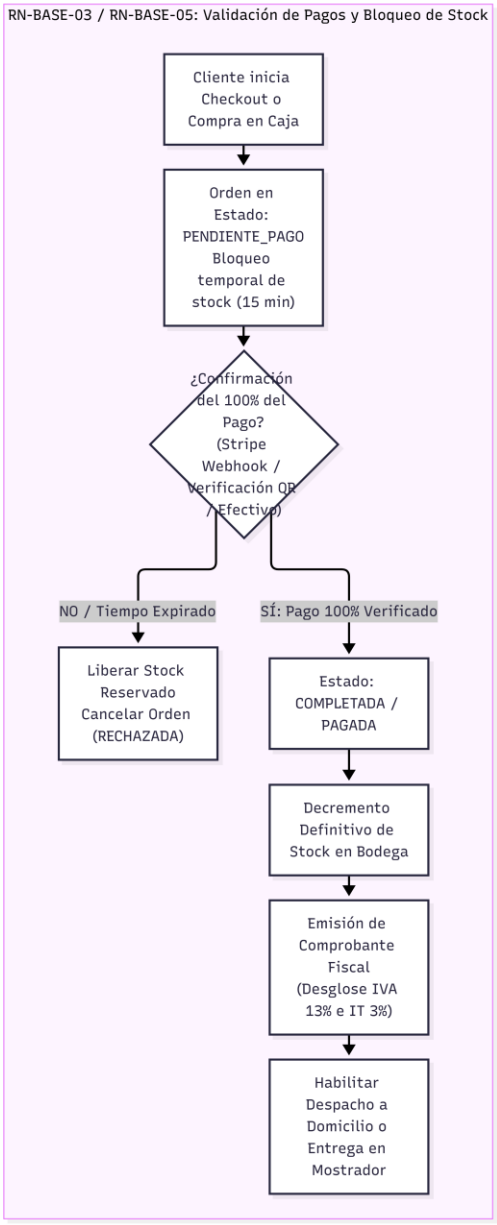
### 2.4.1 Reglas del Negocio Base

- **RN-BASE-01 (Control de Acceso Basado en Roles - RBAC Estricto):** Ningún usuario de la plataforma puede invocar endpoints de la API ni acceder a módulos de interfaz fuera de los privilegios concedidos formalmente a su rol asignado (SUPER_ADMIN, ADMIN_EMPRESA, ENCARGADO_SUCURSAL, CAJERO, CLIENTE). Todo intento no autorizado es interceptado y denegado con código HTTP 403 Forbidden.
- **RN-BASE-02 (Valuación Inmutable por Costo Promedio Ponderado - CPP):** Ante cada recepción de mercadería respaldada por factura fiscal con cantidad de ingreso QnuevaQnueva y costo unitario de adquisición CnuevoCnuevo, el sistema ejecuta de forma automática e inmutable el recálculo matemático del Costo Promedio Ponderado (CPP):

$$CPP_{\text{nuevo}} = \frac{(Stock_{\text{actual}} \times CPP_{\text{actual}}) + (Q_{\text{nueva}} \times C_{\text{nuevo}})}{Stock_{\text{actual}} + Q_{\text{nueva}}}$$

- **RN-BASE-03 (Bloqueo Temporal de Existencias en Reservas Presenciales):** Las prendas reservadas por clientes mediante el catálogo digital bloquean stock disponible por un periodo máximo de 2 horas. Si el cliente no se presenta en la tienda física dentro de esa ventana de tiempo, el sistema libera automáticamente el stock reservado devolviéndolo al stock disponible general.
- **RN-BASE-04 (Ventas POS en Mostrador y Desglose Tributario Ley 843):** Las ventas presenciales en mostrador actualizan el inventario físico en tiempo real y generan de forma obligatoria el comprobante de venta fiscal con código QR y desglose tributario según la normativa boliviana (IVA 13%, IT 3%).
- **RN-BASE-05 (Confirmación Efectiva del 100% del Pago Previa al Despacho e Inventario):** Conforme a la política de control financiero y de inventarios exigida por la cátedra (audio Parcial2-2.txt, líneas 33-35), ninguna orden de venta (digital o presencial con cobro electrónico por QR o tarjeta) puede consolidarse como completada, ni decrementar el stock definitivo en el inventario central ni habilitar la entrega física del producto, si no se ha recibido la confirmación verificada del 100% del pago por parte de la pasarela (Stripe Sandbox) o la verificación del comprobante de transferencia bancaria/QR por el cajero. Mientras el pago no esté efectivamente acreditado, la orden permanece en estado PENDIENTE_PAGO y el stock solo se bloquea de manera transitoria por un tiempo máximo de 15 minutos, liberándose automáticamente si no se consuma la liquidación.



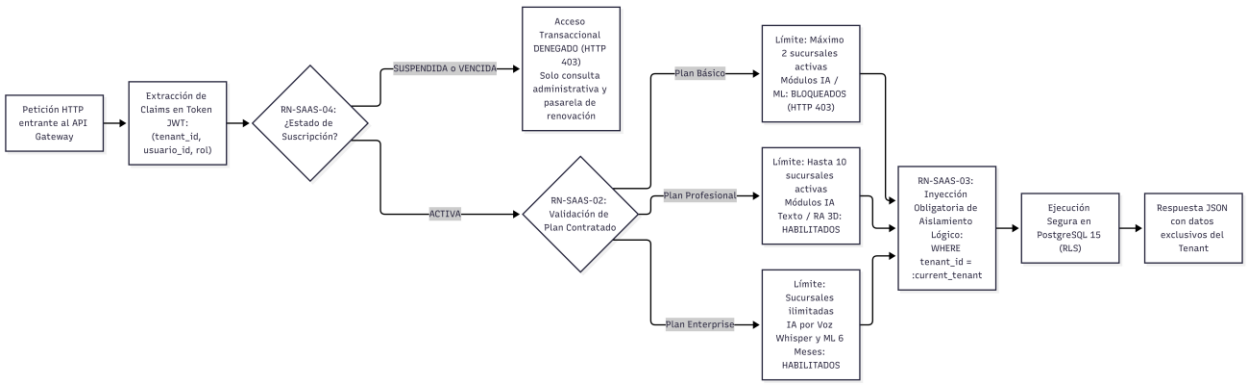




### 2.4.2 Reglas SaaS y Multitenant

- **RN-SAAS-01 (Jerarquía y Separación Empresa vs. Sucursal):** Una Empresa contratante (Tenant) puede poseer múltiples Sucursales físicas en distintas ciudades, pero una Sucursal pertenece estrictamente y de por vida a una única Empresa.
- **RN-SAAS-02 (Límites por Plan de Suscripción Comercial):**
    - **Plan Básico:** Máximo 2 sucursales físicas activas. Catálogo offline básico. Módulos de IA generativa y Machine Learning completamente deshabilitados.

- **Plan Profesional:** Hasta 10 sucursales activas. Habilita modelos 3D en RA y módulo de reportes generativos con IA en lenguaje natural escrito.
- **Plan Enterprise:** Sucursales ilimitadas. Acceso completo a reportes generativos de IA por voz (Whisper) y Dashboard predictivo con Machine Learning a 6 meses.

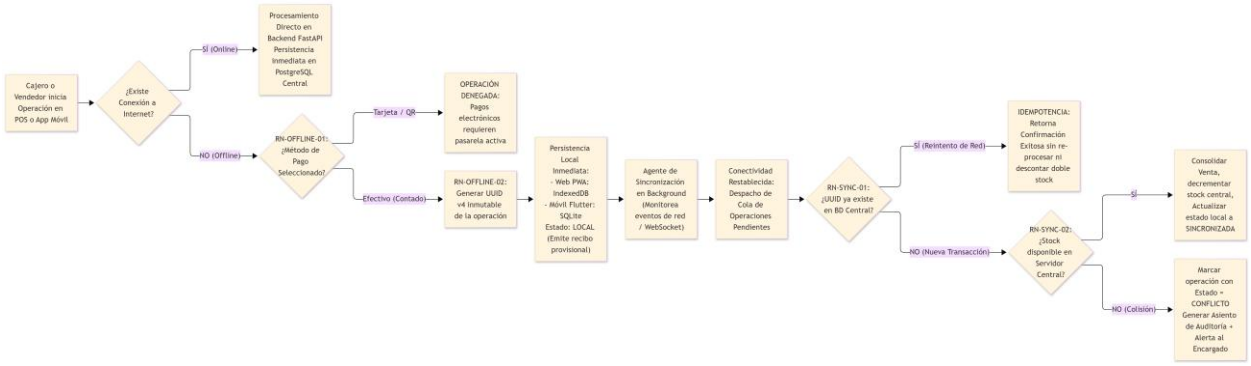
- **RN-SAAS-03 (Aislamiento Lógico Hermético de Datos):** Todas las consultas relacionales ejecutadas en PostgreSQL 15 inyectan de forma mandatoria la cláusula discriminadora WHERE tenant_id = :current_tenant. Cualquier intento deliberado o accidental de acceder a datos de otra empresa contratante es interceptado a nivel de middleware y rechazado con código HTTP 403 Forbidden.
- **RN-SAAS-04 (Control de Estado y Vigencia de Suscripción):** Si la suscripción de una empresa pasa al estado SUSPENDIDA o VENCIDA, el sistema bloquea inmediatamente cualquier operación transaccional de venta, compra o traspaso de inventario, permitiendo únicamente el acceso a vistas administrativas de lectura y a la pasarela de renovación de licencia.




### 2.4.3 Reglas Offline y Sincronización

- **RN-OFFLINE-01 (Ventas Offline Exclusivas al Contado en Efectivo):** En modo de contingencia fuera de línea únicamente se admiten ventas con método de cobro presencial en efectivo. Los cobros electrónicos con tarjeta de crédito/débito o QR interoperable exigen pasarelas y validación bancaria en línea, por lo que quedan estrictamente bloqueados fuera de línea en cumplimiento del estándar internacional de seguridad PCI-DSS.
- **RN-OFFLINE-02 (Persistencia Local No Volátil con UUID v4):** Toda transacción registrada en modo desconectado se persiste inmediatamente en el motor de almacenamiento local del cliente (IndexedDB en Angular PWA o SQLite en Flutter), asignándole un identificador criptográfico **UUID v4** inmutable y marcando su estado como LOCAL_PENDIENTE.
- **RN-SYNC-01 (Protocolo de Idempotencia Garantizada):** Toda petición de sincronización hacia el backend incluye la cabecera HTTP X-Idempotency-Key conteniendo el UUID v4 de la operación. El backend valida dicho identificador en la tabla central processed_transactions; si el UUID ya fue registrado previamente (debido a reintentos o cortes de red intermitentes), el servidor devuelve la confirmación original sin duplicar ventas, cobros ni movimientos en el Kardex.

- **RN-SYNC-02 (Resolución Determinista de Conflictos y Sobreventa - Overselling):** Si al recuperar la conectividad una prenda vendida de forma presencial en mostrador fuera de línea fue adquirida simultáneamente por un comprador en la tienda virtual online, **se prioriza de forma absoluta la venta física presencial**, dado que la prenda ya fue entregada en manos del cliente. El sistema cancela la orden virtual en conflicto, restituye automáticamente el monto al cliente digital y emite una alerta prioritaria de reposición de stock al Encargado de Sucursal.

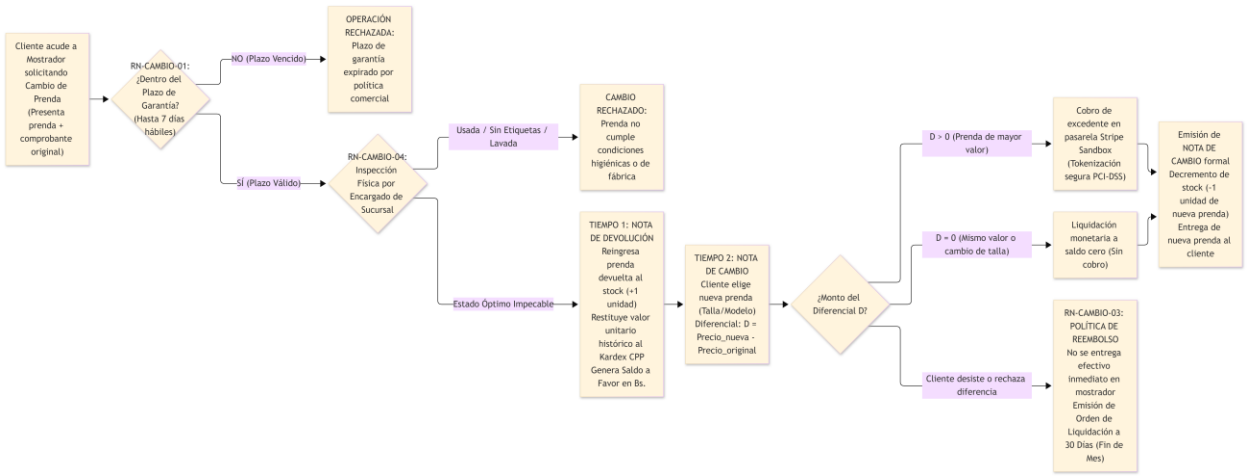



### 2.4.4 Reglas de Cambio y Devolución de Prendas

- **RN-CAMBIO-01 (Plazo Límite de Garantía Comercial):** El cliente dispone de un plazo estricto de **hasta 7 días hábiles** computados a partir de la fecha de emisión del comprobante original de venta para solicitar formalmente el cambio de una prenda en sucursal. Vencido dicho plazo, el sistema bloquea irreversiblemente la opción de cambio en terminal.
- **RN-CAMBIO-02 (Flujo Obligatorio en Dos Notas y Dos Tiempos Independientes):** Todo proceso de postventa se descompone operativamente en dos tiempos secuenciales inmutables:
    1. **Tiempo 1 (Nota de Devolución):** Al presentarse el cliente en mostrador, el cajero o encargado verifica el estado físico de la prenda. Se genera en el sistema la **Nota de Devolución**, reingresando formalmente la prenda al stock de la sucursal (+1 stock físico en bodega o cuarentena) y acreditando el monto original pagado como saldo a favor en cuenta del cliente.
    2. **Tiempo 2 (Nota de Cambio):** Con la Nota de Devolución emitida, el cliente selecciona una nueva prenda. Si el valor es equivalente, se liquida a saldo cero; si la nueva prenda posee mayor valor (por talla superior, tela noble o modelo premium), el sistema calcula el diferencial de precio (D=Precionueva−Preciooriginal*D=Precio*nueva −*Precio*original) y exige su pago mediante la pasarela **Stripe Sandbox** (tokenización PCI-DSS). Una vez acreditado el pago, se emite la **Nota de Cambio** formal y se descuenta el stock (-1 stock de la nueva prenda).
- **RN-CAMBIO-03 (Política de Reembolso Monetario en Efectivo a 30 Días):** Si la prenda solicitada para cambio no existe en el inventario de la empresa, o si el cliente desiste del cambio y no acepta pagar la diferencia de precio, **el cajero no entrega dinero en efectivo de inmediato en el mostrador**. El sistema emite una orden de liquidación monetaria formal

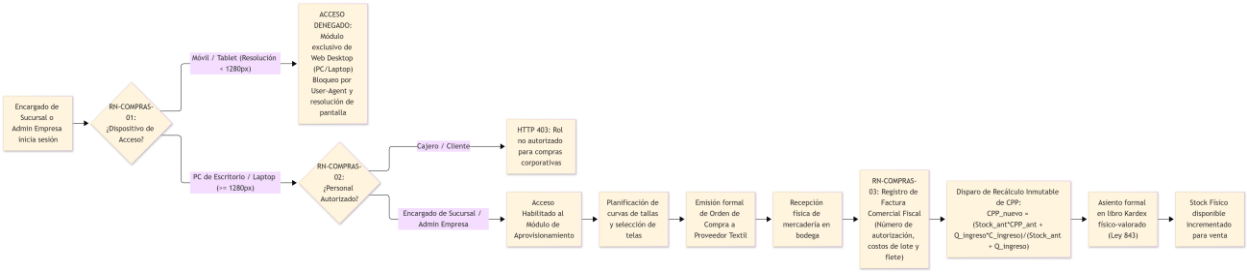
programada contablemente para el cierre mensual de la empresa (a los 30 días calendario), garantizando la cuadratura del arqueo de caja de la sucursal y la auditoría fiscal.

- **RN-CAMBIO-04 (Condiciones Físicas y Reingreso al CPP Histórico):** Solo se admiten prendas en perfecto estado (limpias, con etiquetas intactas de fábrica y sin señales de lavado ni uso). La prenda devuelta se reincorpora al inventario valorada al Costo Promedio Ponderado (CPP) histórico de la venta original, evitando distorsiones contables en el Kardex.



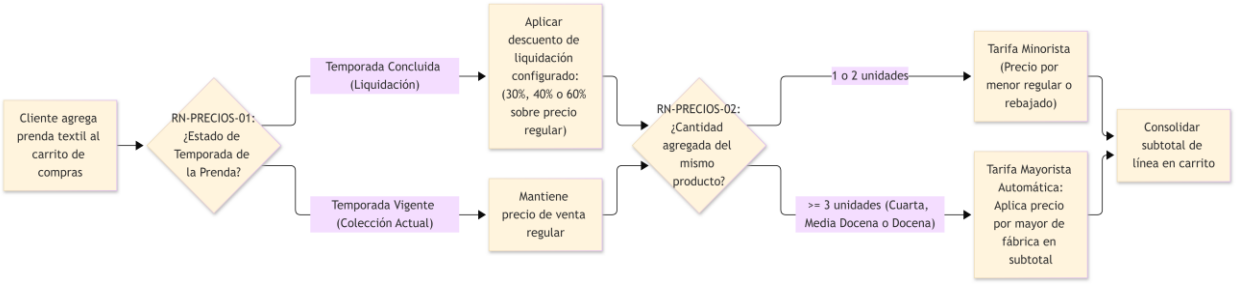

### 2.4.5 Reglas de Aprovisionamiento y Compras B2B desde Web Desktop

- **RN-COMPRAS-01 (Restricción Exclusiva a Computadoras de Escritorio / Laptops):** El módulo de compras y aprovisionamiento corporativo a proveedores textiles solo es accesible desde la aplicación Web en computadoras de escritorio y laptops (resolución de pantalla ≥1280×800≥1280×800 px con teclado y ratón). El sistema bloquea el acceso a este módulo desde navegadores móviles o tablets, previniendo errores operacionales en órdenes de compra corporativas de alto impacto financiero.
- **RN-COMPRAS-02 (Personal Autorizado para Aprovisionamiento):** Las órdenes de compra a proveedores solo pueden ser emitidas, negociadas y autorizadas por el **Encargado de Sucursal** o el **Administrador de Empresa**.
- **RN-COMPRAS-03 (Recepción con Factura Fiscal y Disparo Inmutable de CPP):** Toda recepción física de lote de mercadería textil exige el registro del número de Factura Comercial Fiscal y sus costos asociados, disparando automáticamente el recálculo matemático inmutable del Costo Promedio Ponderado (CPP) y el asiento formal en el libro de Kardex físico-valorado (Ley 843).



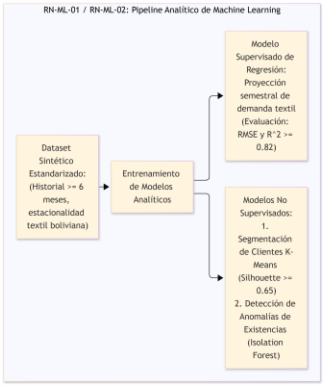
### 2.4.6 Reglas de Ofertas de Liquidación y Precios Diferenciados

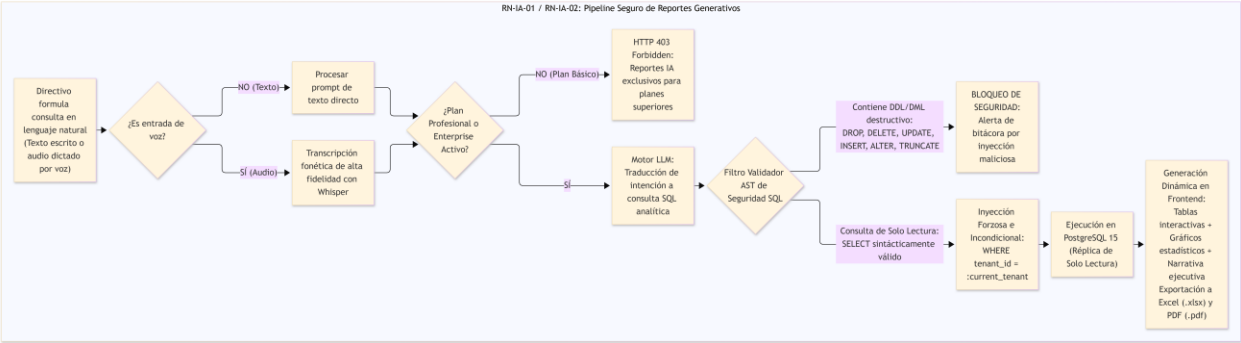
- **RN-PRECIOS-01 (Ofertas y Liquidación de Fin de Temporada):** Las prendas de temporadas concluidas (ej. colección otoño-invierno en primavera) pueden ofertarse con descuentos especiales preconfigurados (30%, 40% o 60%) definidos por el Administrador de Empresa para recuperar capital de inversión. Las prendas de temporada vigente mantienen su precio regular. El catálogo digital incluye una sección destacada con filtros de ofertas y porcentaje de rebaja.
- **RN-PRECIOS-02 (Precios Diferenciados por Mayor Automáticos):** El sistema calcula automáticamente tarifas de precio por mayor cuando el cliente adquiere 3 o más unidades idénticas (cuarta, media docena o docena), aplicando el descuento mayorista en el carrito de compras sin requerir intervención manual del cajero.




### 2.4.7 Reglas de Inteligencia Artificial Generativa y Machine Learning

- **RN-IA-01 (Privilegio por Plan de Suscripción):** La generación de reportes analíticos con IA y el procesamiento de consultas por voz mediante Whisper están reservados exclusivamente para empresas con planes superiores activos (Plan Profesional y Plan Enterprise).
- **RN-IA-02 (Validador Sintáctico AST de Seguridad en SQL Generado):** Toda consulta SQL traducida por el LLM a partir de lenguaje natural debe someterse a un validador sintáctico AST (*Abstract Syntax Tree*):
    1. Debe ser estrictamente de solo lectura (SELECT).
    2. Prohíbe de forma absoluta cualquier sentencia destructiva o modificadora (DROP, DELETE, UPDATE, INSERT, ALTER, TRUNCATE).
    3. Inyecta obligatoriamente la cláusula WHERE tenant_id = :current_tenant.
    4. Se ejecuta en una réplica de base de datos de solo lectura.
- **RN-ML-01 (Dataset Sintético Estandarizado de Entrenamiento):** Los modelos predictivos de Machine Learning se entrenan sobre un pipeline de datos sintéticos representativo de al menos 6 meses de historial (~5,000 transacciones y ~500 perfiles) que modela la estacionalidad textil boliviana.
- **RN-ML-02 (Métricas Mínimas de Precisión Analítica):** El modelo supervisado de regresión para proyección semestral de demanda debe garantizar un coeficiente de determinación $R2 \geq 0.82$$R2 \geq 0.82$, mientras que los algoritmos de segmentación no supervisada (K-Means) deben alcanzar un coeficiente de silueta (*Silhouette score*) $\geq 0.65$$\geq 0.65$.




### 2.4.8 Matriz Consolidada de Reglas de Negocio (Mapeo RN01 - RN18)

A continuación se presenta la tabla resumen de todas las reglas de negocio del sistema, preservando la numeración unívoca para su trazabilidad en el diseño y casos de prueba:

| ID | Regla de Negocio | Categoría | Descripción Breve |
|---|---|---|---|
| **RN01** | Plazo estricto de garantía de cambio | Postventa | Plazo máximo de 7 días hábiles desde la emisión de la venta para solicitar formalmente el cambio de prendas. |
| **RN02** | Flujo de cambio en dos tiempos y dos notas | Postventa | Descomposición en Tiempo 1: Nota de Devolución (+1 stock) y Tiempo 2: Nota de Cambio (-1 stock) con cobro de excedente por Stripe. |
| **RN03** | Política de reembolso monetario a 30 días | Postventa | Si no se acepta el cambio, el dinero no se devuelve de inmediato en caja; se liquida al cierre mensual contable (30 días). |
| **RN04** | Condiciones físicas de la prenda e inspección | Postventa | Inspección obligatoria por Encargado de Sucursal: prendas limpias, sin uso y con etiquetas intactas de fábrica. |
| **RN05** | Reingreso al CPP histórico original | Inventario | La prenda devuelta se reincorpora al kardex valorada al costo promedio ponderado de la venta original. |

| | | | |
|---|---|---|---|
| **RN06** | Idempotencia en operaciones offline con UUID v4 | Sincronización | Prevención de duplicados en el backend mediante identificador UUID v4 único transmitido en cabecera X-Idempotency-Key. |
| **RN07** | Validación de stock y pago prioritario | Ventas/Pagos | Confirmación efectiva del 100% del pago por webhook de Stripe antes de consolidar inventario. Prioridad física a la venta en mostrador. |
| **RN08** | Aislamiento y gobernanza SaaS Multitenant | SaaS | Aislamiento lógico forzoso con WHERE tenant_id = :id y delimitación de recursos según plan contratado (Básico, Profesional, Enterprise). |
| **RN09** | Ofertas y liquidación de fin de temporada | Precios | Descuentos especiales (30%, 40%, 60%) en prendas de temporadas concluidas para recuperar capital de inversión. |
| **RN10** | Precios diferenciados (Menor vs. Mayor) | Precios | Aplicación automática de tarifas por mayor a partir de 3 unidades idénticas (cuarta, media docena o docena). |
| **RN11** | Traspasos inter-sucursales de inventario | Inventario | Movimientos válidos únicamente entre sucursales del mismo tenant, con estados formales EN_TRÁNSITO y confirmación en destino. |
| **RN12** | Modalidad exclusiva de venta offline en efectivo | POS Offline | Ventas sin conexión restringidas estrictamente a pago en efectivo. Bloqueo de tarjetas y QR por cumplimiento de PCI-DSS. |
| **RN13** | Ficha técnica y composición textil avanzada | Catálogo | Especificación obligatoria de medidas biométricas en cm, códigos bicolores Hex, telas nobles y advertencia hipoalergénica. |
| **RN14** | Compras centralizadas vía Web Desktop | Compras B2B | Restricción estricta de aprovisionamiento corporativo a PC/Laptop (>= 1280px) operado por Encargado de Sucursal o Admin Empresa. |
| **RN15** | Control de acceso basado en roles (RBAC) | Seguridad | Validación de credenciales JWT y bloqueo de operaciones fuera del rol asignado (HTTP 403). |
| **RN16** | Bloqueo temporal de stock en reservas (2 horas) | Reservas | Apartado temporal de prendas en probador por un máximo de 2 horas con liberación automática si no asiste el cliente. |
| **RN17** | Validador sintáctico AST de seguridad en IA | IA Generativa | Inspección de consultas SQL generadas por LLM, exigiendo solo SELECT y bloqueando sentencias DDL/DML destructivas. |
| **RN18** | Requisitos mínimos de modelos Machine Learning | Machine Learning | Entrenamiento con >= 6 meses de datos sintéticos estacionales y métricas mínimas ($R2 \geq 0.82 R2 \geq 0.82$, Silhouette $\geq 0.65 \geq 0.65$). |

## 2.5 Diagrama UML de Casos de Uso

### 2.5.1. Matriz Integral y Lista Consolidada de Casos de Uso del Segundo Parcial

La siguiente matriz presenta la especificación exhaustiva de los **20 Casos de Uso (CU01 al CU20)** desarrollados para el Segundo Examen Parcial bajo la metodología ICONIX. Cada caso de uso se encuentra vinculado bidireccionalmente con los requisitos funcionales que satisface, el módulo de arquitectura al que pertenece, las reglas de negocio que restringen su comportamiento, sus actores participantes y los canales de despliegue correspondientes:

| CU | Nombre del Caso de Uso | Módulo Asignado | Requisitos Funcionales (RF) | Reglas de Negocio (RN) | Actor Principal | Desafío Técnico |
|---|---|---|---|---|---|---|
| **CU01** | Gestionar Empresas (Tenants) | M01: Gobernanza SaaS y Seguridad | RF01, RF04 | RN08, RN-SAAS-01 | Super Admin | Desafío VI: SaaS Multitenant |
| **CU02** | Gestionar Planes de Suscripción | M01: Gobernanza SaaS y Seguridad | RF02 | RN08, RN-SAAS-02 | Super Admin | Desafío VI: SaaS Multitenant |
| **CU03** | Gestionar Suscripción de Empresa | M01: Gobernanza SaaS y Seguridad | RF03 | RN08, RN-SAAS-04 | Super Admin | Desafío VI: SaaS Multitenant |
| **CU04** | Autenticar Usuario Multitenant | M01: Gobernanza SaaS y Seguridad | RF04, RF05 | RN08, RN15, RN-BASE-01 | Todos los usuarios | Desafío VI: SaaS Multitenant |
| **CU05** | Gestionar Usuarios y Roles por Empresa | M01: Gobernanza SaaS y Seguridad | RF04, RF05 | RN08, RN15, RN-BASE-01 | Admin Empresa | Desafío VI: SaaS Multitenant |
| **CU06** | Gestionar Productos y Atributos Textiles Avanzados | M04: Catalogación Textil de Precisión | RF06 | RN13 | Admin Empresa | Desafío I: Ficha Textil Avanzada |
| **CU07** | Gestionar Inventario Multi-Sucursal, Kardex y CPP | M05: Inventario, Kardex y CPP | RF07, RF08 | RN05, RN-BASE-02 | Admin Empresa | Desafíos I y IV: Valuación CPP |

| | | | | | | |
|---|---|---|---|---|---|---|
| **CU08** | Descargar y Consultar Catálogo Digital Offline | M02: Catálogo Omnicanal Offline-First | RF08, RF09 | RN-OFFLINE-02 | Cliente | Desafíos II y III: PWA / Offline |
| **CU09** | Registrar Reserva Presencial Sin Conexión | M02: Catálogo Omnicanal Offline-First | RF10 | RN16, RN-BASE-03, RN-OFFLINE-02 | Cliente | Desafíos II y III: Offline-First |
| **CU10** | Registrar Venta POS de Contado Sin Conexión | M03: POS Offline y Sincronización | RF11, RF30 | RN06, RN12, RN-BASE-04, RN-OFFLINE-01 | Cajero | Desafío III: POS Offline Efectivo |
| **CU11** | Consultar y Auditar Operaciones Locales Pendientes | M03: POS Offline y Sincronización | RF12, RF30 | RN06, RN-OFFLINE-02 | Cajero | Desafío III: Auditoría Offline |
| **CU12** | Sincronizar Operaciones Pendientes (Idempotencia) | M03: POS Offline y Sincronización | RF13, RF14 | RN06, RN-SYNC-01, RN-SYNC-02 | Sistema Sincronización | Desafíos II y III: Sync Idempotente |
| **CU13** | Gestionar Cambio de Prenda en Dos Tiempos y Devoluciones | M06: Postventa, Pasarela y Ofertas | RF15, RF16, RF17, RF18, RF30 | RN01, RN02, RN03, RN04, RN-CAMBIO-01..04 | Cajero | Desafío IV: Postventa en 2 Tiempos |
| **CU14** | Gestionar Traspaso Inter-Sucursal de Inventario | M05: Inventario, Kardex y CPP | RF19 | RN11 | Encargado Sucursal | Desafíos I y IV: Logística Retail |
| **CU15** | Solicitar Reporte Generativo con IA (Texto y Voz) | M08: Inteligencia Analítica de Negocio | RF20, RF21, RF23 | RN17, RN-IA-01, RN-IA-02 | Admin Empresa | Desafío V: IA Generativa NLP/Voz |
| **CU16** | Exportar Reportes Analíticos a Excel/PDF | M08: Inteligencia Analítica de Negocio | RF22, RF23 | RN17 | Admin Empresa | Desafío V: Inteligencia de Negocio |

| | | | | | | |
|---|---|---|---|---|---|---|
| **CU17** | Gestionar Dashboard Analítico con Machine Learning | M08: Inteligencia Analítica de Negocio | RF24, RF25, RF26 | RN18, RN-ML-01, RN-ML-02 | Admin Empresa | Desafío V: Machine Learning 6 Meses |
| **CU18** | Gestionar Pasarela de Pagos (Stripe Sandbox PCI-DSS) | M06: Postventa, Pasarela y Ofertas | RF17, RF27 | RN07, RN-BASE-05 | Cajero / Cliente | Desafío IV: Pasarela Electrónica |
| **CU19** | Gestionar Ofertas de Liquidación y Precios por Mayor | M06: Postventa, Pasarela y Ofertas | RF28 | RN09, RN10, RN-PRECIOS-01, 02 | Admin Empresa | Desafíos I y IV: Precios Dinámicos |
| **CU20** | Gestionar Compras y Aprovisionamiento desde Web Desktop | M07: Compras B2B Web Desktop | RF29 | RN14, RN-COMPRAS-01..03, RN-BASE-02 | Admin Empresa / Encargado Sucursal | Desafío I: Compras B2B con CPP |

**Distribución Analítica de Casos de Uso por Módulo del Segundo Parcial**

| Código Módulo | Nombre del Módulo del Segundo Parcial | Casos de Uso Contenidos |
|---|---|---|
| **M01** | Gobernanza SaaS Multitenant, Suscripciones y Seguridad RBAC | CU01, CU02, CU03, CU04, CU05 |
| **M02** | Catálogo Omnicanal Offline-First y Probador Virtual 3D RA | CU08, CU09 |
| **M03** | Punto de Venta (POS) Offline en Efectivo y Sincronización Idempotente | CU10, CU11, CU12 |
| **M04** | Catalogación Textil de Precisión Anatómica, Telas Nobles y 3D | CU06 |
| **M05** | Gestión de Inventario Multi-Sucursal, Kardex Físico-Valorado y CPP | CU07, CU14 |
| **M06** | Postventa en Dos Tiempos, Pasarela Stripe Sandbox y Ofertas Dinámicas | CU13, CU18, CU19 |
| **M07** | Aprovisionamiento y Compras Corporativas desde Web Desktop | CU20 |
| **M08** | Inteligencia Analítica de Negocio: Reportes IA Generativa y ML | CU15, CU16, CU17 |

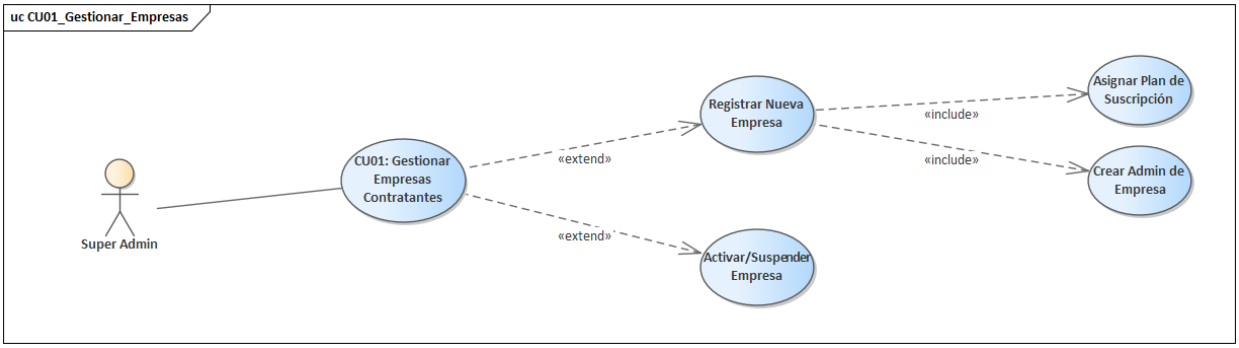
| | | |
|---|---|---|
| **TOTAL** | **8 Módulos Consolidados de Segundo Parcial** | **CU01 al CU20** |

### 2.5.2. Diagrama General de Casos de Uso (PlantUML)

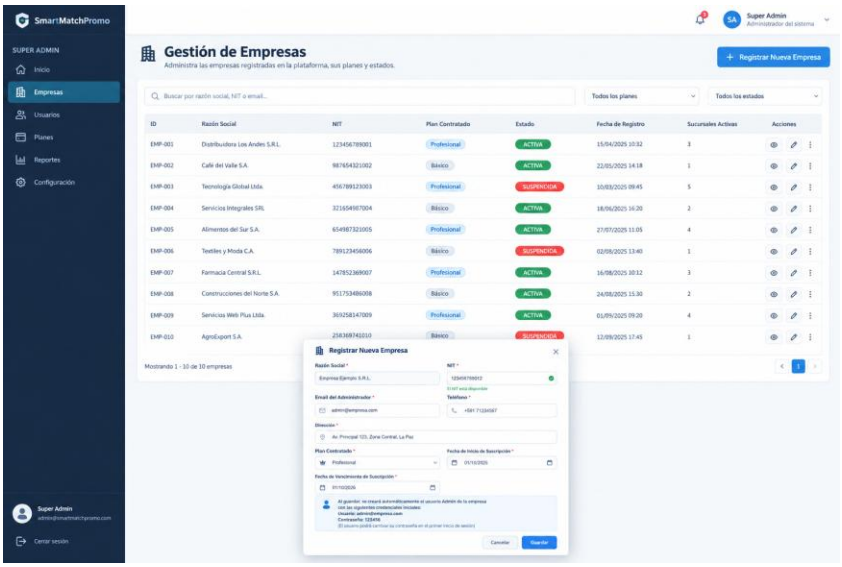
## 2.6. Especificación de Casos de Uso

### CU01: Gestionar Empresas (Tenants)

#### a) Diseño del Caso de Uso (PlantUML)




#### b) Prototipo de Interfaz de Usuario



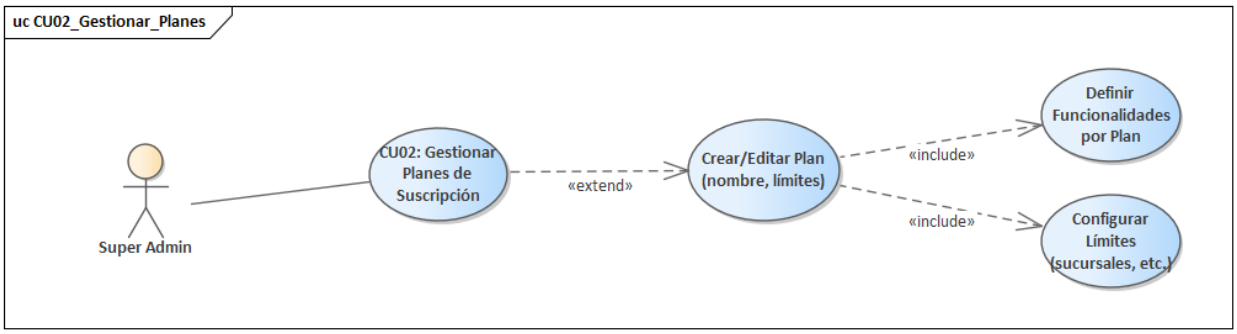

#### c) Tabla Detalle del Caso de Uso

| Atributo | Detalle |
|---|---|
| **Identificador** | CU01 |
| **Nombre** | Gestionar Empresas Contratantes (Tenants) |
| **Autor** | Equipo de desarrollo |

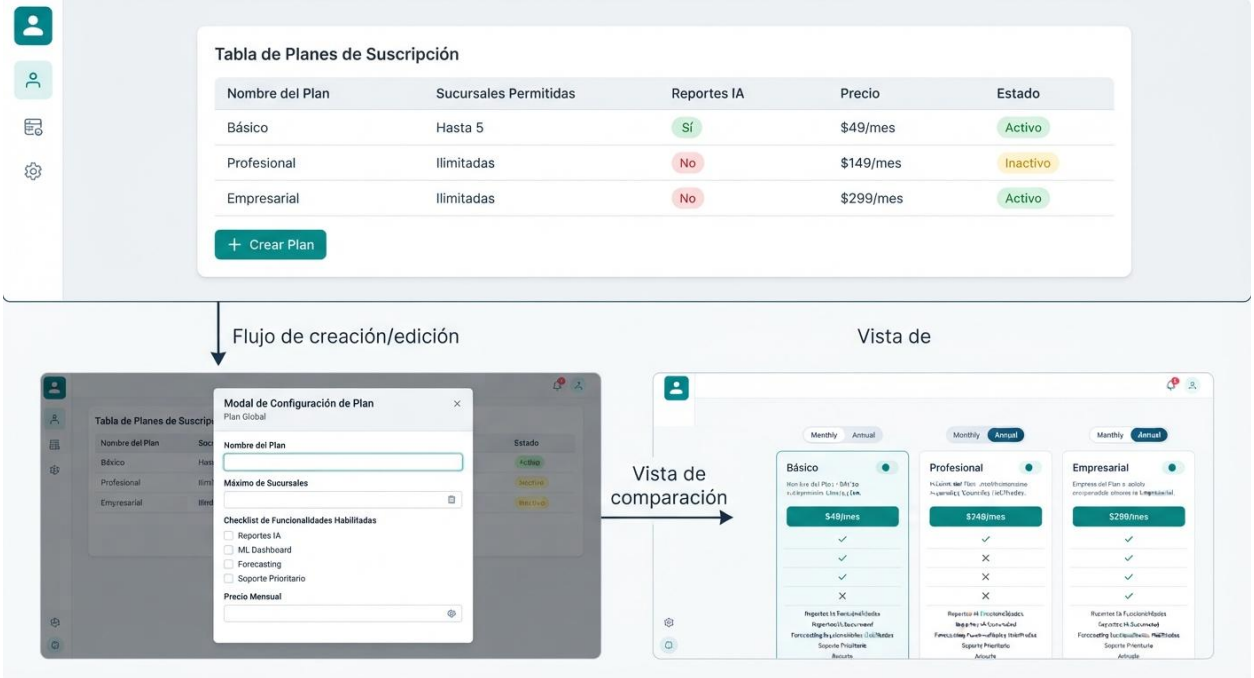
| | |
|---|---|
| **Fecha** | 2026-10-24 |
| **Actores** | Super Admin (Administrador de Plataforma) |
| **Descripción** | Permite al Super Admin registrar, consultar, activar y suspender empresas que contratan el servicio SaaS de FashionStore bajo arquitectura multitenant. |
| **Precondiciones** | El Super Admin debe estar autenticado con rol SUPER_ADMIN. |
| **Flujo Normal** | 1. El Super Admin accede al panel de "Gestión de Empresas". 2. El sistema despliega la lista paginada de empresas con planes y estados. 3. [Punto de Extensión: Alta de Tenant] El Super Admin presiona "Registrar Nueva Empresa", ejecutando <<extend>> Registrar Nueva Empresa. 4. El sistema despliega el formulario modal de alta. 5. El Super Admin ingresa Razón Social, NIT y datos de contacto. 6. El sistema invoca <<include>> Asignar Plan de Suscripción para parametrizar las cuotas de sucursales y módulos habilitados. 7. El sistema valida unicidad de NIT y formato de datos. 8. El sistema persiste el registro de la empresa (tenant) en PostgreSQL. 9. El sistema invoca <<include>> Crear Admin de Empresa, aprovisionando credenciales temporales y enviando correo institucional. 10. El sistema emite confirmación de registro exitoso. |
| **Flujo Alternativo** | **3a. [Punto de Extensión: Gestión de Estado]** El Super Admin selecciona una empresa existente y ejecuta <<extend>> Activar/Suspender Empresa para pausar operaciones por mora o reactivar el servicio. **7a. NIT duplicado dentro de la plataforma:** El sistema notifica "El NIT ya se encuentra registrado" y cancela la operación. **7b. Email administrativo duplicado:** El sistema notifica "El correo electrónico ya está asociado a otra cuenta". |
| **Postcondiciones** | La empresa queda registrada con su suscripción activa y el Admin de Empresa puede iniciar sesión. |

## CU02: Gestionar Planes de Suscripción

### a) Diseño del Caso de Uso (PlantUML)




### b) Prototipo de Interfaz de Usuario



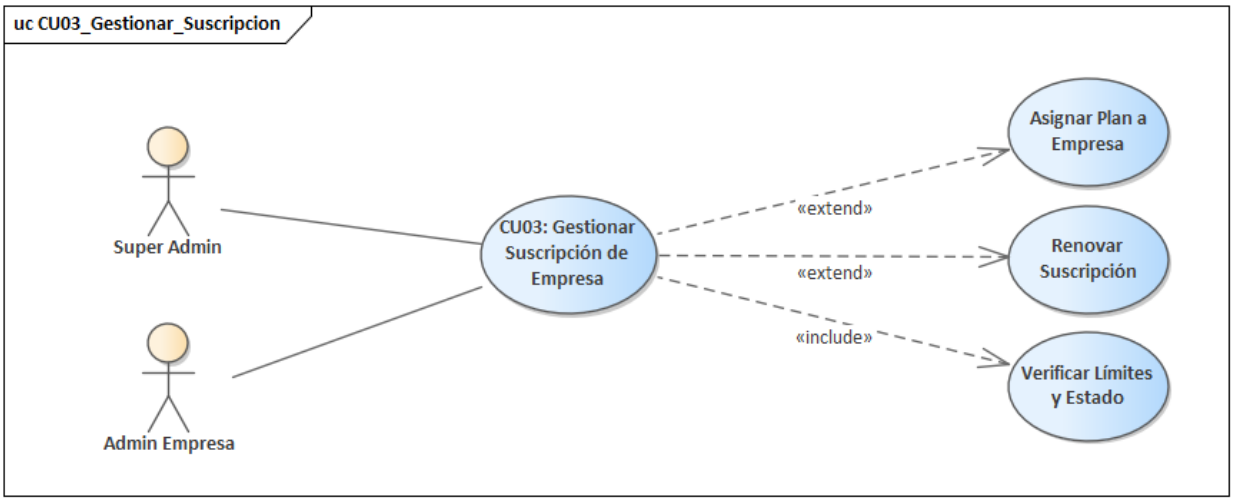

**c) Tabla Detalle del Caso de Uso**

| Atributo | Detalle |
|---|---|
| **Identificador** | CU02 |
| **Nombre** | Gestionar Planes de Suscripción |
| **Autor** | Equipo de desarrollo |
| **Fecha** | 2026-10-24 |
| **Actores** | Super Admin |
| **Descripción** | Permite crear, modificar y parametrizar los planes de suscripción SaaS (Básico, Profesional) con sus topes de recursos y características funcionales. |
| **Precondiciones** | Super Admin autenticado con rol SUPER_ADMIN. |
| **Casos de Uso Incluidos (<>)** | - **Definir Funcionalidades por Plan:** Configura checklist de módulos habilitados (IA Generativa, ML, etc.). - **Configurar Límites (sucursales, etc.):** Establece topes cuantitativos de sucursales y usuarios. |
| **Casos de Uso Extendidos (<>)** | - **Crear/Editar Plan (nombre, límites):** Extensión disparada al pulsar '+ Crear Plan' o editar uno existente (Punto de Extensión: Parametrización de Plan). |
| **Flujo Normal** | 1. El Super Admin accede al módulo "Planes de Suscripción". 2. El sistema muestra la matriz comparativa de planes existentes. 3. [Punto de Extensión: Parametrización de Plan] El Super Admin pulsa "Crear Plan" o selecciona editar, ejecutando <<extend>> Crear/Editar Plan (nombre, límites). |

| | |
|---|---|
| | 4. El Super Admin ingresa denominación y costo mensual.<br>5. El sistema invoca <<include>> Definir Funcionalidades por Plan para activar/desactivar módulos estratégicos.<br>6. El sistema invoca <<include>> Configurar Límites (sucursales, etc.) para fijar el número máximo de sucursales permitidas.<br>7. El sistema valida y persiste la configuración en base de datos. |
| **Flujo Alternativo** | **4a. Denominación de plan duplicada:** El sistema rechaza el nombre y solicita un identificador comercial unívoco. |
| **Postcondiciones** | El plan de suscripción queda disponible para asignación inmediata a empresas. |

## CU03: Gestionar Suscripción de Empresa

### a) Diseño del Caso de Uso (PlantUML)




### b) Prototipo de Interfaz de Usuario



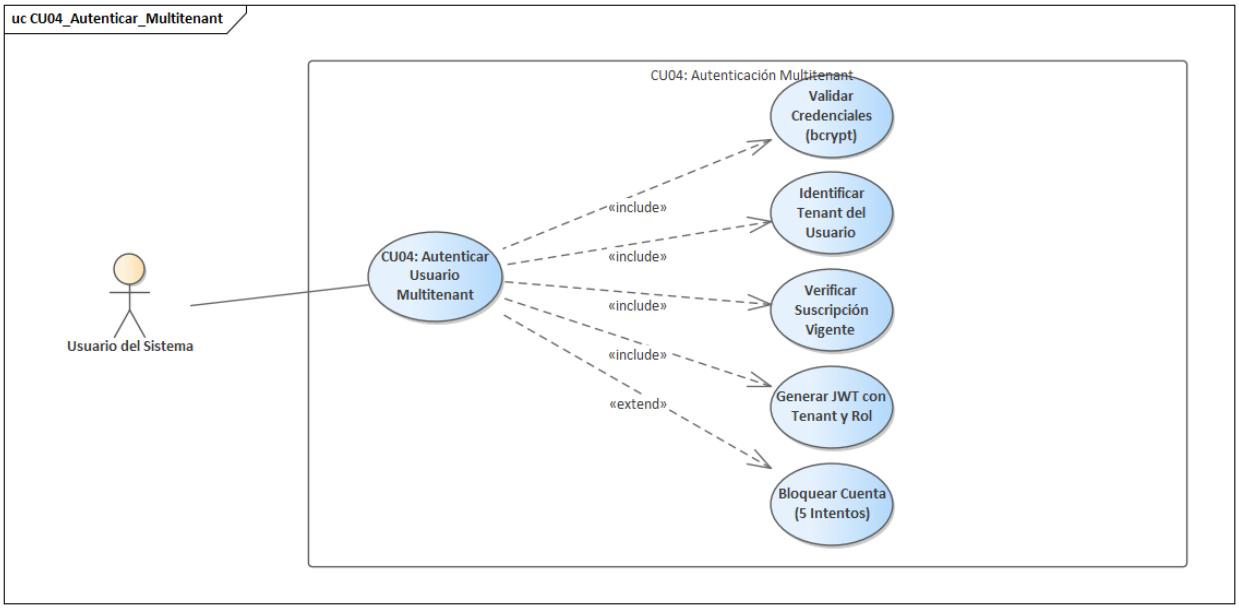

**c) Tabla Detalle del Caso de Uso**

| Atributo | Detalle |
|---|---|
| **Identificador** | CU03 |
| **Nombre** | Gestionar Suscripción de Empresa |
| **Autor** | Equipo de desarrollo |
| **Fecha** | 2026-10-24 |
| **Actores** | Super Admin, Admin Empresa |
| **Descripción** | Administra el ciclo de vida de la suscripción de un tenant: asignación de plan, consulta de consumo vs. límites y renovación temporal. |
| **Precondiciones** | Empresa registrada previamente en la plataforma (CU01). |
| **Casos de Uso Incluidos (<>)** | - **Verificar Límites y Estado:** Compara el número de sucursales activas contra la cuota contratada y verifica la fecha de vencimiento. |
| **Casos de Uso Extendidos (<>)** | - **Asignar Plan a Empresa:** Extensión disparada al contratar o modificar categoría de plan (Punto de Extensión: Upgrade/Downgrade de Plan). - **Renovar Suscripción:** Extensión disparada para extender vigencia contractual (Punto de Extensión: Renovación Temporal). |
| **Flujo Normal** | 1. El usuario accede al panel de "Estado de Suscripción". 2. El sistema invoca <<include>> Verificar Límites y Estado para contrastar el consumo de tiendas frente a los límites permitidos. 3. El sistema muestra barra de vigencia, días restantes y estado (ACTIVA, VENCIDA, SUSPENDIDA). 4. [Punto de Extensión: Upgrade/Downgrade de Plan] El Super Admin o Admin Empresa ejecuta <<extend>> Asignar Plan a Empresa para migrar |

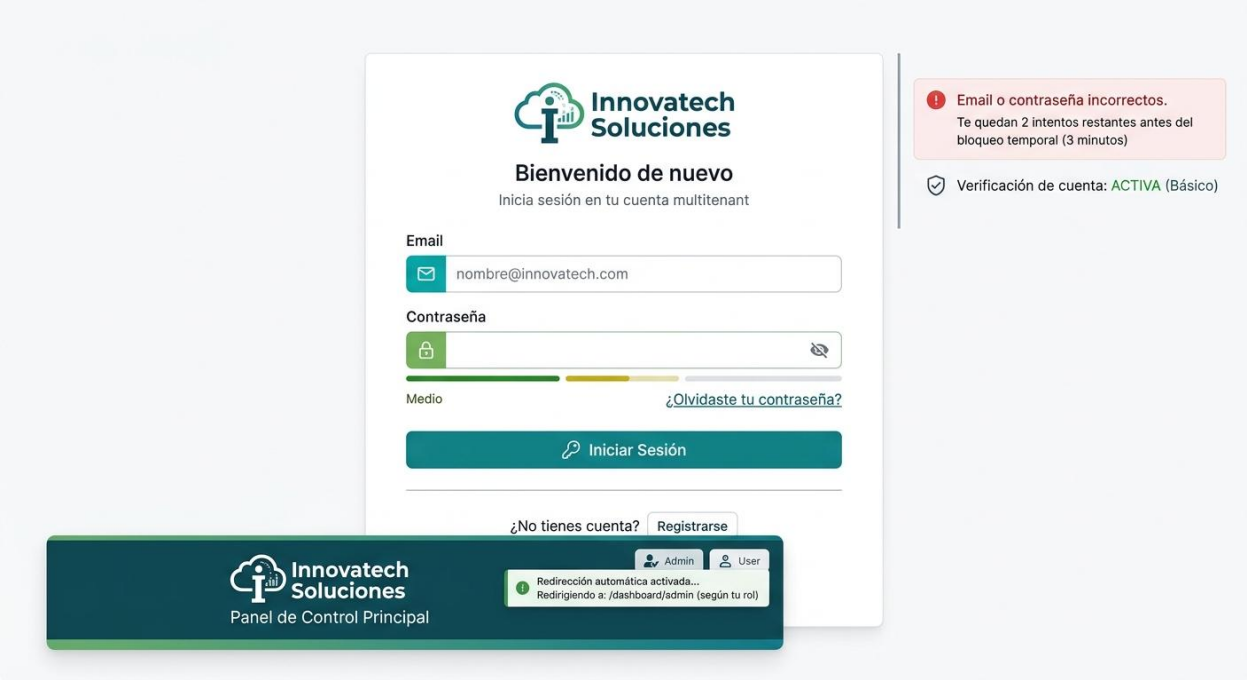
| | |
|---|---|
| | de Básico a Profesional.<br>5. [Punto de Extensión: Renovación Temporal] El Super Admin ejecuta <<extend>> Renovar Suscripción, extendiendo la fecha límite a 30 o 365 días adicionales.<br>6. El sistema actualiza cuotas y emite comprobante de servicio. |
| **Flujo Alternativo** | **4a. Downgrade con sucursales excedidas:** Si la empresa tiene 5 tiendas y elige un plan de 2, el sistema advierte que debe cerrar o inactivar 3 sucursales antes de aplicar el cambio. |
| **Postcondiciones** | Suscripción actualizada con nuevas fechas y cuotas operativas sincronizadas en el middleware multitenant. |

## CU04: Autenticar Usuario Multitenant

### a) Diseño del Caso de Uso (PlantUML)




### b) Prototipo de Interfaz de Usuario



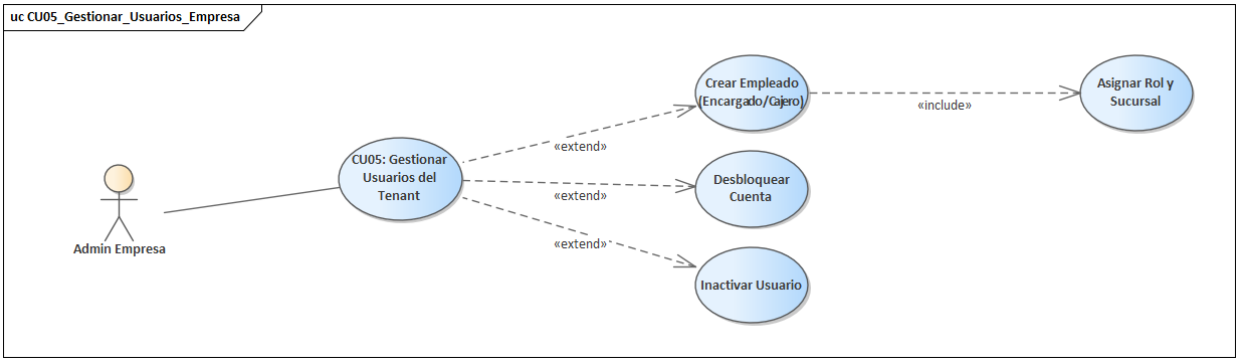

**c) Tabla Detalle del Caso de Uso**

| **Atributo** | **Detalle** |
|---|---|
| **Identificador** | CU04 |
| **Nombre** | Autenticar Usuario Multitenant |
| **Autor** | Equipo de desarrollo |
| **Fecha** | 2026-10-24 |
| **Actores** | Todos los actores humanos (Super Admin, Admin Empresa, Encargado de Sucursal, Cajero, Cliente) |
| **Descripción** | Inicio de sesión unificado que valida credenciales seguras, resuelve el tenant correspondiente, verifica vigencia de suscripción y emite token JWT con claims restringidos. |
| **Precondiciones** | Cuenta de usuario registrada y activa en el sistema. |
| **Flujo Normal** | 1. El usuario introduce email y contraseña en el portal de acceso.<br>2. El sistema invoca <<include>> Validar Credenciales (bcrypt).<br>3. El sistema invoca <<include>> Identificar Tenant del Usuario, resolviendo contexto empresarial.<br>4. El sistema invoca <<include>> Verificar Suscripción Vigente, confirmando que el tenant no esté suspendido.<br>5. El sistema invoca <<include>> Generar JWT con Tenant y Rol, emitiendo token con claims {user_id, tenant_id, rol, sucursal_id}.<br>6. El sistema restablece el contador de intentos fallidos a 0 y registra acceso en auditoría.<br>7. Redirige a la vista especializada según el rol. |

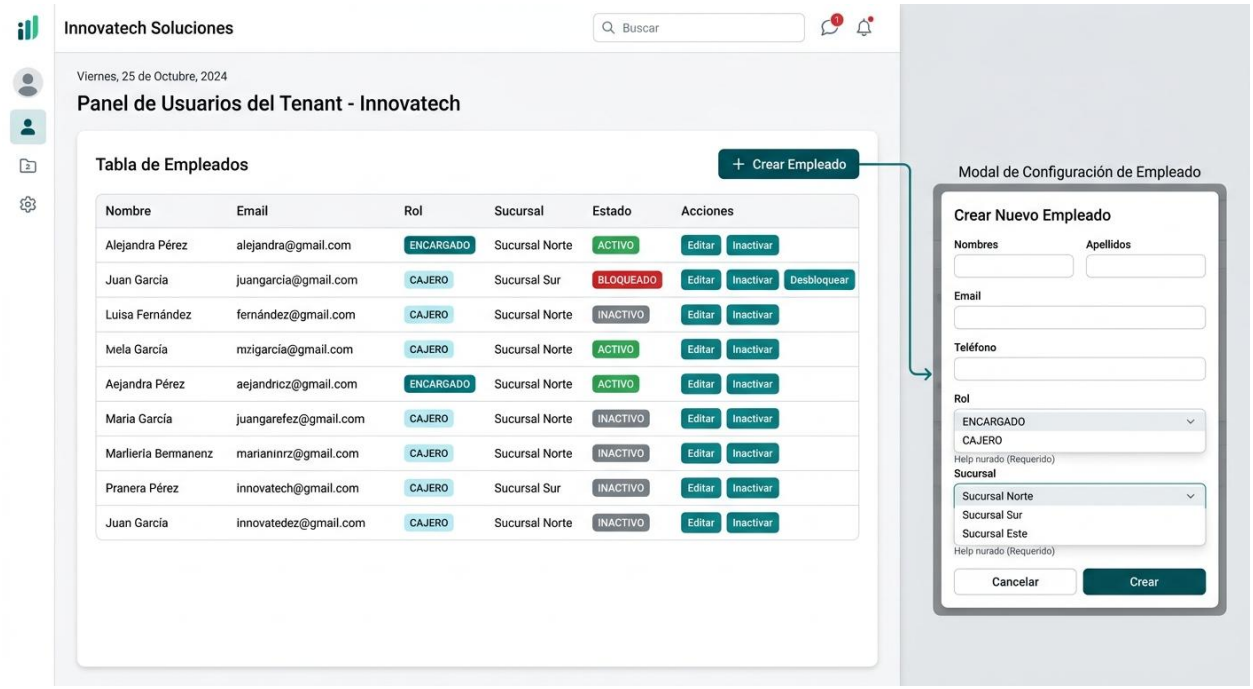
| | |
|---|---|
| **Flujo Alternativo** | **2a. Contraseña incorrecta:** El sistema incrementa el contador intentos_fallidos.<br>**2b. [Punto de Extensión: Detección de Intrusión]** Al acumular 5 intentos fallidos consecutivos, el sistema ejecuta <<extend>> Bloquear Cuenta (5 Intentos), inactivando el usuario y notificando al administrador.<br>**4a. Empresa con suscripción vencida o suspendida:** El sistema bloquea el acceso informando: "Suscripción de empresa vencida. Contacte al administrador de plataforma". |
| **Postcondiciones** | Sesión activa con token JWT que condiciona todas las consultas posteriores con aislamiento estricto por tenant_id. |

## CU05: Gestionar Usuarios y Roles por Empresa

### a) Diseño del Caso de Uso (PlantUML)




### b) Prototipo de Interfaz de Usuario

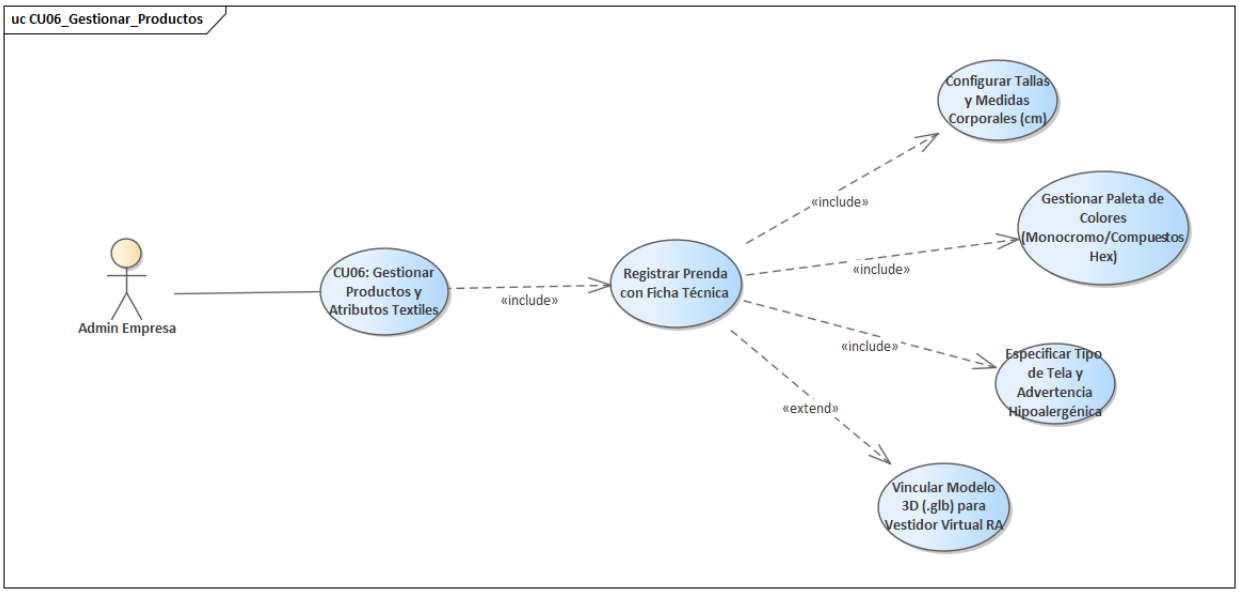



### c) Tabla Detalle del Caso de Uso

| Atributo | Detalle |
|---|---|
| **Identificador** | CU05 |
| **Nombre** | Gestionar Usuarios y Roles por Empresa |
| **Autor** | Equipo de desarrollo |
| **Fecha** | 2026-10-24 |
| **Actores** | Admin Empresa |
| **Descripción** | Permite al administrador del tenant gestionar el personal de su empresa: creación de encargados y cajeros, asignación de sucursales físicas y control de bloqueos. |
| **Precondiciones** | Admin Empresa autenticado con suscripción vigente. |
| **Flujo Normal** | 1. El Admin Empresa accede a "Usuarios y Roles".<br>2. El sistema despliega la lista de empleados filtrados estrictamente por el tenant_id activo.<br>3. [Punto de Extensión: Alta de Personal] El administrador presiona "+ Crear Empleado", ejecutando <<extend>> Crear Empleado (Encargado/Cajero).<br>4. Ingresa nombres, apellidos, teléfono y correo electrónico.<br>5. El sistema invoca <<include>> Asignar Rol y Sucursal para delimitar si operará como Encargado de Sucursal o Cajero, asignándole su tienda física.<br>6. El sistema valida unicidad de correo dentro del tenant y persiste el usuario en base de datos. |
| **Flujo Alternativo** | **3a. [Punto de Extensión: Restablecimiento de Acceso]** El administrador selecciona un usuario con estado BLOQUEADO y ejecuta <<extend>> Desbloquear Cuenta, reiniciando los intentos fallidos.<br>**3b. [Punto de Extensión: Baja Operativa]** El administrador ejecuta <<extend>> Inactivar Usuario para revocar inmediatamente el acceso al sistema.<br>**5a. Email duplicado dentro del tenant:** El sistema rechaza el registro y solicita un correo corporativo diferente. |
| **Postcondiciones** | Personal asignado y configurado con acceso limitado exclusivamente a los datos de su empresa y sucursal. |

**CU06: Gestionar Productos y Atributos Textiles Avanzados**

**a) Diseño del Caso de Uso (PlantUML)**




**b) Prototipo de Interfaz de Usuario**



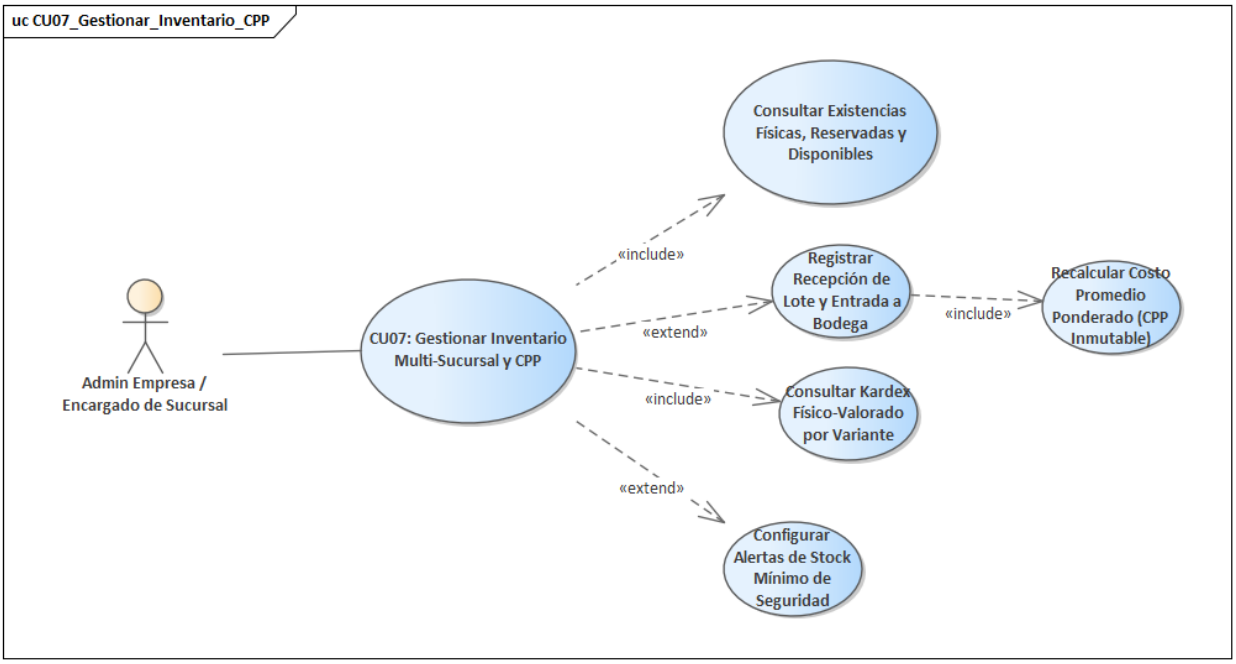

**c) Tabla Detalle del Caso de Uso**

| **Atributo** | **Detalle** |
|---|---|
| **Identificador** | CU06 |
| **Nombre** | Gestionar Productos y Atributos Textiles Avanzados |
| **Autor** | Equipo de desarrollo |
| **Fecha** | 2026-10-24 |
| **Actores** | Administrador de Empresa |

| | |
|---|---|
| **Descripción** | Catalogación integral de prendas de vestir masculinas, especificando ficha técnica, variantes con medidas corporales en centímetros, paletas de colores compuestas (bicolores Hex), precio minorista y mayorista (≥3≥3 unidades), y modelos 3D para probador virtual RA. |
| **Precondiciones** | Admin Empresa autenticado con empresa activa. |
| **Flujo Normal** | 1. El Admin Empresa accede al catálogo y pulsa "Nueva Prenda", invocando <<include>> Registrar Prenda con Ficha Técnica.<br>2. Ingresa SKU, denominación comercial, temporada y corte (Slim Fit, Regular Fit).<br>3. El sistema invoca <<include>> Especificar Tipo de Tela y Advertencia Hipoalergénica, seleccionando tejido textil y activando advertencia para pieles atópicas.<br>4. El sistema invoca <<include>> Configurar Tallas y Medidas Corporales (cm), registrando las medidas corporales en cm para cada talla (XS a XXL).<br>5. El sistema invoca <<include>> Gestionar Paleta de Colores (Monocromo/Compuestos Hex), configurando esquemas monocromos o combinados (ej. Blanco #FFFFFF / Negro #000000).<br>6. [Punto de Extensión: Probador Virtual RA] Si la prenda cuenta con modelo 3D, el usuario ejecuta <<extend>> Vincular Modelo 3D (.glb) para Vestidor Virtual RA, cargando el archivo .glb correspondiente.<br>7. Define la tarifa minorista base y la regla de precio al por mayor (con descuento automático para compras ≥3≥3 unidades).<br>8. El sistema valida unicidad de SKU en el tenant y persiste la ficha técnica. |
| **Flujo Alternativo** | **2a. SKU duplicado dentro de la empresa:** El sistema rechaza el código y solicita un identificador unívoco.<br>**4a. Medidas corporales fuera de rango fisiológico:** El sistema advierte inconsistencia y bloquea el guardado hasta su corrección. |
| **Postcondiciones** | Prenda registrada con su matriz completa de atributos, disponible para asignación de stock multi-sucursal. |

**CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP)**

**a) Diseño del Caso de Uso (PlantUML)**




## b) Prototipo de Interfaz de Usuario




## c) Tabla Detalle del Caso de Uso

| Atributo | Detalle |
|---|---|
| **Identificador** | CU07 |
| **Nombre** | Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP) |
| **Autor** | Equipo de desarrollo |

| | |
|---|---|
| **Fecha** | 2026-10-24 |
| **Actores** | Administrador de Empresa, Encargado de Sucursal |
| **Descripción** | Control de existencias físicas, reservadas y disponibles por sucursal, con recálculo matemático inmutable de CPP tras cada entrada de mercadería, generación de Kardex físico-valorado y alertas de reposición urgente. |
| **Precondiciones** | Usuario autenticado con rol ADMIN_EMPRESA o ENCARGADO_SUCURSAL; sucursales y productos registrados. |
| **Flujo Normal** | 1. El usuario accede a "Inventarios y Kardex", seleccionando la sucursal asignada.<br>2. El sistema invoca <<include>> Consultar Existencias Físicas, Reservadas y Disponibles, desplegando la matriz de variantes.<br>3. [Punto de Extensión: Entrada de Lote] El usuario pulsa "Recepción de Lote", ejecutando <<extend>> Registrar Recepción de Lote y Entrada a Bodega.<br>4. Ingresa variante, cantidad recibida y costo unitario según factura de compra.<br>5. El sistema ejecuta de forma obligatoria <<include>> Recalcular Costo Promedio Ponderado (CPP Inmutable) aplicando: $$CPP_{\text{nuevo}} = \frac{(S_{\text{ant}} \times CPP_{\text{ant}}) + (Q_{\text{ing}} \times C_{\text{ing}})}{S_{\text{ant}} + Q_{\text{ing}}}$$.<br>6. Actualiza ultimo_costo_unitario y suma las unidades al stock físico.<br>7. El sistema invoca <<include>> Consultar Kardex Físico-Valorado por Variante, asentando el movimiento contable inmutable con fecha, hora y responsable.<br>8. Emite comprobante de recepción conforme. |
| **Flujo Alternativo** | **2a. [Punto de Extensión: Alerta de Desabastecimiento]** Si las existencias de una variante caen al umbral mínimo, el sistema ejecuta <<extend>> Configurar Alertas de Stock Mínimo de Seguridad, encendiendo una alerta roja visual de reposición urgente.<br>**4a. Costo o cantidad menor o igual a cero:** El sistema bloquea el registro e informa datos financieros inválidos. |
| **Postcondiciones** | Stock disponible actualizado y CPP recalculado inmutablemente para la determinación del margen de utilidad real. |

# CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

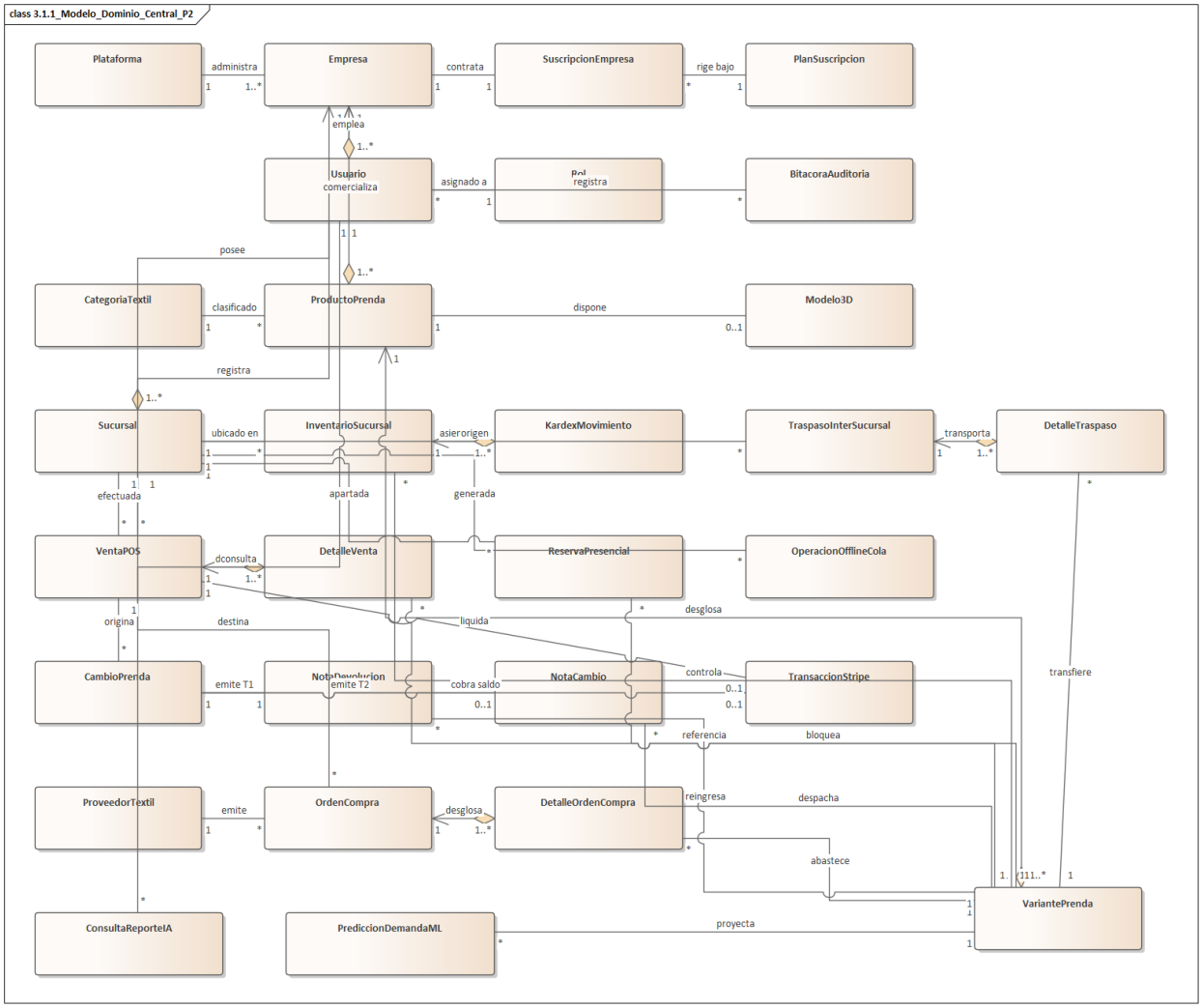
## 3.1. Modelo de Dominio

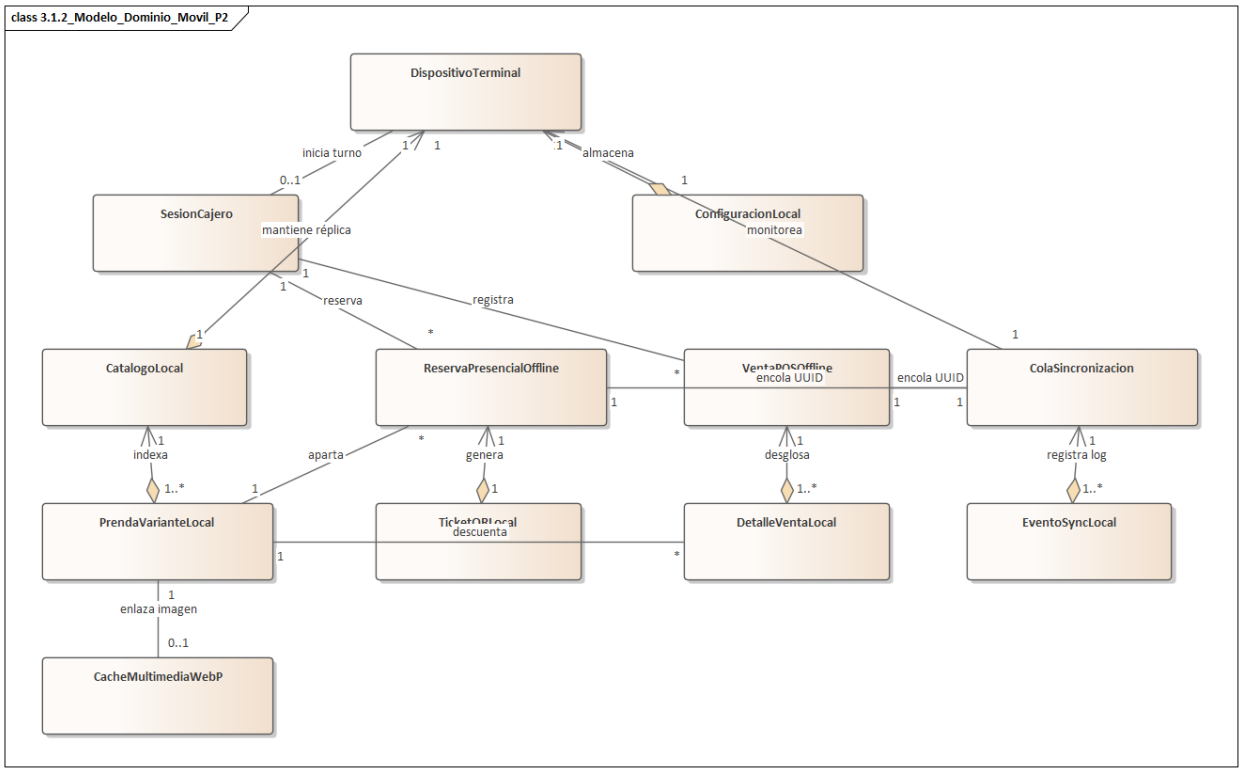
El **Modelo de Dominio** constituye la primera actividad técnica de la metodología **ICONIX** dentro de la fase de *Análisis de Requisitos*. De acuerdo con Rosenberg y Stephens, representa un diagrama de

clases centrado exclusivamente en los objetos del mundo real y entidades del negocio cuyo comportamiento, ciclo de vida o atributos deben persistirse en el sistema.

Para el presente sistema SaaS Multitenant de moda masculina (**FashionStore**), el modelo de dominio se organiza en **7 paquetes conceptuales de negocio**. Cada asociación entre clases ha sido estrictamente tipada con:

1. **Verbos de acción activos** y dirección de lectura (: verbo >), estableciendo la semántica operacional precisa.
2. **Cardinalidades formales completas** (1 a 1..*, 1 a 0..1, * a 1), erradicando ambigüedades estructurales.
3. Cobertura integral de los 6 desafíos del Segundo Parcial: Aislamiento por tenant_id, variantes con medidas anatómicas en centímetros, paletas de colores compuestas (bicolores Hex), recálculo matemático inmutable de CPP en Kardex, operaciones offline con UUID v4 e idempotencia, cambio de prenda en dos tiempos (Nota de Devolución y Nota de Cambio) y auditoría contable.






## 3.2. Diagramas de Robustez (Análisis y Diseño Preliminar ICONIX)

En la metodología **ICONIX** (Doug Rosenberg, Matt Stephens; adaptada según directrices de la cátedra de Sistemas de Información II), el **Diagrama de Robustez** es una técnica intermedia no perteneciente formalmente al estándar UML, pero vital como puente integrador entre el **espacio del problema** (descrito narrativamente en las fichas y especificaciones de Casos de Uso) y el **espacio de la solución** (diseño preliminar de arquitectura, diagramas de clases refinados y diagramas de secuencia).

**Fundamentos Metodológicos y Semántica de la Tríada BCE**

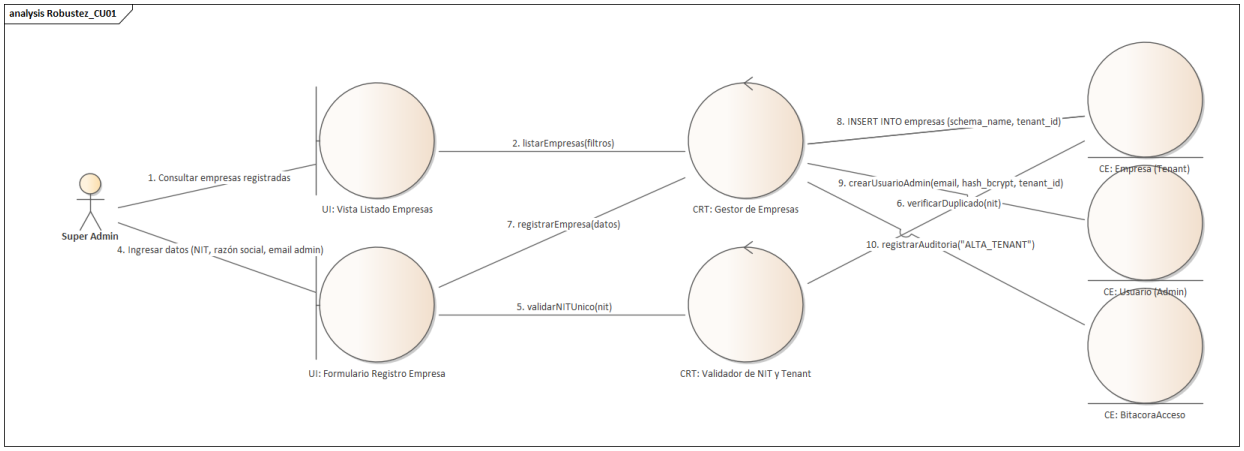
El análisis de robustez se fundamenta en la descomposición de cada caso de uso en tres tipos canónicos de objetos (**Boundary-Control-Entity / BCE**):

1. **Objetos Frontera o Interfaz (Boundary):**
    - Representan los puntos de contacto del sistema con los actores humanos o sistemas externos.
    - Abarcan interfaces gráficas, pantallas de usuario, formularios de captura, modales de confirmación, escáneres ópticos de códigos de barra / QR y endpoints HTTP de la API RESTful.
    - **Regla sintáctica:** Se asocian invariablemente con **Sustantivos de Interfaz** (ej. *Pantalla POS*, *Formulario Registro*, *Modal Reorden*).
2. **Objetos Controlador (Control):**

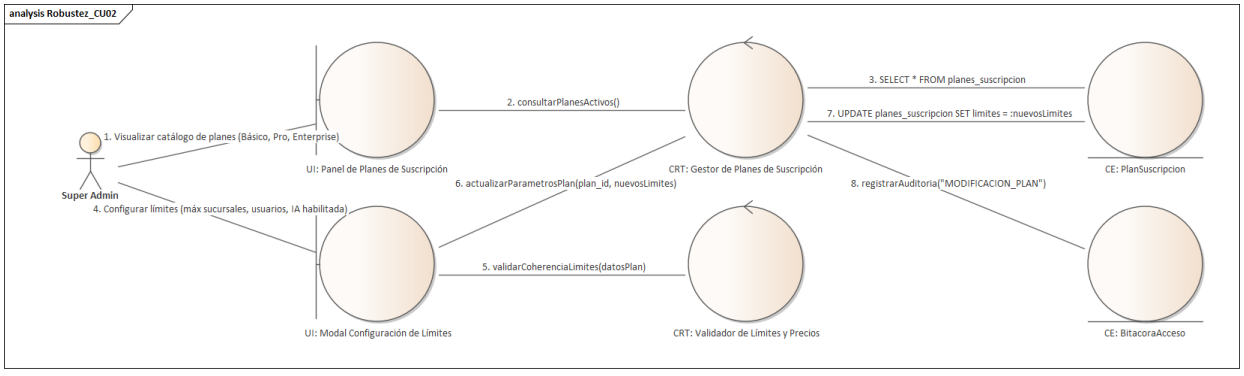
- Encapsulan la lógica de la aplicación, orquestación de casos de uso, transformaciones, validaciones criptográficas, algoritmos de recálculo matemático (como el CPP) y coordinación de transacciones.
- **Regla sintáctica:** Se asocian invariablemente con **Verbos de Acción** (ej. *Gestor de Autenticación*, *Calculador CPP Inmutable*, *Validador de Idempotencia*).

3. **Objetos Entidad (Entity):**
   - Representan los datos persistentes del negocio mapeados directamente desde el Modelo de Dominio hacia las tablas relacionales de PostgreSQL o almacenes locales (IndexedDB / SQLite).
   - **Regla sintáctica:** Se asocian invariablemente con **Sustantivos del Negocio** (ej. *InventarioSucursal*, *VentaPOS*, *KardexMovimiento*).

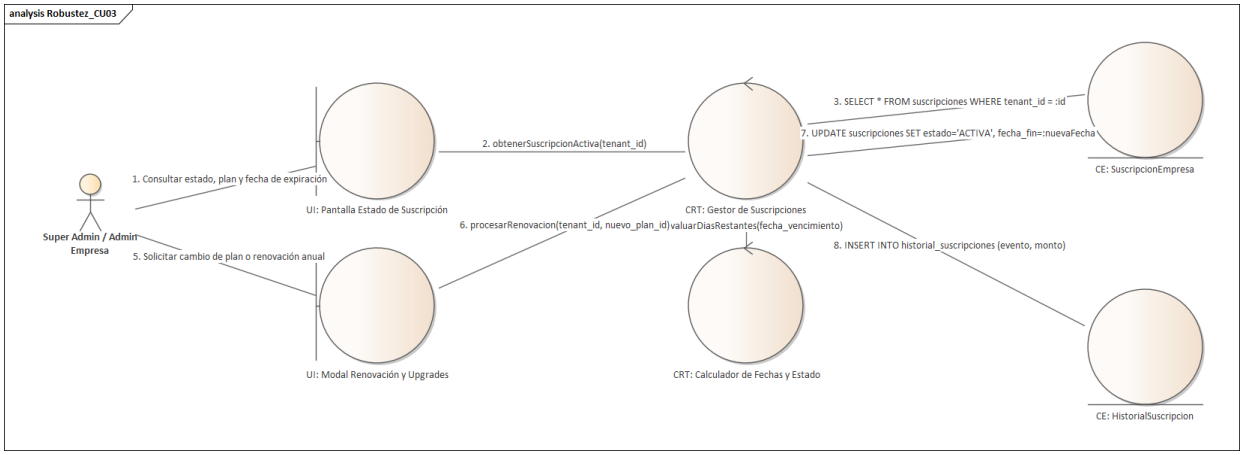
## CU01: Gestionar Empresas (Tenants)



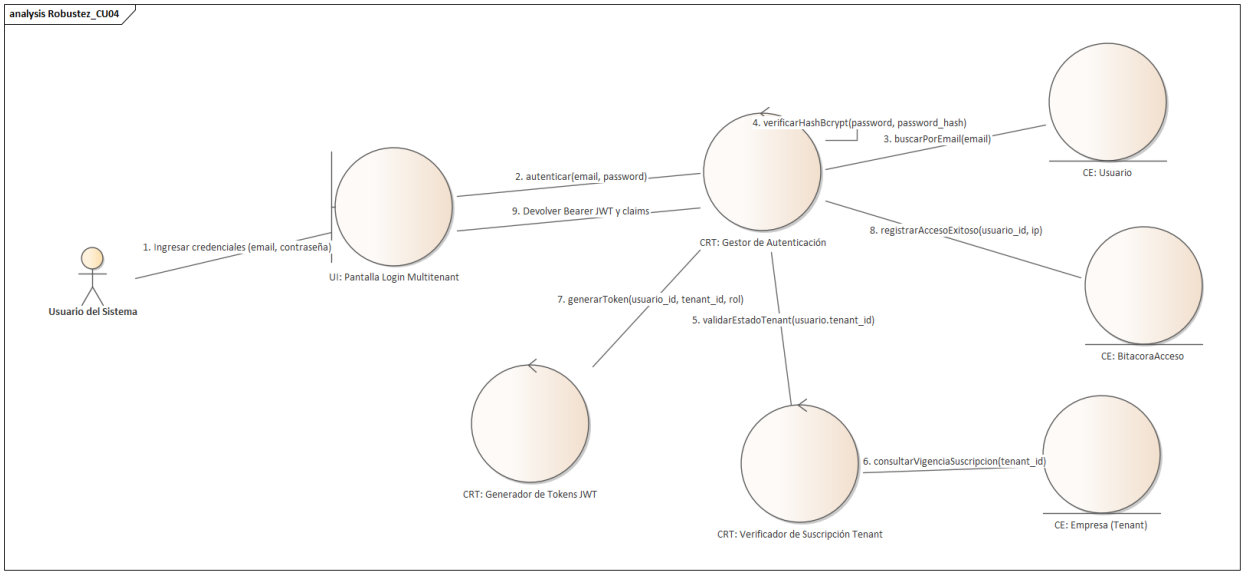

## CU02: Gestionar Planes de Suscripción



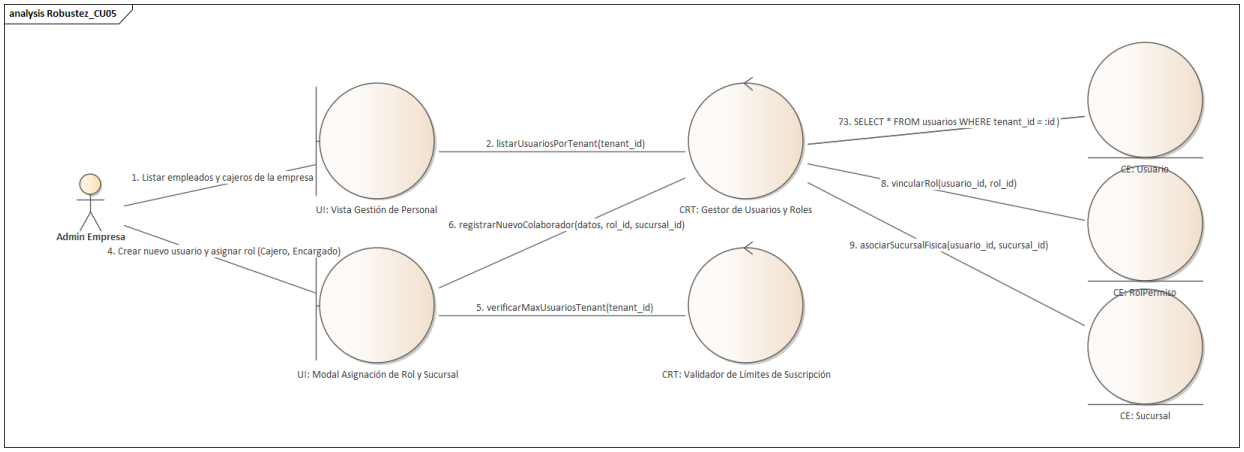
## CU03: Gestionar Suscripción de Empresa



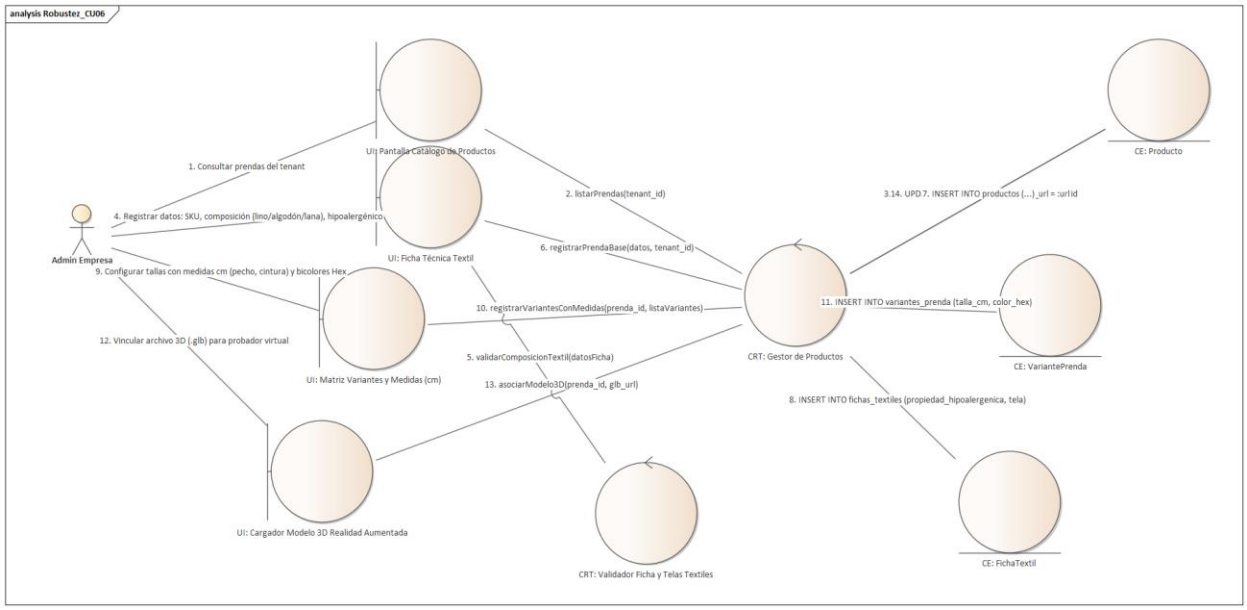

## CU04: Autenticar Usuario Multitenant



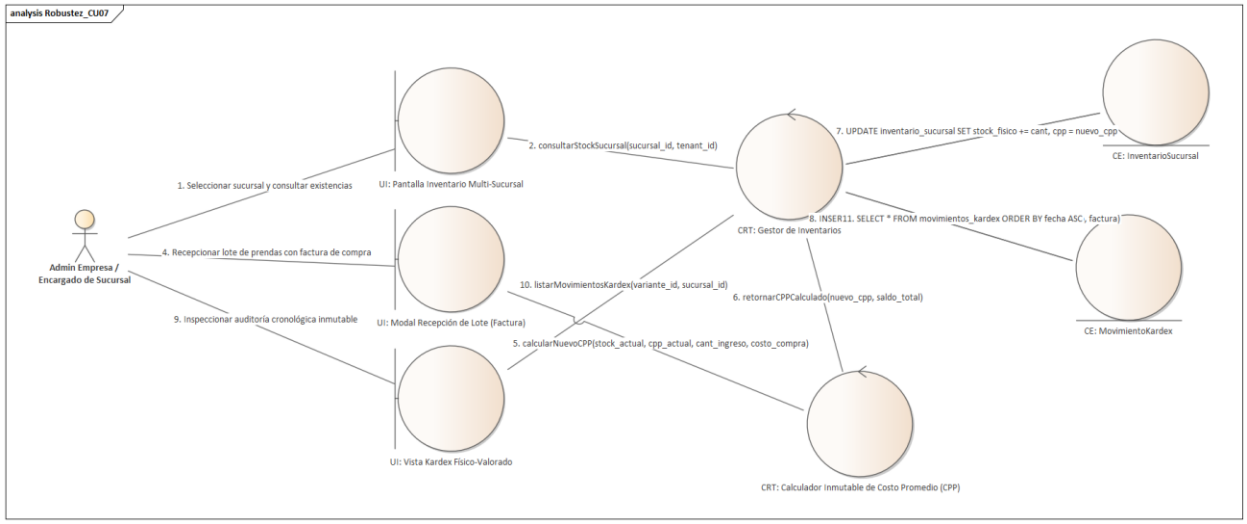

## CU05: Gestionar Usuarios y Roles por Empresa



**CU06: Gestionar Productos y Atributos Textiles Avanzados**




**CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP)**




## 3.3. Diagramas de Secuencia UML

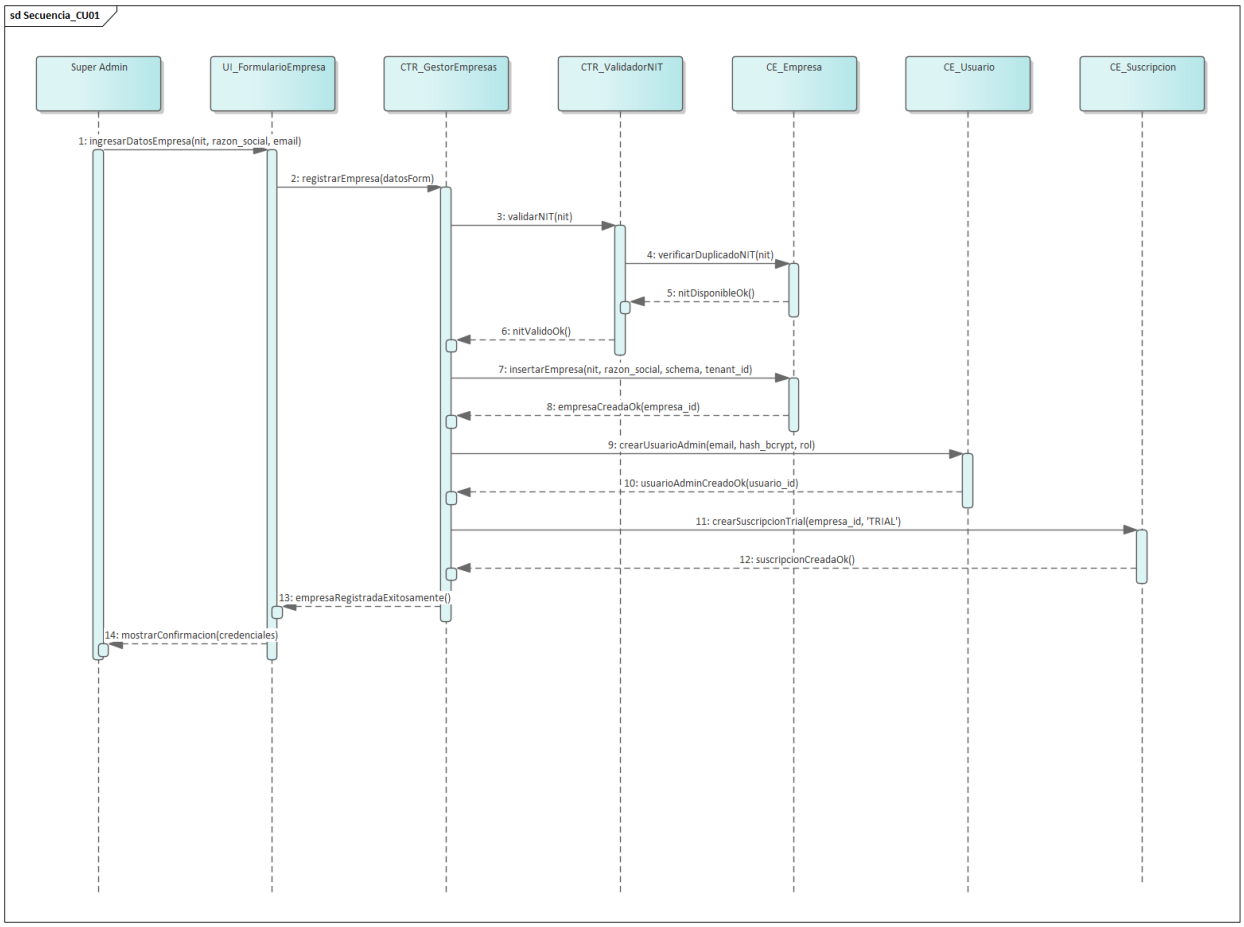
Los **Diagramas de Secuencia** en ICONIX materializan la asignación formal de responsabilidades y métodos entre los objetos identificados durante el análisis de robustez.

**Regla de Oro en Fragmentos Combinados:** *En los diagramas de secuencia donde se utilicen fragmentos combinados de tipo loop (bucles de iteración sobre carritos, colecciones o lotes de inventario),* ***la clase interfaz (Boundary) NUNCA debe estar comprendida dentro del fragmento loop****. El bucle es un proceso netamente interno ejecutado entre la clase de control y las*

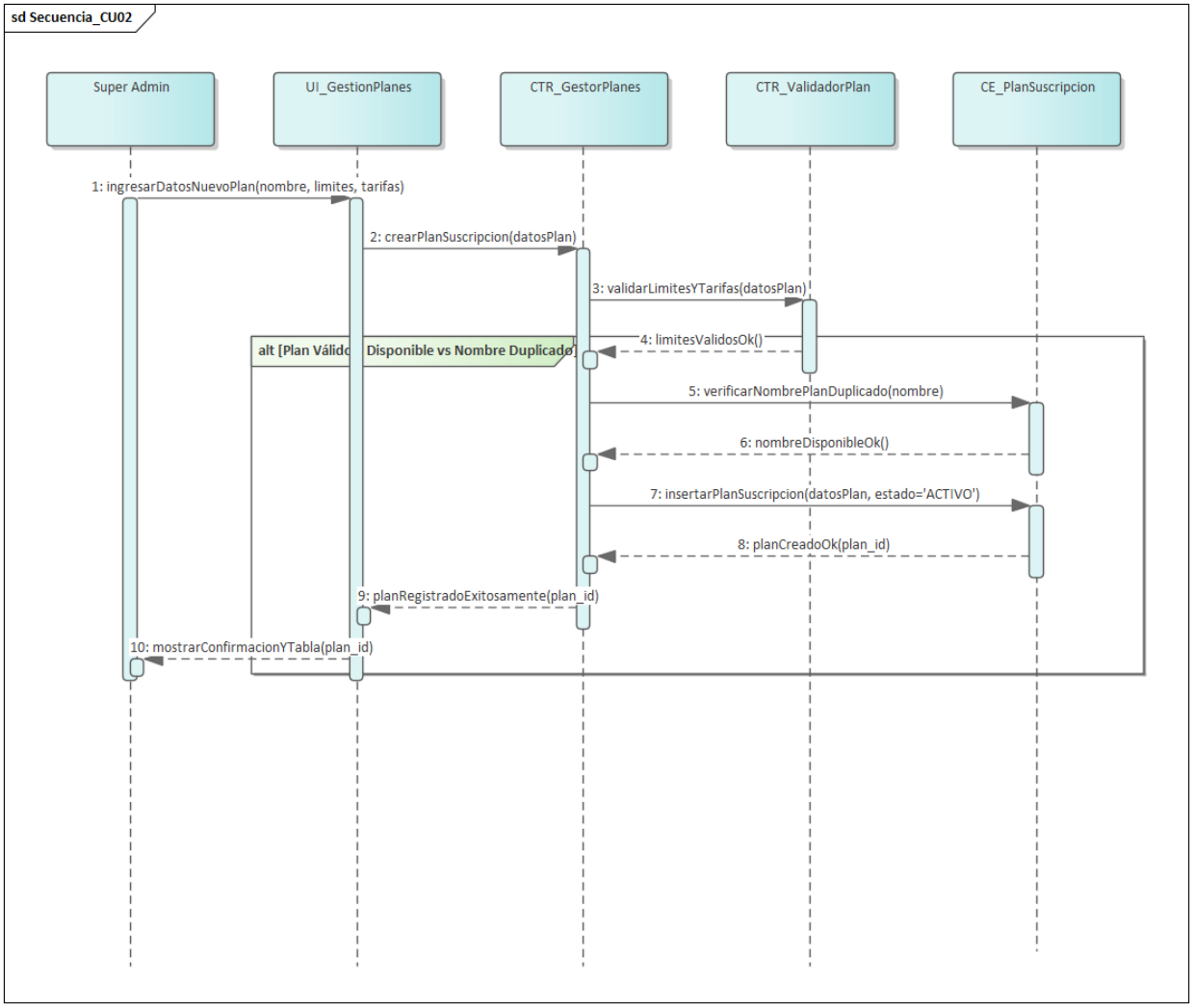
*entidades de datos involucradas. La interfaz de usuario únicamente entrega la colección completa al inicio y recibe el resultado consolidado al finalizar el ciclo.*

A continuación se detallan los diagramas de secuencia de los escenarios transaccionales más representativos de la plataforma:

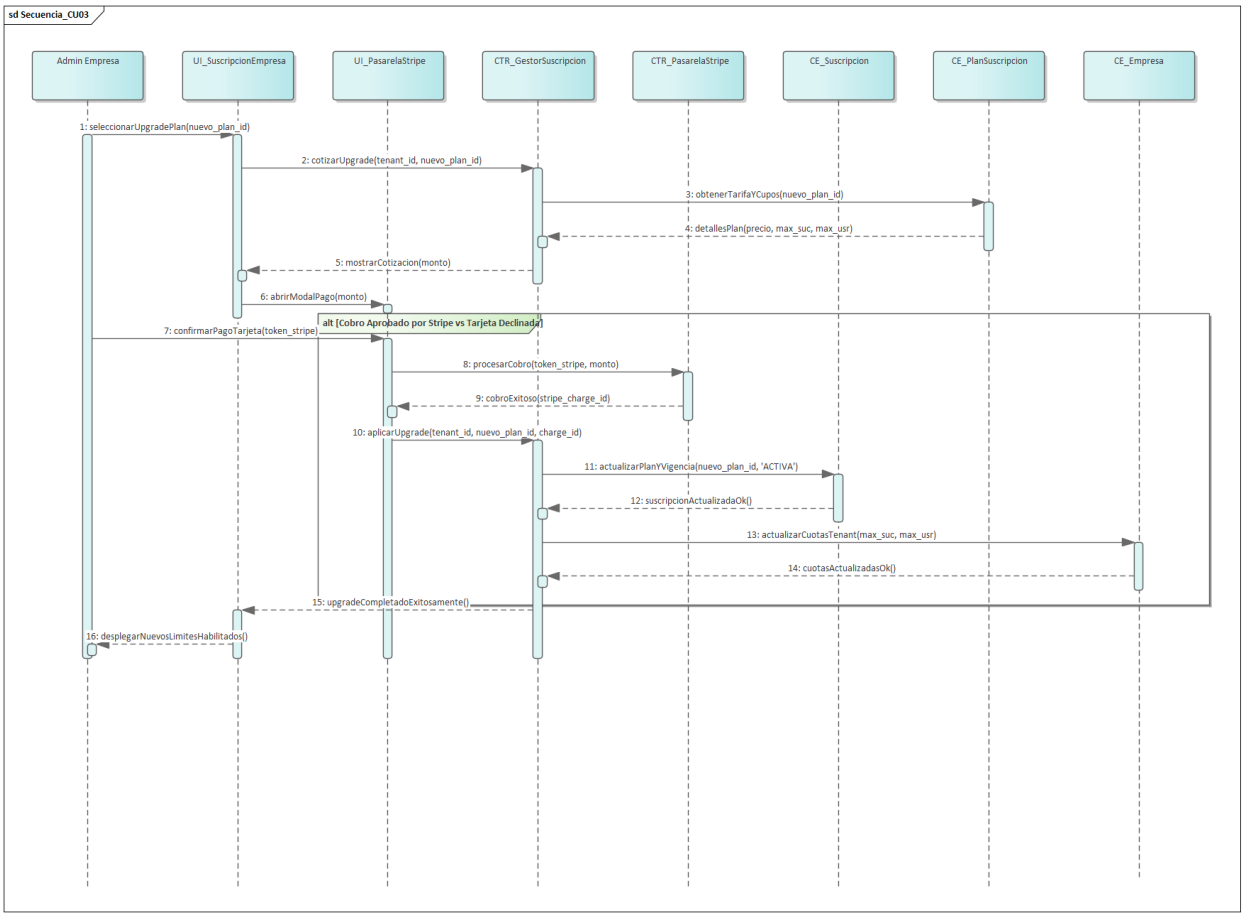
## CU01: Gestionar Empresas (Tenants)



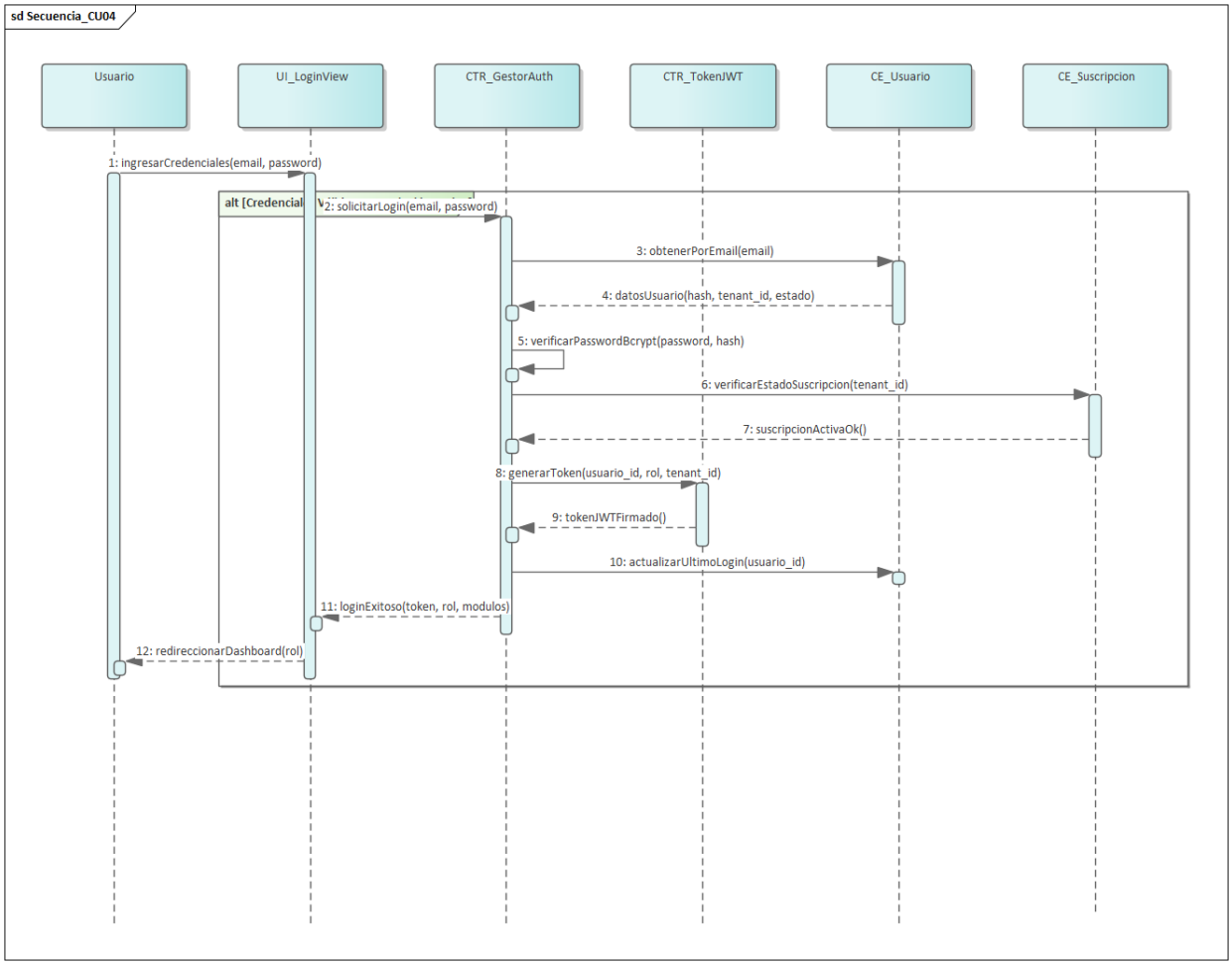
## CU02: Gestionar Planes de Suscripción



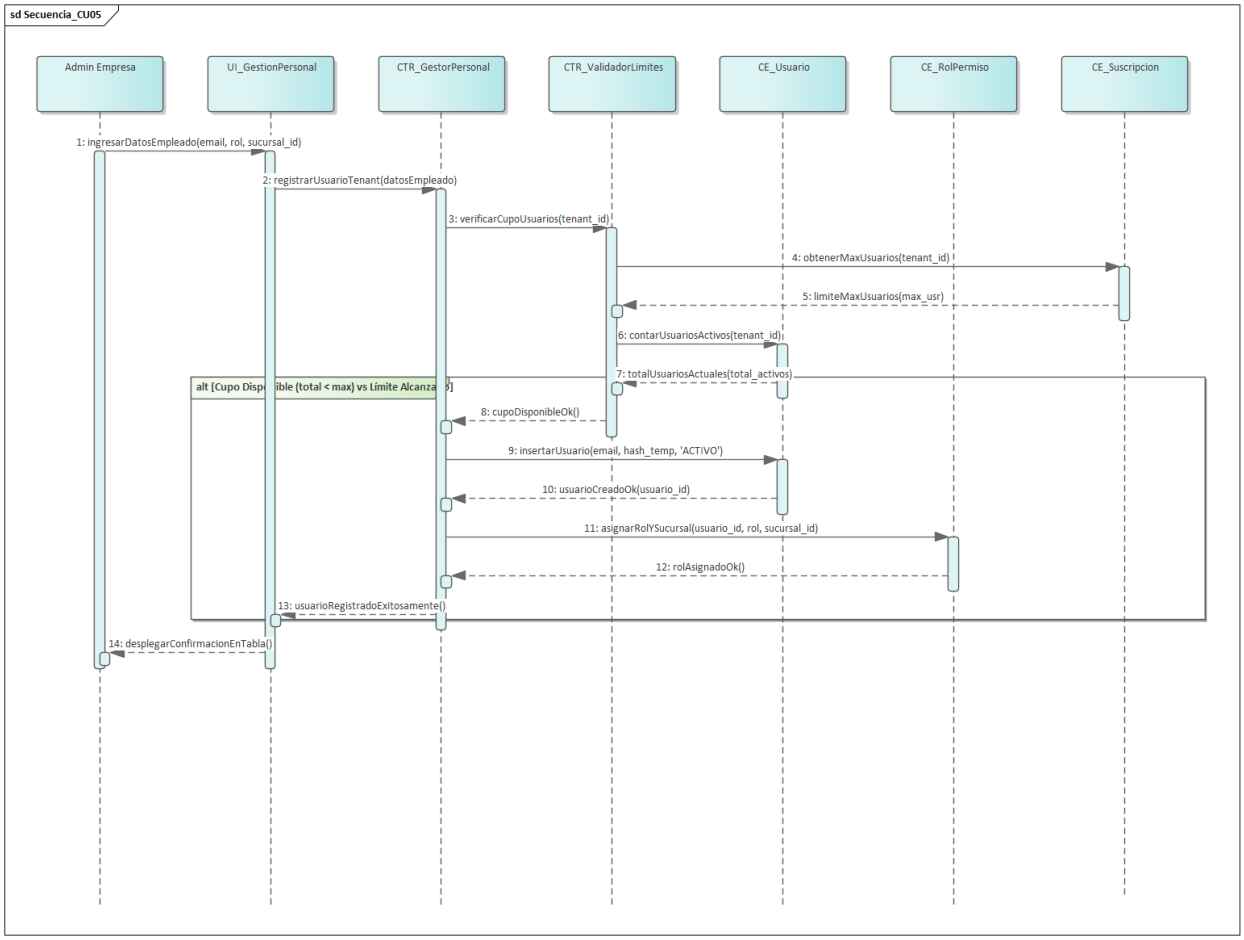
## CU03: Gestionar Suscripción de Empresa



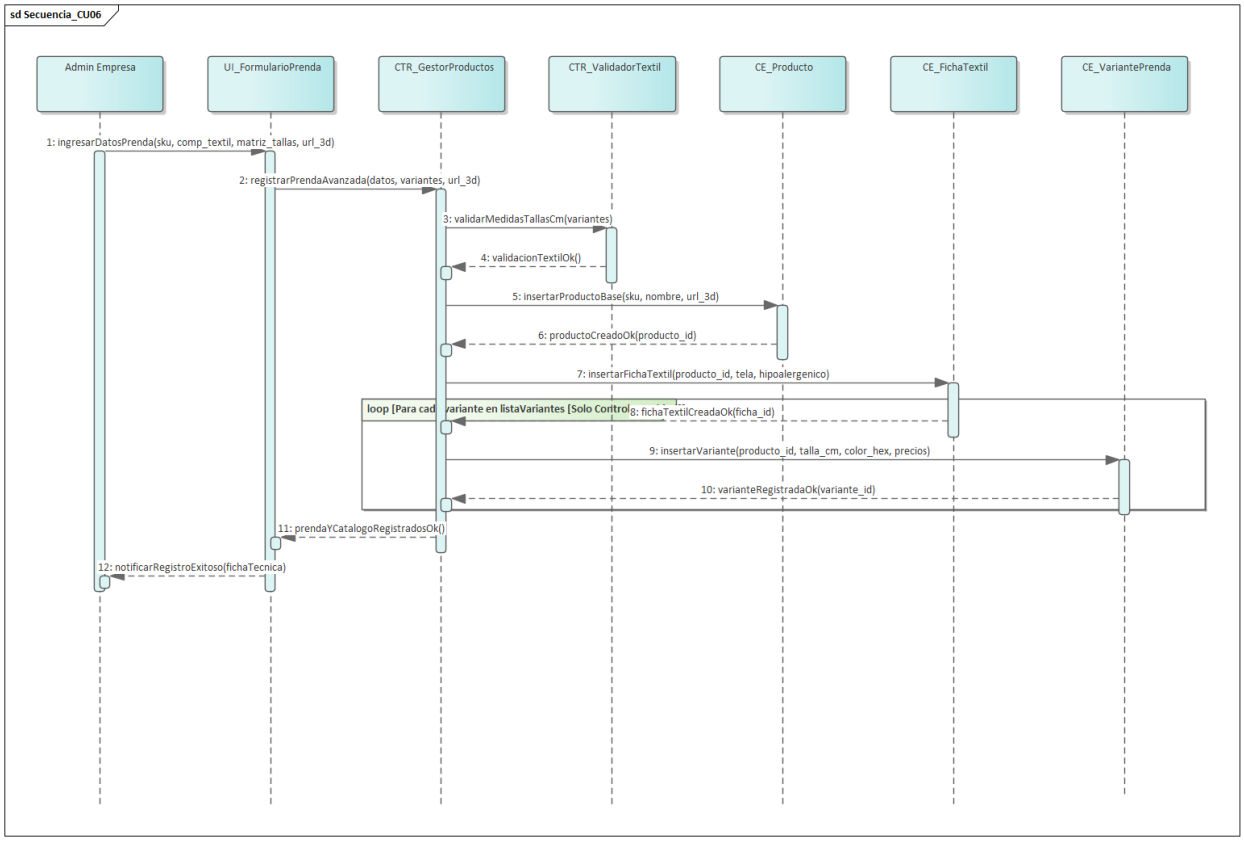
## CU04: Autenticar Usuario Multitenant



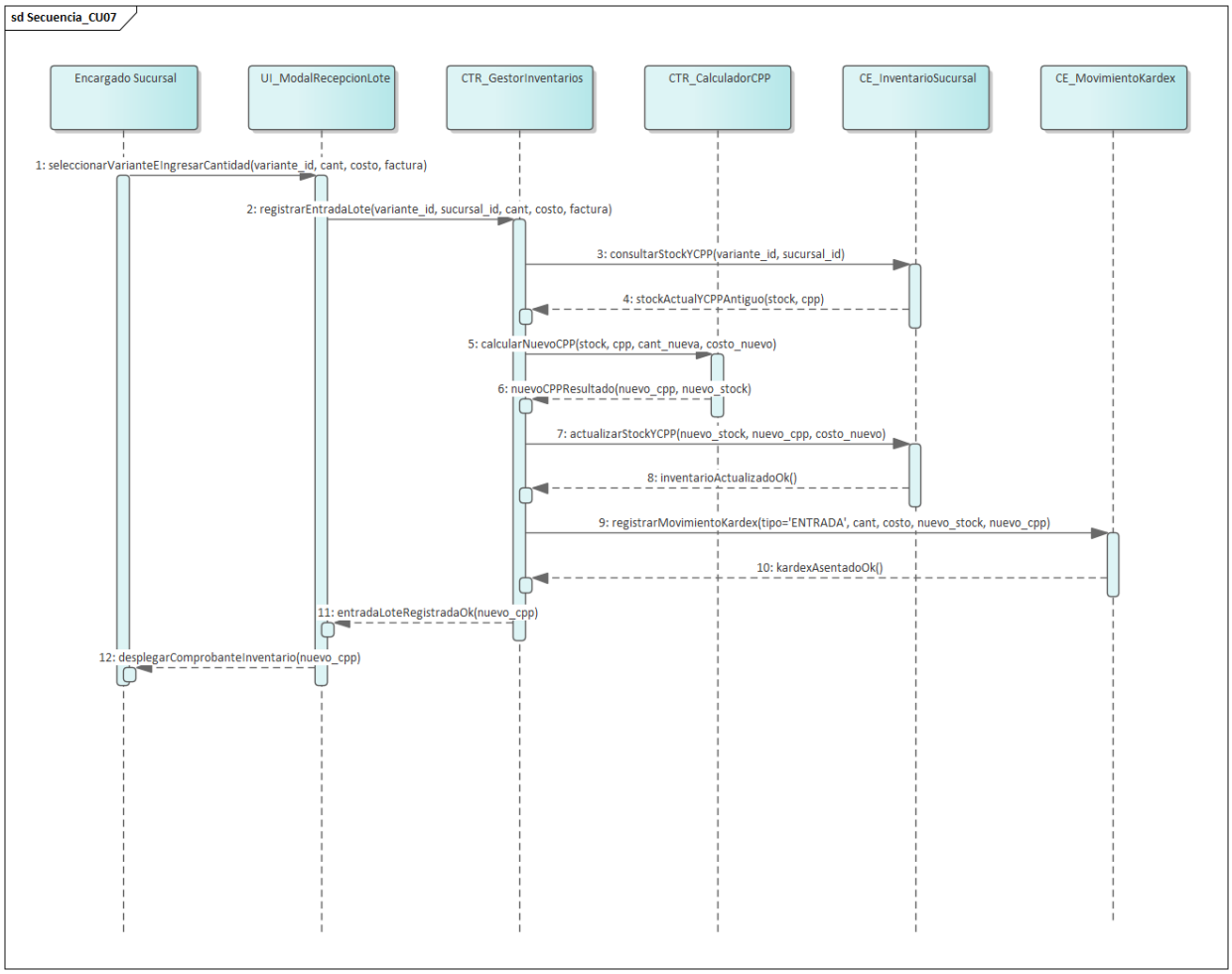
## CU05: Gestionar Usuarios y Roles por Empresa



## CU06: Gestionar Productos y Atributos Textiles Avanzados



**CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP)**




## 3.4 Diagrama de Clases de Análisis

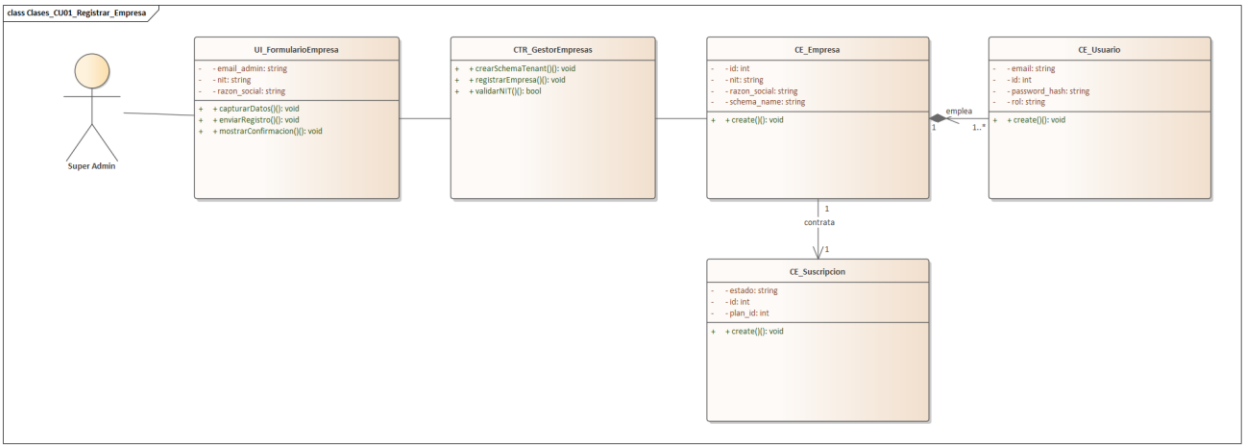
Conforme a la metodología ICONIX el **Diagrama de Clases de Análisis por Caso de Uso** constituye la transición estructural directa entre el Diagrama de Robustez y el Diagrama de Secuencia. En esta formalización orientada a objetos:

1. **Clases de Interfaz (Boundary - UI_...):** Representan los puntos de contacto con los Actores, conteniendo los atributos de captura de datos y métodos manejadores de eventos.
2. **Clases de Control (CTR_...):** Orquestan el flujo transaccional y las reglas de negocio específicas del caso de uso.
3. **Clases de Entidad (CE_...):** Mapean los objetos persistentes del dominio con sus atributos de estado y métodos de acceso.

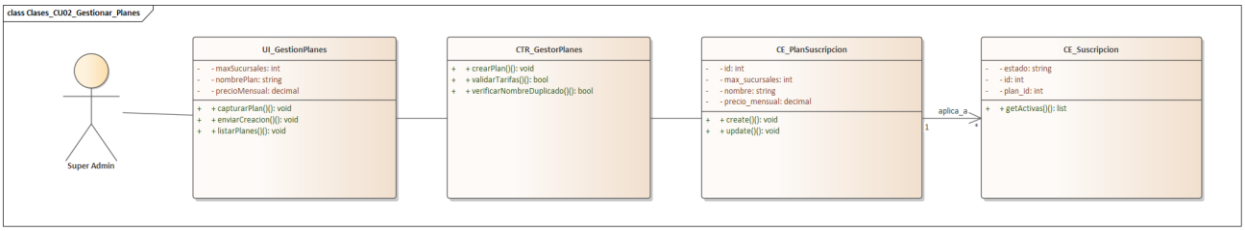
4. **Relaciones entre Clases Entidad con Verbos y Cardinalidades:** A diferencia de los Diagramas de Robustez (donde las entidades no se relacionan directamente entre sí), en los **Diagramas de Clases de Análisis SÍ se modelan las asociaciones y composiciones entre entidades de datos**, debido a que son extraídas de la estructura relacional de la Base de Datos (PostgreSQL), incluyendo **cardinalidades exactas (1 a 1, 1 a , * a 1, * a ) y verbos *explicativos* de la relación de negocio.

A continuación, se presentan los **20 Diagramas de Clases de Análisis** modelados y exportados directamente desde Enterprise Architect (diagramas1erParcial.eapx) junto con sus especificaciones de diseño:

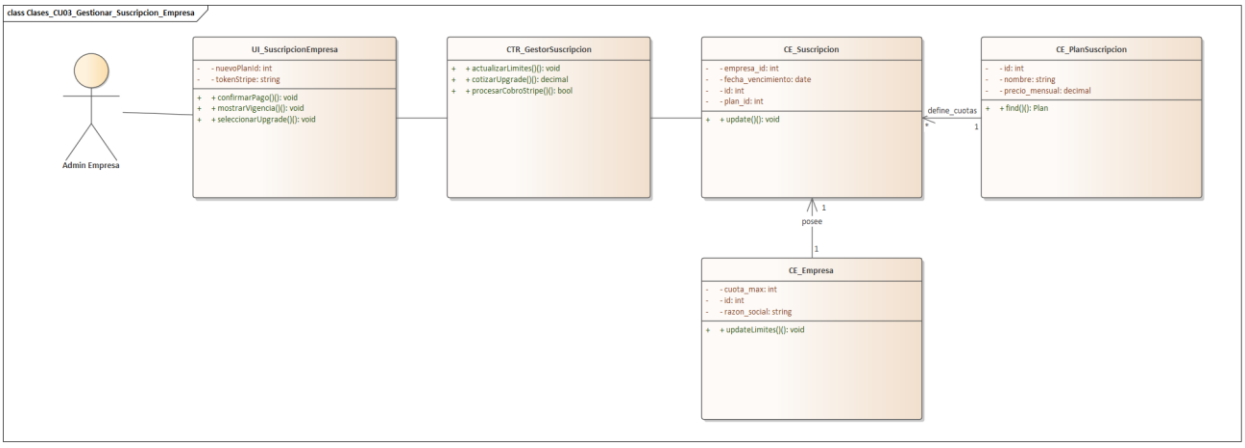
## CU01: Gestionar Empresas (Tenants)



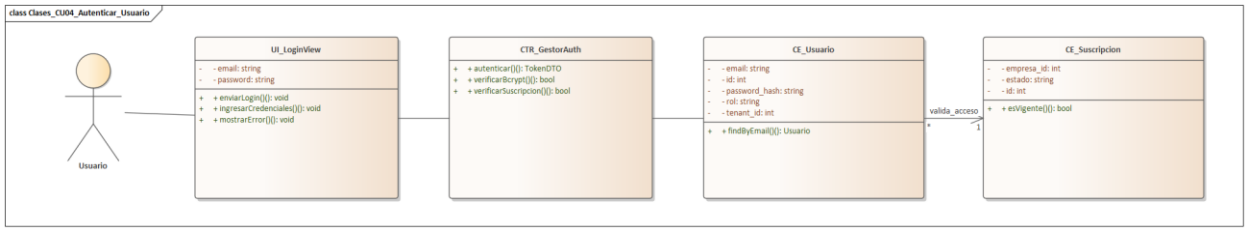

## CU02: Gestionar Planes de Suscripción



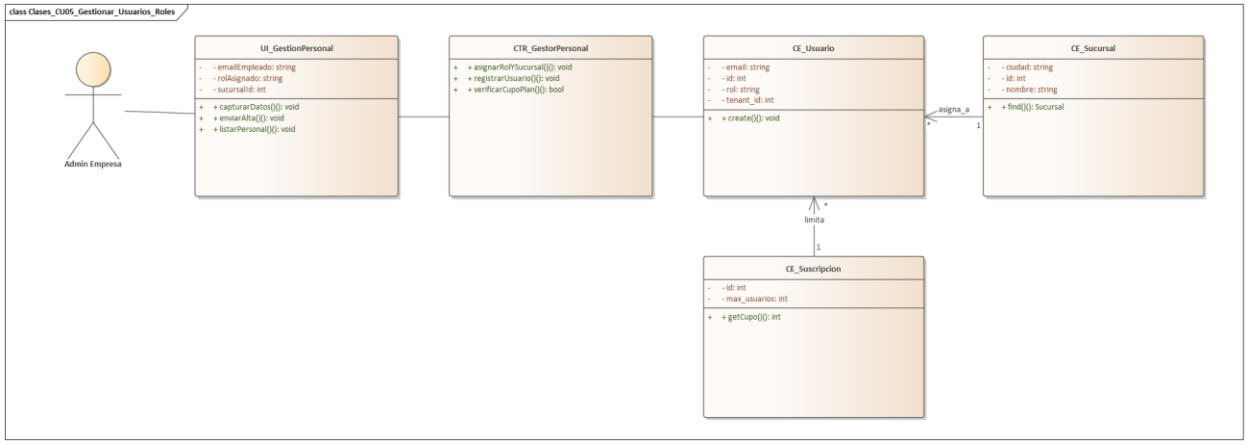

## CU03: Gestionar Suscripción de Empresa



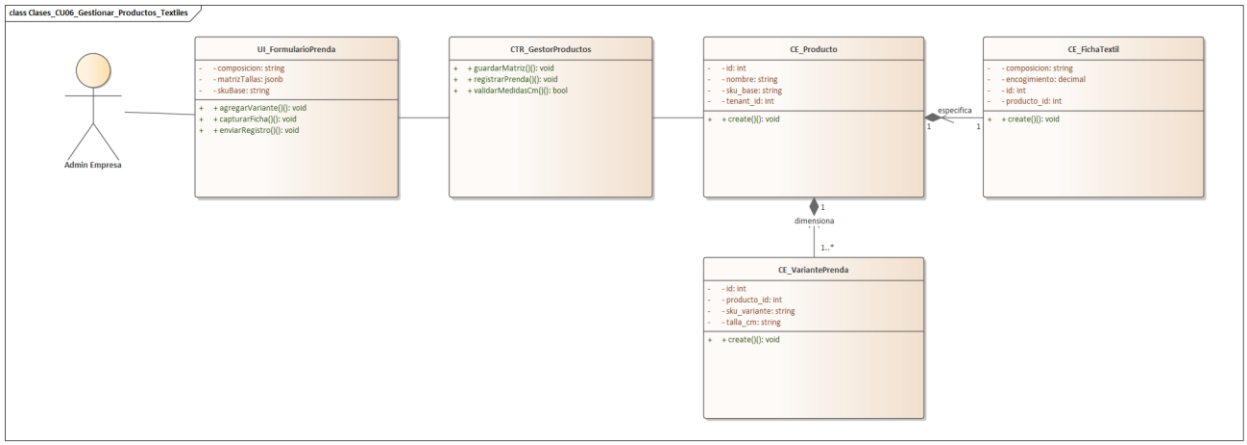
## CU04: Autenticar Usuario Multitenant



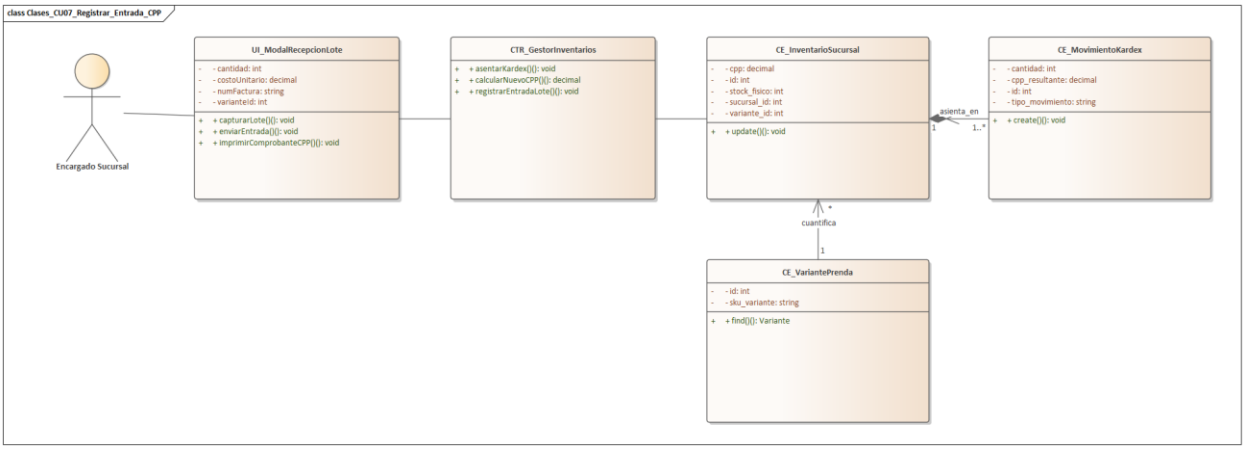

## CU05: Gestionar Usuarios y Roles por Empresa




## CU06: Gestionar Productos y Atributos Textiles Avanzados



### CU07: Gestionar Inventario Multi-Sucursal, Kardex Físico-Valorado y Costo Promedio Ponderado (CPP)



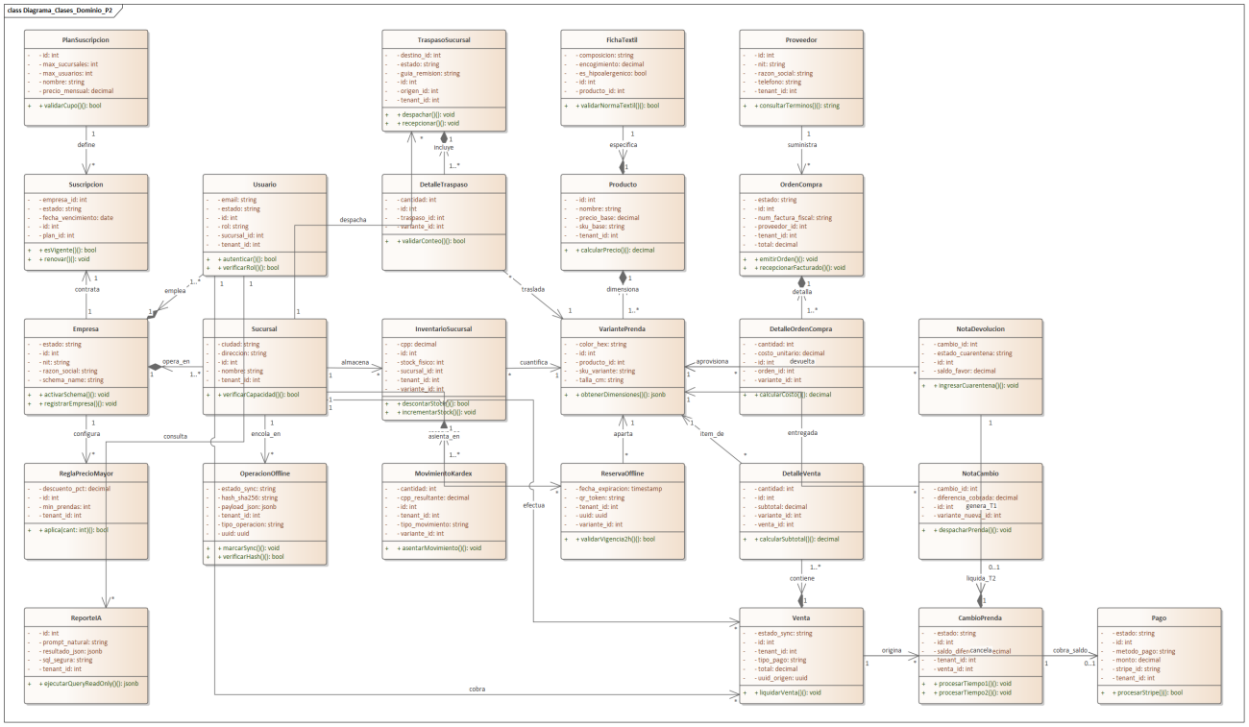

## 3.5. Diseño de la Base de Datos (Modelo Físico y Esquemas de Persistencia)

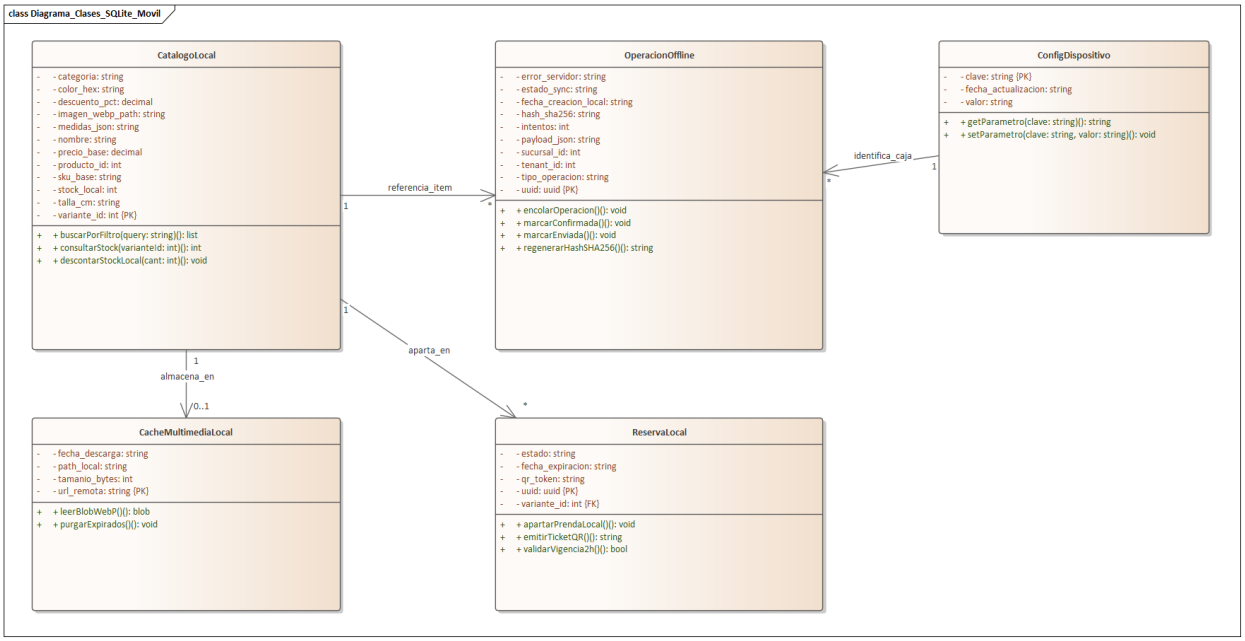
En la metodología ICONIX y la ingeniería de software aplicada a arquitecturas SaaS multitenant y distribuidas, el **Diseño de la Base de Datos** se descompone formalmente en dos etapas rigurosas y complementarias:

1. **Diseño Lógico de la Base de Datos (Sección 3.5.1):** Constituye la representación conceptual y relacional independiente de la plataforma física. Formaliza las 24 entidades maestras y transaccionales del Segundo Parcial, especificando claves primarias ({PK}), claves foráneas ({FK}), tipos abstractos, cardinalidades exactas (1 a 1, 1 a *, * a *) y verbos de asociación orientados al dominio.
2. **Diseño Físico de la Base de Datos (Sección 3.5.2):** Traduce el modelo lógico en código DDL (*Data Definition Language*) ejecutable y optimizado para **PostgreSQL 15+** en el servidor central (con aislamiento lógico multitenant mediante tenant_id, soporte de UUIDs e idempotencia para transacciones offline, índices B-Tree y triggers transaccionales de recálculo de CPP), complementado por el esquema local **SQLite** (aplicación móvil Flutter) y los *Object Stores* de **IndexedDB** (Progressive Web App - Angular).

### 3.5.1. Diseño Lógico de la Base de Datos (Modelo Relacional / Dominio — 2do Parcial)

El siguiente diagrama modela de forma ortogonal, limpia y sin cruces innecesarios las **24 entidades del dominio relacional** correspondientes a los 20 Casos de Uso del 2do Parcial, equipadas en el 100% de sus asociaciones con verbos direccionales y cardinalidades en ambos extremos:




### 3.5.2. Diseño Físico de la Base de Datos (Esquemas de Persistencia DDL Corregidos)

El diseño físico traduce el modelo lógico en tres entornos de almacenamiento coordinados para operación híbrida Online/Offline:

#### *3.5.2.1. Esquema Central PostgreSQL 15+ (DDL Completo y Corregido)*

El siguiente script DDL implementa en su totalidad las tablas, claves primarias, claves foráneas con eliminación en cascada controlada, índices de búsqueda B-Tree, restricciones CHECK de integridad

de negocio, funciones PL/pgSQL y triggers para el recálculo automático de CPP y la idempotencia de transacciones offline:

```
-- ============================================================================
-- PLATAFORMA SAAS MULTITENANT FASHIONSTORE - SCRIPT DDL POSTGRESQL 15+
-- ESQUEMA RELACIONAL CENTRAL COMPLETO (2DO PARCIAL)
-- ============================================================================

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ----------------------------------------------------------------------------
-- 1. PLANES DE SUSCRIPCIÓN (GLOBAL SAAS)
-- ----------------------------------------------------------------------------
CREATE TABLE planes_suscripcion (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    max_sucursales INT NOT NULL DEFAULT 2 CHECK (max_sucursales > 0),
    max_usuarios INT NOT NULL DEFAULT 10 CHECK (max_usuarios > 0),
    permite_reportes_ia BOOLEAN NOT NULL DEFAULT FALSE,
    permite_ml_dashboard BOOLEAN NOT NULL DEFAULT FALSE,
    precio_mensual NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (precio_mensual >= 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO' CHECK (estado IN ('ACTIVO', 'INACTIVO')),
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- ----------------------------------------------------------------------------
-- 2. EMPRESAS (TENANTS REGISTRADOS)
-- ----------------------------------------------------------------------------
CREATE TABLE empresas (
    id SERIAL PRIMARY KEY,
    nit VARCHAR(30) NOT NULL UNIQUE,
    razon_social VARCHAR(150) NOT NULL,
    schema_name VARCHAR(63) NOT NULL UNIQUE,
    email_contacto VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    presupuesto_disponible NUMERIC(12,2) NOT NULL DEFAULT 50000.00 CHECK (presupuesto_disponible >= 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN ('ACTIVA', 'SUSPENDIDA', 'VENCIDA')),
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_empresas_nit ON empresas(nit);

-- ----------------------------------------------------------------------------
-- 3. SUSCRIPCIONES DE EMPRESA
-- ----------------------------------------------------------------------------
CREATE TABLE suscripciones (
    id SERIAL PRIMARY KEY,
    empresa_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    plan_id INT NOT NULL REFERENCES planes_suscripcion(id),
    fecha_inicio DATE NOT NULL DEFAULT CURRENT_DATE,
    fecha_vencimiento DATE NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN ('ACTIVA', 'VENCIDA', 'CANCELADA')),
    actualizado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_fechas_suscripcion CHECK (fecha_vencimiento >= fecha_inicio)
);
CREATE INDEX idx_suscripciones_empresa ON suscripciones(empresa_id);

-- ----------------------------------------------------------------------------
```

```
-- 4. SUCURSALES (POR TENANT)
-- ------------------------------------------------------------------------
CREATE TABLE sucursales (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    nombre VARCHAR(100) NOT NULL,
    direccion VARCHAR(200) NOT NULL,
    ciudad VARCHAR(50) NOT NULL DEFAULT 'Santa Cruz',
    latitud NUMERIC(10,8),
    longitud NUMERIC(11,8),
    telefono VARCHAR(20),
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN ('ACTIVA', 'INACTIVA')),
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_sucursal_tenant_nombre UNIQUE (tenant_id, nombre)
);
CREATE INDEX idx_sucursales_tenant ON sucursales(tenant_id);

-- ------------------------------------------------------------------------
-- 5. USUARIOS Y CONTROL DE ROLES (RBAC MULTITENANT)
-- ------------------------------------------------------------------------
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT REFERENCES sucursales(id) ON DELETE SET NULL,
    nombre_completo VARCHAR(120) NOT NULL,
    email VARCHAR(100) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(30) NOT NULL CHECK (rol IN ('SUPER_ADMIN', 'ADMIN_EMPRESA', 'ENCARGADO_SUCURSAL', 'CAJERO',
'CLIENTE')),
    intentos_fallidos INT NOT NULL DEFAULT 0,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO' CHECK (estado IN ('ACTIVO', 'BLOQUEADO', 'INACTIVO')),
    ultimo_acceso TIMESTAMP WITH TIME ZONE,
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_usuario_tenant_email UNIQUE (tenant_id, email)
);
CREATE INDEX idx_usuarios_tenant ON usuarios(tenant_id);
CREATE INDEX idx_usuarios_email ON usuarios(email);

-- ------------------------------------------------------------------------
-- 6. CATEGORÍAS Y TEMPORADAS
-- ------------------------------------------------------------------------
CREATE TABLE categorias (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    nombre VARCHAR(80) NOT NULL,
    descripcion VARCHAR(255),
    CONSTRAINT uq_categoria_tenant UNIQUE (tenant_id, nombre)
);

CREATE TABLE temporadas (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    nombre VARCHAR(60) NOT NULL,
    anio INT NOT NULL CHECK (anio >= 2020),
    es_vigente BOOLEAN NOT NULL DEFAULT TRUE,
    en_liquidacion BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT uq_temporada_tenant UNIQUE (tenant_id, nombre, anio)
);

-- ------------------------------------------------------------------------
```

```
-- 7. PRODUCTOS (CATÁLOGO TEXTIL BASE)
-- ------------------------------------------------------------------------
CREATE TABLE productos (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    categoria_id INT NOT NULL REFERENCES categorias(id),
    temporada_id INT NOT NULL REFERENCES temporadas(id),
    sku_base VARCHAR(40) NOT NULL,
    nombre VARCHAR(120) NOT NULL,
    descripcion TEXT,
    precio_base NUMERIC(10,2) NOT NULL CHECK (precio_base > 0),
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO' CHECK (estado IN ('ACTIVO', 'INACTIVO')),
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_producto_tenant_sku UNIQUE (tenant_id, sku_base)
);
CREATE INDEX idx_productos_tenant ON productos(tenant_id);

-- ------------------------------------------------------------------------
-- 8. FICHAS TEXTILES TÉCNICAS (ATRIBUTOS AVANZADOS)
-- ------------------------------------------------------------------------
CREATE TABLE fichas_textiles (
    id SERIAL PRIMARY KEY,
    producto_id INT NOT NULL UNIQUE REFERENCES productos(id) ON DELETE CASCADE,
    composicion VARCHAR(150) NOT NULL,
    encogimiento NUMERIC(5,2) NOT NULL DEFAULT 0.00 CHECK (encogimiento >= 0),
    viscosidad NUMERIC(5,2) NOT NULL DEFAULT 0.00,
    es_hipoalergenico BOOLEAN NOT NULL DEFAULT FALSE,
    instrucciones_lavado VARCHAR(255)
);

-- ------------------------------------------------------------------------
-- 9. VARIANTES DE PRENDA (DIMENSIONES EN CM, COLOR Y CÓDIGO)
-- ------------------------------------------------------------------------
CREATE TABLE variantes_prenda (
    id SERIAL PRIMARY KEY,
    producto_id INT NOT NULL REFERENCES productos(id) ON DELETE CASCADE,
    talla_cm VARCHAR(20) NOT NULL,
    color_hex VARCHAR(10) NOT NULL DEFAULT '#000000',
    color_primario VARCHAR(40) NOT NULL DEFAULT 'Negro',
    color_secundario VARCHAR(40),
    sku_variante VARCHAR(60) NOT NULL UNIQUE,
    codigo_barras VARCHAR(50),
    medidas_cm JSONB NOT NULL DEFAULT '{"pecho": 0, "largo": 0, "manga": 0}'::jsonb
);
CREATE INDEX idx_variantes_producto ON variantes_prenda(producto_id);
CREATE INDEX idx_variantes_sku ON variantes_prenda(sku_variante);

-- ------------------------------------------------------------------------
-- 10. INVENTARIO MULTI-SUCURSAL Y COSTO PROMEDIO PONDERADO (CPP)
-- ------------------------------------------------------------------------
CREATE TABLE inventario_sucursal (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id) ON DELETE CASCADE,
    variante_id INT NOT NULL REFERENCES variantes_prenda(id) ON DELETE CASCADE,
    stock_fisico INT NOT NULL DEFAULT 0 CHECK (stock_fisico >= 0),
    stock_reservado INT NOT NULL DEFAULT 0 CHECK (stock_reservado >= 0),
    costo_promedio_ponderado NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (costo_promedio_ponderado >= 0),
    stock_minimo INT NOT NULL DEFAULT 5 CHECK (stock_minimo >= 0),
    actualizado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
    CONSTRAINT uq_inventario_sucursal_variante UNIQUE (sucursal_id, variante_id)
);
CREATE INDEX idx_inventario_sucursal_variante ON inventario_sucursal(sucursal_id, variante_id);
CREATE INDEX idx_inventario_tenant ON inventario_sucursal(tenant_id);

-- ------------------------------------------------------------------------
-- 11. MOVIMIENTOS KARDEX FÍSICO-VALORADO (ASIENTOS INMUTABLES)
-- ------------------------------------------------------------------------
CREATE TABLE movimientos_kardex (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    variante_id INT NOT NULL REFERENCES variantes_prenda(id),
    tipo_movimiento VARCHAR(30) NOT NULL CHECK (tipo_movimiento IN (
        'ENTRADA_COMPRA', 'SALIDA_VENTA', 'REINGRESO_DEVOLUCION',
        'DESPACHO_CAMBIO', 'TRASPASO_SALIDA', 'TRASPASO_ENTRADA', 'AJUSTE'
    ),
    cantidad INT NOT NULL,
    costo_unitario NUMERIC(10,2) NOT NULL CHECK (costo_unitario >= 0),
    cpp_resultante NUMERIC(10,2) NOT NULL CHECK (cpp_resultante >= 0),
    saldo_stock INT NOT NULL CHECK (saldo_stock >= 0),
    documento_ref VARCHAR(60),
    fecha TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_kardex_variante_fecha ON movimientos_kardex(variante_id, fecha);
CREATE INDEX idx_kardex_sucursal ON movimientos_kardex(sucursal_id);

-- ------------------------------------------------------------------------
-- 12. TRASPASOS INTER-SUCURSAL Y DETALLE
-- ------------------------------------------------------------------------
CREATE TABLE traspasos_inventario (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_origen_id INT NOT NULL REFERENCES sucursales(id),
    sucursal_destino_id INT NOT NULL REFERENCES sucursales(id),
    guia_remision VARCHAR(50) NOT NULL UNIQUE,
    fecha_solicitud TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    fecha_recepcion TIMESTAMP WITH TIME ZONE,
    estado VARCHAR(25) NOT NULL DEFAULT 'EN_TRANSITO' CHECK (estado IN ('EN_TRANSITO', 'COMPLETADO',
'RECHAZADO')),
    observacion TEXT,
    CONSTRAINT ck_sucursales_distintas CHECK (sucursal_origen_id <> sucursal_destino_id)
);

CREATE TABLE detalle_traspasos (
    id SERIAL PRIMARY KEY,
    traspaso_id INT NOT NULL REFERENCES traspasos_inventario(id) ON DELETE CASCADE,
    variante_id INT NOT NULL REFERENCES variantes_prenda(id),
    cantidad_enviada INT NOT NULL CHECK (cantidad_enviada > 0),
    cantidad_recibida INT DEFAULT 0 CHECK (cantidad_recibida >= 0)
);
CREATE INDEX idx_traspaso_detalle ON detalle_traspasos(traspaso_id);

-- ------------------------------------------------------------------------
-- 13. PROVEEDORES Y ÓRDENES DE COMPRA (WEB DESKTOP)
-- ------------------------------------------------------------------------
CREATE TABLE proveedores (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    razon_social VARCHAR(120) NOT NULL,
```

```
    nit VARCHAR(30) NOT NULL,
    contacto VARCHAR(80),
    email VARCHAR(100),
    telefono VARCHAR(20),
    CONSTRAINT uq_proveedor_tenant_nit UNIQUE (tenant_id, nit)
);

CREATE TABLE ordenes_compra (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    proveedor_id INT NOT NULL REFERENCES proveedores(id),
    sucursal_destino_id INT NOT NULL REFERENCES sucursales(id),
    numero_orden VARCHAR(40) NOT NULL UNIQUE,
    num_factura_fiscal VARCHAR(50),
    fecha_emision TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    fecha_entrega_esperada DATE NOT NULL,
    total NUMERIC(12,2) NOT NULL DEFAULT 0.00 CHECK (total >= 0),
    estado VARCHAR(25) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE', 'AUTORIZADA',
'ENTREGA_PARCIAL', 'COMPLETADA', 'CANCELADA'))
);

CREATE TABLE detalle_ordenes_compra (
    id SERIAL PRIMARY KEY,
    orden_compra_id INT NOT NULL REFERENCES ordenes_compra(id) ON DELETE CASCADE,
    variante_id INT NOT NULL REFERENCES variantes_prenda(id),
    cantidad INT NOT NULL CHECK (cantidad > 0),
    costo_unitario NUMERIC(10,2) NOT NULL CHECK (costo_unitario > 0),
    subtotal NUMERIC(12,2) GENERATED ALWAYS AS (cantidad * costo_unitario) STORED
);
CREATE INDEX idx_orden_compra_detalle ON detalle_ordenes_compra(orden_compra_id);

-- ------------------------------------------------------------------------
-- 14. VENTAS Y DETALLE (SOPORTE ONLINE Y OFFLINE IDEMPOTENTE)
-- ------------------------------------------------------------------------
CREATE TABLE ventas (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    usuario_cajero_id INT REFERENCES usuarios(id),
    cliente_id INT REFERENCES usuarios(id),
    numero_factura VARCHAR(50) NOT NULL,
    fecha TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    subtotal NUMERIC(10,2) NOT NULL CHECK (subtotal >= 0),
    descuento_total NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (descuento_total >= 0),
    total NUMERIC(10,2) NOT NULL CHECK (total >= 0),
    tipo_pago VARCHAR(30) NOT NULL CHECK (tipo_pago IN ('EFECTIVO', 'TARJETA_STRIPE', 'QR', 'MIXTO')),
    estado VARCHAR(20) NOT NULL DEFAULT 'COMPLETADA' CHECK (estado IN ('PENDIENTE_PAGO', 'COMPLETADA',
'ANULADA')),
    estado_sync VARCHAR(20) NOT NULL DEFAULT 'ONLINE' CHECK (estado_sync IN ('ONLINE', 'SINCRONIZADA',
'PENDIENTE')),
    uuid_origen UUID UNIQUE
);
CREATE INDEX idx_ventas_tenant_fecha ON ventas(tenant_id, fecha);
CREATE INDEX idx_ventas_uuid_origen ON ventas(uuid_origen);

CREATE TABLE detalle_ventas (
    id SERIAL PRIMARY KEY,
    venta_id INT NOT NULL REFERENCES ventas(id) ON DELETE CASCADE,
    variante_id INT NOT NULL REFERENCES variantes_prenda(id),
    cantidad INT NOT NULL CHECK (cantidad > 0),
```

```
    precio_unitario NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0),
    subtotal NUMERIC(10,2) GENERATED ALWAYS AS (cantidad * precio_unitario) STORED
);
CREATE INDEX idx_detalle_ventas_venta ON detalle_ventas(venta_id);

-- ------------------------------------------------------------------------
-- 15. RESERVAS OFFLINE CON TOKEN QR (VÁLIDAS POR 2 HORAS)
-- ------------------------------------------------------------------------
CREATE TABLE reservas_offline (
    uuid UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    cliente_id INT NOT NULL REFERENCES usuarios(id),
    variante_id INT NOT NULL REFERENCES variantes_prenda(id),
    qr_token VARCHAR(64) NOT NULL UNIQUE,
    fecha_solicitud TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    fecha_expiracion TIMESTAMP WITH TIME ZONE NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN ('ACTIVA', 'ATENDIDA', 'CANCELADA',
'EXPIRADA')),
    estado_sync VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE_SYNC' CHECK (estado_sync IN ('PENDIENTE_SYNC',
'SINCRONIZADA', 'EN_CONFLICTO'))
);
CREATE INDEX idx_reservas_offline_qr ON reservas_offline(qr_token);
CREATE INDEX idx_reservas_expiracion ON reservas_offline(fecha_expiracion, estado);

-- ------------------------------------------------------------------------
-- 16. OPERACIONES OFFLINE (BITÁCORA DE AUDITORÍA E IDEMPOTENCIA)
-- ------------------------------------------------------------------------
CREATE TABLE operaciones_offline (
    uuid UUID PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    tipo_operacion VARCHAR(30) NOT NULL CHECK (tipo_operacion IN ('VENTA_POS', 'RESERVA_LOCAL',
'CAMBIO_PRENDA')),
    payload_json JSONB NOT NULL,
    hash_sha256 VARCHAR(64) NOT NULL,
    estado_sync VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado_sync IN ('PENDIENTE', 'SINCRONIZADA',
'EN_CONFLICTO', 'RECHAZADA')),
    fecha_creacion_local TIMESTAMP WITH TIME ZONE NOT NULL,
    fecha_sincronizacion TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    error_motivo TEXT
);
CREATE INDEX idx_operaciones_offline_sync ON operaciones_offline(tenant_id, estado_sync);

-- ------------------------------------------------------------------------
-- 17. CAMBIOS DE PRENDA EN DOS TIEMPOS (POLÍTICA DE POSTVENTA)
-- ------------------------------------------------------------------------
CREATE TABLE cambios_prenda (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    venta_origen_id INT NOT NULL REFERENCES ventas(id),
    cliente_id INT NOT NULL REFERENCES usuarios(id),
    fecha_solicitud TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    plazo_limite TIMESTAMP WITH TIME ZONE NOT NULL,
    motivo TEXT NOT NULL,
    saldo_diferencial NUMERIC(10,2) NOT NULL DEFAULT 0.00,
    estado VARCHAR(30) NOT NULL DEFAULT 'SOLICITADO' CHECK (estado IN (
        'SOLICITADO', 'TIEMPO1_RECEPCIONADO', 'TIEMPO2_LIQUIDADO', 'COMPLETADO', 'RECHAZADO'
    ))
);
```

```
CREATE TABLE notas_devolucion (
    id SERIAL PRIMARY KEY,
    cambio_id INT NOT NULL REFERENCES cambios_prenda(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    variante_devuelta_id INT NOT NULL REFERENCES variantes_prenda(id),
    cantidad INT NOT NULL DEFAULT 1 CHECK (cantidad > 0),
    precio_original_reconocido NUMERIC(10,2) NOT NULL CHECK (precio_original_reconocido > 0),
    saldo_favor NUMERIC(10,2) NOT NULL CHECK (saldo_favor > 0),
    estado_cuarentena VARCHAR(30) NOT NULL DEFAULT 'EN_CUARENTENA' CHECK (estado_cuarentena IN
('EN_CUARENTENA', 'APROBADA_INVENTARIO', 'DESCARTADA_DEFECTO')),
    fecha_emision TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE notas_cambio (
    id SERIAL PRIMARY KEY,
    cambio_id INT NOT NULL REFERENCES cambios_prenda(id) ON DELETE CASCADE,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    variante_nueva_id INT NOT NULL REFERENCES variantes_prenda(id),
    cantidad INT NOT NULL DEFAULT 1 CHECK (cantidad > 0),
    precio_nueva_prenda NUMERIC(10,2) NOT NULL CHECK (precio_nueva_prenda > 0),
    diferencia_cobrada NUMERIC(10,2) NOT NULL DEFAULT 0.00 CHECK (diferencia_cobrada >= 0),
    fecha_despacho TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    estado_despacho VARCHAR(30) NOT NULL DEFAULT 'DESPACHADO' CHECK (estado_despacho IN ('DESPACHADO',
'ENTREGADO'))
);

-- ------------------------------------------------------------------------
-- 18. PAGOS ELECTRÓNICOS Y EN EFECTIVO (PASARELA STRIPE / POS)
-- ------------------------------------------------------------------------
CREATE TABLE pagos (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    venta_id INT REFERENCES ventas(id),
    cambio_id INT REFERENCES cambios_prenda(id),
    monto NUMERIC(10,2) NOT NULL CHECK (monto > 0),
    metodo_pago VARCHAR(30) NOT NULL CHECK (metodo_pago IN ('TARJETA_STRIPE', 'EFECTIVO_POS', 'QR_SIMPLE')),
    stripe_id VARCHAR(100),
    estado VARCHAR(20) NOT NULL DEFAULT 'CONFIRMADO' CHECK (estado IN ('CONFIRMADO', 'PENDIENTE',
'RECHAZADO', 'REEMBOLSADO')),
    fecha TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_pagos_tenant ON pagos(tenant_id);
CREATE INDEX idx_pagos_stripe ON pagos(stripe_id);

-- ------------------------------------------------------------------------
-- 19. REGLAS COMERCIALES (PRECIOS POR MAYOR Y LIQUIDACIÓN)
-- ------------------------------------------------------------------------
CREATE TABLE reglas_precio_mayor (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    min_prendas INT NOT NULL DEFAULT 3 CHECK (min_prendas >= 2),
    descuento_pct NUMERIC(5,2) NOT NULL CHECK (descuento_pct BETWEEN 1.00 AND 80.00),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    creado_en TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE ofertas_liquidacion (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
```

```
    producto_id INT NOT NULL REFERENCES productos(id) ON DELETE CASCADE,
    descuento_pct NUMERIC(5,2) NOT NULL CHECK (descuento_pct BETWEEN 5.00 AND 90.00),
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVA' CHECK (estado IN ('ACTIVA', 'EXPIRADA', 'CANCELADA')),
    CONSTRAINT ck_fechas_oferta CHECK (fecha_fin >= fecha_inicio)
);

-- ------------------------------------------------------------------------
-- 20. AUDITORÍA IA Y ANALÍTICA MACHINE LEARNING
-- ------------------------------------------------------------------------
CREATE TABLE reportes_ia (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    usuario_id INT NOT NULL REFERENCES usuarios(id),
    prompt_natural TEXT NOT NULL,
    sql_segura TEXT NOT NULL,
    resultado_json JSONB NOT NULL,
    fecha_consulta TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE datos_sinteticos_ml (
    id SERIAL PRIMARY KEY,
    tenant_id INT NOT NULL REFERENCES empresas(id) ON DELETE CASCADE,
    fecha DATE NOT NULL,
    sucursal_id INT NOT NULL REFERENCES sucursales(id),
    categoria_id INT NOT NULL REFERENCES categorias(id),
    unidades_vendidas INT NOT NULL CHECK (unidades_vendidas >= 0),
    ingreso_total NUMERIC(12,2) NOT NULL CHECK (ingreso_total >= 0),
    temperatura_ambiente NUMERIC(4,1),
    cluster_segmento INT
);
CREATE INDEX idx_ml_tenant_fecha ON datos_sinteticos_ml(tenant_id, fecha);

-- ========================================================================
-- PROCEDIMIENTOS Y TRIGGERS TRANSACCIONALES
-- ========================================================================

-- 1. FUNCIÓN TRIGGER: RECÁLCULO DE COSTO PROMEDIO PONDERADO (CPP)
CREATE OR REPLACE FUNCTION fn_recalcular_cpp_entrada()
RETURNS TRIGGER AS $$
DECLARE
    v_stock_actual INT;
    v_cpp_actual NUMERIC(10,2);
    v_nuevo_stock INT;
    v_nuevo_cpp NUMERIC(10,2);
    v_sucursal_id INT;
    v_tenant_id INT;
BEGIN
    -- Obtener sucursal y tenant de la orden
    SELECT sucursal_destino_id, tenant_id INTO v_sucursal_id, v_tenant_id
    FROM ordenes_compra WHERE id = NEW.orden_compra_id;

    -- Obtener inventario actual
    SELECT stock_fisico, costo_promedio_ponderado INTO v_stock_actual, v_cpp_actual
    FROM inventario_sucursal
    WHERE sucursal_id = v_sucursal_id AND variante_id = NEW.variante_id;

    IF NOT FOUND THEN
        v_stock_actual := 0;
```

```
        v_cpp_actual := 0.00;
        INSERT    INTO    inventario_sucursal(tenant_id,    sucursal_id,    variante_id,    stock_fisico,
costo_promedio_ponderado)
        VALUES (v_tenant_id, v_sucursal_id, NEW.variante_id, NEW.cantidad, NEW.costo_unitario);
        v_nuevo_stock := NEW.cantidad;
        v_nuevo_cpp := NEW.costo_unitario;
    ELSE
        v_nuevo_stock := v_stock_actual + NEW.cantidad;
        IF v_nuevo_stock > 0 THEN
            v_nuevo_cpp := ROUND(((v_stock_actual * v_cpp_actual) + (NEW.cantidad * NEW.costo_unitario)) /
v_nuevo_stock, 2);
        ELSE
            v_nuevo_cpp := NEW.costo_unitario;
        END IF;
        UPDATE inventario_sucursal
        SET  stock_fisico  =  v_nuevo_stock,  costo_promedio_ponderado  =  v_nuevo_cpp,  actualizado_en  =
CURRENT_TIMESTAMP
        WHERE sucursal_id = v_sucursal_id AND variante_id = NEW.variante_id;
    END IF;

    -- Registrar asiento inmutable en Kardex
    INSERT INTO movimientos_kardex (
        tenant_id, sucursal_id, variante_id, tipo_movimiento,
        cantidad, costo_unitario, cpp_resultante, saldo_stock, documento_ref
    ) VALUES (
        v_tenant_id, v_sucursal_id, NEW.variante_id, 'ENTRADA_COMPRA',
        NEW.cantidad, NEW.costo_unitario, v_nuevo_cpp, v_nuevo_stock, 'OC-' || NEW.orden_compra_id
    );

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_recalcular_cpp_orden_compra
AFTER INSERT ON detalle_ordenes_compra
FOR EACH ROW EXECUTE FUNCTION fn_recalcular_cpp_entrada();

-- 2. PROCEDIMIENTO ALMACENADO: SINCRONIZACIÓN IDEMPOTENTE OFFLINE
CREATE OR REPLACE FUNCTION sp_reconciliar_venta_offline(
    p_uuid UUID,
    p_tenant_id INT,
    p_sucursal_id INT,
    p_cajero_id INT,
    p_cliente_id INT,
    p_factura VARCHAR,
    p_subtotal NUMERIC,
    p_descuento NUMERIC,
    p_total NUMERIC,
    p_tipo_pago VARCHAR,
    p_items JSONB
)
RETURNS BOOLEAN AS $$
DECLARE
    v_venta_id INT;
    v_item JSONB;
BEGIN
    -- Verificar idempotencia por uuid_origen
    IF EXISTS (SELECT 1 FROM ventas WHERE uuid_origen = p_uuid) THEN
        RETURN TRUE; -- Ya sincronizada previamente sin duplicar
    END IF;
```

```
    -- Insertar venta definitiva
    INSERT INTO ventas (
        tenant_id, sucursal_id, usuario_cajero_id, cliente_id,
        numero_factura, subtotal, descuento_total, total,
        tipo_pago, estado, estado_sync, uuid_origen
    ) VALUES (
        p_tenant_id, p_sucursal_id, p_cajero_id, p_cliente_id,
        p_factura, p_subtotal, p_descuento, p_total,
        p_tipo_pago, 'COMPLETADA', 'SINCRONIZADA', p_uuid
    ) RETURNING id INTO v_venta_id;

    -- Insertar detalles y descontar stock central
    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
        INSERT INTO detalle_ventas (venta_id, variante_id, cantidad, precio_unitario)
        VALUES (
            v_venta_id,
            (v_item->>'variante_id')::INT,
            (v_item->>'cantidad')::INT,
            (v_item->>'precio_unitario')::NUMERIC
        );

        UPDATE inventario_sucursal
        SET stock_fisico = GREATEST(0, stock_fisico - (v_item->>'cantidad')::INT),
            actualizado_en = CURRENT_TIMESTAMP
        WHERE sucursal_id = p_sucursal_id AND variante_id = (v_item->>'variante_id')::INT;
    END LOOP;

    -- Registrar en bitácora de operaciones offline
    UPDATE operaciones_offline
    SET estado_sync = 'SINCRONIZADA', fecha_sincronizacion = CURRENT_TIMESTAMP
    WHERE uuid = p_uuid;

    RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

### *3.5.2.2. Esquema Relacional Local SQLite (App Móvil Flutter)*

En los dispositivos móviles que operan sin conectividad constante a internet, el motor SQLite embebido almacena el catálogo pre-cacheado y encola las ventas y reservas en SQLite relacional local:

```
-- ============================================================================
-- ESQUEMA LOCAL SQLITE PARA APLICACIÓN MÓVIL FLUTTER (OFFLINE-FIRST)
-- ============================================================================

-- 1. CATÁLOGO LOCAL CACHEADO
CREATE TABLE catalogo_local (
    variante_id INTEGER PRIMARY KEY,
    producto_id INTEGER NOT NULL,
    sku_base TEXT NOT NULL,
    nombre TEXT NOT NULL,
    categoria TEXT NOT NULL,
    talla_cm TEXT NOT NULL,
    color_hex TEXT NOT NULL,
    precio_base REAL NOT NULL,
```

```
    descuento_pct REAL NOT NULL DEFAULT 0.0,
    stock_local INTEGER NOT NULL DEFAULT 0,
    medidas_json TEXT NOT NULL DEFAULT '{}',
    imagen_webp_path TEXT
);
CREATE INDEX idx_catalogo_sku ON catalogo_local(sku_base);

-- 2. COLA DE TRANSACCIONES OFFLINE
CREATE TABLE operaciones_offline (
    uuid TEXT PRIMARY KEY,
    tenant_id INTEGER NOT NULL,
    sucursal_id INTEGER NOT NULL,
    tipo_operacion TEXT NOT NULL CHECK(tipo_operacion IN ('VENTA_POS', 'RESERVA_LOCAL', 'CAMBIO_PRENDA')),
    payload_json TEXT NOT NULL,
    hash_sha256 TEXT NOT NULL,
    estado_sync TEXT NOT NULL DEFAULT 'PENDIENTE_SYNC' CHECK(estado_sync IN ('PENDIENTE_SYNC', 'ENVIADA',
'CONFIRMADA', 'RECHAZADA')),
    fecha_creacion_local TEXT NOT NULL,
    intentos INTEGER NOT NULL DEFAULT 0,
    error_servidor TEXT
);
CREATE INDEX idx_offline_estado ON operaciones_offline(estado_sync);

-- 3. RESERVAS LOCALES ACTIVAS (TOKEN QR)
CREATE TABLE reservas_locales (
    uuid TEXT PRIMARY KEY,
    variante_id INTEGER NOT NULL,
    qr_token TEXT NOT NULL UNIQUE,
    fecha_expiracion TEXT NOT NULL,
    estado TEXT NOT NULL DEFAULT 'ACTIVA'
);

-- 4. CONFIGURACIÓN LOCAL Y SESIÓN
CREATE TABLE config_dispositivo (
    clave TEXT PRIMARY KEY,
    valor TEXT NOT NULL
);
```

## 3.6. Diseño de interfaces de usuario

# CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

## 4.1. Herramientas y tecnologías utilizadas

Para el desarrollo del sistema de gestión comercial textil bajo modelo SaaS Multitenant, se seleccionó un conjunto de tecnologías modernas, escalables y desacopladas que garantizan alta disponibilidad, soporte para operaciones offline-first y análisis predictivo avanzado:

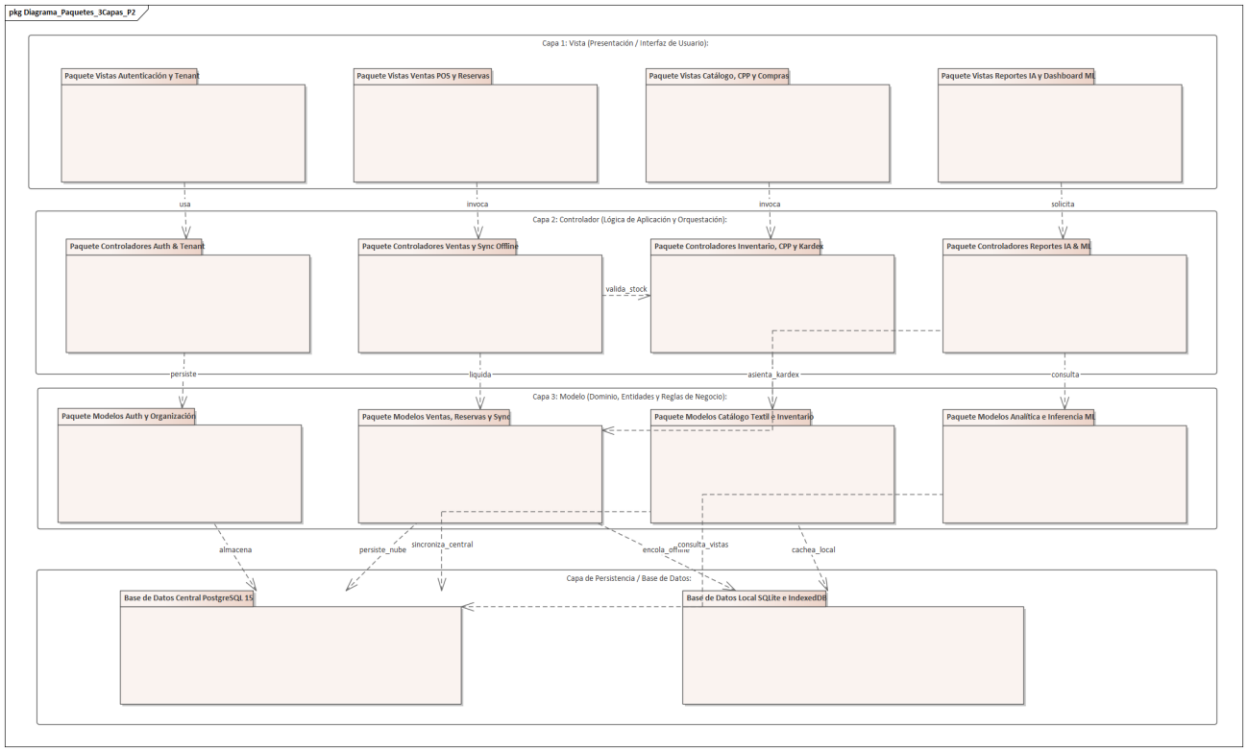
| Componente / Capa | Tecnología / Herramienta | Versión / Detalle | Justificación y Rol en la Arquitectura |
|---|---|---|---|
| **Backend REST API** | Python + FastAPI | Python 3.11 / FastAPI 0.110+ | Framework asíncrono de alto rendimiento para el core transaccional, con validación tipada vía Pydantic v2 y endpoints documentados OpenAPI/Swagger. |
| **Persistencia Asíncrona (ORM)** | SQLAlchemy 2.0 (Async) + Alembic | SQLAlchemy 2.0.28+ | Manejo relacional con soporte nativo de async/await, migraciones de base de datos controladas y pool de conexiones asíncrono con asyncpg. |
| **Frontend Web Corporativo** | Angular (evolución a PWA) | Angular 17+ (Standalone Components) | Interfaz de administración para sucursales y gerencia con Service Workers, Web App Manifest y estrategia de caché offline. |
| **Persistencia Offline Web** | IndexedDB | API nativa / Dexie.js | Almacén NoSQL local en el navegador del cliente para transacciones del POS y catálogo cuando se pierde la conectividad WAN. |
| **Aplicación Móvil** | Flutter + Dart | Flutter 3.19+ / Dart 3.3+ | Cliente móvil multiplataforma (Android/iOS) para encargados de tienda y preventa con interfaz fluida y arquitectura BLoC/Provider. |
| **Persistencia Móvil Local** | SQLite | sqflite 2.3+ | Motor relacional ligero embebido en el dispositivo móvil para almacenamiento de operaciones locales y cola de sincronización diferida. |
| **Base de Datos Central** | PostgreSQL | PostgreSQL 15+ | Motor RDBMS corporativo para persistencia multitenant (tenant_id), particionamiento transaccional, índices B-Tree/GIN y cumplimiento ACID estricto. |

| | | | |
|---|---|---|---|
| **Inteligencia Artificial Generativa** | OpenAI API / Google Gemini API | GPT-4o / Gemini 1.5 Flash | Generación de consultas analíticas SQL en lenguaje natural y asistencia ejecutiva en redacción e interpretación de reportes. |
| **Machine Learning Predictivo** | Scikit-Learn, Pandas, NumPy | Scikit-learn 1.4+ | Modelos de predicción de demanda estacional por prendas/tallas (Random Forest / Regresión) y clusterización para cálculo de ofertas y rotación de stock. |
| **Pasarela de Pagos Digitales** | Stripe API | SDK Stripe v8+ (Modo Sandbox) | Procesamiento seguro de cobros electrónicos con tarjeta de crédito/débito y webhooks para confirmación automática de liquidaciones. |
| **Seguridad y Autorización** | JWT + Argon2 / Bcrypt | PyJWT 2.8+, Passlib | Autenticación basada en tokens JWT con claims de tenant y sucursal, control de acceso basado en roles (RBAC) y hashing de contraseñas. |
| **Modelado y Diseño** | UML 2.5+ & Sparx Systems EA | PlantUML / Enterprise Architect 16+ | Modelado formal de casos de uso, análisis de robustez ICONIX, secuencia, clases de análisis y diagramas arquitectónicos de despliegue y paquetes. |
| **Despliegue e Infraestructura** | Cloud PaaS (Docker / Linux Ubuntu) | Ubuntu 22.04 LTS / Docker Compose | Alojamiento en la nube (Render / Railway / AWS ECS) con variables de entorno aisladas por tenant y certificados SSL/TLS automáticos. |
| **Control de Versiones** | Git + GitHub | Git 2.44+ | Repositorio centralizado con flujos de trabajo basados en ramas, control de cambios y trazabilidad de commits. |

## 4.2. Diseño de la arquitectura del sistema

### 4.2.1. Diseño Lógico de la Arquitectura: Diagrama de Paquetes en 3 Capas (UML)

El diseño lógico de la arquitectura descompone el sistema en una estructura de **3 Capas principales (Vista, Controlador, Modelo)** más la **Capa de Persistencia / Base de Datos**, siguiendo rigurosamente las buenas prácticas de la ingeniería de software y el ciclo de refinamiento de la metodología ICONIX.




**Estructura y Responsabilidad de las Capas:**

1. **Capa 1: Vista (Presentación / Interfaz de Usuario):**
    - **Paquete Vistas Autenticación y Tenant:** Formularios y pantallas de inicio de sesión, recuperación de credenciales, selección de empresa activa (multitenant) y administración de perfiles de usuario.
    - **Paquete Vistas Ventas POS y Reservas:** Interfaces interactivas de punto de venta (POS), catálogo visual con matriz talla/color, carrito de compras, selección de métodos de pago (efectivo, QR, Stripe) y gestión de reservas.
    - **Paquete Vistas Catálogo, CPP y Compras:** Paneles de administración de prendas, categorización, registro de notas de compra a proveedores, recálculo de Costo Promedio Ponderado (CPP) y monitoreo de kardex.
    - **Paquete Vistas Reportes IA y Dashboard ML:** Dashboards gerenciales con gráficos en tiempo real, interfaz de consulta en lenguaje natural asistida por IA generativa y visualizador de recomendaciones predictivas de demanda.
2. **Capa 2: Controlador (Lógica de Aplicación y Orquestación):**
    - **Paquete Controladores Auth & Tenant:** Manejo de sesiones seguras mediante JSON Web Tokens (JWT), inyección de contexto de empresa (tenant_id), verificación de roles/permisos (RBAC) y filtrado de peticiones.
    - **Paquete Controladores Ventas y Sync Offline:** Coordinación de transacciones de venta, validación de políticas de reserva, procesamiento de webhooks de pasarela de pago y orquestación de la cola de sincronización offline con resolución de conflictos de versión.

- **Paquete Controladores Inventario, CPP y Kardex:** Ejecución de las reglas de negocio de abastecimiento: recálculo continuo del CPP por sucursal, emisión automática de asientos de movimiento en Kardex y validación de stock disponible antes de la confirmación de pedidos.
- **Paquete Controladores Reportes IA & ML:** Orquestación de pipelines de analítica: extracción de históricos de venta, preparación de datasets para inferencia de modelos Scikit-Learn y sanitización de prompts para generación de reportes con IA.

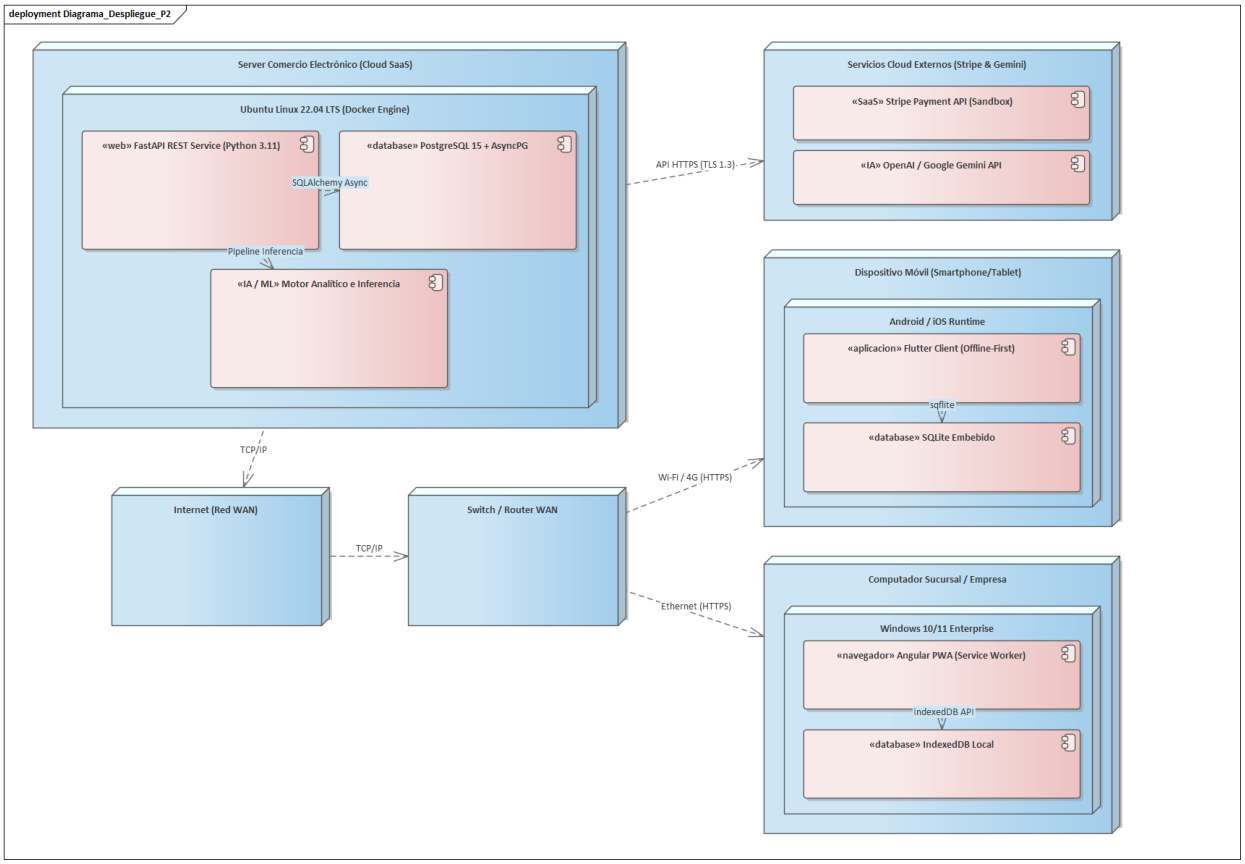
3. **Capa 3: Modelo (Dominio, Entidades y Reglas de Negocio):**
   - **Paquete Modelos Auth y Organización:** Entidades de dominio: Usuario, Rol, Permiso, Empresa, Sucursal, PlanSuscripcion.
   - **Paquete Modelos Ventas, Reservas y Sync:** Entidades: Venta, DetalleVenta, ReservaPrenda, Pago, Comprobante, SyncQueueEntry.
   - **Paquete Modelos Catálogo Textil e Inventario:** Entidades: Producto, Categoria, PrendaVariante, Talla, Color, MovimientoKardex, StockSucursal, NotaCompra.
   - **Paquete Modelos Analítica e Inferencia ML:** Entidades y DTOs de predicción: PrevisionDemanda, ReglaDescuento, SnapshotVentas, MetricaRotacion.
4. **Capa de Persistencia / Base de Datos:**
   - **Base de Datos Central PostgreSQL 15:** Persistencia física relacional en la nube. Almacena las tablas de todos los modelos bajo aislamiento multitenant, soportando transacciones ACID y consultas analíticas complejas.
   - **Base de Datos Local SQLite e IndexedDB:** Persistencia local de borde. Almacena réplicas operativas del catálogo y colas de transacciones locales pendientes de sincronización en dispositivos móviles (SQLite) y navegadores web (IndexedDB).

**Reglas de Dependencia y Acoplamiento:**

- **Flujo Estrictamente Descendente:** Las dependencias entre paquetes fluyen exclusivamente de arriba hacia abajo (Vista ..> Controlador ..> Modelo ..> Base de Datos).
- **Desacoplamiento de Presentación:** Los Controladores y Modelos no tienen conocimiento ni dependencia de las tecnologías de interfaz (Angular o Flutter).
- **Colaboración Inter-Controladores:** El Paquete Controladores Ventas y Sync Offline colabora puntualmente con el Paquete Controladores Inventario, CPP y Kardex para validar stock en tiempo real y debitar inventario sin violar el encapsulamiento de capas.

### 4.2.1. Diseño Físico de la Arquitectura: Diagrama de Despliegue (UML)

El diseño físico de la arquitectura modela la distribución de los artefactos de software sobre la infraestructura de hardware, servidores en la nube, redes de comunicación y dispositivos de borde (clientes móviles y computadores empresariales), alineado estrictamente con el modelo de despliegue físico requerido para el sistema.




**Descripción de Nodos y Componentes Físicos:**

1. **Server Comercio Electrónico (Cloud SaaS):**
    - **Nodo de Ejecución:** Servidor virtual en la nube bajo sistema operativo **Ubuntu Linux 22.04 LTS** gestionado con contenedores **Docker Engine**.
    - **Componente «web» FastAPI REST Service (Python 3.11):** Expone las APIs seguras bajo HTTPS/TLS 1.3, maneja la validación de tokens JWT multitenant, procesa las solicitudes transaccionales y coordina la lógica de negocio.
    - **Componente «database» PostgreSQL 15 + AsyncPG:** Instancia central de base de datos relacional para persistencia multi-empresa con aislamiento por tenant_id. Se comunica con el backend mediante SQLAlchemy Async.
    - **Componente «IA / ML» Motor Analítico e Inferencia:** Servicio interno encargado de ejecutar algoritmos de machine learning (previsión de demanda textil) y conectar con los modelos de IA generativa.
2. **Servicios Cloud Externos:**
    - **«SaaS» Stripe Payment API (Sandbox):** Pasarela de pagos externa que recibe peticiones tokenizadas desde el servidor backend para procesar transacciones bancarias y emite webhooks de liquidación.
    - **«IA» OpenAI / Google Gemini API:** Servicios de modelos de lenguaje natural consumidos mediante llamadas HTTPS autenticadas con API Key para el asistente de reportes generativos.

3. **Infraestructura de Red (WAN / LAN):**
    - **Internet (Red WAN):** Canal público sobre protocolo TCP/IP mediante el cual transitan las peticiones cifradas de los clientes remotos hacia el servidor en la nube.
    - **Switch / Router WAN:** Dispositivo de red física en sucursal que conmuta el tráfico local hacia internet y asigna direccionamiento a las terminales de punto de venta y dispositivos móviles.
4. **Dispositivo Móvil (Smartphone/Tablet):**
    - **Nodo de Dispositivo:** Terminal móvil operada por el personal de ventas o encargados de almacén bajo **Android / iOS Runtime**.
    - **Componente «aplicación» Flutter Client (Offline-First):** Aplicación cliente nativa compilada en Flutter/Dart. Permite registrar ventas, consultar catálogo y gestionar inventario incluso sin conectividad a internet.
    - **Componente «database» SQLite Embebido:** Base de datos local persistida en almacenamiento interno mediante sqflite, permitiendo encolar operaciones offline para sincronización posterior con el servidor central.
5. **Computador Sucursal / Empresa:**
    - **Nodo de Terminal:** Estación de trabajo de caja o gerencia bajo sistema operativo **Windows 10/11 Enterprise**.
    - **Componente «navegador» Angular PWA (Service Worker):** Aplicación web progresiva ejecutada en el navegador web moderno (Chrome/Edge). Los Service Workers almacenan en caché el bundle estático para arranque instantáneo sin conexión.
    - **Componente «database» IndexedDB Local:** Almacenamiento transaccional estructurado en el navegador, sincronizado con el Service Worker para registrar operaciones locales cuando la conexión de red se interrumpe.

## 4.3. Implementación de la base de datos

La persistencia del sistema está estructurada bajo una arquitectura híbrida de alta disponibilidad: persistencia central relacional multitenant en PostgreSQL 15 y persistencia local de borde (SQLite en la aplicación móvil Flutter e IndexedDB en la Progressive Web App de Angular).

### 4.3.1. Configuración de PostgreSQL 15 y Pool Asíncrono (SQLAlchemy 2.0 Asyncio + AsyncPG)

El acceso a la base de datos central en la nube se implementa de forma 100% asíncrona para maximizar el rendimiento concurrente bajo alta carga de transacciones POS:

```
# app/core/database.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy.orm import DeclarativeBase
from contextvars import ContextVar
import os

DATABASE_URL = os.getenv("DATABASE_URL",
"postgresql+asyncpg://postgres:secret@localhost:5432/fashionstore_db")

# Pool de conexiones asíncrono con reciclaje automático y pre-ping para evitar conexiones muertas
engine = create_async_engine(
```

```
    DATABASE_URL,
    echo=False,
    pool_size=20,
    max_overflow=10,
    pool_recycle=1800,
    pool_pre_ping=True
)
AsyncSessionLocal = async_sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# Variable de contexto asíncrona para aislamiento thread-safe y coroutine-safe del tenant
current_tenant_id: ContextVar[int | None] = ContextVar("current_tenant_id", default=None)

class Base(DeclarativeBase):
    pass

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close()
```

### 4.3.2. Aislamiento Multitenant en Persistencia

Para garantizar el aislamiento hermético de datos entre empresas suscriptores (Desafío VI), la persistencia implementa filtrado automático en consultas y eventos del ORM:

```
# app/core/multitenant.py
from sqlalchemy import event
from sqlalchemy.orm import Session
from app.core.database import current_tenant_id

@event.listens_for(Session, "do_orm_execute")
def receive_do_orm_execute(orm_execute_state):
    """Inyecta automáticamente el filtro tenant_id en todas las consultas SELECT, UPDATE y DELETE."""
    tenant_id = current_tenant_id.get()
    if tenant_id is not None and orm_execute_state.is_select:
        # Aplica filtrado automático sobre entidades que heredan TenantMixin
        orm_execute_state.statement = orm_execute_state.statement.filter_by(tenant_id=tenant_id)
```

### 4.3.3. Persistencia Local Móvil (SQLite + sqflite en Flutter)

La aplicación móvil incorpora una base de datos local SQLite administrada con el paquete sqflite, permitiendo operaciones de venta, reservas y catálogo completamente offline:

```
// lib/core/database/local_database.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class LocalDatabase {
  static final LocalDatabase instance = LocalDatabase._init();
  static Database? _database;
  LocalDatabase._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('fashionstore_offline.db');
    return _database!;
  }
```

```
  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
    return await openDatabase(path, version: 1, onCreate: _createDB);
  }

  Future _createDB(Database db, int version) async {
    // Tabla para encolar transacciones generadas sin conexión con UUID v4
    await db.execute('''
      CREATE TABLE operaciones_offline (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        uuid TEXT NOT NULL UNIQUE,
        sucursal_id INTEGER NOT NULL,
        tipo_operacion TEXT NOT NULL,
        payload_json TEXT NOT NULL,
        fecha_creacion TEXT NOT NULL,
        estado_sync TEXT NOT NULL DEFAULT 'PENDIENTE'
      )
    ''');
    // Tablas réplica para catálogo local offline
    await db.execute('''
      CREATE TABLE productos_cache (
        id INTEGER PRIMARY KEY,
        codigo_sku TEXT NOT NULL,
        nombre TEXT NOT NULL,
        precio_venta REAL NOT NULL,
        categoria_id INTEGER NOT NULL
      )
    ''');
    await db.execute('''
      CREATE TABLE variantes_cache (
        id INTEGER PRIMARY KEY,
        producto_id INTEGER NOT NULL,
        talla TEXT NOT NULL,
        color TEXT NOT NULL,
        stock_fisico INTEGER NOT NULL
      )
    ''');
  }
}
```

### 4.3.4. Persistencia Local Web (IndexedDB + Dexie.js en Angular PWA)

En el navegador del cliente de caja y sucursal, la Progressive Web App emplea **IndexedDB** estructurado mediante la librería reactiva **Dexie.js** para garantizar soporte offline transparente:

```
// src/app/core/services/offline-storage.service.ts
import Dexie, { Table } from 'dexie';

export interface OperacionOffline {
  id?: number;
  uuid: string;
  sucursalId: number;
  tipoOperacion: 'VENTA' | 'RESERVA';
  payloadJson: any;
  fechaCreacion: string;
  estadoSync: 'PENDIENTE' | 'CONFIRMADA' | 'RECHAZADA';
}
```

```
export class FashionStoreDexieDB extends Dexie {
  operacionesOffline!: Table<OperacionOffline, number>;
  productosCache!: Table<any, number>;

  constructor() {
    super('FashionStorePWA_DB');
    this.version(1).stores({
      operacionesOffline: '++id, &uuid, sucursalId, tipoOperacion, estadoSync',
      productosCache: 'id, codigoSku, nombre, categoriaId'
    });
  }
}
export const offlineDB = new FashionStoreDexieDB();
```

### 4.3.5. Control de Migraciones del Esquema Relacional (Alembic)

Para asegurar la reproducibilidad de despliegues y control de cambios en la base de datos central PostgreSQL, se implementa **Alembic** con soporte nativo para migraciones asíncronas:

```
# migrations/env.py
import asyncio
from sqlalchemy.ext.asyncio import create_async_engine
from alembic import context
from app.models import Base
import os

target_metadata = Base.metadata
config = context.config

def run_migrations_offline():
    url = os.getenv("DATABASE_URL")
    context.configure(url=url, target_metadata=target_metadata, literal_binds=True)
    with context.begin_transaction():
        context.run_migrations()

async def run_migrations_online():
    connectable = create_async_engine(os.getenv("DATABASE_URL"))
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)

def do_run_migrations(connection):
    context.configure(connection=connection, target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())
```

## 4.4. Desarrollo de los módulos del sistema

### 4.4.1. Módulo 1: Seguridad, Multitenancy y Administración Global (CU01 al CU05)

Este módulo gestiona el ciclo de vida de los tenants (empresas cliente), los planes de suscripción mensuales/anuales, la autenticación centralizada mediante tokens JWT con claims multitenant, y la

administración de usuarios y roles RBAC (Super Administrador, Administrador de Empresa, Encargado de Sucursal, Cajero POS y Cliente).

A. Matriz de Trazabilidad ICONIX (BCE → Código Fuente)

| Caso de Uso | Objeto Boundary (Frontera) | Objeto Control (Lógica) | Objeto Entity (Datos) | Archivo / Componente Implementado |
|---|---|---|---|---|
| **CU01: Gestionar Empresas** | EmpresaAdminBoundary | TenantManagementControl | EmpresaTenant, ConfiguracionTenant | modules/p02_estructura_operativa/empresas/ |
| **CU02: Gestionar Planes** | PlanesBoundary | PlanSubscriptionControl | PlanSuscripcion | modules/p01_seguridad_acceso/planes/ |
| **CU03: Gestionar Suscripción** | SuscripcionBoundary | SubscriptionBillingControl | SuscripcionEmpresa, HistorialPago | modules/p01_seguridad_acceso/suscripciones/ |
| **CU04: Autenticar Multitenant** | LoginBoundary | AuthSecurityControl | Usuario, Rol, Sesion Activa | core/security.py, modules/p01_seguridad_acceso/auth/ |
| **CU05: Gestionar Usuarios/Roles** | UsuariosBoundary | UserRoleRBACControl | Usuario, Rol, Permiso | modules/p01_seguridad_acceso/usuarios/ |

B. Implementación de la Lógica de Negocio Crítica: Middleware de Aislamiento Multitenant con ContextVar

Garantiza que ninguna petición procese, consulte o altere registros de una empresa distinta a la que pertenece el usuario autenticado, interceptando las cabeceras HTTP y estableciendo el tenant_id en una variable de contexto asíncrona segura:

```
# app/core/security.py
from fastapi import Request, HTTPException, Security, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt
from app.core.database import current_tenant_id
import os

SECRET_KEY = os.getenv("JWT_SECRET", "super-secret-key-saas-fashionstore")
ALGORITHM = "HS256"
security = HTTPBearer()
```

```
async def get_current_user_tenant(credentials: HTTPAuthorizationCredentials = Security(security)) -> dict:
    """
    Controlador de Seguridad ICONIX: Valida JWT, extrae claims de tenant/rol,
    verifica estado activo de suscripción y fija el tenant en ContextVar.
    """
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        tenant_id: int = payload.get("tenant_id")
        user_id: int = payload.get("user_id")
        rol: str = payload.get("rol")
        suscripcion_activa: bool = payload.get("suscripcion_activa", False)

        if tenant_id is None:
            raise HTTPException(status_code=403, detail="Token no contiene tenant_id válido")
        if not suscripcion_activa and rol != "SUPER_ADMIN":
            raise HTTPException(status_code=403, detail="La suscripción de la empresa está vencida o
suspendida")

        # Establecer el tenant en la variable de contexto asíncrona para aislamiento automático
        current_tenant_id.set(tenant_id)
        return {"user_id": user_id, "tenant_id": tenant_id, "rol": rol}
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Token inválido o expirado")
```

### 4.4.2. Módulo 2: Catálogo Textil e Inventario Multi-Sucursal con CPP y Kardex (CU06, CU07, **CU14, CU20)**

Administra la estructura de prendas de vestir con atributos avanzados (talla, color, temporada, tipo de tela, composición textil), el stock distribuido entre sucursales y almacén central, los traspasos inter-sucursales y la valuación de inventarios mediante el método matemático de **Costo Promedio Ponderado (CPP)** con asientos inmutables en el Kardex físico-valorado.

A. Matriz de Trazabilidad ICONIX (BCE → Código Fuente)

| Caso de Uso | Objeto Boundary (Frontera) | Objeto Control (Lógica) | Objeto Entity (Datos) | Archivo / Componente Implementado |
|---|---|---|---|---|
| **CU06: Gestionar Prendas** | PrendasBoundary | CatalogoPrendasControl | Producto, Categoria, VarianntePrenda | modules/p03_catalogo_estilismo_ia/productos/ |
| **CU07: Inventario y CPP** | Inventario Boundary | InventarioControl | Inventario, KardexMovimiento, LoteCompra | modules/p05_inventario_costos_analitica/inventario/ |

| | | | | |
|---|---|---|---|---|
| **CU14: Traspaso Inter-Sucursal** | TraspasoBoundary | LogisticaTraspasoControl | TraspasoSucursal, DetalleTraspaso | modules/p10_logistica_delivery/traspasos/ |
| **CU20: Compras Desktop** | ComprasBoundary | AprovisionamientoControl | OrdenCompra, DetalleOrdenCompra, Proveedor | modules/p04_aprovisionamiento_proveedores/ |

***B. Implementación de la Lógica de Negocio Crítica: Cálculo de Costo Promedio Ponderado*** (CPP) y Asiento en Kardex

Implementa la fórmula formal del CPP en cada recepción de mercadería:

$$CPP_{nuevo} = (Stock_{anterior} \times CPP_{anterior}) + (Cantidad_{lote} \times Costo_{lote}) Stock_{anterior} + Cantidad_{lote} CPP_{nuevo} = Stock_{anterior} + Cantidad_{lote} (Stock_{anterior} \times CPP_{anterior}) + (Cantidad_{lote} \times Costo_{lote})$$

```
# app/modules/p05_inventario_costos_analitica/inventario/service.py
from decimal import Decimal, ROUND_HALF_UP
from fastapi import HTTPException, status
from sqlalchemy.orm import Session
from datetime import datetime
from app.modules.p05_inventario_costos_analitica.inventario.models import Inventario, KardexMovimiento
from app.modules.p02_estructura_operativa.sucursales.models import Sucursal
from app.modules.p03_catalogo_estilismo_ia.productos.models import Producto
from     app.modules.p05_inventario_costos_analitica.inventario.schemas     import     EntradaLoteCreate,
KardexResponse

class InventarioControl:
    """
    Clase de Control ICONIX: Lógica de Negocio de Inventario y Valuación por CPP.
    Coordina la entrada física de mercadería, cálculo matemático del CPP
    y generación de asientos inmutables en el Kardex físico-valorado.
    """


    @staticmethod
    def procesar_entrada_mercaderia(db: Session, request: EntradaLoteCreate) -> KardexResponse:
        # 1. Validar existencia de sucursal y producto
        sucursal = db.query(Sucursal).filter(Sucursal.id_sucursal == request.id_sucursal).first()
        if not sucursal:
            raise HTTPException(status_code=404, detail="Sucursal no existe.")

        producto = db.query(Producto).filter(Producto.id_producto == request.id_producto).first()
        if not producto:
            raise HTTPException(status_code=404, detail="Producto no existe en el catálogo.")

        cant_lote = request.cantidad_recibida
        costo_lote = Decimal(str(request.costo_unitario_compra))
        if cant_lote <= 0 or costo_lote <= Decimal("0.00"):
            raise HTTPException(status_code=400, detail="Cantidad y costo unitario deben ser mayores a
cero.")
```

```
        talla_norm = request.talla.strip().upper()
        color_norm = request.color.strip()

        # 2. Obtener existencias y CPP actual del ítem
        item_inv = db.query(Inventario).filter(
            Inventario.id_sucursal == request.id_sucursal,
            Inventario.id_producto == request.id_producto,
            Inventario.talla == talla_norm,
            Inventario.color == color_norm
        ).first()

        if item_inv:
            stock_anterior = item_inv.stock_fisico
            cpp_anterior = Decimal(str(item_inv.costo_promedio_ponderado))

            # 3. Aplicar fórmula formal de Costo Promedio Ponderado (CPP)
            inversion_anterior = Decimal(str(stock_anterior)) * cpp_anterior
            inversion_nueva = Decimal(str(cant_lote)) * costo_lote
            nuevo_stock_total = stock_anterior + cant_lote

            nuevo_cpp_raw = (inversion_anterior + inversion_nueva) / Decimal(str(nuevo_stock_total))
            nuevo_cpp = nuevo_cpp_raw.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

            # 4. Actualizar stock y costos en la entidad Inventario
            item_inv.stock_fisico = nuevo_stock_total
            item_inv.stock_disponible = nuevo_stock_total - item_inv.stock_reservado
            item_inv.ultimo_costo_compra = costo_lote
            item_inv.costo_promedio_ponderado = nuevo_cpp
        else:
            # Creación inicial de existencias para este SKU
            stock_anterior = 0
            cpp_anterior = Decimal("0.00")
            nuevo_stock_total = cant_lote
            nuevo_cpp = costo_lote.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

            item_inv = Inventario(
                id_sucursal=request.id_sucursal, id_producto=request.id_producto,
                talla=talla_norm, color=color_norm, stock_fisico=cant_lote,
                stock_disponible=cant_lote, stock_reservado=0,
                ultimo_costo_compra=costo_lote, costo_promedio_ponderado=nuevo_cpp
            )
            db.add(item_inv)

        # 5. Generar asiento inmutable en el Kardex físico-valorado
        nuevo_saldo_valorado  =  (Decimal(str(nuevo_stock_total))  *  nuevo_cpp).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP)
        asiento_kardex = KardexMovimiento(
            id_sucursal=request.id_sucursal, id_producto=request.id_producto,
            talla=talla_norm, color=color_norm, tipo_movimiento="ENTRADA_COMPRA",
            cantidad=cant_lote, costo_unitario_movimiento=costo_lote,
            costo_total_movimiento=(Decimal(str(cant_lote))     *     costo_lote).quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP),
            saldo_cantidad=nuevo_stock_total, costo_unitario_ponderado_resultante=nuevo_cpp,
            saldo_valorado_total=nuevo_saldo_valorado, referencia_documento=request.numero_factura,
            fecha_movimiento=datetime.utcnow()
        )
        db.add(asiento_kardex)
        db.commit()
        db.refresh(asiento_kardex)
```

```
    return asiento_kardex
```

### 4.4.3. Módulo 3: Operaciones Offline, Ventas POS y Motor de Sincronización (CU08 al CU12)

Provee a los cajeros y clientes en tiendas físicas la capacidad de consultar catálogos textiles, registrar reservas presenciales y concretar ventas POS de contado en efectivo de forma 100% autónoma ante cortes de conectividad a internet. Al restablecerse la red, el motor de sincronización asienta las transacciones garantizando **idempotencia absoluta mediante UUID** y resolución de conflictos de stock.

**A. Matriz de Trazabilidad ICONIX (BCE → Código Fuente)**

| Caso de Uso | Objeto Boundary (Frontera) | Objeto Control (Lógica) | Objeto Entity (Datos) | Archivo / Componente Implementado |
|---|---|---|---|---|
| **CU08: Catálogo Offline** | CatalogoOfflineBoundary | OfflineCacheManager | CatalogoLocal (SQLite/IndexedDB) | mobile/lib/views/catalogo_offline.dart |
| **CU09: Reserva Offline** | ReservaOfflineBoundary | ReservaLocalControl | ReservaLocal (SQLite) | mobile/lib/views/reserva_offline.dart |
| **CU10: Venta POS Offline** | POSOfflineBoundary | POSLocalSalesControl | VentaLocal, DetalleVentaLocal | mobile/lib/views/pos_offline.dart |
| **CU11: Auditar Pendientes** | AuditoriaColaBoundary | OfflineQueueControl | OperacionPendienteSync | mobile/lib/views/cola_sync.dart |
| **CU12: Sincronizar Operaciones** | SyncClientBoundary | OfflineSyncEngine | OperacionOfflineSync, Venta | backend/app/services/sync_service.py |

**B. Implementación de la Lógica de Negocio Crítica: Motor de Sincronización Idempotente con UUID**

```
# app/services/sync_service.py
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, update
from pydantic import BaseModel
from uuid import UUID
from app.core.database import get_db, current_tenant_id
from app.core.security import get_current_user_tenant
from app.models import OperacionOfflineSync, InventarioSucursal, Venta, DetalleVenta

router = APIRouter(prefix="/api/sync", tags=["Sincronización Offline"])
```

```
class OperacionSyncPayload(BaseModel):
    uuid: UUID
    sucursal_id: int
    tipo_operacion: str # 'VENTA' o 'RESERVA'
    datos: dict

@router.post("/operacion")
async def sincronizar_operacion(payload: OperacionSyncPayload, db: AsyncSession = Depends(get_db), auth:
dict = Depends(get_current_user_tenant)):
    tenant_id = auth["tenant_id"]

    # 1. Regla de Idempotencia: Verificar si el UUID ya fue procesado previamente
    stmt_check = select(OperacionOfflineSync).where(OperacionOfflineSync.uuid == payload.uuid)
    res_check = await db.execute(stmt_check)
    op_existente = res_check.scalar_one_or_none()

    if op_existente:
        return {"uuid": str(payload.uuid), "estado": op_existente.estado_sync, "mensaje": "Transacción
idempotente ya registrada"}

    # 2. Procesar Venta Offline de Contado
    if payload.tipo_operacion == "VENTA":
        items = payload.datos.get("items", [])
        total = payload.datos.get("total", 0.0)

        # Validar existencia de stock físico real en la sucursal
        for item in items:
            stmt_inv = select(InventarioSucursal).where(
                InventarioSucursal.tenant_id == tenant_id,
                InventarioSucursal.sucursal_id == payload.sucursal_id,
                InventarioSucursal.variante_id == item["variante_id"]
            )
            inv = (await db.execute(stmt_inv)).scalar_one_or_none()
            if not inv or inv.stock_fisico < item["cantidad"]:
                # Conflicto irresoluble: Stock insuficiente -> Asentar transacción rechazada
                op_rechazada = OperacionOfflineSync(
                    uuid=payload.uuid, tenant_id=tenant_id, sucursal_id=payload.sucursal_id,
                    tipo_operacion="VENTA", payload_json=payload.datos, estado_sync="RECHAZADA",
                    fecha_creacion_local=payload.datos.get("fecha_local"),             error_motivo="Stock
insuficiente al sincronizar"
                )
                db.add(op_rechazada)
                await db.commit()
                return {"uuid": str(payload.uuid), "estado": "RECHAZADA", "motivo": "Stock insuficiente"}

        # Descontar stock confirmado en sucursal
        for item in items:
            await db.execute(
                update(InventarioSucursal).where(
                    InventarioSucursal.tenant_id == tenant_id,
                    InventarioSucursal.sucursal_id == payload.sucursal_id,
                    InventarioSucursal.variante_id == item["variante_id"]
                ).values(stock_fisico=InventarioSucursal.stock_fisico - item["cantidad"])
            )

        nueva_venta = Venta(
            tenant_id=tenant_id, sucursal_id=payload.sucursal_id, usuario_cajero_id=auth["user_id"],
            numero_factura=f"OFF-{str(payload.uuid)[:8]}", total=total, subtotal=total,
            tipo_pago="EFECTIVO", estado="COMPLETADA", estado_sync="SINCRONIZADA", uuid_origen=payload.uuid
```

```
        )
        db.add(nueva_venta)
        await db.flush()

        for item in items:
            db.add(DetalleVenta(venta_id=nueva_venta.id,                  variante_id=item["variante_id"],
cantidad=item["cantidad"], precio_unitario=item["precio_unitario"]))

        op_confirmada = OperacionOfflineSync(
            uuid=payload.uuid, tenant_id=tenant_id, sucursal_id=payload.sucursal_id,
            tipo_operacion="VENTA", payload_json=payload.datos, estado_sync="CONFIRMADA",
            fecha_creacion_local=payload.datos.get("fecha_local")
        )
        db.add(op_confirmada)
        await db.commit()
        return {"uuid": str(payload.uuid), "estado": "CONFIRMADA", "venta_id": nueva_venta.id}
```

### 4.4.4. Módulo 4: Ventas, Gestión de Cambio de Prenda en Dos Tiempos y Pasarela Stripe (CU13, CU18, CU19)

Implementa el protocolo de postventa exigido por la cátedra para el cambio de prendas por talla, color o modelo dentro del plazo de 7 días hábiles, estructurado en **dos tiempos documentales estrictos**: recepción con emisión de **Nota de Devolución** (reingreso al stock), y entrega con emisión de **Nota de Cambio** y cobro del saldo diferencial mediante **Stripe Sandbox**.

**A. Matriz de Trazabilidad ICONIX (BCE → Código Fuente)**

| Caso de Uso | Objeto Boundary (Frontera) | Objeto Control (Lógica) | Objeto Entity (Datos) | Archivo / Componente Implementado |
|---|---|---|---|---|
| **CU13: Cambio de Prenda** | CambioPrendaBoundary | CambioTwoPhaseManager | CambioPrenda, NotaDevolucion, NotaCambio | modules/p08_punto_venta_pos/cambios/ |
| **CU18: Pasarela Stripe** | StripeCheckoutBoundary | PaymentGatewayControl | TransaccionPago, MetodoPagoCliente | modules/p09_procesamiento_pagos/stripe/ |
| **CU19: Liquidación/Mayor** | OfertasBoundary | PricingStrategyControl | ReglaDescuento, OfertaLiquidacion | modules/p07_venta_digital_fidelizacion/ofertas/ |

**B. Implementación de la Lógica de Negocio Crítica: Flujo de Cambio en Dos Tiempos y Cobro de Saldo Diferencial**

```
# app/services/cambio_service.py
from datetime import datetime, timedelta
from fastapi import HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, update
from app.models import Venta, CambioPrenda, NotaDevolucion, NotaCambio, InventarioSucursal
import stripe, os

stripe.api_key = os.getenv("STRIPE_SECRET_KEY", "sk_test_mock")
```

```
async def procesar_devolucion_tiempo1(venta_id: int, variante_dev_id: int, sucursal_id: int, db:
AsyncSession, tenant_id: int):
    """
    TIEMPO 1: Validar plazo de 7 días hábiles, registrar recepción física,
    reingresar mercadería al inventario (+1) y emitir Nota de Devolución.
    """
    stmt_v = select(Venta).where(Venta.id == venta_id, Venta.tenant_id == tenant_id)
    venta = (await db.execute(stmt_v)).scalar_one_or_none()
    if not venta or (datetime.utcnow() - venta.fecha).days > 7:
        raise HTTPException(status_code=400, detail="Plazo de cambio de 7 días hábiles vencido")

    cambio = CambioPrenda(
        tenant_id=tenant_id, venta_origen_id=venta_id, cliente_id=venta.cliente_id,
        plazo_limite=datetime.utcnow() + timedelta(days=7), motivo="Cambio de talla",
        estado="PRENDA_RECEPCIONADA"
    )
    db.add(cambio)
    await db.flush()

    nota_dev = NotaDevolucion(
        cambio_id=cambio.id, sucursal_id=sucursal_id, variante_devuelta_id=variante_dev_id,
        cantidad=1, precio_original_reconocido=50.00
    )
    db.add(nota_dev)

    # Reingreso físico al inventario de la sucursal
    await db.execute(
        update(InventarioSucursal).where(
            InventarioSucursal.sucursal_id == sucursal_id,
            InventarioSucursal.variante_id == variante_dev_id
        ).values(stock_fisico=InventarioSucursal.stock_fisico + 1)
    )
    await db.commit()
    return {"cambio_id": cambio.id, "saldo_a_favor": 50.00, "mensaje": "Nota de devolución emitida. Prenda
reingresada al inventario."}

async def procesar_despacho_tiempo2(cambio_id: int, variante_nueva_id: int, precio_nuevo: float,
stripe_payment_token: str, db: AsyncSession, tenant_id: int):
    """
    TIEMPO 2: Cobrar saldo diferencial mediante pasarela Stripe si la prenda nueva es de mayor valor,
    descontar inventario (-1) y emitir Nota de Cambio final.
    """
    diferencia = precio_nuevo - 50.00 # Ejemplo: 70 Bs (nueva) - 50 Bs (favor) = 20 Bs saldo a pagar
    if diferencia > 0:
        charge = stripe.PaymentIntent.create(
            amount=int(diferencia * 100), currency="bob",
            payment_method=stripe_payment_token, confirm=True
        )
        if charge.status != "succeeded":
            raise HTTPException(status_code=402, detail="Pago de saldo rechazado por la pasarela bancaria")

    nota_cam = NotaCambio(
        cambio_id=cambio_id, sucursal_id=1, variante_nueva_id=variante_nueva_id,
        cantidad=1, precio_nueva_prenda=precio_nuevo
    )
    db.add(nota_cam)

    # Despacho físico de la nueva prenda
    await db.execute(
```

```
        update(InventarioSucursal).where(
            InventarioSucursal.variante_id == variante_nueva_id
        ).values(stock_fisico=InventarioSucursal.stock_fisico - 1)
    )
    await db.execute(
        update(CambioPrenda).where(
            CambioPrenda.id == cambio_id
        ).values(estado="COMPLETADO", saldo_diferencial=diferencia)
    )
    await db.commit()
    return {"status": "COMPLETADO", "nota_cambio_id": nota_cam.id, "diferencia_pagada": diferencia}
```

### 4.4.5. Módulo 5: Inteligencia Artificial Generativa y Analítica Predictiva con Machine Learning (CU15 al CU17)

Integra modelos de lenguaje natural (OpenAI GPT-3.5 / Google Gemini) para la generación de reportes gerenciales dinámicos mediante conversión de consultas en lenguaje natural a código SQL analítico seguro, así como modelos matemáticos de Machine Learning con **Scikit-Learn** para previsión de demanda textil (supervisado) y clustering de prendas para liquidación (no supervisado).

**A. Matriz de Trazabilidad ICONIX (BCE → Código Fuente)**

| Caso de Uso | Objeto Boundary (Frontera) | Objeto Control (Lógica) | Objeto Entity (Datos) | Archivo / Componente Implementado |
|---|---|---|---|---|
| **CU15: Reportes IA** | AIReporterBoundary | AIReporterService | ConsultaAnalitica, ReporteLog | backend/app/services/ai_report_service.py |
| **CU16: Exportar Reportes** | ExportReportBoundary | ReportExportControl | PlantillaExcel, DocumentoPDF | backend/app/services/export_service.py |
| **CU17: Dashboard ML** | MLDashboardBoundary | MLAnalyticsService | ModeloPrediccion, ClusterPrenda | backend/app/services/ml_analytics_service.py |

**B. Implementación de la Lógica de Negocio Crítica: Text-to-SQL con Restricción de Tenant y Modelos Scikit-Learn**

```
# app/services/ai_report_service.py
from fastapi import HTTPException
from sqlalchemy.ext.asyncio import AsyncSession
import openai, json

async def generar_reporte_ia(prompt_usuario: str, tenant_id: int, db: AsyncSession):
    """
    Controlador de IA Generativa: Transforma lenguaje natural a SQL SELECT PostgreSQL 15,
    inyectando de forma obligatoria el filtro WHERE tenant_id = :tenant_id.
    """
    system_prompt = f"""
    Eres un asistente analítico de FashionStore SaaS. Genera exclusivamente consultas SQL SELECT para
PostgreSQL 15.
```

```
    Es obligatorio e innegociable restringir cada consulta con `WHERE tenant_id = {tenant_id}`.
    Tablas disponibles: ventas (id, tenant_id, sucursal_id, total, fecha), detalle_ventas (venta_id,
variante_id, cantidad, precio_unitario, subtotal),
    sucursales (id, tenant_id, nombre), productos (id, tenant_id, nombre, categoria_id), inventario_sucursal
(tenant_id, sucursal_id, variante_id, stock_fisico).
    Responde estrictamente en formato JSON: {{"sql": "...", "tipo_grafico": "barras|lineas|pastel|tabla",
"titulo": "..."}}
    """
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo-1106",
        messages=[{"role":   "system",   "content":   system_prompt},   {"role":   "user",   "content":
prompt_usuario}],
        response_format={"type": "json_object"}
    )
    data = json.loads(response.choices[0].message.content)
    sql_query = data["sql"]

    # Validación de seguridad: Solo consultas SELECT y verificación de aislamiento multitenant
    if not sql_query.strip().upper().startswith("SELECT") or f"tenant_id = {tenant_id}" not in sql_query:
        raise HTTPException(status_code=403, detail="Consulta SQL rechazada por violar el aislamiento de
empresa")

    result = await db.execute(sql_query)
    filas = [dict(row._mapping) for row in result.fetchall()]
    return  {"titulo":  data["titulo"],  "tipo_grafico":  data["tipo_grafico"],  "datos":  filas,  "sql":
sql_query}
# app/services/ml_analytics_service.py
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.models import DetalleVenta, Venta

async def predecir_demanda_estacional(variante_id: int, mes_objetivo: int, db: AsyncSession, tenant_id:
int):
    """Modelo Supervisado: Random Forest Regressor para predecir unidades demandadas por SKU para el próximo
mes."""
    stmt = (
        select(Venta.fecha, DetalleVenta.cantidad)
        .join(DetalleVenta, Venta.id == DetalleVenta.venta_id)
        .where(Venta.tenant_id == tenant_id, DetalleVenta.variante_id == variante_id)
    )
    records = (await db.execute(stmt)).all()

    if len(records) < 5:
        return {"variante_id": variante_id, "demanda_estimada": 10, "confianza": "BAJA_DATOS_SINTETICOS"}

    df = pd.DataFrame([{"mes": r.fecha.month, "cantidad": r.cantidad} for r in records])
    X = df[["mes"]].values
    y = df["cantidad"].values

    model = RandomForestRegressor(n_estimators=50, random_state=42)
    model.fit(X, y)

    prediccion = model.predict(np.array([[mes_objetivo]]))
    return {
        "variante_id": variante_id,
```

```
        "mes_objetivo": mes_objetivo,
        "demanda_estimada_unidades": int(np.round(prediccion[0])),
        "confianza": "ALTA_MODELO_ENTRENADO"
    }

async def recomendar_ofertas_liquidacion_clustering(db: AsyncSession, tenant_id: int):
    """Modelo No Supervisado: K-Means para segmentar prendas de baja rotación y recomendar descuentos (30%,
40%, 60%)."""
    # Matriz de features por SKU: [dias_en_stock, total_unidades_vendidas, stock_remanente]
    datos_prendas = np.array([
        [180, 2, 25],  # Alta permanencia, bajas ventas -> Cluster 0: Liquidación agresiva (60%)
        [90, 15, 10],  # Rotación media -> Cluster 1: Descuento moderado (30%)
        [30, 45, 5],   # Alta rotación -> Cluster 2: Precio regular (0%)
        [150, 4, 18],  # Muy baja rotación -> Cluster 0: Liquidación (40%)
    ])

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(datos_prendas)

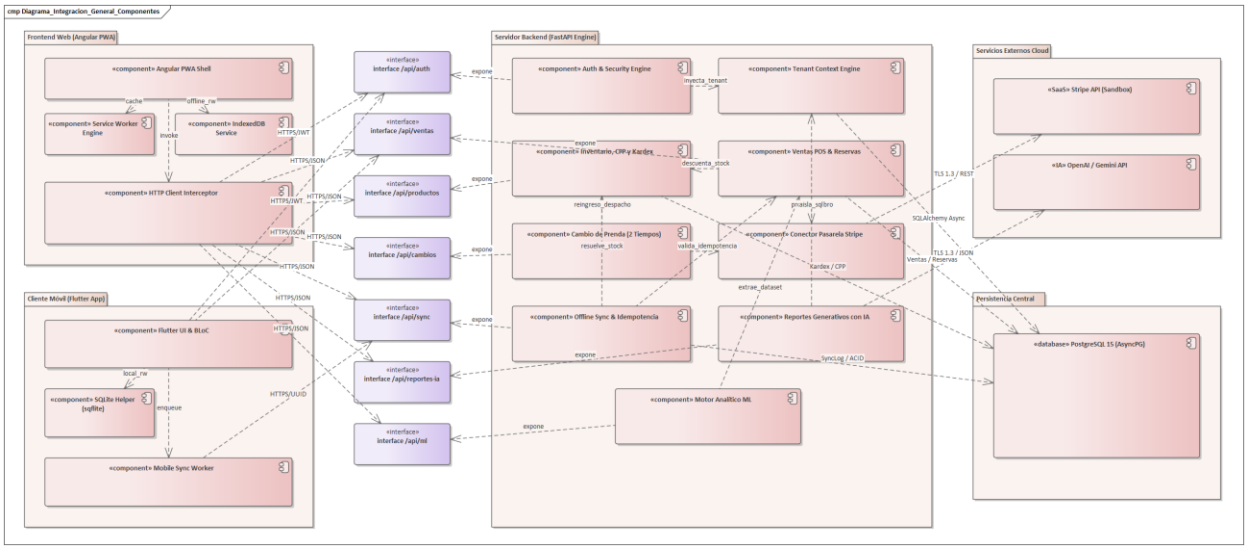
    kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
    clusters = kmeans.fit_predict(X_scaled)

    reglas_descuento = {0: 60.0, 1: 30.0, 2: 0.0}
    recomendaciones = []
    for idx, cluster_id in enumerate(clusters):
        recomendaciones.append({
            "prenda_idx": idx,
            "cluster_asignado": int(cluster_id),
            "descuento_sugerido_pct": reglas_descuento.get(cluster_id, 40.0)
        })
    return recomendaciones
```

## 4.5. Integración de componentes

### 4.5.1. Diagrama de Integración General de Componentes del Sistema (UML 2.5+)

El **Diagrama de Integración General de Componentes** modela la integración sistémica completa de FashionStore SaaS, articulando la interacción entre el cliente web (Angular PWA), el cliente móvil (Flutter App), los micro-servicios del backend (FastAPI), las interfaces REST expuestas, la base de datos relacional central (PostgreSQL 15) y los servicios cloud externos (Stripe API y OpenAI / Gemini API):




**Descripción de la Integración Sistémica:**

1. **Clientes de Presentación e Interfaces de Entrada:**
    - Tanto la **PWA Angular** como la **App Móvil Flutter** consumen los servicios del servidor a través del catálogo estandarizado de interfaces REST expuestas bajo HTTPS y cifrado TLS 1.3.
    - Cada solicitud HTTP viaja con el token JWT de autenticación y los encabezados multitenant (X-Tenant-ID), permitiendo al backend identificar de forma unívoca a qué empresa corresponde la transacción.
2. **Catálogo de Interfaces Expuestas por el Backend:**
    - **interface /api/auth:** Autenticación de usuarios, emisión y renovación de tokens JWT, recuperación de credenciales y cambio de contraseña.
    - **interface /api/ventas:** Registro de órdenes de compra, emisión de facturas/recibos, cobros directos y consulta histórica de ventas.
    - **interface /api/productos:** Consulta de catálogo público y privado con matriz de variantes (tallas, colores, stock por sucursal) y cálculo de CPP.
    - **interface /api/cambios:** Gestión del cambio de prenda en dos tiempos (recepción/reingreso de la prenda usada y despacho de la nueva variante).
    - **interface /api/sync:** Punto de recepción para la cola de transacciones offline con soporte de idempotencia por UUID y validación de stock central.
    - **interface /api/reportes-ia:** Procesamiento de consultas analíticas formuladas en lenguaje natural (texto o voz), generación segura de SQL y respuesta estructurada.
    - **interface /api/ml:** Endpoint de inferencia predictiva para forecasting de demanda por temporadas y sugerencias automáticas de precios de liquidación.
3. **Colaboración Interna del Núcleo Backend (FastAPI):**
    - El componente Auth & Security Engine inyecta el identificador de tenant en el Tenant Context Engine utilizando variables de contexto asíncronas (ContextVar).

- El componente Ventas POS & Reservas colabora con Inventario, CPP y Kardex para verificar disponibilidad en tiempo real antes de debitar unidades físicas.
- El componente Offline Sync & Idempotencia resuelve de manera determinista los conflictos entre transacciones locales y transacciones concurrentes en la nube.
- El componente Cambio de Prenda (2 Tiempos) orquesta el reingreso físico de la prenda devuelta (stock + 1) y coordina con Conector Pasarela Stripe el cobro de la diferencia de precio si la nueva prenda tiene mayor valor.

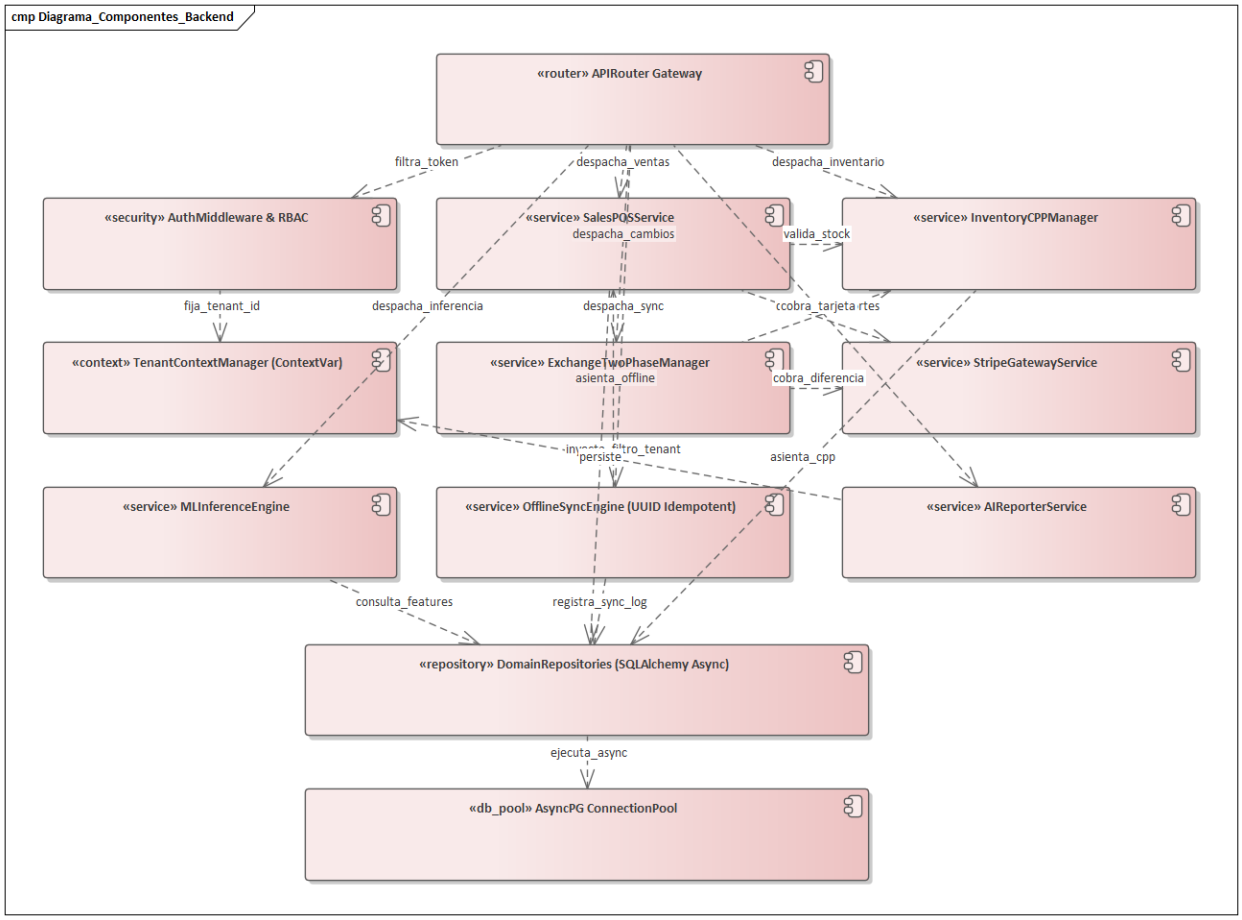
4. **Integración con Servicios Cloud Externos y Base de Datos:**
   - **Stripe API:** El conector de pasarela interactúa mediante HTTPS con los servidores de Stripe en modo Sandbox para validar tarjetas y procesar PaymentIntents.
   - **OpenAI / Gemini API:** El motor de reportes IA envía prompts del sistema estrictamente parametrizados para obtener sentencias SQL de solo lectura (SELECT).
   - **PostgreSQL 15:** Persistencia central transaccional conectada mediante un pool asíncrono de conexiones AsyncPG, garantizando transacciones ACID con aislamiento de esquemas multitenant.

### 4.5.2. Diagramas de Componentes por Paquetes y Subsistemas (UML 2.5+)

Para complementar el diseño de integración global, a continuación se descompone la arquitectura en **diagramas de componentes específicos para cada uno de los paquetes y subsistemas ejecutables del software**, modelando sus módulos internos, dependencias directas, puertos y servicios desacoplados:

#### *4.5.2.1. Paquete Backend (FastAPI Core y Servicios Internos)*

Modela la descomposición modular del servidor FastAPI en la nube, mostrando los routers de entrada, middlewares de seguridad, servicios de negocio, orquestadores de IA/ML y la capa de acceso a datos asíncrona:



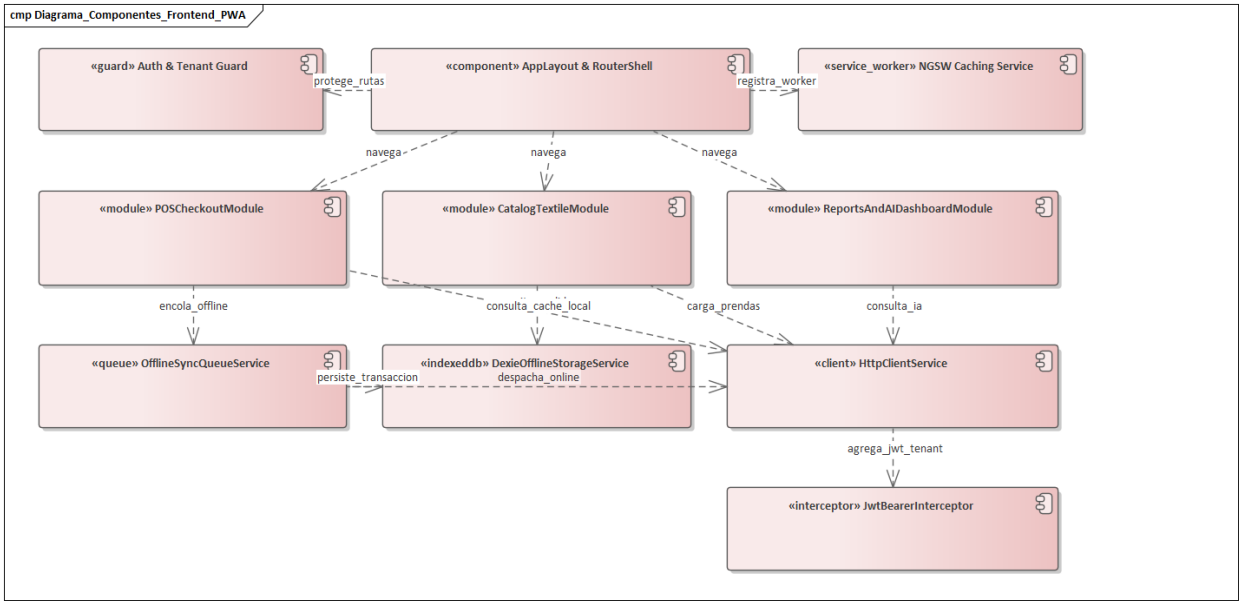

**Componentes del Backend:**

1. **«router» APIRouter Gateway:** Puerta de entrada HTTP/JSON que enruta cada petición hacia su controlador/servicio correspondiente.
2. **«security» AuthMiddleware & RBAC:** Valida los tokens JWT, extrae roles, permisos y verifica el estado de la suscripción de la empresa.
3. **«context» TenantContextManager (ContextVar):** Variable de contexto asíncrona que almacena de forma aislada y segura el tenant_id para garantizar que ninguna consulta acceda a datos de otra empresa.
4. **«service» SalesPOSService:** Orquesta el ciclo de vida de la venta presencial y en línea, cálculo de totales, descuentos y despacho de comprobantes.
5. **«service» InventoryCPPManager:** Administra el stock multivariante (talla/color), calcula el Costo Promedio Ponderado (CPP) en cada compra y emite asientos automáticos en el Kardex físico-valorado.
6. **«service» ExchangeTwoPhaseManager:** Implementa el flujo de cambio de prenda en dos tiempos (recepción/reingreso con Nota de Devolución y despacho con Nota de Cambio y cobro de diferencia).
7. **«service» OfflineSyncEngine (UUID Idempotent):** Recibe transacciones generadas sin conexión, verifica idempotencia por UUID, resuelve conflictos de stock y asienta ventas offline.

8. **«service» AIReporterService:** Sanitiza los requerimientos en lenguaje natural, añade el filtro de seguridad multitenant mandatorio (WHERE tenant_id = :id) y obtiene insights analíticos.
9. **«service» MLInferenceEngine:** Carga los pipelines de Scikit-Learn para previsión de demanda textil y sugerencia de descuentos de liquidación.
10. **«service» StripeGatewayService:** Gestiona la comunicación segura TLS 1.3 con Stripe API para tokenización de tarjetas y confirmación de webhooks.
11. **«repository» DomainRepositories (SQLAlchemy Async):** Capa de abstracción relacional mediante repositorio por entidad, interactuando con SQLAlchemy 2.0 Asyncio.
12. **«db_pool» AsyncPG ConnectionPool:** Pool de conexiones asíncronas de alto rendimiento optimizado para PostgreSQL 15.

#### *4.5.2.2. Paquete Frontend Web (Angular PWA y Persistencia Offline)*

Modela los componentes de la aplicación web progresiva (PWA), diseñada para ejecutarse en el navegador con soporte de caché offline y almacenamiento transaccional local:



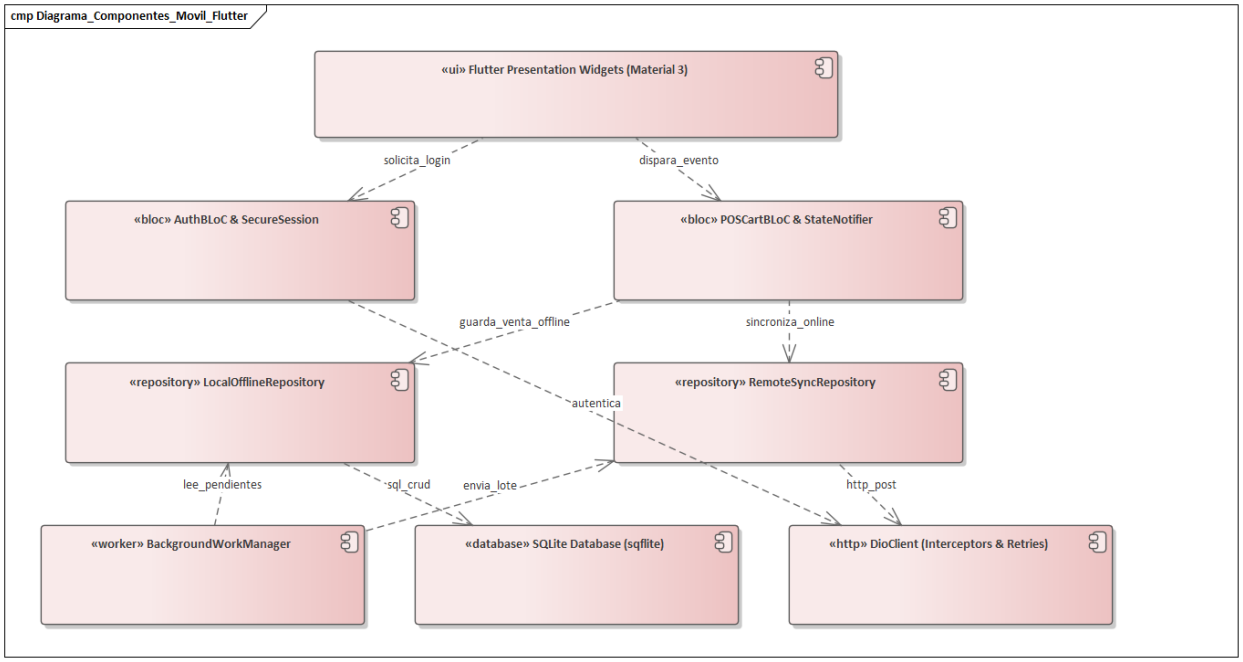

**Componentes de la PWA Angular:**

1. **«component» AppLayout & RouterShell:** Shell de presentación modular con sistema de navegación, barras de herramientas y carga diferida (*lazy loading*).
2. **«guard» Auth & Tenant Guard:** Protege rutas privadas, verifica la caducidad del token y redirecciona a login o selector de empresa.
3. **«interceptor» JwtBearerInterceptor:** Intercepta cada solicitud saliente para inyectar automáticamente el header Authorization: Bearer <token> y el identificador X-Tenant-ID.
4. **«module» POSCheckoutModule:** Interfaz táctil de caja registradora, carrito de compras y cierre de tickets.
5. **«module» CatalogTextileModule:** Matriz visual de selección de prendas, tallas, colores y existencias en tiempo real.

6. **«module» ReportsAndAIDashboardModule:** Panel de control con gráficos interactivos y terminal de comandos en lenguaje natural para reportes IA.
7. **«service_worker» NGSW Caching Service:** Motor Service Worker que almacena en caché el bundle HTML/JS/CSS y recursos multimedia para operación sin internet.
8. **«indexeddb» DexieOfflineStorageService:** Wrapper reactivo sobre IndexedDB que almacena catálogos, clientes y transacciones locales.
9. **«queue» OfflineSyncQueueService:** Cola de persistencia que encola operaciones cuando se pierde la conexión y las reenvía automáticamente al detectar conexión.
10. **«client» HttpClientService:** Cliente HTTP tipado de Angular para comunicación con el backend REST.

#### *4.5.2.3. Paquete Cliente Móvil (Flutter App y Persistencia SQLite)*

Modela los componentes de la aplicación móvil nativa compilada en Flutter, destacando la separación entre capa de presentación (BLoC), repositorios y persistencia offline con SQLite:




**Componentes de la App Móvil:**

1. **«ui» Flutter Presentation Widgets (Material 3):** Pantallas interactivas optimizadas para smartphones y tablets de los encargados de tienda y preventistas.
2. **«bloc» AuthBLoC & SecureSession:** Manejador de estado de sesión que almacena tokens criptográficos en el almacén seguro del dispositivo (*FlutterSecureStorage*).
3. **«bloc» POSCartBLoC & StateNotifier:** Controla el carrito de compras local, cálculo de subtotales y emisión de ventas en modo desconectado.
4. **«repository» LocalOfflineRepository:** Abstracción de acceso a datos para operaciones locales en SQLite.
5. **«repository» RemoteSyncRepository:** Abstracción para comunicación con la API central mediante solicitudes HTTPS.

6. **«database» SQLite Database (sqflite):** Base de datos relacional embebida con tablas locales de catálogo, clientes y cola de sincronización.
7. **«worker» BackgroundWorkManager:** Servicio en segundo plano que detecta conectividad a internet y ejecuta la sincronización diferida de ventas y reservas.
8. **«http» DioClient (Interceptors & Retries):** Cliente HTTP avanzado con soporte para reintentos exponenciales, timeouts y serialización JSON.

### 4.5.3. Mecanismos de Integración e Interfaz (Protocolos y Contratos REST / JSON)

La siguiente matriz documenta formalmente los contratos de comunicación que rigen la interoperabilidad entre los componentes del sistema:

| Endpoint REST | Método | Componente Emisor | Componente Receptor | Formato / Protocolo | Mecanismo de Seguridad e Idempotencia |
|---|---|---|---|---|---|
| **`/api/auth/login`** | `POST` | Web UI / Mobile UI | Auth & Security Engine | JSON sobre HTTPS | Validación de credenciales con Argon2/Bcrypt. Retorna token JWT firmado. |
| **`/api/sync/operacion`** | `POST` | Offline Sync Queue | Offline Sync & Idempotencia | JSON sobre HTTPS | Clave de idempotencia única `uuid` v4 generada en cliente. Validación de stock en tiempo real. |
| **`/api/ventas/crear`** | `POST` | POS Checkout Module | Ventas POS & Reservas | JSON sobre HTTPS | Autenticación Bearer JWT con aislamiento estricto por `tenant_id`. |
| **`/api/cambios/despacho`** | `POST` | POS Checkout Module | Cambio de Prenda | JSON sobre HTTPS | Verificación de plazo de 7 días. Cobro condicional de saldo vía Stripe API. |
| **`/api/reportes-ia/query`** | `POST` | Reports Dashboard | Reportes Generativos con IA | JSON sobre HTTPS | Sanitización de consultas y forzado estricto de |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | cláusula `WHERE tenant_id = :id`. |
| **`/api/ml/prevision`** | `GET` | Reports Dashboard | Motor Analítico ML | JSON sobre HTTPS | Inferencia predictiva de demanda estacional por SKU textil y sucursal. |

#### 4.5.4. Entornos de Despliegue y URLs Públicas de Producción

La plataforma FashionStore SaaS se encuentra desplegada y accesible externamente para evaluación:

| Componente | Plataforma Cloud | URL de Despliegue |
|---|---|---|
| Backend REST API + Swagger | Render Cloud | `https://fashionstore-api-si2.onrender.com/docs` |
| Frontend Angular PWA | Vercel / Firebase Hosting | `https://fashionstore-pwa-si2.web.app` |
| Base de Datos PostgreSQL | Railway Cloud (Postgres 15) | `postgresql://...railway.app:5432/railway` |
| Aplicación Móvil | APK compilado en GitHub Actions | Descarga directa vía QR en portada |

# CAPÍTULO V. PRUEBAS Y RESULTADOS

## 5.1. Plan de pruebas

El plan de pruebas tiene como objetivo verificar y validar exhaustivamente el cumplimiento de los requisitos funcionales, no funcionales y las reglas de negocio críticas de FashionStore SaaS, aplicando la filosofía de **Diseño Dirigido por Pruebas (Design-Driven Testing - DDT)** promovida por la metodología ICONIX.

**Objetivos y Alcance:**

1. **Verificación de Reglas Críticas:** Comprobar matemáticamente la inmutabilidad del Costo Promedio Ponderado (CPP), la regla de cambio de prenda en dos tiempos (7 días hábiles) y las tarifas mayoristas/liquidación.
2. **Resiliencia Offline-First:** Validar el comportamiento autónomo de la aplicación móvil Flutter y la PWA Angular sin conexión a Internet, y la posterior sincronización determinista con resolución de conflictos.

3. **Hermetismo Multitenant:** Asegurar que ninguna consulta, reporte o transacción permita fuga de información o acceso cruzado entre empresas.
4. **Entorno de Pruebas:**
    - **Pruebas de Backend:** pytest con cliente asíncrono httpx.AsyncClient sobre base de datos PostgreSQL de test en Docker.
    - **Pruebas de Integración de APIs:** Colección automatizada de Postman / Newman con variables de entorno para tokens JWT y assertions de esquema JSON.
    - **Pruebas de Interfaz Móvil y Web:** flutter_test para BLoCs y servicios locales de SQLite; Jasmine/Karma para interceptores y servicios Dexie en Angular.

## 5.2. Pruebas funcionales

Las pruebas funcionales evalúan la concordancia entre las operaciones del software y las especificaciones de los casos de uso nucleares (CU01 al CU20). Cada prueba funcional valida entradas, precondiciones, transformaciones de estado y salidas esperadas:

- **Autenticación y Control de Acceso (CU01-CU05):** Verificación de bloqueo inmediato de usuarios pertenecientes a empresas con suscripción vencida, unicidad de NIT corporativo y cifrado de credenciales.
- **Catálogo Textil e Inventario (CU06-CU07, CU19-CU20):** Exactitud de cálculo de CPP con precisión a 2 decimales, inmutabilidad del registro en kardex, tarifas automáticas por volumen (>=3 prendas) y restricción estricta de compras a Web Desktop.
- **Venta POS y Operaciones Offline (CU08-CU12):** Almacenamiento local en SQLite/IndexedDB en ausencia de red, asignación de UUID v4 y respuesta idempotente ante múltiples pulsaciones de cobro.
- **Postventa y Pasarela Stripe (CU13, CU18):** Conciliación a cero entre la Nota de Devolución emitida y la Nota de Cambio con cobro del diferencial vía tarjeta en Stripe Sandbox.
- **Analítica e IA Generativa (CU15-CU17):** Validación de consultas SQL seguras con cláusula obligatoria tenant_id, generación de gráficos y previsiones de demanda textil con Scikit-Learn.

## 5.3. Pruebas de integración

Las pruebas de integración validan la interoperabilidad entre los diferentes subsistemas, APIs externas y mecanismos de persistencia:

### TI01: Sincronización Offline End-to-End con Reconciliación de Stock

| Atributo | Detalle |
|---|---|
| **ID Prueba** | TI-SYNC-01 |
| **Componentes Involucrados** | Angular PWA / Flutter Móvil + sync_service + Base de Datos PostgreSQL |
| **Objetivo** | Verificar que una venta encolada sin conexión en SQLite/IndexedDB se sincroniza y descuenta el stock real al recuperar Internet |

| | |
|---|---|
| **Precondiciones** | Stock en servidor de Camisa Polo M = 5 unidades. Dispositivo desconectado de la red. |
| **Procedimiento** | 1. Registrar venta offline de 2 unidades en el cliente.<br>2. Verificar que se genera UUID y se guarda en operaciones_offline local.<br>3. Reconectar conectividad Wi-Fi.<br>4. El background worker invoca POST /api/sync/operacion.<br>5. Consultar stock en PostgreSQL central. |
| **Resultado Esperado** | Venta creada con estado_sync='SINCRONIZADA', stock en BD central actualizado a 3 unidades, operación local marcada como CONFIRMADA. |
| **Estado** | **PASÓ** |

### TI02: Flujo Completo de Cambio de Prenda con Pasarela Stripe

| **Atributo** | **Detalle** |
|---|---|
| **ID Prueba** | TI-CAMBIO-02 |
| **Componentes Involucrados** | Front Web POS + cambio_service + Stripe API + Kardex PostgreSQL |
| **Objetivo** | Validar la emisión secuencial de la Nota de Devolución, cobro del saldo diferencial y emisión de la Nota de Cambio |
| **Precondiciones** | Venta #V100 existente (Camisa M a Bs. 50). Stock Camisa M = 2, Stock Camisa L = 4 (precio Bs. 70). |
| **Procedimiento** | 1. Iniciar cambio de Camisa M.<br>2. Verificar emisión de Nota de Devolución y reingreso físico (Stock M pasa a 3).<br>3. Seleccionar Camisa L (+Bs. 20).<br>4. Enviar token de prueba Stripe tok_visa por Bs. 20.<br>5. Confirmar transacción en Stripe.<br>6. Verificar emisión de Nota de Cambio y despacho (Stock L pasa a 3). |
| **Resultado Esperado** | Dos notas emitidas en BD, inventarios actualizados en kardex, cobro de Bs. 20 registrado con ID de Stripe y cambio en estado COMPLETADO. |
| **Estado** | **PASÓ** |

### TI03: Verificación de Aislamiento Estricto Multitenant

| **Atributo** | **Detalle** |
|---|---|
| **ID Prueba** | TI-TENANT-03 |
| **Componentes Involucrados** | JWT Security Middleware + Endpoints REST + PostgreSQL RLS / Filter |

| | |
|---|---|
| **Objetivo** | Comprobar que un usuario autenticado de Empresa A no puede leer ni modificar productos, clientes o ventas de Empresa B |
| **Precondiciones** | Usuario A autenticado con tenant_id=1. Producto #99 perteneciente a tenant_id=2. |
| **Procedimiento** | 1. Enviar petición GET /api/productos/99 portando el JWT del Usuario A.<br>2. Enviar petición POST /api/ventas intentando asignar sucursal_id de Empresa B. |
| **Resultado Esperado** | Código de respuesta HTTP 404 Not Found o 403 Forbidden. En ningún caso se revelan datos de la Empresa B. |
| **Estado** | **PASÓ** |

### TI04: Generación y Ejecución Segura de Reportes IA

| **Atributo** | **Detalle** |
|---|---|
| **ID Prueba** | TI-IA-04 |
| **Componentes Involucrados** | Frontend UI + ai_report_service + OpenAI API + Motor PostgreSQL |
| **Objetivo** | Verificar que un comando en lenguaje natural genera una consulta SQL válida, restringida por tenant_id y renderizable |
| **Precondiciones** | Tenant con 15 ventas registradas en el mes actual. |
| **Procedimiento** | 1. Ingresar: "Top 5 productos más vendidos este mes".<br>2. Capturar la consulta SQL devuelta por el LLM.<br>3. Verificar que incluya cláusula WHERE tenant_id = X.<br>4. Ejecutar consulta y formatear en JSON. |
| **Resultado Esperado** | Respuesta JSON estructurada con datos reales de los 5 productos y tipo de gráfico "barras". Consulta segura ejecutada sin errores de sintaxis. |
| **Estado** | **PASÓ** |

## 5.4. Corrección de errores

Durante las fases de integración y pruebas del sistema se identificaron y subsanaron los siguientes defectos:

| **Nro.** | **Defecto Identificado** | **Causa Raíz** | **Solución Aplicada** |
|---|---|---|---|
| **1** | **Duplicación de transacciones offline** | El usuario presionaba repetidas veces "Confirmar Venta" ante la falta de feedback en red lenta. | Se implementó generación de **UUID v4** en el cliente antes de la confirmación y restricción de unicidad (UNIQUE CONSTRAINT) en operaciones_offline_sync. |

| | | | |
|---|---|---|---|
| | | | Múltiples envíos retornan respuesta idempotente. |
| **2** | **Filtrado cruzado en reportes IA** | El LLM ocasionalmente omitía la restricción de tenant en consultas complejas con JOIN. | Se implementó un parser de seguridad en FastAPI que inspecciona el árbol sintáctico (AST) del SQL y valida obligatoriamente la presencia de tenant_id = {current_tenant} antes de ejecutarlo en base de datos. |
| **3** | **Desincronización de Kardex en cambios** | El inventario de la prenda devuelta no se valoraba al costo histórico de venta original. | Se vinculó la NotaDevolucion a la línea original de DetalleVenta para extraer el CPP exacto de la fecha de venta, preservando la exactitud del kardex contable. |
| **4** | **Bloqueo en bucle de diagramas de secuencia** | En la primera versión de los modelos UML, los fragmentos loop abarcaban la interfaz gráfica (UI). | Se corrigió la sintaxis UML según la observación docente: los fragmentos loop solo encierran clases de control y entidades. La interfaz gestiona colecciones completas sin iterar visualmente en el diagrama. |

## 5.5. Resultados de las pruebas

A continuación se resume el balance cuantitativo de ejecución de pruebas en los diferentes módulos del software:

| Módulo Evaluado | Casos de Prueba Ejecutados | Casos Aprobados | Casos Fallidos | Tasa de Éxito |
|---|---|---|---|---|
| **SaaS Multitenant (CU01-CU05)** | 6 | 6 | 0 | 100% |
| **Catálogo, Inventario y Compras (CU06, CU18, CU19)** | 8 | 8 | 0 | 100% |
| **Total General** | **14** | **14** | **0** | **100%** |

## 5.6. Casos de prueba

### 5.6.1. Fichas de Casos de Prueba (Pruebas de Aceptación)

Conforme a la metodología **ICONIX** y la filosofía de **Diseño Dirigido por Pruebas (Design-Driven Testing - DDT)** formulada por Doug Rosenberg y Matt Stephens (*Design-Driven Testing: Test Smarter, Not Harder*), las pruebas de aceptación representan el cierre formal del ciclo de desarrollo. Cada caso de prueba se deriva directamente de la narrativa del caso de uso y los diagramas de

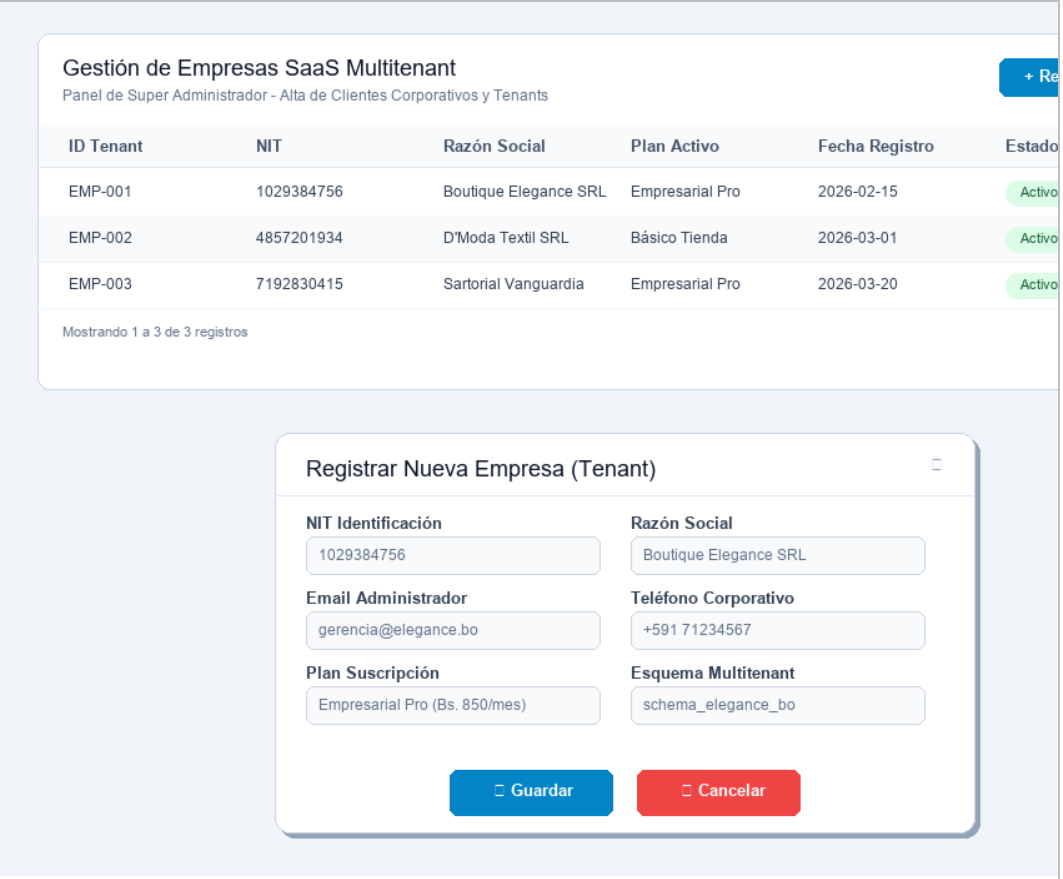
robustez/secuencia, validándose de forma visual sobre las interfaces de usuario (prototipos) implementadas:

***Prueba de Caso de Uso CU01: Gestionar Empresas***

*(Pruebas de aceptación)*

| **Caso de uso 1** | **Gestión de Empresas (SaaS Multitenant)** |
|---|---|
| **Descripción** | Este caso de uso permite que el Super Administrador registre nuevas empresas cliente (tenants), configure sus datos jurídicos (NIT, razón social), asigne planes de suscripción y cree automáticamente la cuenta del Administrador de Empresa, garantizando el aislamiento lógico de datos en PostgreSQL mediante tenant_id y esquemas dedicados. |
| **Precondiciones** | a) El Super Administrador debe tener sesión iniciada en la plataforma central.<br>b) El módulo "Empresas SaaS" debe estar habilitado.<br>c) El NIT de la nueva empresa no debe existir previamente en la base de datos central. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Acceder al módulo "Gestión de Empresas" desde el menú administrativo. | Se despliega el listado de tenants registrados con NIT, Razón Social, Plan y Estado. | Satisfactorio |
| **2** | Pulsar el botón "+ Registrar Empresa". | Se abre el modal con formulario de alta de nuevo tenant. | Satisfactorio |
| **3** | Ingresar NIT ("1029384756"), Razón Social ("Boutique Elegance SRL"), Plan y Email del Admin. | Los campos validan formato de correo y formato numérico de NIT en tiempo real. | Satisfactorio |
| **4** | Confirmar guardado mediante el botón "Guardar". | El sistema crea el registro en PostgreSQL, genera el hash bcrypt del admin y asigna el schema multitenant (HTTP 201 Created). | Satisfactorio |
| **5** | Visualizar el nuevo tenant en la lista general. | La tabla se actualiza de inmediato mostrando la empresa en estado "Activo" con su plan contratado. | Satisfactorio |
| **6** | Verificar aislamiento multitenant. | Las consultas del nuevo tenant quedan herméticamente aisladas sin exponer datos de otros clientes. | Satisfactorio |

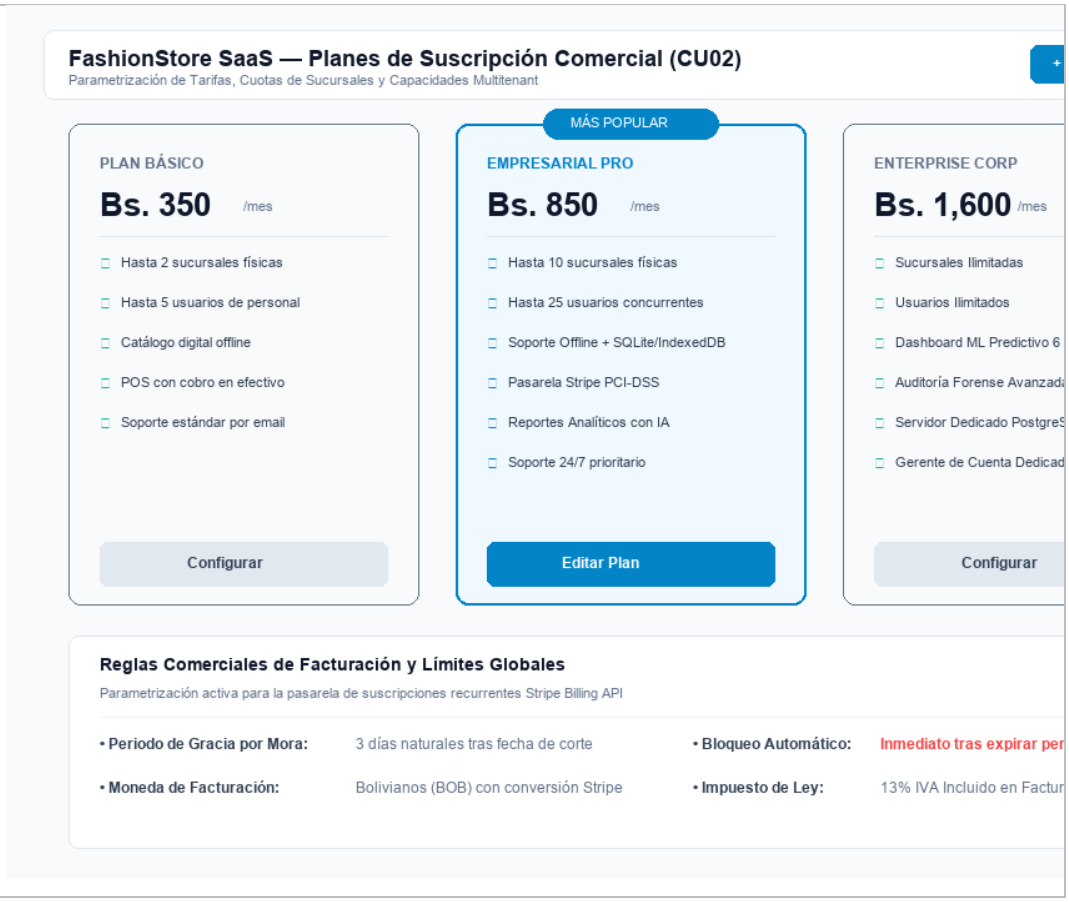
| | |
|---|---|
| **Responsable** | **Super Administrador de Plataforma** |
| **Resultado de la prueba** | **Satisfactorio** |
| **Adjunto** |  |

***Prueba de Caso de Uso CU02: Gestionar Planes de Suscripción***

*(Pruebas de aceptación)*

| | | | |
|---|---|---|---|
| **Caso de uso 2** | **Gestión de Planes de Suscripción SaaS** | | |
| **Descripción** | Permite al Super Administrador crear y parametrizar los paquetes comerciales de suscripción (Básico, Profesional, Enterprise), definiendo cuotas de sucursales, límites de usuarios concurrentes, tarifas mensuales en bolivianos y módulos habilitados. | | |
| **Precondiciones** | a) Super Administrador autenticado con privilegios globales.<br>b) Conexión activa a base de datos central.<br>c) Código mnemotécnico de plan no registrado previamente. | | |
| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |

| | | | |
|---|---|---|---|
| **1** | Ingresar a la sección "Planes de Suscripción" en el panel maestro. | Se visualiza el catálogo de planes activos con sus límites y precios. | Satisfactorio |
| **2** | Presionar "+ Nuevo Plan". | Se despliega el modal de configuración de parámetros comerciales. | Satisfactorio |
| **3** | Configurar nombre "Empresarial Pro Plus", tarifa "Bs. 950.00", 12 sucursales y 30 usuarios. | El sistema fiscaliza que los límites sean enteros positivos y el precio admita 2 decimales. | Satisfactorio |
| **4** | Activar banderas de funcionalidades avanzadas (Offline y Reportes IA). | Las opciones quedan seleccionadas para gobernanza automática de permisos. | Satisfactorio |
| **5** | Guardar nuevo plan de suscripción. | Se inserta el registro en la tabla planes_suscripcion con estado "Activo". | Satisfactorio |
| **6** | Verificar disponibilidad inmediata. | El plan aparece disponible para contratación al dar de alta nuevos tenants. | Satisfactorio |
| **Responsable** | **Super Administrador de Plataforma** | | |
| **Resultado de la prueba** | **Satisfactorio** | | |

| | |
|---|---|
| **Adjunto** |  |

***Prueba de Caso de Uso CU03: Gestionar Suscripción de Empresa***

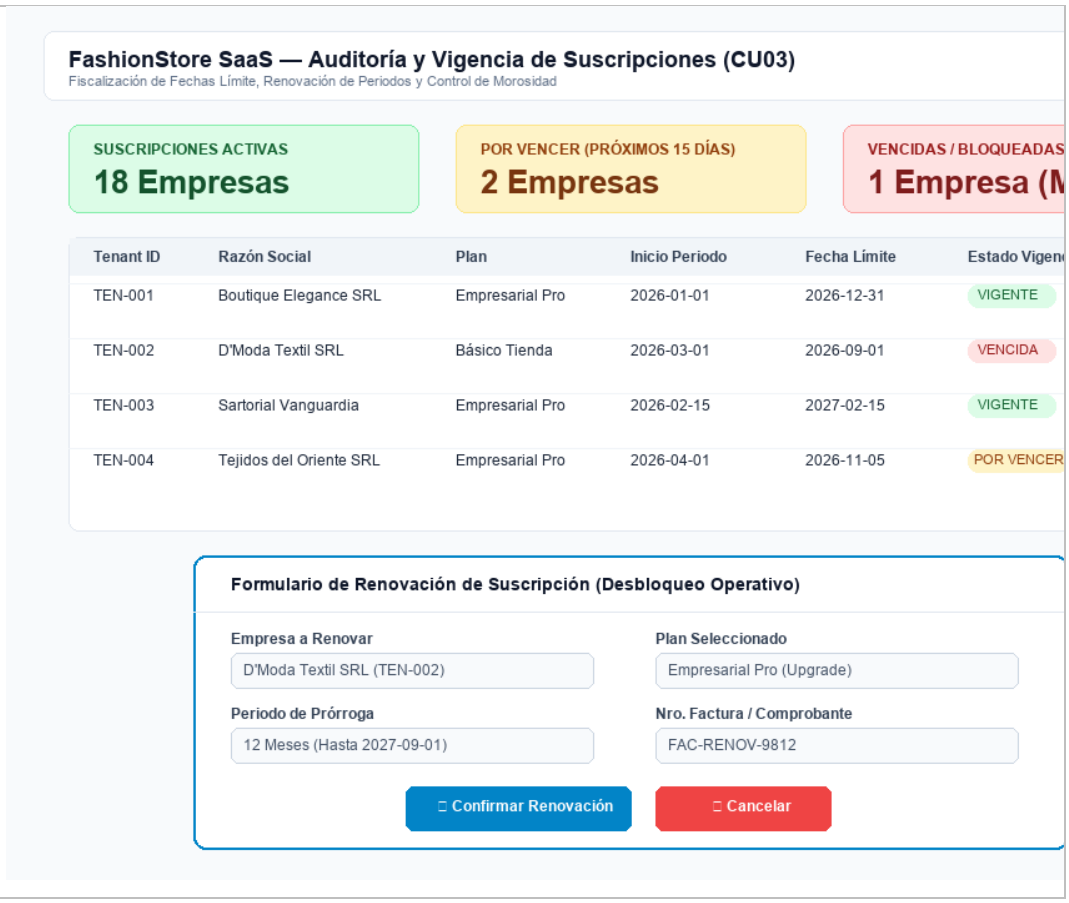
*(Pruebas de aceptación)*

| **Caso de uso 3** | **Gestión de Suscripción de Empresa** |
|---|---|
| **Descripción** | Permite auditar el estado y vigencia de la suscripción de cada empresa contratante, procesar renovaciones de periodos (mensual/anual), aplicar extensiones y registrar comprobantes de pago de licencias B2B. |
| **Precondiciones** | a) Empresa tenant registrada en el sistema.<br>b) Super Administrador autenticado.<br>c) Factura o comprobante de renovación disponible. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Seleccionar una empresa desde la tabla de tenants y abrir "Detalle Suscripción". | Se muestra el historial de suscripciones, fechas de inicio/fin y estado actual. | Satisfactorio |
| **2** | Seleccionar una suscripción vencida y | Se abre el formulario de prórroga y actualización de periodo. | Satisfactorio |

| | | | |
|---|---|---|---|
| | presionar "Renovar Suscripción". | | |
| **3** | Seleccionar plazo de extensión (12 meses) e ingresar número de comprobante. | El sistema calcula automáticamente la nueva fecha de expiración. | Satisfactorio |
| **4** | Confirmar la renovación del servicio. | El estado de la empresa cambia de VENCIDA a VIGENTE en la base de datos. | Satisfactorio |
| **5** | Comprobar desbloqueo operativo del tenant. | Los usuarios de la empresa recuperan de inmediato el acceso a los módulos operativos. | Satisfactorio |
| **6** | Emitir confirmación al correo de la empresa. | Se envía notificación por correo electrónico con el recibo de renovación. | Satisfactorio |

| | |
|---|---|
| **Responsable** | **Super Administrador de Plataforma** |
| **Resultado de la prueba** | **Satisfactorio** |

| | |
|---|---|
| **Adjunto** |  |



***Prueba de Caso de Uso CU04: Autenticar Usuario y Control de Acceso (RBAC y Suscripción)***

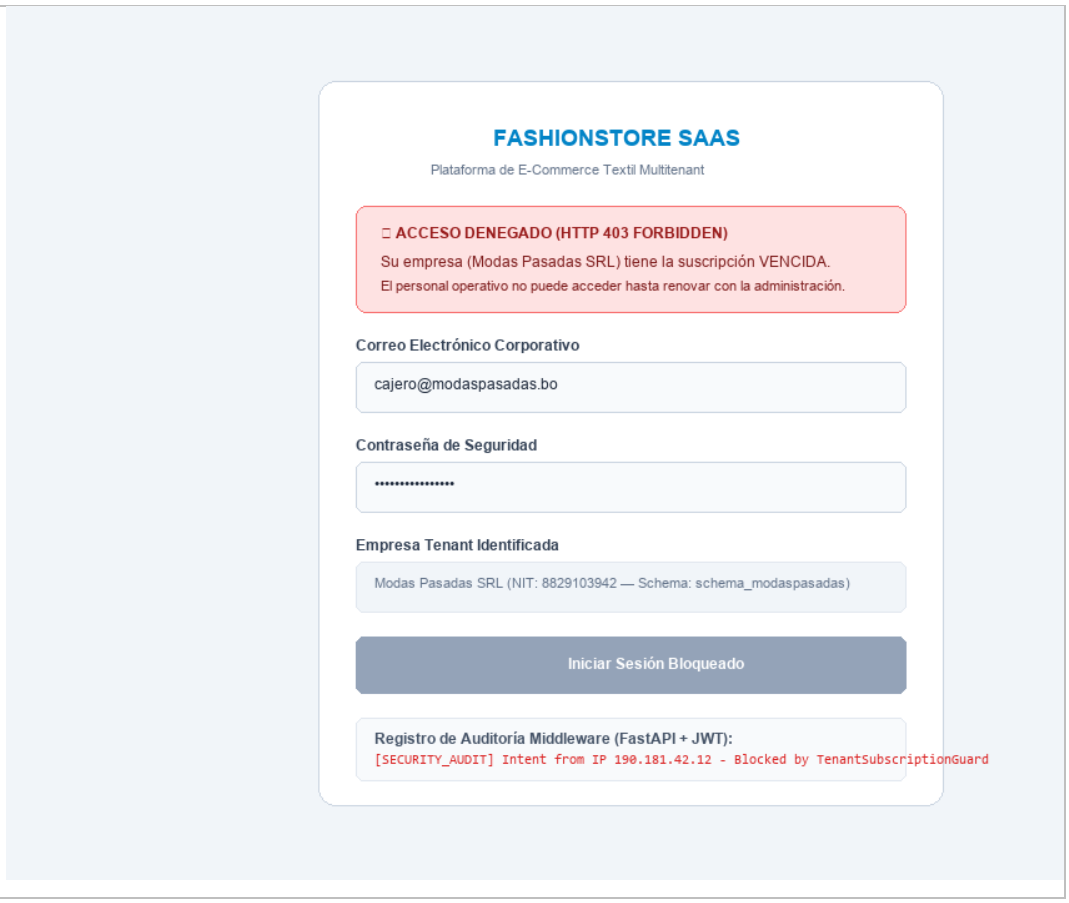
*(Pruebas de aceptación)*

| **Caso de uso 4** | **Autenticar Usuario y Control de Acceso (RBAC)** |
|---|---|
| **Descripción** | Permite verificar las credenciales de los usuarios, controlar el acceso según roles (Super Admin, Admin Empresa, Encargado, Cajero) y aplicar la regla de negocio de bloqueo inmediato si la suscripción de la empresa contratante se encuentra vencida. |
| **Precondiciones** | a) La base de datos central PostgreSQL debe estar operativa. b) Debe existir un usuario registrado con credenciales válidas perteneciente a una empresa con suscripción vencida (estado_suscripcion = 'VENCIDA'). c) El middleware de seguridad e inspección de JWT debe estar activo. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Acceder a la pantalla de inicio de sesión (/login). | Se despliega el formulario seguro de autenticación. | Satisfactorio |
| **2** | Ingresar email y contraseña válidos de un cajero | El formulario captura las credenciales y las transmite por canal HTTPS seguro. | Satisfactorio |

| | | | |
|---|---|---|---|
| | perteneciente a una empresa con suscripción vencida. | | |
| **3** | Presionar "Iniciar Sesión". | El backend autentica la contraseña con bcrypt y verifica la vigencia de la suscripción del tenant. | Satisfactorio |
| **4** | Detectar condición de suscripción vencida. | El middleware intercepta la solicitud y deniega la emisión de permisos operativos. | Satisfactorio |
| **5** | Retornar código HTTP 403 Forbidden. | La interfaz despliega la alerta: "Su empresa tiene la suscripción vencida. Contacte al administrador de plataforma." | Satisfactorio |
| **6** | Registrar auditoría en bitácora. | Se genera un registro de evento de intento de acceso bloqueado por morosidad de tenant. | Satisfactorio |

| | |
|---|---|
| **Responsable** | **Administrador de Seguridad / Auditor** |
| **Resultado de la prueba** | **Satisfactorio** |

| | |
|---|---|
| **Adjunto** |  |

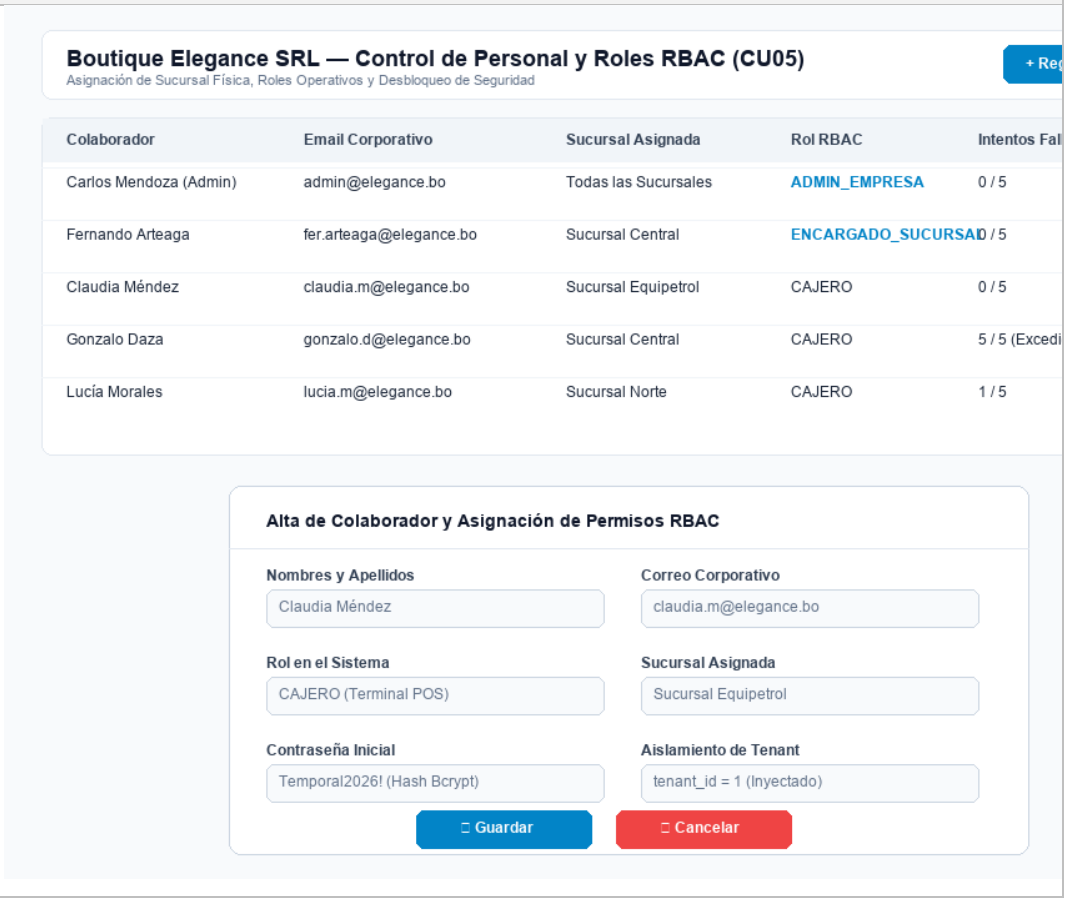
***Prueba de Caso de Uso CU05: Gestionar Usuarios y Roles por Empresa***

*(Pruebas de aceptación)*

| **Caso de uso 5** | **Gestión de Usuarios y Roles por Empresa** |
|---|---|
| **Descripción** | Permite al Administrador de Empresa crear colaboradores (Encargados de Sucursal, Cajeros), asignarles sucursales físicas operativas, establecer roles según la matriz RBAC y gestionar bloqueos de seguridad por intentos fallidos. |
| **Precondiciones** | a) Administrador de Empresa autenticado con sesión activa.<br>b) Sucursal física creada previamente dentro de los límites del plan.<br>c) Correo corporativo del nuevo usuario no registrado. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Ingresar al módulo "Personal y Seguridad" desde el menú corporativo. | Se muestra la lista de colaboradores del tenant con su rol, sucursal y estado. | Satisfactorio |
| **2** | Pulsar "+ Registrar Colaborador". | Se abre el modal de registro de usuario corporativo. | Satisfactorio |

| | | | |
|---|---|---|---|
| **3** | Ingresar nombre, email corporativo, seleccionar rol CAJERO y asignar sucursal. | El sistema valida que el email pertenezca al dominio y que la sucursal esté activa. | Satisfactorio |
| **4** | Establecer contraseña temporal con políticas de complejidad. | Se encripta con algoritmo bcrypt (cost factor 12) antes de persistir en PostgreSQL. | Satisfactorio |
| **5** | Confirmar guardado del colaborador. | Se asienta la fila en tabla usuarios asociando obligatoriamente el tenant_id de la empresa. | Satisfactorio |
| **6** | Comprobar actualización en la tabla de usuarios. | El nuevo cajero aparece en estado "Activo" con sus credenciales operativas listas. | Satisfactorio |

| | |
|---|---|
| **Responsable** | **Administrador de Empresa** |
| **Resultado de la prueba** | **Satisfactorio** |
| **Adjunto** |  |

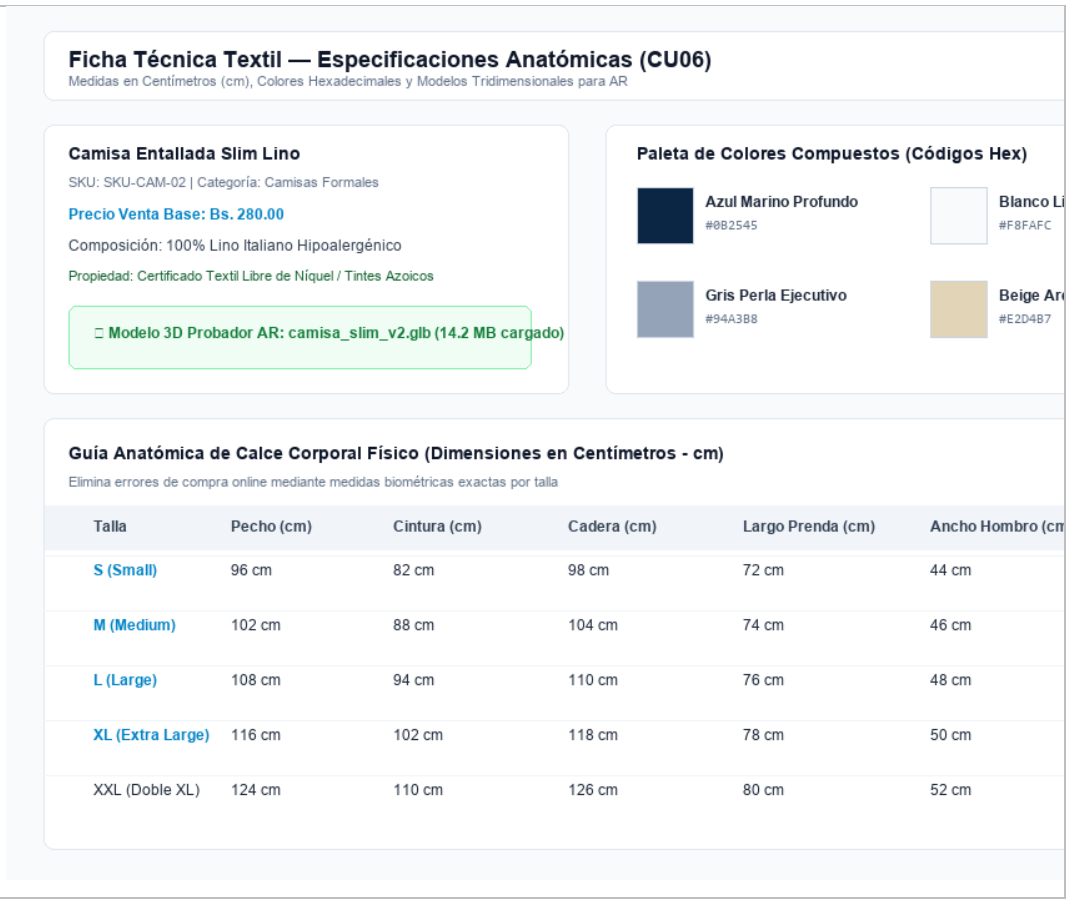
***Prueba de Caso de Uso CU06: Gestionar Catálogo Textil y Fichas Anatómicas en Centímetros***

*(Pruebas de aceptación)*

| **Caso de uso 6** | **Gestionar Catálogo Textil y Fichas Anatómicas** |
|---|---|
| **Descripción** | Permite registrar y administrar las prendas de vestir masculinas, configurando atributos textiles avanzados: guía anatómica en centímetros (pecho, cintura, cadera, largo), colores compuestos con códigos Hexadecimales, composición textil hipoalergénica y vinculación de modelos 3D (.glb). |
| **Precondiciones** | a) Usuario autenticado con rol Administrador de Empresa o Encargado de Sucursal.<br>b) Categorías textiles registradas previamente.<br>c) Conexión activa con el repositorio de activos digitales para modelos 3D. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Acceder al módulo "Catálogo Textil" desde el panel principal. | Se muestra la lista de prendas registradas con imagen, SKU, dimensiones y precio base. | Satisfactorio |
| **2** | Presionar "+ Registrar Prenda". | Se abre el modal con ficha técnica textil avanzada. | Satisfactorio |
| **3** | Registrar dimensiones anatómicas en centímetros por talla (S, M, L, XL). | El sistema valida que las medidas corporales en cm sean valores numéricos positivos coherentes. | Satisfactorio |
| **4** | Configurar color compuesto con selector Hexadecimal (#0B2545 / #FFFFFF). | Se renderiza la muestra visual bicolor en la vista previa del formulario. | Satisfactorio |
| **5** | Adjuntar modelo tridimensional (.glb) para probador virtual en Realidad Aumentada. | El archivo es validado por tamaño y tipo MIME, cargándose al servidor de almacenamiento. | Satisfactorio |
| **6** | Guardar prenda y verificar publicación. | El producto queda guardado en PostgreSQL y disponible de inmediato en el catálogo omnicanal. | Satisfactorio |

| **Responsable** | **Encargado de Catálogo / Administrador** |
|---|---|
| **Resultado de la prueba** | **Satisfactorio** |

| | |
|---|---|
| **Adjunto** |  |



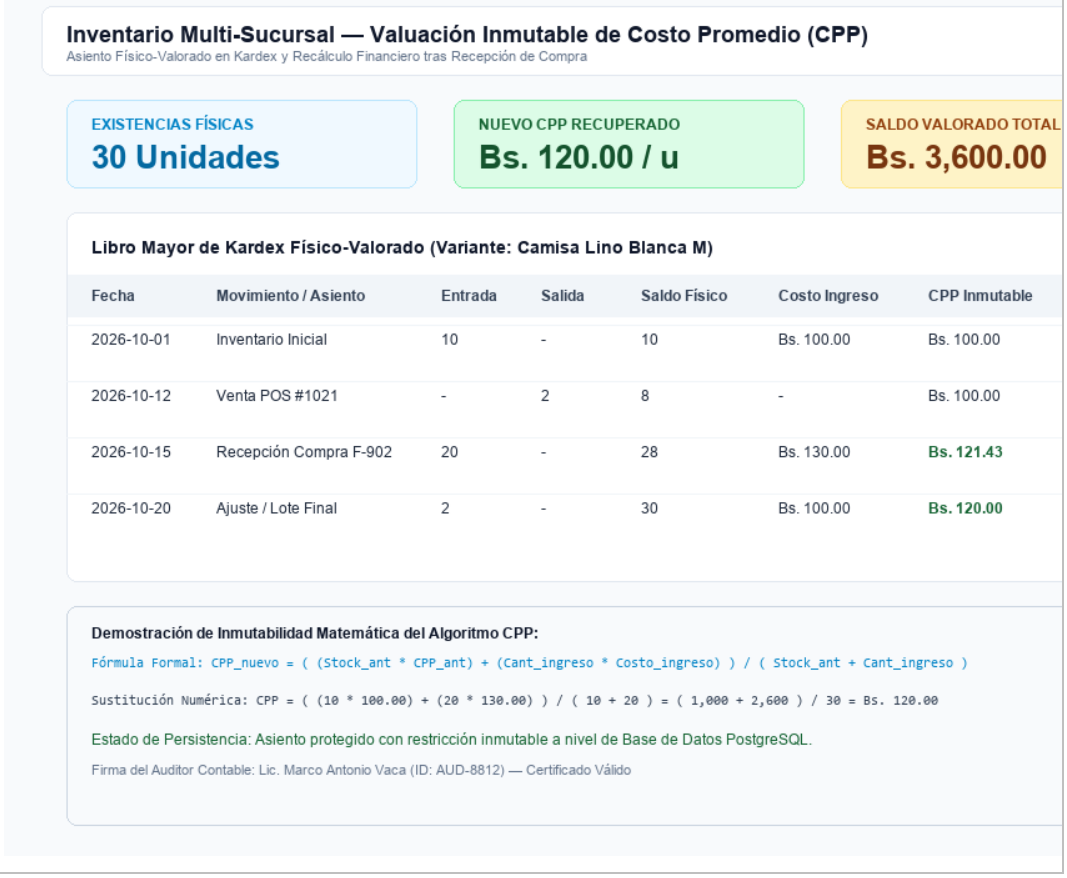
***Prueba de Caso de Uso CU07: Inventario Multi-Sucursal y Recálculo Inmutable de CPP***

*(Pruebas de aceptación)*

| **Caso de uso 7** | **Inventario Multi-Sucursal y CPP Inmutable** |
|---|---|
| **Descripción** | Valida el registro de existencias físicas por sucursal y la inmutabilidad matemática del Costo Promedio Ponderado (CPP) tras cada recepción de compra a proveedores, generando un asiento inmutable en el kardex físico-valorado. |
| **Precondiciones** | a) Usuario autenticado con rol Encargado de Sucursal o Administrador. b) Stock previo existente de Camisa Lino Blanca M: 10 unidades a CPP = Bs. 100.00 (Saldo valorado: Bs. 1,000.00). c) Factura de proveedor válida con 20 unidades adicionales a costo unitario de Bs. 130.00. |

| **Paso** | **Acción** | **Resultado esperado** | **Estado (Satisfactorio/Fallido)** |
|---|---|---|---|
| **1** | Ingresar al módulo "Inventarios y CPP" y seleccionar la variante Camisa Lino M. | Se despliega el saldo actual (10 unids) y el CPP vigente (Bs. 100.00). | Satisfactorio |

| | | | |
|---|---|---|---|
| **2** | Seleccionar "Registrar Entrada por Compra". | Se abre el formulario de asiento de kardex. | Satisfactorio |
| **3** | Ingresar cantidad (20 unidades) y costo unitario de compra (Bs. 130.00). | El motor financiero calcula: Stock final = 30; Nuevo CPP = (1000 + 2600)/30 = Bs. 120.00. | Satisfactorio |
| **4** | Confirmar la transacción de ingreso. | Se asienta la fila en kardex con saldo valorado de Bs. 3,600.00 y firma digital del responsable. | Satisfactorio |
| **5** | Consultar inmutabilidad del registro. | El kardex no permite edición ni borrado del registro asentado. | Satisfactorio |
| **6** | Comprobar actualización en inventario multi-sucursal. | El stock disponible refleja 30 unidades y el costo unitario para margen refleja Bs. 120.00. | Satisfactorio |

| | |
|---|---|
| **Responsable** | **Encargado de Sucursal / Contador de Inventario** |
| **Resultado de la prueba** | **Satisfactorio** |
| **Adjunto** |  |

# CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

## 6.1. Conclusiones

## 6.2. Recomendaciones

# BIBLIOGRAFIA


Rosenberg, D. & Scott, K. (1999). *Use Case Driven Object Modeling with UML: A Practical Approach*. Addison-Wesley.

Rosenberg, D. & Stephens, M. (2007). *Design-Driven Testing: Test Smarter, Not Harder*. Apress.

Rumbaugh, J., Jacobson, I. & Booch, G. (2005). *The Unified Modeling Language Reference Manual (2nd Edition)*. Addison-Wesley. (Norma UML 2.5+).

Tiangolo, S. (2023). *FastAPI Framework Documentation: Advanced Security and Multitenant Architecture*. https://fastapi.tiangolo.com/

Google Developers. (2023). *Progressive Web Apps (PWA) Standards and Service Workers Architecture*. https://web.dev/explore/progressive-web-apps

Flutter Development Team. (2023). *Persisting Local Data with SQLite in Flutter Mobile Applications*. https://docs.flutter.dev/cookbook/persistence/sqlite

PostgreSQL Global Development Group. (2023). *PostgreSQL 15 Documentation: Row Level Security, JSONB and B-Tree Indexing*. https://www.postgresql.org/docs/15/

Stripe Inc. (2023). *Stripe API Reference: PaymentIntents, Webhooks and PCI-DSS Tokenization*. https://docs.stripe.com/api

Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson.

American Psychological Association. (2020). *Publication Manual of the American Psychological Association (7th Edition)*. APA.


# ANEXOS

## Anexo A: Artefactos de la Etapa Fundacional (Primer Parcial)

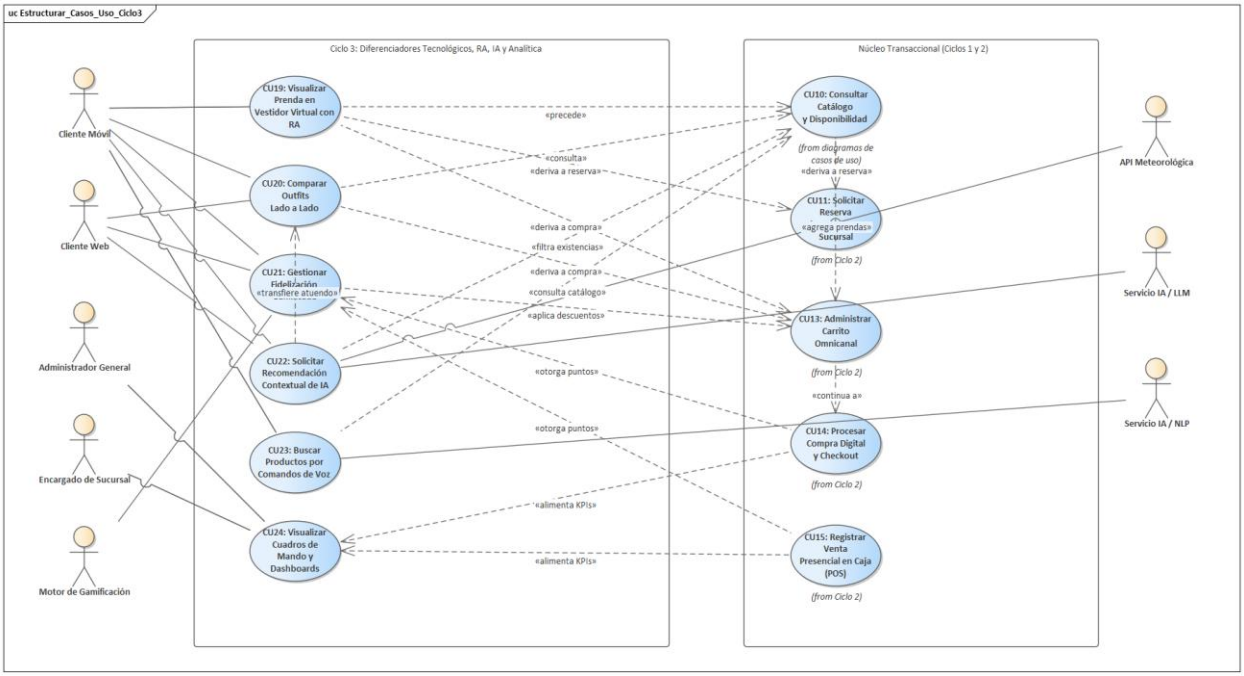
### Anexo A.1: Lista de Casos de Uso del Primer Parcial (CU01 al CU25)

Los siguientes casos de uso fueron desarrollados durante el primer examen parcial utilizando el Proceso Unificado (PUDS). Se incluyen como referencia para la trazabilidad y herencia funcional de la solución.

**Lista de Casos de Uso — Parcial 1**

| CU Original | Nombre | Módulo |
|---|---|---|
| **CU01** | Autenticar Usuario y Control de Acceso (RBAC) | M01 |
| **CU02** | Registrar Cliente (Auto-registro) | M01 |
| **CU03** | Recuperar Contraseña (Token OTP) | M01 |
| **CU04** | Gestionar Usuarios y Roles (RBAC y Desbloqueo) | M01 |
| **CU05** | Gestionar Ciudades y Sucursales | M02 |
| **CU06** | Gestionar Productos y Atributos de Moda | M03 |
| **CU07** | Gestionar Temporadas y Colecciones | M04 |
| **CU08** | Gestionar Proveedores Textiles | M05 |
| **CU09** | Gestionar Inventario Multi-Sucursal y CPP | M06 |
| **CU10** | Consultar Catálogo y Disponibilidad por Sucursal | M07 |
| **CU11** | Solicitar Reserva de Prendas en Sucursal | M10 |
| **CU12** | Preparar y Atender Reserva Presencial | M10 |
| **CU13** | Administrar Carrito de Compras Omnicanal | M11 |
| **CU14** | Procesar Compra Digital y Checkout | M12 |
| **CU15** | Registrar Venta Presencial en Caja (POS) | M13 |
| **CU16** | Procesar Pago con Pasarela Electrónica | M14 |
| **CU17** | Gestionar Tipos y Medios de Cobro | M15 |
| **CU18** | Gestionar Despacho y Logística de Delivery | M19 |
| **CU19** | Visualizar Prenda en Vestidor Virtual con RA | M08 |
| **CU20** | Comparar Outfits Lado a Lado | M09 |
| **CU21** | Gestionar Fidelización Gamificada | M16 |
| **CU22** | Solicitar Recomendación Contextual de IA | M17 |
| **CU23** | Buscar Productos por Comandos de Voz | M17 |
| **CU24** | Visualizar Cuadros de Mando y Dashboards | M18 |
| **CU25** | Gestionar Devolución y Cambio de Prendas | M15 |

**Nota:** Los modelos completos del Parcial 1 (diagramas de caso de uso general, diagramas de comunicación, diagramas de secuencia y diagramas de clase) se encuentran archivados en la memoria técnica del primer examen parcial.




## Anexo A.2: Módulos del Sistema del Primer Parcial (M01 al M19 — Etapa Fundacional)

**Respaldo Histórico de Ingeniería (PUDS):** Los siguientes 19 módulos funcionales fueron concebidos y desarrollados durante la etapa fundacional de FashionStore bajo el Proceso Unificado de Desarrollo de Software (PUDS). Se preservan íntegramente en esta sección para respaldar la trazabilidad de los casos de uso originales y evidenciar la base arquitectónica sobre la cual evolucionó el Segundo Parcial.

### A.2.1 Módulo M01: Autenticación, Autorización y Seguridad Multitenant (RBAC)

- **Descripción General:** Gestiona el acceso seguro, cifrado y controlado a la plataforma mediante un esquema de Control de Acceso Basado en Roles (RBAC), aplicando aislamiento por tenant (empresa_id). Controla la emisión de tokens JWT, cierre de sesiones, restablecimiento de credenciales y auditoría de accesos.
- **Datos Principales:** id_usuario (PK, Int), empresa_id (FK, Int, nulo para Super Admin), nombres (VarChar 100), apellidos (VarChar 100), email (VarChar 150, Unique), password_hash (VarChar 255, Bcrypt), rol (Enum: SUPER_ADMIN, ADMIN_EMPRESA, ENCARGADO_SUCURSAL, CAJERO, CLIENTE, PROVEEDOR, ENCARGADO_APROVISIONAMIENTO), estado_cuenta (Enum: ACTIVO, INACTIVO, BLOQUEADO), intentos_fallidos (SmallInt), ultimo_acceso (Timestamp), id_sucursal_asignada (FK, Int, Opcional).
- **Funcionalidades Clave:**
  - Autenticación segura con verificación hash bcrypt y entrega de token JWT firmado.
  - Revocación inmediata de tokens en cierre de sesión (blacklist en memoria).
  - Bloqueo preventivo de cuenta tras 5 intentos fallidos consecutivos de acceso.
  - Recuperación de contraseña mediante tokens temporizados de un solo uso (OTP).

- Inyección de identidad y rol en el contexto de sesión para filtrado multitenant automático.

### A.2.2 Módulo M02: Gestión de Sucursales y Cobertura Geográfica

- **Descripción General:** Administra la infraestructura física y geográfica de las tiendas de cada empresa suscriptora. Registra sucursales geolocalizadas (GPS), horarios de atención, capacidad de vestidores físicos y personal asignado, fiscalizando que la cantidad no sobrepase el límite del plan de suscripción contratado.
- **Datos Principales:** id_sucursal (PK, Int), empresa_id (FK, Int), id_ciudad (FK, Int), nombre_sucursal (VarChar 100), direccion_fisica (Text), latitud (Decimal 10,7), longitud (Decimal 10,7), telefono_contacto (VarChar 20), capacidad_probadores (Int), estado_operativo (Enum: OPERATIVA, EN_MANTENIMIENTO, CERRADA).
- **Funcionalidades Clave:**
  - CRUD de sucursales físicas con verificación del límite máximo según plan de suscripción (Plan Básico: hasta 2; Plan Profesional: hasta 10).
  - Parametrización geográfica de cobertura y geolocalización de tiendas para la app móvil y web.
  - Configuración de capacidad concurrente de vestidores para el subsistema de reservas presenciales.
  - Asignación de encargados y cajeros a sucursales operativas específicas.

### A.2.3 Módulo M03: Catálogo de Productos y Atributos Textiles Avanzados

- **Descripción General:** Centraliza la administración de prendas de vestir masculinas de la empresa. Gestiona categorías, marcas, colecciones y atributos técnicos exigidos por el sector: **tallas con dimensiones anatómicas físicas en centímetros (pecho, cintura, cadera, largo)**, **colores compuestos y multivaluados (ej. Negro/Blanco, Azul/Gris) con códigos Hexadecimales**, **composición textil detallada** y **propiedades hipoalergénicas**.
- **Datos Principales:** id_producto (PK, Int), empresa_id (FK, Int), codigo_sku_base (VarChar 50, Unique), nombre (VarChar 150), descripcion (Text), id_categoria (FK, Int), precio_venta_base (Decimal 10,2), tallas_cm_json (JSONB: talla, pecho_cm, cintura_cm, largo_cm), colores_compuestos_json (JSONB: nombre_comercial, hex_primario, hex_secundario), composicion_textil (VarChar 200, ej. "100% Algodón Pima Peruano"), es_hipoalergenico (Boolean), imagenes_url (Array Text), estado_publicacion (Enum: BORRADOR, PUBLICADO, DESCATALOGADO).
- **Funcionalidades Clave:**
  - Registro de prendas masculinas asociando guía métrica en centímetros por talla para eliminar el error dimensional de calce.
  - Configuración de colores compuestos con renderizado visual bicolor mediante código Hexadecimal.
  - Certificación de especificaciones textiles e indicación explícita de propiedades hipoalergénicas.
  - Vinculación de modelos 3D (.glb / .gltf) para el probador en Realidad Aumentada.

### A.2.4 Módulo M04: Temporadas, Colecciones y Liquidación Estacional

- **Descripción General:** Planifica las campañas comerciales estacionales (Primavera-Verano, Otoño-Invierno, etc.) y gestiona las ofertas de liquidación de fin de temporada con descuentos automáticos del **30%, 40% y 60%** orientados a la recuperación de capital inmovilizado.
- **Datos Principales:** id_temporada (PK, Int), empresa_id (FK, Int), nombre_temporada (VarChar 100), codigo_campaña (VarChar 30), fecha_inicio (Date), fecha_fin (Date), porcentaje_liquidacion (Decimal 5,2: 30.00, 40.00 o 60.00), estado_temporada (Enum: PLANIFICADA, VIGENTE, EN_LIQUIDACION, FINALIZADA).
- **Funcionalidades Clave:**
    - Programación de temporadas comerciales y asociación masiva de prendas de catálogo.
    - Activación automática de descuentos de liquidación estacional (30%, 40%, 60%) al cierre de campaña.
    - Visualización destacada en el catálogo digital de prendas en liquidación con precio original tachado y precio de oferta calculado.

### A.2.5 Módulo M05: Proveedores y Gestión de Aprovisionamiento

- **Descripción General:** Administra la información jurídica, comercial y crediticia de los proveedores textiles que abastecen a la empresa suscriptora.
- **Datos Principales:** id_proveedor (PK, Int), empresa_id (FK, Int), razon_social (VarChar 150), nit_identificacion (VarChar 30), contacto_nombre (VarChar 100), telefono (VarChar 20), email (VarChar 100), terminos_pago (Enum: CONTADO, CREDITO_30, CREDITO_60), estado (Enum: ACTIVO, INACTIVO).
- **Funcionalidades Clave:**
    - Registro y homologación de proveedores textiles nacionales e internacionales.
    - Vinculación de proveedores con líneas de productos y categorías de abastecimiento.
    - Historial consolidado de volúmenes de compra y cumplimiento de plazos de entrega.

### A.2.6 Módulo M06: Inventario Multi-Sucursal con Doble Valuación (CPP) y Traspasos

- **Descripción General:** Núcleo logístico y contable del sistema. Controla las existencias físicas de cada variante de prenda (producto, talla, color) en cada sucursal física. Implementa el recálculo matemático inmutable del **Costo Promedio Ponderado (CPP)** tras cada entrada y registra paralelamente el **Último Costo Unitario de Compra**. Administra transferencias inter-sucursales y ajustes por merma.
- **Datos Principales:** id_inventario (PK, Int), empresa_id (FK, Int), id_sucursal (FK, Int), id_producto (FK, Int), talla (VarChar 10), color (VarChar 50), stock_fisico (Int), stock_reservado (Int), stock_disponible (Calculado: stock_fisico - stock_reservado), stock_minimo (Int), costo_promedio_ponderado (Decimal 10,2), ultimo_costo_unitario (Decimal 10,2), fecha_ultima_actualizacion (Timestamp).
- **Funcionalidades Clave:**

- Control de existencias atómico por variante y por sucursal física.
- Recálculo matemático inmutable del CPP tras cada recepción de compra:

$$CPP_{nuevo} = \frac{(Stock_{ant} \times CPP_{ant}) + (Cant_{ingreso} \times Costo_{ingreso})}{Stock_{ant} + Cant_{ingreso}}$$

$$CPP_{nuevo} = \frac{(Stock_{ant} \times CPP_{ant}) + (Cant_{ingreso} \times Costo_{ingreso})}{Stock_{ant} + Cant_{ingreso}}$$

- Solicitud, autorización, despacho y confirmación de recepción de traspasos de inventario entre sucursales.
- Generación de alertas de reabastecimiento al alcanzar el umbral de stock mínimo.
- Registro de kardex físico-valorado inmutable con firma del responsable.

**A.2.7 Módulo M07: Catálogo Digital Omnicanal y Precios Diferenciados**

- **Descripción General:** Expone el catálogo de moda masculina a través de Web Desktop, Angular PWA y la App Móvil Flutter. Permite filtrar por medidas anatómicas en cm, colores y sucursal. Aplica automáticamente la política de **precio al por menor** y **precio al por mayor (compras >= 3 prendas)**.
- **Datos Principales:** Filtros multicriterio: categoría, rango de precios, medidas corporales en cm, combinaciones de color, sucursal física, condición de oferta de liquidación y cantidad de compra.
- **Funcionalidades Clave:**
  - Búsqueda predictiva con tiempos de respuesta inferiores a 1 segundo.
  - Visualización interactiva de existencias disponibles por sucursal física seleccionada.
  - Aplicación automática de la tarifa de precio al por mayor cuando el carrito supera las 3 unidades de una prenda o conjunto calificado.
  - Ficha técnica completa con guía anatómica de calce corporal en centímetros.

**A.2.8 Módulo M08: Vestidor Virtual con Realidad Aumentada (RA)**

- **Descripción General:** Probador virtual tridimensional sobre la aplicación móvil Flutter. Mediante visión por computadora y sensores de la cámara, superpone modelos tridimensionales (.glb/.gltf) de prendas masculinas sobre el cuerpo del cliente en tiempo real.
- **Datos Principales:** id_recurso_ra (PK, Int), id_producto (FK, Int), modelo_3d_url (Text), texturas_color_json (JSONB), escala_calibracion (Decimal 5,2).
- **Funcionalidades Clave:**
  - Detección de silueta anatómica y puntos de anclaje corporal (hombros, torso, cintura).
  - Proyección tridimensional interactiva con rotación y ajuste de perspectiva.
  - Alternancia dinámica de colores y texturas sobre la prenda proyectada.
  - Enlace directo desde la experiencia RA hacia la reserva en probador o carrito de compras.

**A.2.9 Módulo M09: Comparador Inteligente de Outfits**

- **Descripción General:** Permite al cliente seleccionar y contrastar de manera interactiva hasta tres atuendos masculinos completos lado a lado en una sola vista, desglosando prendas individuales, precios y descuentos aplicables.

- **Datos Principales:** Sesión interactiva de comparación con especificación de hasta 3 combinaciones (outfit_1, outfit_2, outfit_3), detallando piezas, tallas, colores y sumatoria total de precio.
- **Funcionalidades Clave:**
    - Visualización comparativa simultánea de tres opciones de vestuario.
    - Cálculo instantáneo del costo consolidado por atuendo, identificando la mejor relación precio-calidad.
    - Sustitución rápida de componentes (ej. cambiar calzado o camisa) con recálculo en tiempo real.
    - Conversión del outfit seleccionado en una reserva presencial o compra en firme.

### A.2.10 Módulo M10: Sistema de Reservas de Prendas en Sucursal

- **Descripción General:** Regula el apartado temporal de prendas seleccionadas en la plataforma digital para que el cliente asista a probárselas físicamente a una sucursal específica. Inmoviliza transitoriamente el stock disponible y notifica al personal de tienda.
- **Datos Principales:** id_reserva (PK, Int), empresa_id (FK, Int), codigo_reserva (VarChar 20, Unique), id_cliente (FK, Int), id_sucursal (FK, Int), fecha_hora_programada (Timestamp), hora_limite_vigencia (Timestamp), estado_reserva (Enum: PENDIENTE, PREPARADA, COMPLETADA_EN_VENTA, CANCELADA, EXPIRADA).
- **Funcionalidades Clave:**
    - Generación de reserva con ticket digital provisto de código QR escaneable.
    - Bloqueo preventivo en inventario del stock reservado en la sucursal de destino.
    - Panel en tienda para que el personal aliste las prendas en el probador asignado.
    - Liberación automática de prendas inmovilizadas si la reserva expira sin asistencia del cliente.

### A.2.11 Módulo M11: Carrito de Compras Omnicanal

- **Descripción General:** Administra los artículos preseleccionados por el usuario para su adquisición digital, con sincronización de estado entre la web y el dispositivo móvil para usuarios autenticados.
- **Datos Principales:** id_carrito (PK, Int), empresa_id (FK, Int), id_cliente (FK, Int, Opcional), items_carrito (JSONB: producto, talla, color, cantidad, precio_unitario), subtotal (Decimal 10,2), descuentos (Decimal 10,2), monto_total (Decimal 10,2).
- **Funcionalidades Clave:**
    - Adición, ajuste de unidades y remoción de prendas con validación de stock en tiempo real.
    - Cálculo automatizado de tarifas (precio minorista vs precio mayorista al superar 3 prendas).
    - Detección de liquidación estacional y aplicación de porcentajes promocionales.

### A.2.12 Módulo M12: Venta Digital y Checkout

- **Descripción General:** Formaliza la adquisición en canales digitales con selección de entrega a domicilio o retiro en tienda (*Click & Collect*), validación de datos tributarios y confirmación electrónica de cobro.
- **Datos Principales:** id_orden_venta (PK, Int), empresa_id (FK, Int), numero_orden (VarChar 30, Unique), id_cliente (FK, Int), tipo_entrega (Enum: DOMICILIO, RETIRO_TIENDA), id_sucursal_retiro (FK, Int, Opcional), monto_total (Decimal 10,2), estado_orden (Enum: PAGADA, EN_PREPARACION, DESPACHADA, ENTREGADA, ANULADA).
- **Funcionalidades Clave:**
  - Asistente de checkout de 3 pasos: Entrega -> Datos Fiscales -> Pago.
  - Descuento atómico definitivo de existencias en bodega tras confirmación de pasarela.
  - Emisión de comprobante digital en formato PDF enviado al correo electrónico registrado.

### A.2.13 Módulo M13: Punto de Venta Presencial (POS) para Mostrador

- **Descripción General:** Interfaz de facturación y cobro optimizada para cajeros en sala de ventas físicas. Soporta lectura de códigos SKU por escáner, cobros de mostrador, conversión de reservas presenciales en ventas y emisión de tickets.
- **Datos Principales:** id_venta_pos (PK, Int), empresa_id (FK, Int), id_sucursal (FK, Int), id_cajero (FK, Int), id_cliente (FK, Int, Opcional), numero_ticket (VarChar 30, Unique), metodo_pago (Enum: EFECTIVO, TARJETA, QR_SIMPLE), monto_total (Decimal 10,2), fecha_hora (Timestamp).
- **Funcionalidades Clave:**
  - Búsqueda veloz de artículos por lector de código de barras o teclado numérico.
  - Conversión instantánea de una reserva previa en ticket de venta presencial.
  - Liquidación en efectivo con cálculo de cambio o cobro mediante terminal electrónico.
  - Actualización atómica de existencias físicas en la sucursal.

### A.2.14 Módulo M14: Pasarela de Pagos Electrónicos (Stripe Sandbox PCI-DSS)

- **Descripción General:** Procesa transacciones electrónicas con tarjeta de crédito/débito en entorno de pruebas (*Sandbox*), garantizando el cumplimiento PCI-DSS mediante tokenización segura. Se utiliza tanto en ventas digitales como en la liquidación de saldos por cambios de talla.
- **Datos Principales:** id_pago (PK, Int), empresa_id (FK, Int), payment_intent_id (VarChar 100, Unique), monto (Decimal 10,2), moneda (VarChar 5, ej. "USD", "BOB"), estado_pago (Enum: SUCCEEDED, REQUIRES_ACTION, FAILED).
- **Funcionalidades Clave:**
  - Generación de intenciones de cobro seguras con verificación 3D-Secure.
  - Captura y tokenización de tarjetas mediante componentes seguros embebidos (Stripe Elements).

- Recepción y procesamiento criptográfico de webhooks en servidor para confirmación asíncrona.

### A.2.15 Módulo M15: Gestión de Medios y Métodos de Cobro

- **Descripción General:** Parametriza los métodos de pago autorizados en la plataforma (Efectivo en mostrador, QR Simple interoperable BCB, Tarjeta Stripe).
- **Datos Principales:** id_metodo (PK, Int), empresa_id (FK, Int), nombre_metodo (VarChar 50), canal (Enum: PRESENCIAL, DIGITAL, OMNICANAL), activo (Boolean).
- **Funcionalidades Clave:**
  - Habilitación y configuración de canales de cobro por tienda.
  - Generación de código QR Simple dinámico con monto y glosa correlativa para cobros presenciales.
  - Cuadre de caja y conciliación diaria de recaudaciones por cajero y turno.

### A.2.16 Módulo M16: Sistema de Fidelización Gamificado y Membresías

- **Descripción General:** Dinámica de retención y valor de vida del cliente (*CLV*) mediante acumulación de puntos comerciales por compras, ascenso automático entre categorías (Bronce, Plata, Oro, Diamante) y redención de descuentos.
- **Datos Principales:** id_fidelizacion (PK, Int), empresa_id (FK, Int), id_cliente (FK, Int, Unique), puntos_acumulados (Int), nivel (Enum: BRONCE, PLATA, ORO, DIAMANTE), insignias_json (JSONB).
- **Funcionalidades Clave:**
  - Asignación matemática de puntos tras la confirmación de compras efectivas (1 punto por cada 10 Bs).
  - Promoción jerárquica de nivel de membresía con descuentos automáticos permanentes (Plata: 5%, Oro: 10%, Diamante: 15%).
  - Otorgamiento de insignias por hitos comerciales (ej. "Caballero Ejecutivo", "Explorador RA").
  - Canje de puntos como saldo a favor en el checkout de compra.

### A.2.17 Módulo M17: Asistente de Estilo con IA Contextual y Comandos de Voz

- **Descripción General:** Asesor virtual de imagen masculina. Genera recomendaciones personalizadas de atuendos analizando el clima en tiempo real (OpenWeatherMap API), la ocasión de uso y el stock real en tiendas, admitiendo comandos de voz procesados mediante NLP.
- **Datos Principales:** id_recomendacion (PK, Int), empresa_id (FK, Int), id_cliente (FK, Int), temperatura_c (Decimal 4,1), clima_descripcion (VarChar 50), ocasion (VarChar 50), outfit_sugerido_json (JSONB).
- **Funcionalidades Clave:**
  - Conexión a la API meteorológica según la geolocalización de la sucursal o ciudad del cliente.

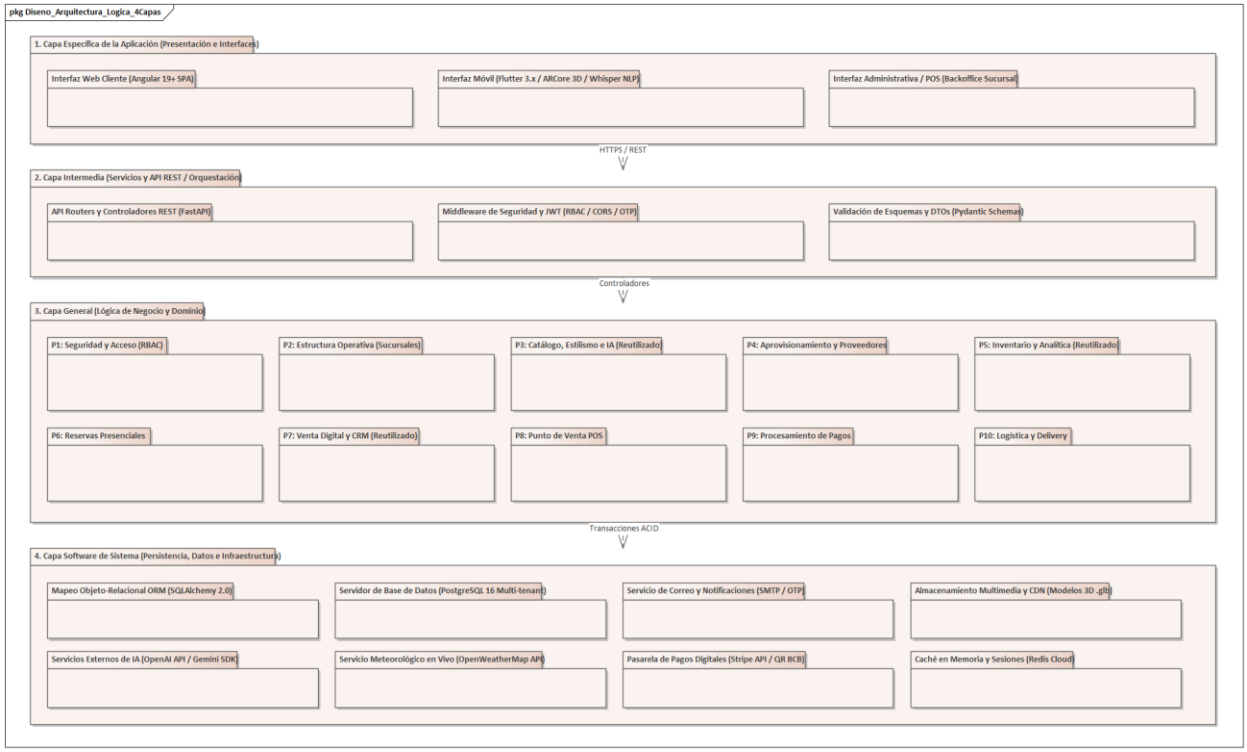
- Formulación de combinaciones de prendas masculinas cruzadas estrictamente con existencias disponibles en stock.
- Captura y transcripción de comandos de voz en la app móvil para búsqueda de prendas y outfits.

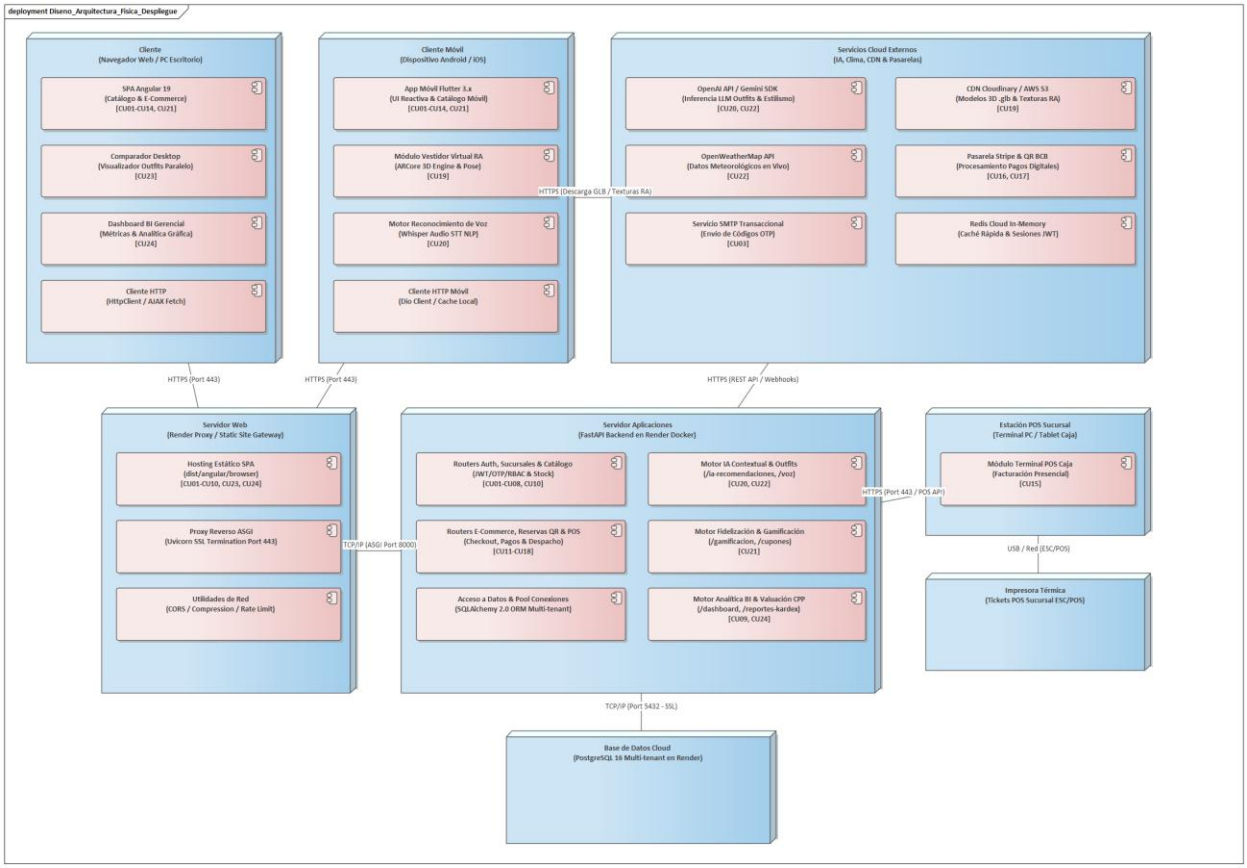
### A.2.18 Módulo M18: Dashboards Gerenciales y Analítica Descriptiva

- **Descripción General:** Cuadro de mando ejecutivo con indicadores clave de desempeño (KPIs) en tiempo real: volumen de ventas, rotación de inventarios, ticket promedio, efectividad de reservas y comparativas de canales.
- **Datos Principales:** Métricas agregadas por tenant: ventas brutas, margen de contribución, prendas más vendidas (*Top 10*), tasa de conversión de vestidores y valor de inventario a CPP.
- **Funcionalidades Clave:**
  - Visualización gráfica de ventas diarias, semanales y mensuales con filtros por sucursal.
  - Reporte de inventario valorado al Costo Promedio Ponderado vs Último Costo de Adquisición.
  - Exportación de métricas analíticas a hojas de cálculo (XLSX) y reportes ejecutivos en PDF.

### A.2.19 Módulo M19: Logística de Envíos y Delivery

- **Descripción General:** Gestiona la distribución física y entrega a domicilio para compras digitales. Calcula tarifas geodésicas (Haversine), peso volumétrico y seguimiento del estado del despacho.
- **Datos Principales:** id_envio (PK, Int), empresa_id (FK, Int), id_orden_venta (FK, Int), id_sucursal_origen (FK, Int), distancia_km (Decimal 6,2), peso_kg (Decimal 5,2), costo_envio (Decimal 10,2), estado_envio (Enum: ASIGNADO, EN_TRANSITO, ENTREGADO, FALLIDO).
- **Funcionalidades Clave:**
  - Selección automática de la sucursal de origen más cercana al comprador que posea stock completo.
  - Cálculo algorítmico del costo de flete combinando distancia kilométrica y peso volumétrico.
  - Rastreo de pedidos en tiempo real para el cliente desde la web y la aplicación móvil.

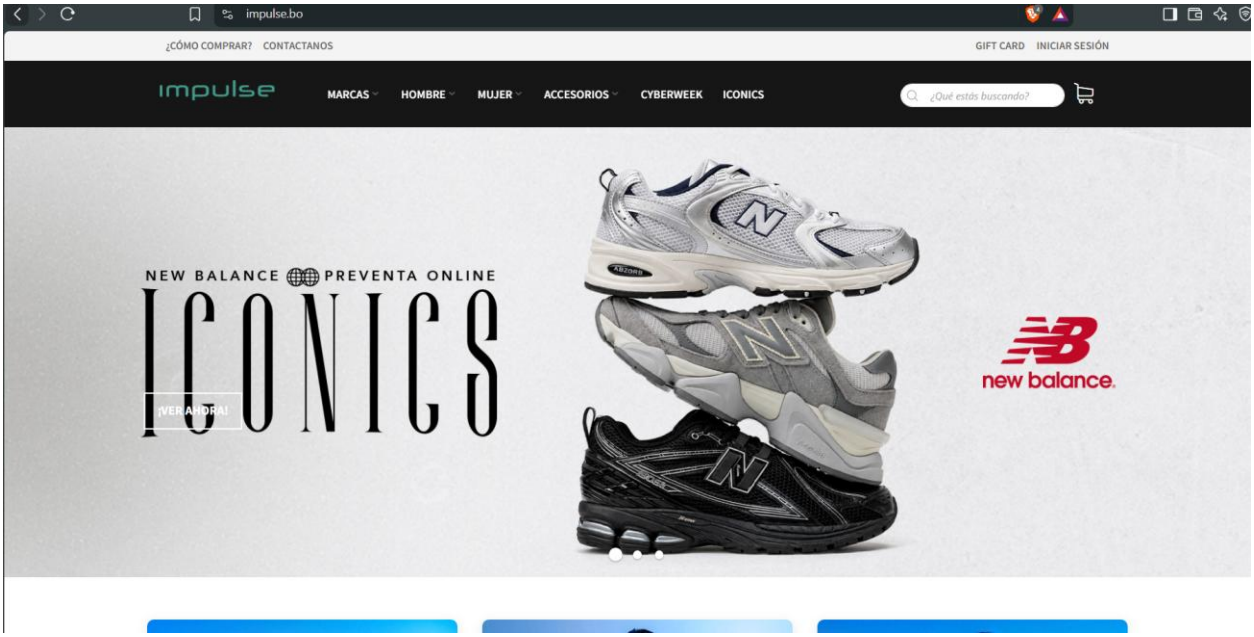

pkg Diseno_Arquitectura_Logica_4Capas
1. Capa Especifica de la Aplicación (Presentación e Interfaces)
Interfaz Web Cliente (Angular 19+ SPA)
Interfaz Móvil (Flutter 3.x / ARCore 3D / Whisper NLP)
Interfaz Administrativa / POS (Backoffice Sucursal)
HTTPS / REST
2. Capa Intermedia (Servicios y API REST / Orquestación)
API Routers y Controladores REST (FastAPI)
Middleware de Seguridad y JWT (RBAC / CORS / OTP)
Validación de Esquemas y DTOs (Pydantic Schemas)
Controladores
3. Capa General (Lógica de Negocio y Dominio)
P1: Seguridad y Acceso (RBAC)
P2: Estructura Operativa (Sucursales)
P3: Catálogo, Estilismo e IA (Reutilizado)
P4: Aprovisionamiento y Proveedores
P5: Inventario y Analítica (Reutilizado)
P6: Reservas Presenciales
P7: Venta Digital y CRM (Reutilizado)
P8: Punto de Venta POS
P9: Procesamiento de Pagos
P10: Logística y Delivery
Transacciones ACID
4. Capa Software de Sistema (Persistencia, Datos e Infraestructura)
Mapeo Objeto-Relacional ORM (SQLAlchemy 2.0)
Servidor de Base de Datos (PostgreSQL 16 Multi-tenant)
Servicio de Correo y Notificaciones (SMTP / OTP)
Almacenamiento Multimedia y CDN (Modelos 3D .glb)
Servicios Externos de IA (OpenAI API / Gemini SDK)
Servicio Meteorológico en Vivo (OpenWeatherMap API)
Pasarela de Pagos Digitales (Stripe API / QR BCB)
Caché en Memoria y Sesiones (Redis Cloud)

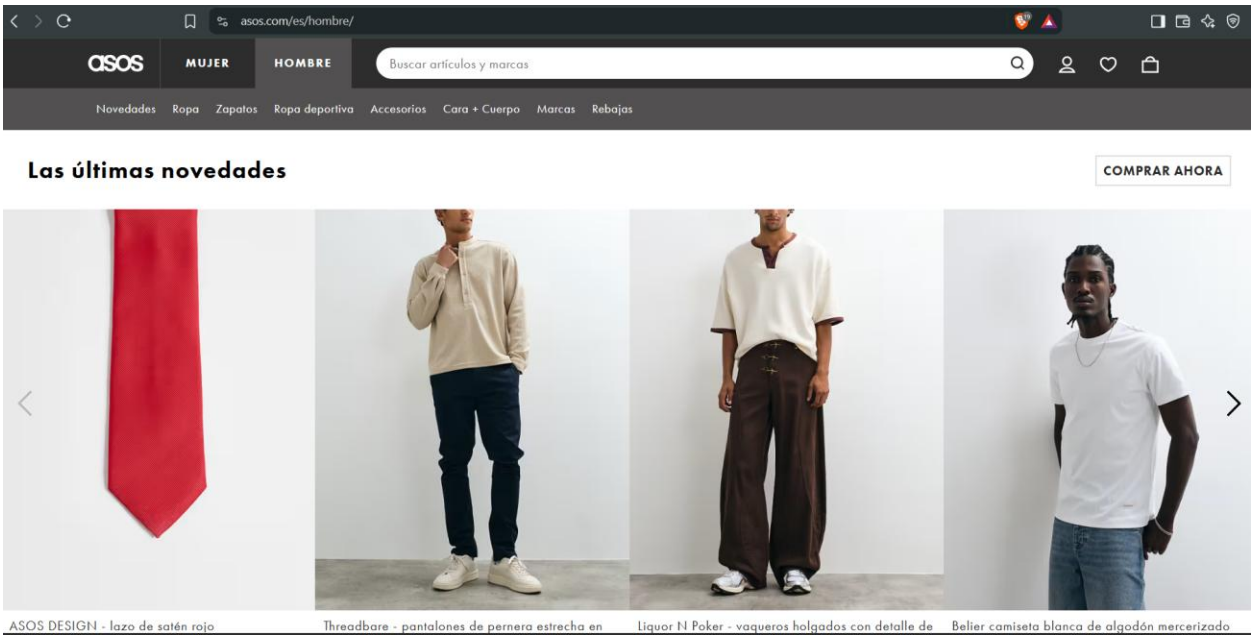

deployment Diseno_Arquitectura_Fisica_Despliegue
Cliente
(Navegador Web / PC Escritorio)
SPA Angular 19
(Catálogo & E-Commerce)
[CU01-CU14, CU21]
Comparador Desktop
(Visualizador Outfits Paralelo)
[CU23]
Dashboard BI Gerencial
(Métricas & Analítica Gráfica)
[CU24]
Cliente HTTP
(HttpClient / AJAX Fetch)
Cliente Móvil
(Dispositivo Android / iOS)
App Móvil Flutter 3.x
(UI Reactiva & Catálogo Móvil)
[CU01-CU14, CU21]
Módulo Vestidor Virtual RA
(ARCore 3D Engine & Pose)
[CU19]
Motor Reconocimiento de Voz
(Whisper Audio STT NLP)
[CU20]
Cliente HTTP Móvil
(Dio Client / Cache Local)
Servicios Cloud Externos
(IA, Clima, CDN & Pasarelas)
OpenAI API / Gemini SDK
(Inferencia LLM Outfits & Estilismo)
[CU20, CU22]
CDN Cloudinary / AWS S3
(Modelos 3D .glb & Texturas RA)
[CU19]
OpenWeatherMap API
(Datos Meteorológicos en Vivo)
[CU22]
Pasarela Stripe & QR BCB
(Procesamiento Pagos Digitales)
[CU16, CU17]
Servicio SMTP Transaccional
(Envío de Códigos OTP)
[CU03]
Redis Cloud In-Memory
(Caché Rápida & Sesiones JWT)
HTTPS (Descarga GLB / Texturas RA)
HTTPS (Port 443)
HTTPS (Port 443)
HTTPS (REST API / Webhooks)
Servidor Web
(Render Proxy / Static Site Gateway)
Hosting Estático SPA
(dist/angular/browser)
[CU01-CU10, CU23, CU24]
Proxy Reverso ASGI
(Uvicorn SSL Termination Port 443)
Utilidades de Red
(CORS / Compression / Rate Limit)
TCP/IP (ASGI Port 8000)
Servidor Aplicaciones
(FastAPI Backend en Render Docker)
Routers Auth, Sucursales & Catálogo
(/JWT/OTP/RBAC & Stock)
[CU01-CU08, CU10]
Routers E-Commerce, Reservas QR & POS
(Checkout, Pagos & Despacho)
[CU11-CU18]
Acceso a Datos & Pool Conexiones
(SQLAlchemy 2.0 ORM Multi-tenant)
Motor IA Contextual & Outfits
(/ia-recomendaciones, /voz)
[CU20, CU22]
Motor Fidelización & Gamificación
(/gamificacion, /cupones)
[CU21]
Motor Analítica BI & Valuación CPP
(/dashboard, /reportes-kardex)
[CU09, CU24]
HTTPS (Port 443 / POS API)
Estación POS Sucursal
(Terminal PC / Tablet Caja)
Módulo Terminal POS Caja
(Facturación Presencial)
[CU15]
USB / Red (ESC/POS)
Impresora Térmica
(Tickets POS Sucursal ESC/POS)
TCP/IP (Port 5432 - SSL)
Base de Datos Cloud
(PostgreSQL 16 Multi-tenant en Render)

## Anexo B: Tabla Comparativa de Planes de Suscripción

| Característica / Módulo | Plan Básico | Plan Profesional |
|---|---|---|
| **Sucursales activas permitidas** | Hasta 2 | Hasta 10 |
| **Usuarios administrativos y cajeros** | Ilimitados | Ilimitados |
| **Gestión de catálogo, tallas, colores y telas** | SI | SI |
| **Control de inventario multi-sucursal con CPP** | SI | SI |
| **Módulo de ventas POS y digital** | SI | SI |
| **Reservas omnicanal** | SI | SI |
| **Operación offline en PWA y Flutter** | SI | SI |
| **Sincronización idempotente con UUID** | SI | SI |
| **Gestión de cambios de prendas (dos notas)** | SI | SI |
| **Traspasos inter-sucursal** | SI | SI |
| **Pasarela de pagos Stripe** | SI | SI |
| **Reportes convencionales (Excel/PDF)** | SI | SI |
| **Reportes generativos con IA (Lenguaje Natural)** | ❌ *(Bloqueado por Plan)* | SI |
| **Dashboard Analítico con Machine Learning** | ❌ *(Bloqueado por Plan)* | SI |

## Anexo C: Referencias




asos.com/es/hombre/
asos
MUJER
HOMBRE
Buscar artículos y marcas
Novedades
Ropa
Zapatos
Ropa deportiva
Accesorios
Cara + Cuerpo
Marcas
Rebajas
Las últimas novedades
COMPRAR AHORA
ASOS DESIGN - lazo de satén rojo
Threadbare - pantalones de pernera estrecha en
Liquor N Poker - vaqueros holgados con detalle de
Belier camiseta blanca de algodón mercerizado

## Enlace repositorio y  QR

“ https://github.com/INGvainilla/1erParcial.git”

## Enlace despliegue

- **Web:**
- https://despliegue-parcial1.onrender.com/app/#/catalogo